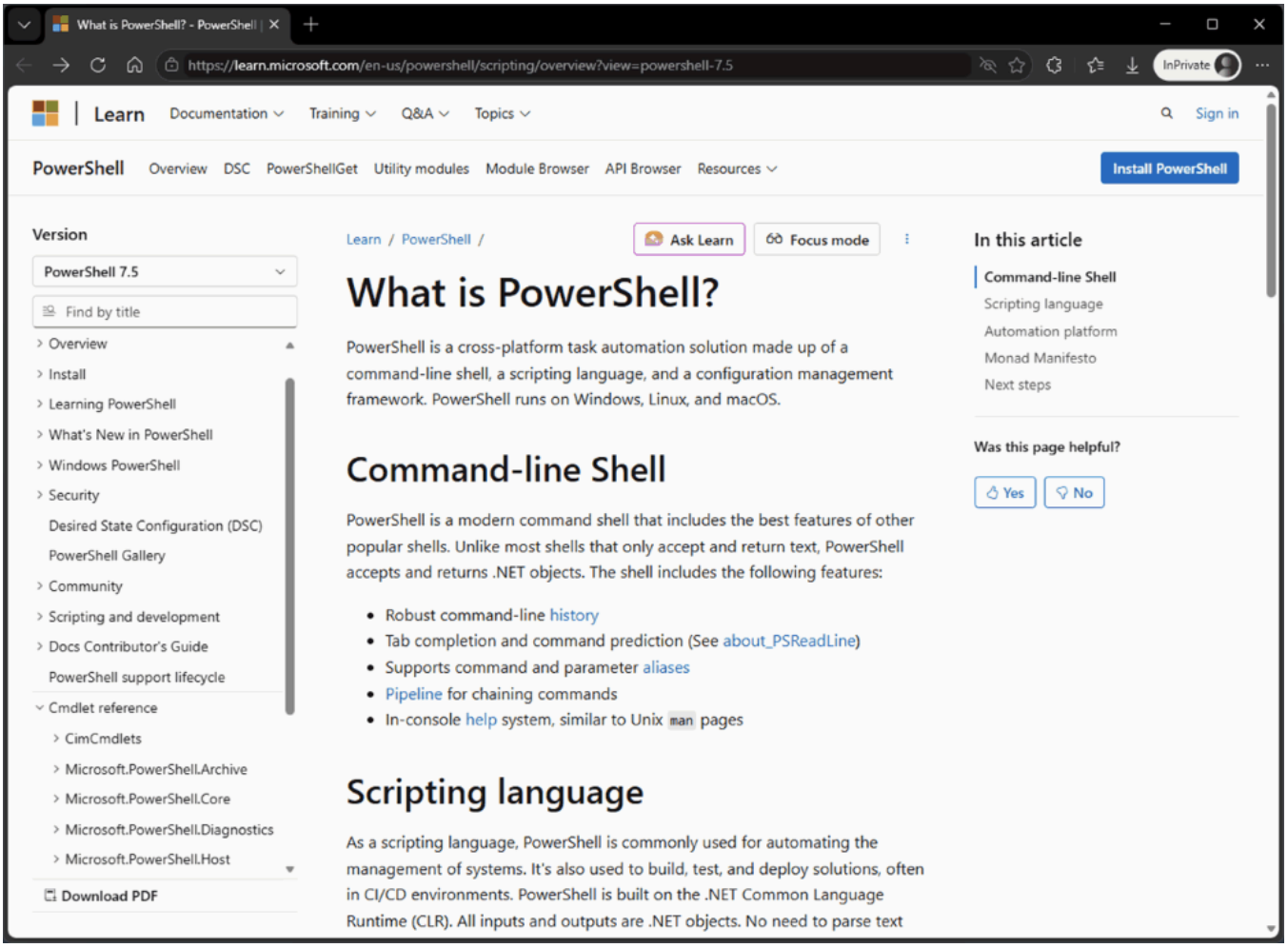
How to use the PowerShell documentation

Welcome to the PowerShell online documentation.

Features of the Learn platform

The web page contains multiple elements that help you use and navigate the documentation.



Navigation elements

- **Site level navigation** - The site level navigation appears at the top of the page. It contains links to other content on the Microsoft Learn platform.
- **Related content navigation** - The related content bar is immediately below the site level navigation. It contains links to content related to the current documentation set, which is PowerShell in this case.
- **Article navigation** - The article navigation appears at the top right of the article. It contains links to sections within the article.

Table of Contents

- **Version selector** - The version selector appears above the Table of Contents (TOC) and controls which version of the **cmdlet reference** appears in the TOC.
- The **Filter and search** box allows you to quickly find articles in the TOC by filtering on words that appear in the title of an article.
- **Conceptual content** - The top section of the TOC contains conceptual articles that aren't version-specific.
- **Cmdlet reference** - The bottom section of the TOC contains the cmdlet reference for the version of PowerShell selected in the version selector.

Action and menu buttons

These buttons provide other ways of interacting with the content.

- The **Ask Learn** button opens an AI chat pane where you can ask questions and get help about the content.
- The **Focus mode** button hides the TOC and other distractions, allowing you to focus on the content.
- The **Menu** button provides a way to add content to a collection, provide feedback, edit the content, or share the content with others.

Feedback elements

There are two ways to provide feedback on the documentation.

- **Anonymous feedback** - The **Was this helpful?** section on the right side of the article allows you to provide a thumbs-up or thumbs-down rating. You can also enter more feedback in a text box.
- **Feedback on GitHub** - At the bottom of each article, you can provide feedback on the documentation or the product. These links take you to the GitHub repository where you can open an issue and give feedback.

Filter and search for articles

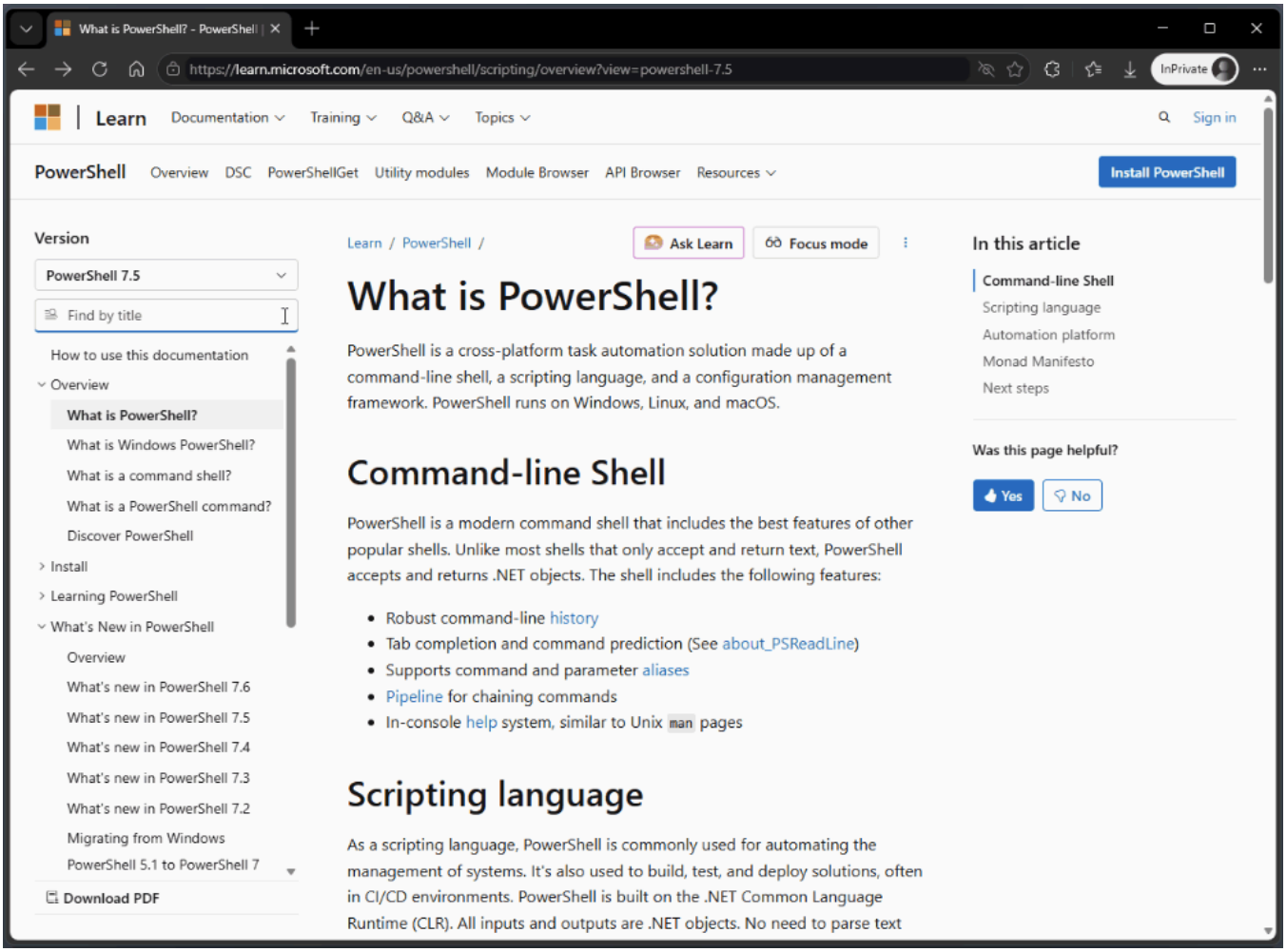
There are two ways to search for content in Docs.

- The filter and search box under the version selector allows filtering articles by words that appear in the title of an article or command aliases. The filter displays a list of matching articles as you type. You can also select the option to search for the words in the body of articles. When you search from here, the search is limited to the PowerShell documentation.

- The search box in the site-level navigation bar searches the entire site. It returns a list of matching articles from all documentation sets.

In the following example, you see how the drop-down list is filtered to show the `Get-ChildItem` command as you type the name. When you enter `gci`, the filter shows the `Get-CimInstance` and `Get-ChildItem` commands because the alias for both commands starts with `gci`.

Next, the word `idempotent` is entered. The filter shows no articles. Clicking the search link searches for `idempotent` in the PowerShell documentation. This search only returns 12 results. Notice the difference when the same word is searched using the site-level search box. The search returns 1,096 articles from the entire Microsoft Learn site.



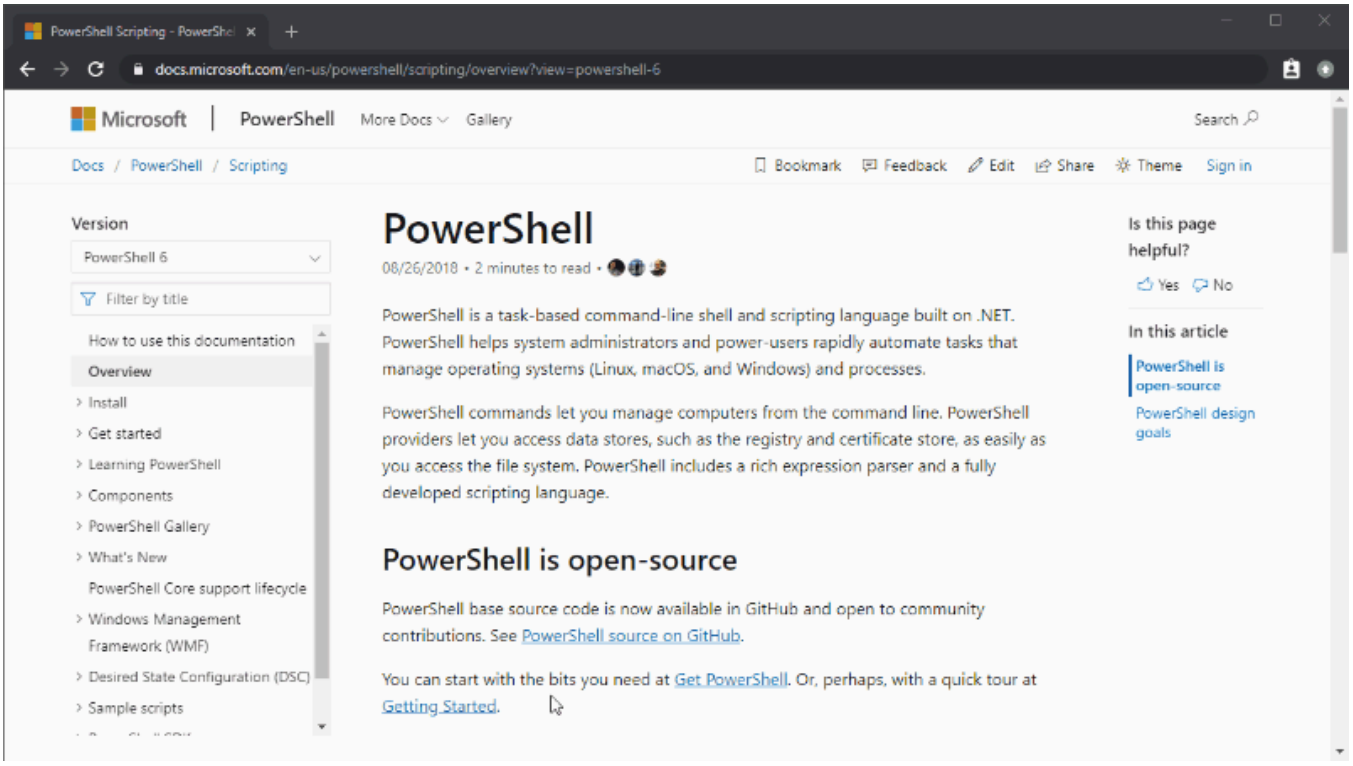
Selecting the version of PowerShell

This site contains cmdlet reference for the following versions of PowerShell:

- PowerShell 7.6 (LTS)
- PowerShell 7.5
- PowerShell 7.4 (LTS)
- PowerShell 5.1

Use the version selector located above the TOC to select the version of PowerShell you want. By default, the page loads with the most current stable release version selected. The version selector controls which version of the cmdlet reference appears in the TOC under the **Reference** node. Some cmdlets work differently in different versions of PowerShell you're using. Be sure you're viewing the documentation for the correct version of PowerShell.

The version selector doesn't affect conceptual documentation. The conceptual documents appear above the **Reference** node in the TOC. The same conceptual articles appear for every version selected. If there are version-specific differences, the documentation makes note of those differences.



You can verify the version of PowerShell you're using by inspecting the `$PSVersionTable.PSVersion` value. The following example shows the output for Windows PowerShell 5.1.

PowerShell

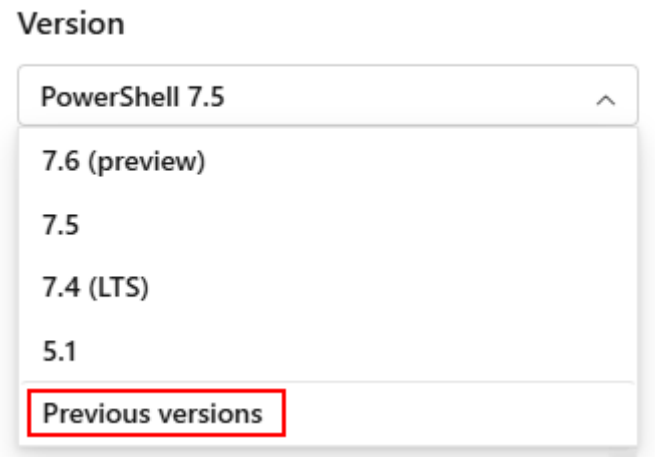
```
$PSVersionTable.PSVersion
```

Output

```
Major  Minor  Build  Revision
-----  -----  -----  --------
5      1      26100  7705
```

Find articles for older versions of PowerShell

Documentation for older versions of PowerShell is archived in our Previous Versions site. You can choose **Previous Versions** from the version selector.

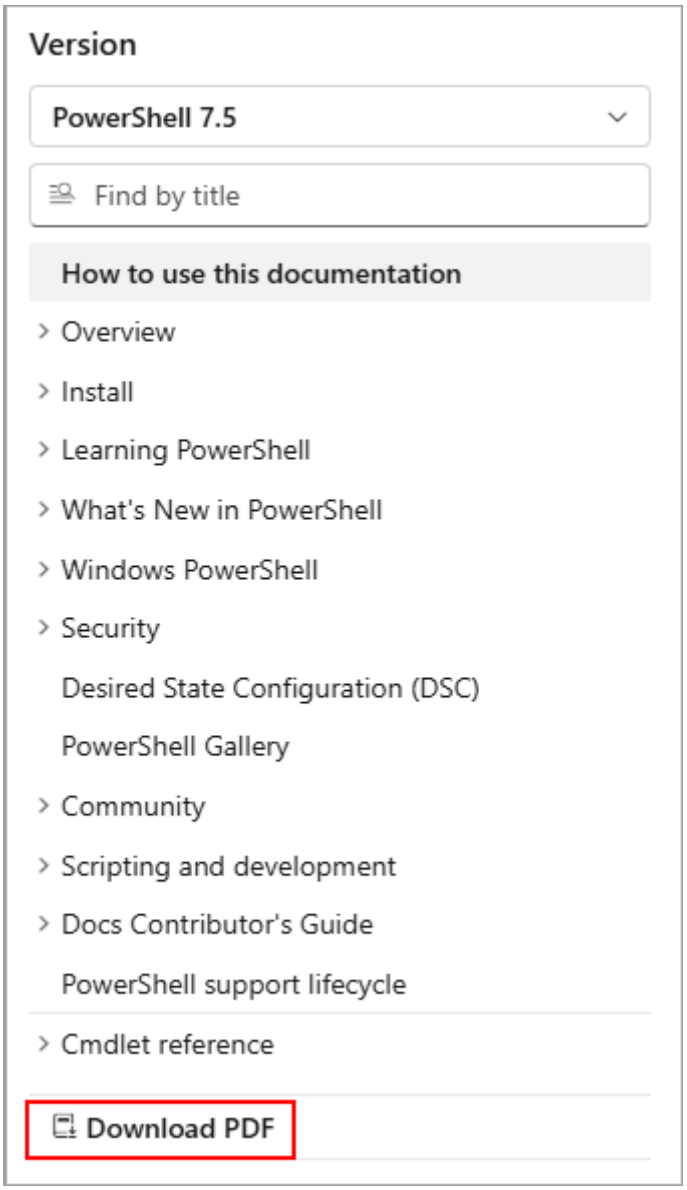


The **Previous versions** option takes you to the site containing documentation for older and unsupported versions of PowerShell:

- PowerShell 7.3
- PowerShell 7.2
- PowerShell 7.1
- PowerShell 7.0
- PowerShell 6
- PowerShell 5.0
  - WMF 5.x Release notes
  - PowerShell ISE object model
  - PowerShell Workflows
  - PowerShell Web Access
- PowerShell 4.0
- PowerShell 3.0

Download the documentation as a PDF

To download the documentation as a PDF, select the **Download PDF** button at the bottom of the TOC.



- If you're viewing a conceptual article, the Learn platform creates a PDF containing the conceptual content for the selected version.
- If you're viewing a reference article, the Learn platform creates a PDF containing the reference content for the selected version.

Last updated on 03/18/2026

What is PowerShell?

PowerShell is a cross-platform task automation solution made up of a command-line shell, a scripting language, and a configuration management framework. PowerShell runs on Windows, Linux, and macOS.

Command-line Shell

PowerShell is a modern command shell that includes the best features of other popular shells. Unlike most shells that only accept and return text, PowerShell accepts and returns .NET objects. The shell includes the following features:

- Robust command-line history
- Tab completion and command prediction (See about_PSReadLine)
- Supports command and parameter aliases
- Pipeline for chaining commands
- In-console help system, similar to Unix `man` pages

Scripting language

As a scripting language, PowerShell is commonly used for automating the management of systems. It's also used to build, test, and deploy solutions, often in CI/CD environments. PowerShell is built on the .NET Common Language Runtime (CLR). All inputs and outputs are .NET objects. No need to parse text output to extract information from output. The PowerShell scripting language includes the following features:

- Extensible through functions, classes, scripts, and modules
- Extensible formatting system for easy output
- Extensible type system for creating dynamic types
- Built-in support for common data formats like CSV, JSON, and XML

Automation platform

The extensible nature of PowerShell provides an ecosystem of PowerShell modules to deploy and manage almost any technology you work with. For example:

Microsoft modules

- Azure
- Windows
- Exchange
- SQL

Third-party modules

- AWS
- VMware
- Oracle Cloud

Configuration management

PowerShell Desired State Configuration (DSC) is a management framework in PowerShell that enables you to manage your enterprise infrastructure with configuration as code. With DSC, you can:

- Create declarative configurations and custom scripts for repeatable deployments
- Enforce configuration settings and report on configuration drift
- Deploy configuration using push or pull models

Monad Manifesto

Jeffrey Snover, the inventor of PowerShell, wrote the Monad Manifesto to explain his vision for PowerShell and how it would change the way we manage systems. Use the following link to download a copy of the Monad Manifesto.

This PDF file is a version of the original Monad Manifesto, which articulated the long-term vision and started the development effort that became PowerShell. PowerShell has delivered on many of the elements described in this document.

Next steps

Getting started

Are you new to PowerShell and don't know where to start? Take a look at these resources.

- Install PowerShell
- Discover PowerShell
- PowerShell 101

- Microsoft Virtual Academy videos
- PowerShell Learn modules

PowerShell in action

Take a look at how PowerShell is being used in different scenarios and on different platforms.

- PowerShell remoting over SSH
- Getting started with Azure PowerShell
- Building a CI/CD pipeline with DSC
- Managing Microsoft Exchange

Last updated on 08/12/2026

What is Windows PowerShell?

Windows PowerShell and ***PowerShell*** are two separate products.

- ***Windows PowerShell*** is the version of PowerShell that ships in Windows. This version of PowerShell uses the full .NET Framework, which only runs on Windows. The latest version is Windows PowerShell 5.1. Microsoft is no longer updating Windows PowerShell with new features. Support for Windows PowerShell is tied to the version of Windows you are using.

- ***PowerShell*** is built on the new versions of .NET instead of the .NET Framework and runs on Windows, Linux, and macOS. Support for PowerShell is based on the version of .NET that it was built on. For more information about the support lifecycle for PowerShell, see the PowerShell support lifecycle documentation.

Further reading

- For a more detailed explanation of the differences between Windows PowerShell and PowerShell, see Differences between Windows PowerShell 5.1 and PowerShell 7.x.
- For information about migrating from Windows PowerShell to PowerShell, see Migrating from Windows PowerShell 5.1 to PowerShell 7.
- For more information about previous versions of Windows PowerShell, see Previous versions of PowerShell.
- For more information about the terminology used in PowerShell documentation, see Product terminology and branding guidelines.

Last updated on 11/20/2025

What is a command shell?

Many people use the terms *command shell*, *command-line tool*, and *terminal* interchangeably, which can be confusing. This article explains the difference between these concepts and provides examples of each.

A **command shell** is an interactive command-line interface for managing a computer, also known as a **Read-Eval-Print Loop** (REPL).

A shell takes input from the keyboard, evaluates that input, and executes the input as a shell command or forwards the input to the operating system to be executed. Most shells can also read commands from a script file, and can include programming features like variables, flow control, and functions.

Types of command shells

There are two main types of command shells:

- General purpose command shells

  General purpose command shells are designed to work with the operating system and allow you to run any command that the operating system supports. They also include shell-specific commands and programming features. The following list contains some examples of general purpose command shells:
  - PowerShell
  - Windows Command Shell
  - bash - popular on Linux
  - zsh - popular on macOS

- Utility command shells

  Utility command shells are designed to work with specific applications or services. These shells can only run commands that are specific to the application or service. Some utility shells support running commands from a batch script, but don't include programming features. Usually, these shells can only be used interactively.
  - AI Shell - An interactive-only shell used to communicate with AI services such as Azure OpenAI.

    - netsh - Network shell (netsh) is a command-line utility that allows you to configure and display the status of various network components on Windows. It's both a command-line tool and a command shell. It also supports running commands from a script file.

Command-line tools

A **command-line tool** is a standalone program that you run from a command shell. Command-line tools are typically designed to perform a specific task, such as managing files, configuring settings, or querying for information. Command-line tools can be used in any shell that supports running external programs.

- Azure CLI - a collection of command-line tools for managing Azure resources that can be run in any supported shell.
- Azure PowerShell - a collection of PowerShell modules for managing Azure resources that can be run in any supported version of PowerShell.
- OpenSSH for Windows - includes a command-line client and a server that provides secure communication over a network.
- Windows Commands - a collection of command-line tools that are built into Windows.

In general, command-line tools don't provide a command shell (REPL) interface. The `netsh` command in Windows is an exception, as it's both a command-line tool and an interactive command shell.

Terminals

A **terminal** is an application that provides a text-based window for hosting command shells. Some terminals are designed to work with a specific shell, while others can host multiple shells. They can also include advanced features such as:

- Ability to create multiple panes within a single window
- Ability to create multiple tabs to host multiple shells
- Ability to change color schemes and fonts
- Support for copy and paste operations

The following list contains some examples of terminal applications:

- Windows Terminal - a modern terminal application for Windows that can host multiple shells.
- Windows Console Host - the default host application on Windows for text-based applications. It can also host the Windows Command Shell or PowerShell.

- Terminal for macOS - the default terminal application on macOS that can host the bash or zsh shell.
- iTerm2 for macOS - a popular 3rd-party terminal application for macOS.
- Azure Cloud Shell - a browser-based terminal application hosted in Microsoft Azure. Azure Cloud shell gives you the choice of using bash or PowerShell. Each shell comes preconfigured with many command-line tools for managing Azure resources.

Last updated on 07/07/2025

What is a PowerShell command (cmdlet)?

Article • 03/07/2024

Commands for PowerShell are known as cmdlets (pronounced command-lets). In addition to cmdlets, PowerShell allows you to run any command available on your system.

What is a cmdlet?

Cmdlets are native PowerShell commands, not stand-alone executables. Cmdlets are collected into PowerShell modules that can be loaded on demand. Cmdlets can be written in any compiled .NET language or in the PowerShell scripting language itself.

Cmdlet names

PowerShell uses a *Verb-Noun* name pair to name cmdlets. For example, the `Get-Command` cmdlet included in PowerShell is used to get all the cmdlets that are registered in the command shell. The verb identifies the action that the cmdlet performs, and the noun identifies the resource on which the cmdlet performs its action.

Next steps

To learn more about PowerShell and how to find other cmdlets, see the PowerShell Bits tutorial Discover PowerShell.

For more information about creating your own cmdlets, see the following resources:

Script-based cmdlets

- about_Functions_Advanced
- about_Functions_CmdletBindingAttribute
- about_Functions_Advanced_Methods

Compiled cmdlets (PowerShell SDK docs)

- Cmdlet overview

Discover PowerShell

PowerShell is a command-line shell and a scripting language in one. PowerShell started out on Windows to help automate administrative tasks. Now, it runs cross platform and can be used for various tasks.

What makes PowerShell unique is that it accepts and returns .NET objects, rather than text. This feature makes it easier to connect different commands in a *pipeline*.

What can PowerShell be used for?

Initially, PowerShell was Windows-only. Now, it's cross-platform and can be used for various tasks like:

- **Cloud management**. PowerShell can be used to manage cloud resources. For example, you can retrieve information, update, or deploy new resources.
- **CI/CD**. It can also be used as part of a Continuous Integration/Continuous Deployment pipeline.
- **Automate tasks for Active Directory and Exchange**. You can use it to automate almost any task on Windows like creating users in Active Directory and mailboxes in Exchange.

Who uses PowerShell?

PowerShell is a powerful tool that can help people working in a multitude of roles. Traditionally, PowerShell was used by the System Administrators. Now it's being used by people calling themselves DevOps, Cloud Ops, and even Developers.

PowerShell cmdlets

PowerShell comes with hundreds of preinstalled commands. PowerShell commands are called cmdlets (pronounced *command-lets*).

The name of each cmdlet consists of a *Verb-Noun* pair. For example, `Get-Process`. This naming convention makes it easier to understand what the cmdlet does. It also makes it easier to find the command you're looking for. When looking for a cmdlet to use, you can filter on the verb or noun.

Using cmdlets to explore PowerShell

When you first pick up PowerShell, it might feel intimidating as there seems to be so much to learn. PowerShell is designed to help you learn a little at a time, as you need it.

PowerShell includes cmdlets that help you discover PowerShell. Using these four cmdlets, you can discover what commands are available, what they do, and what types they operate on.

- `Get-Verb`. Running this command returns a list of verbs that most commands adhere to. The response includes a description of what these verbs do. Since most commands follow this naming convention, it sets expectations on what a command does. This command helps you select the appropriate verb and what to name a command when you create your own commands.
- `Get-Command`. This command retrieves a list of all commands installed on your machine.
- `Get-Member`. It operates on object based output and is able to discover what object, properties and methods are available for a command.
- `Get-Help`. Invoking this command with the name of a command as an argument displays a help page describing various parts of a command.

Using these commands, you can discover almost anything you need to know about PowerShell.

Verb

Verb is an important concept in PowerShell. It's a naming standard that most cmdlets follow. It's also a naming standard you're expected to follow when you write your own commands. The idea is that the *Verb* says what you're trying to do, like read or maybe change data. PowerShell has a standardized list of verbs. To get a full list of all possible verbs, use the `Get-Verb` cmdlet:

PowerShell

```
Get-Verb
```

The cmdlet returns a long list of verbs. The **Description** provides context for what the verb is meant to do. Here's the first few rows of output:

Output

```
Verb    AliasPrefix   Group     Description
----    -----------   -----     -----------
Add     a             Common    Adds a resource to a container, or attaches an item
to another item
Clear   cl            Common    Removes all the resources from a container but does
not delete the container
Close   cs            Common    Changes the state of a resource to make it
inaccessible, unavailable, or unusab…
```

```
Copy    cp            Common    Copies a resource to another name or to another
container
Enter   et            Common    Specifies an action that allows the user to move
into a resource
Exit    ex            Common    Sets the current environment or context to the most
recently used context
...
```

Find commands with Get-Command

The `Get-Command` cmdlet returns a list of all available commands installed on your system. The list can be large. You can limit the amount of information that comes back by filtering the response using parameters or helper cmdlets.

Filter on name

You can filter the output of `Get-Command` using different parameters. Filtering allows you to find commands that have certain properties. The **Name** parameter allows you to find a specific command by name.

PowerShell

```
Get-Command -Name Get-Process
```

Output

```
CommandType     Name              Version    Source
-----------     ----              -------    ------
Cmdlet          Get-Process       7.0.0.0    Microsoft.PowerShell.Management
```

What if you want to find all the commands that work with processes? You can use a wildcard `*` to match other forms of the string. For example:

PowerShell

```
Get-Command -Name *-Process
```

Output

```
CommandType     Name              Version    Source
-----------     ----              -------    ------
Cmdlet          Debug-Process     7.0.0.0    Microsoft.PowerShell.Management
Cmdlet          Get-Process       7.0.0.0    Microsoft.PowerShell.Management
Cmdlet          Start-Process     7.0.0.0    Microsoft.PowerShell.Management
```

```
Cmdlet          Stop-Process       7.0.0.0    Microsoft.PowerShell.Management
Cmdlet          Wait-Process       7.0.0.0    Microsoft.PowerShell.Management
```

Filtering on Noun and Verb

There are other parameters that filter on verb and noun values. The verb part of a command's name is the leftmost part. The verb should be one of the values returned by the `Get-Verb` cmdlet. The rightmost part of a command is the noun part. A noun can be anything.

- **Filter on verb**. In the command `Get-Process`, the verb part is `Get`. To filter on the verb part, use the **Verb** parameter.

  PowerShell

  ```powershell
  Get-Command -Verb 'Get'
  ```

  This example lists all commands that use the verb `Get`.

- **Filter on noun**. In the command `Get-Process`, the noun part is `Process`. To filter on the noun, use the **Noun** parameter. The following example returns all cmdlets that have nouns starting with the letter `U`.

  PowerShell

  ```powershell
  Get-Command -Noun U*
  ```

Also, you can combine parameters to narrow down your search, for example:

PowerShell

```powershell
Get-Command -Verb Get -Noun U*
```

Output

```
CommandType     Name                                Version    Source
-----------     ----                                -------    ------
Cmdlet          Get-UICulture                       7.0.0.0
Microsoft.PowerShell.Utility
Cmdlet          Get-Unique                          7.0.0.0
Microsoft.PowerShell.Utility
Cmdlet          Get-Uptime                          7.0.0.0
Microsoft.PowerShell.Utility
```

Use helper cmdlets to filter results

You can also use other cmdlets to filter results.

- `Select-Object`. This versatile command helps you pick out specific properties from one or more objects. You can also limit the number of items you get back. The following example returns the **Name** and **Source** property values for the first 5 commands available in the current session.

  PowerShell

  ```powershell
  Get-Command | Select-Object -First 5 -Property Name, Source
  ```

  Output

  ```
  Name                       Source
  ----                       ------
  Add-AppPackage             Appx
  Add-AppPackageVolume       Appx
  Add-AppProvisionedPackage Dism
  Add-AssertionOperator      Pester
  Add-ProvisionedAppPackage Dism
  ```

  For more information, see Select-Object.

- `Where-Object`. This cmdlet lets you filter the objects returned based on the values of properties. The command takes an expression that can test the value of a property. The following example returns all processes where the `ProcessName` starts with `p`.

  PowerShell

  ```powershell
  Get-Process | Where-Object {$_.ProcessName -like "p*"}
  ```

  The `Get-Process` cmdlet returns a collection of process objects. To filter the response, *pipe* the output to `Where-Object`. Piping means that two or more commands are connected via a pipe `|` character. The output from one command is sent as the input for the next command. The filter expression for `Where-Object` uses the `-like` operator to match processes that start with the letter `p`.

Explore objects with Get-Member

Once you locate the cmdlet you want, you want to know more about what output it produces. The `Get-Member` cmdlet displays the type, properties, and methods of an object. Pipe the output you want to inspect to `Get-Member`.

PowerShell

```
Get-Process | Get-Member
```

The result displays the returned type as `TypeName` and all the properties and methods of the object. Here's an excerpt of such a result:

Output

```
TypeName: System.Diagnostics.Process

Name        MemberType     Definition
----        ----------     ----------
Handles     AliasProperty  Handles = Handlecount
Name        AliasProperty  Name = ProcessName
...
```

Using the **MemberType** parameter you can limit the information returned.

PowerShell

```
Get-Process | Get-Member -MemberType Method
```

By default PowerShell only displays a few properties. The previous example displayed the `Name`, `MemberType`, and `Definition` members. You can use `Select-Object` to specify properties you want to see. For example, you want to display only the `Name` and `Definition` properties:

PowerShell

```
Get-Process | Get-Member | Select-Object Name, Definition
```

Search by parameter type

`Get-Member` showed us that `Get-Process` returns **Process** type objects. The **ParameterType** parameter of `Get-Command` can be used to find other commands that take **Process** objects as input.

PowerShell

```
Get-Command -ParameterType Process
```

Output

```
CommandType     Name                                Version    Source
-----------     ----                                -------    ------
Cmdlet          Debug-Process                       7.0.0.0
Microsoft.PowerShell.Managem…
Cmdlet          Enter-PSHostProcess                 7.1.0.0    Microsoft.PowerShell.Core
Cmdlet          Get-Process                         7.0.0.0
Microsoft.PowerShell.Managem…
Cmdlet          Get-PSHostProcessInfo               7.1.0.0    Microsoft.PowerShell.Core
Cmdlet          Stop-Process                        7.0.0.0
Microsoft.PowerShell.Managem…
Cmdlet          Wait-Process                        7.0.0.0
Microsoft.PowerShell.Managem…
```

Knowing the output type of a command can help narrow down your search for related commands.

Additional resources

- Get-Command
- Get-Member
- Select-Object

Last updated on 02/24/2026

Install PowerShell on Windows, Linux, and macOS

Learn how to install PowerShell on Windows, Linux, and macOS.

Windows

OVERVIEW

Install PowerShell on Windows

Install PowerShell on Windows IoT and Nano Server

Supported Windows releases

macOS

OVERVIEW

Install on macOS

Supported macOS releases

Alternate install methods

Linux

OVERVIEW

Linux overview

Alpine

Debian

Red Hat Enterprise Linux

Ubuntu

Q&A

GET STARTED

Alternate install methods

Community supported Linux

Using PowerShell in Docker

Arm Processor support

Microsoft Update FAQ for PowerShell

PowerShell Support Lifecycle

Install PowerShell 7 on Windows

PowerShell 7 doesn't replace Windows PowerShell 5.1. It installs to a new directory and runs side-by-side with Windows PowerShell 5.1. There are some Windows PowerShell modules that can be run using the PowerShell 7 Windows Compatibility feature. Other modules require that you run them in Windows PowerShell 5.1. For more information, see PowerShell 7 module compatibility.

There are multiple package versions of PowerShell 7 that can be installed. This article focuses on installing the latest stable release package. For more information about the package versions, see the PowerShell Support Lifecycle article.

Choose an installation method

There are multiple ways to install PowerShell in Windows. Each install method is designed to support different scenarios and workflows. Choose the method that best suits your needs.

- WinGet - Recommended way to install PowerShell on Windows clients
- MSI package - Best choice for Windows Servers and enterprise deployment scenarios
- MSIX package - An easy way to install for casual users of PowerShell but has limitations
- ZIP package - Easiest way to *side load* or install multiple versions or install on Windows Server Core, Windows IoT, and Arm-based systems
- .NET Global tool - A good choice for .NET developers that install and use other global tools

Install PowerShell using WinGet

WinGet, the Windows Package Manager, is a command-line tool that enables you to discover, install, upgrade, remove, and configure applications on Windows client computers. This tool is the client interface to the Windows Package Manager service. The `winget` command-line tool is included in Windows 11 and Windows Server 2025 as part of the **App Installer**. The `winget` command can be run from any command shell, including the Windows Command shell (`cmd.exe`), Windows PowerShell (`powershell.exe`), or PowerShell (`pwsh.exe`).

> **Note**
>
> See the **winget documentation** for a list of system requirements and install instructions. `winget` isn't available on Windows Server 2022 or earlier versions. Windows Server 2025 includes `winget` for **Windows Server with Desktop Experience** only.

Use the following `winget` commands to install PowerShell:

Search for the latest version of PowerShell

PowerShell

```
winget search --id Microsoft.PowerShell --exact
```

Output

```
Name               Id                            Version   Source
-----------------------------------------------------------------
PowerShell         Microsoft.PowerShell          7.6.6.0   winget
```

Beginning with the winget package for PowerShell 7.6.0, winget installs the MSIX package by default.

Install the PowerShell 7 MSIX package:

PowerShell

```
winget install --id Microsoft.PowerShell --source winget
```

Use the following command to install the PowerShell 7 MSI package:

PowerShell

```
winget install --id Microsoft.PowerShell --source winget --installer-type wix
```

Alternatively, you can manually download and install the MSI package.

Use the following command to install PowerShell 7.7-preview packages:

PowerShell

```
winget install --id Microsoft.PowerShell.Preview --source winget
```

Beginning with the PowerShell 7.7.0 release, there is no MSI package available. WinGet only installs the MSIX package.

Install the MSI package

To install PowerShell on Windows, use the following links to download the install package from GitHub.

Latest stable release:

- PowerShell-7.6.6-win-x64.msi
- PowerShell-7.6.6-win-arm64.msi

Once downloaded, double-click the installer file and follow the prompts.

Install the MSI package with command-line options

MSI packages can be installed from the command line allowing administrators to deploy packages without user interaction. The MSI package includes the following properties to control the installation options:

- `USE_MU` - This property has two possible values:
  - `1` (default) - Opts into updating through Microsoft Update, WSUS, or Configuration Manager
  - `0` - Don't opt into updating through Microsoft Update, WSUS, or Configuration Manager
- `ENABLE_MU`
  - `1` (default) - Opts into using Microsoft Update for Automatic Updates
  - `0` - Don't opt into using Microsoft Update

    > **Note**
    >
    > Enabling updates may have been set in a previous installation or manual configuration. Using `ENABLE_MU=0` doesn't remove the existing settings. Also, this setting can be overruled by Group Policy settings controlled by your administrator.

- `ADD_EXPLORER_CONTEXT_MENU_OPENPOWERSHELL` - This property controls the option for adding the `Open PowerShell` item to the context menu in Windows Explorer.
- `ADD_FILE_CONTEXT_MENU_RUNPOWERSHELL` - This property controls the option for adding the `Run with PowerShell` item to the context menu in Windows Explorer.
- `ENABLE_PSREMOTING` - This property controls the option for enabling PowerShell remoting during installation.

- `REGISTER_MANIFEST` - This property controls the option for registering the Windows Event Logging manifest.
- `ADD_PATH` - This property controls the option for adding PowerShell to the Windows PATH environment variable.
- `DISABLE_TELEMETRY` - This property controls the option for disabling PowerShell's telemetry by setting the `POWERSHELL_TELEMETRY_OPTOUT` environment variable.
- `INSTALLFOLDER` - This property controls the installation directory. The default is `$Env:ProgramFiles\PowerShell\`. This is the location where the installer creates the versioned subfolder. You can't change the name of the versioned subfolder.
  - For current releases, the versioned subfolder is `7`
  - For preview releases, the versioned subfolder is `7-preview`

The following example shows how to silently install PowerShell with all the install options enabled.

PowerShell

```
$msiParams = @(
    '/package'
    'PowerShell-7.6.6-win-x64.msi'
    '/quiet'
    'ADD_EXPLORER_CONTEXT_MENU_OPENPOWERSHELL=1'
    'ADD_FILE_CONTEXT_MENU_RUNPOWERSHELL=1'
    'ENABLE_PSREMOTING=1'
    'REGISTER_MANIFEST=1'
    'USE_MU=1'
    'ENABLE_MU=1'
    'ADD_PATH=1'
)
msiexec.exe @msiParams
```

For a full list of command-line options for `Msiexec.exe`, see Command line options.

Install the ZIP package

PowerShell binary ZIP archives are provided to enable advanced deployment scenarios. Download one of the following ZIP archives from the current release page.

- PowerShell-7.6.6-win-x64.zip
- PowerShell-7.6.6-win-arm64.zip

Depending on how you download the file you may need to unblock the file using the `Unblock-File` cmdlet. Unzip the contents to the location of your choice and run `pwsh.exe` from there. Unlike installing the MSI packages, installing the ZIP archive doesn't check for prerequisites. For remoting over WSMan to work properly, ensure that you've met the prerequisites.

Use this method to install the ARM-based version of PowerShell on computers like the Microsoft Surface Pro X. For best results, install PowerShell to the `$Env:ProgramFiles\PowerShell\7` folder. If you are installing an additional version of PowerShell 7 side-by-side with an existing version of PowerShell 7, install the additional version to a different folder. You must manually add a shortcut to the Start Menu and add the location to the PATH environment variable.

Install as a .NET Global tool

If you already have the .NET Core SDK installed, you can install PowerShell as a .NET Global tool.

```
dotnet tool install --global PowerShell
```

The dotnet tool installer adds `$HOME\.dotnet\tools` to your `$Env:PATH` environment variable. However, the currently running shell doesn't have the updated `$Env:PATH`. You can start PowerShell from a new shell by typing `pwsh`.

Install the MSIX package

PowerShell can be installed from the Microsoft Store or by manually downloading the MSIX package.

Benefits of the Microsoft Store package:

- Automatic updates built right into Windows
- Integrates with other software distribution mechanisms like Intune and Configuration Manager
- Can install on Windows systems using x64 or Arm64 processors

To manually install the MSIX package, download one of the following packages from the GitHub releases page and double-click the file to install it.

- Current LTS - PowerShell-7.6.6.msixbundle
- Latest stable - PowerShell-7.5.11.msixbundle
- Previous LTS - PowerShell-7.4.20.msixbundle

- Current preview - PowerShellPreview-7.7.0-preview.5.msixbundle

Alternatively, you can use the following command to install the MSIX package from the command line:

```PowerShell
Add-AppxPackage -Path ".\PowerShell-7.6.6.msixbundle"
```

Limitations of a MSIX-based installation

Store-based installations are installed for a single user. There is no option to install it for all users. Microsoft Store packages run in an application sandbox that virtualizes access to some filesystem and registry locations. Changes to virtualized file and registry locations don't persist outside of the application sandbox.

Store-based installations don't support PowerShell remoting. The application sandbox blocks all changes to the application's root folder. Any system-level configuration settings stored in `$PSHOME` can't be modified. This includes the WSMAN configuration. This prevents remote sessions from connecting to Store-based installs of PowerShell. User-level configurations and SSH remoting for outbound connections are supported.

The following commands aren't supported in a Microsoft Store instance of PowerShell. These commands need write access to `$PSHOME`.

- `Register-PSSessionConfiguration`
- `Update-Help -Scope AllUsers`
- `Enable-ExperimentalFeature -Scope AllUsers`
- `Set-ExecutionPolicy -Scope LocalMachine`

For MSIX-based installations, you can't create or modify the all-users profile scripts `$PROFILE.AllUsersAllHosts` and `$PROFILE.AllUsersCurrentHost` because those profiles must be stored in `$PSHOME`. Current-user profiles are still supported.

The PowerShell package is exempt from file and registry virtualization. Changes to virtualized file and registry locations now persist outside of the application sandbox. However, changes to the application's root folder are still blocked.

For more information, see Understanding how packaged desktop apps run on Windows.

> **Important**

You must be running on Windows build 1903 or higher for this exemption to work.

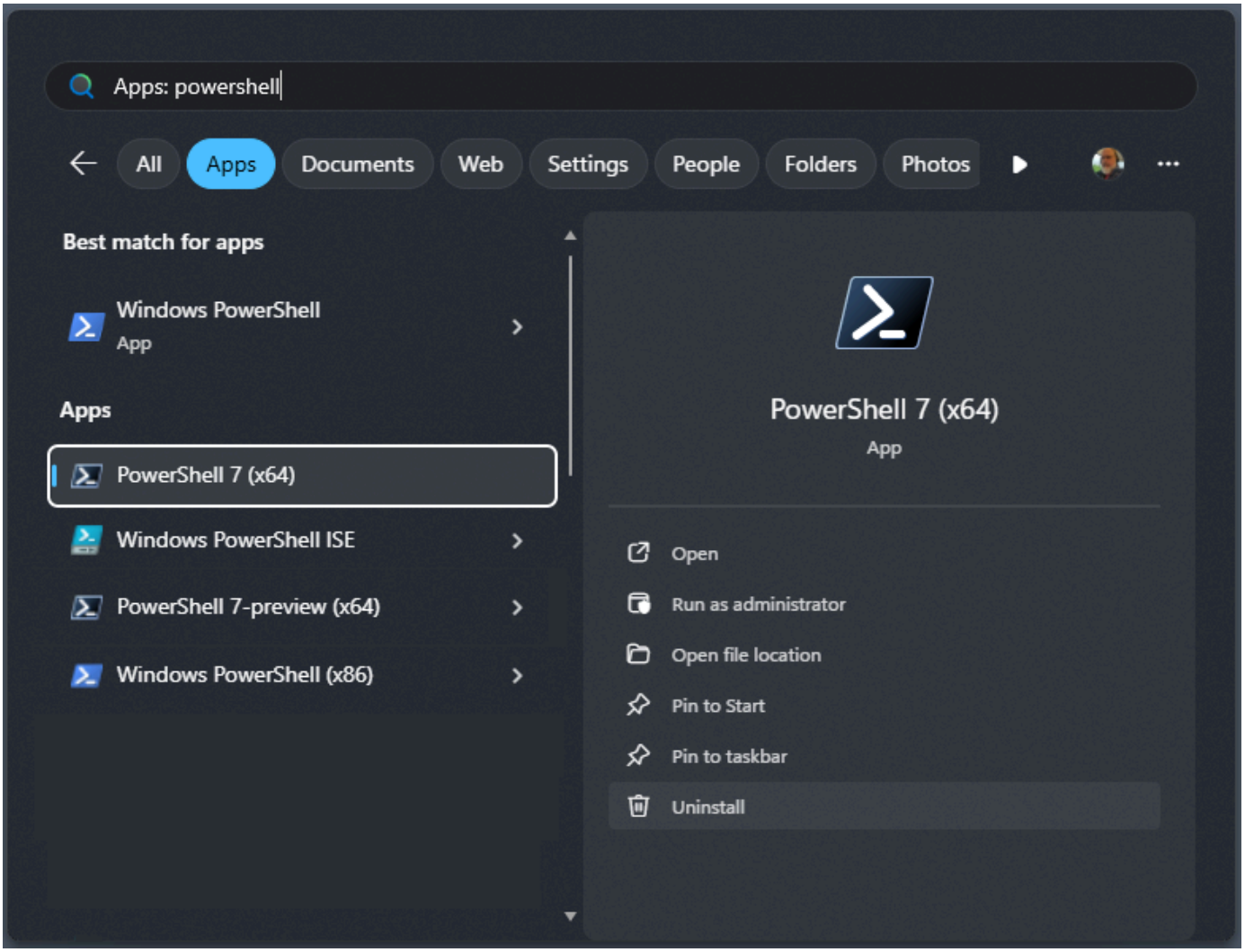
Start PowerShell 7

After installing PowerShell 7, you can start it by running the `pwsh` command or open it from the Start Menu. The installer creates shortcut entries in the Windows Start Menu.

By default, the installer installs the package in `$Env:ProgramFiles\PowerShell\7`. Preview releases of PowerShell 7 install to `$Env:ProgramFiles\PowerShell\7-preview`. The installed location is added to your `$Env:PATH` environment variable.

> **Note**
>
> To run PowerShell 7.5 side-by-side with other versions of PowerShell 7, use the ZIP install method to install the other version to a different folder. When you install using the ZIP method, you must manually add a shortcut to the Start Menu and add the location to the PATH environment variable.

The following screenshot shows multiple versions of PowerShell in the Start Menu. Select the item labeled **PowerShell 7**.



The selected entry is for PowerShell 7. Preview versions of PowerShell 7 install side-by-side with stable versions. Select the item labeled **PowerShell 7-preview** to start the preview version.

The first and last entries shown are for Windows PowerShell 5.1, which are installed by default on Windows. If you choose **Windows PowerShell ISE**, that starts the Windows PowerShell Integrated Scripting Environment (ISE), which is a different application that only works with Windows PowerShell 5.1.

Upgrade PowerShell 7

PowerShell 7 supports updates through Microsoft Update. When you enable this feature, you'll get the latest PowerShell 7 updates in your traditional Microsoft Update (MU) management flow, whether that's with Windows Update for Business, WSUS, Microsoft Endpoint Configuration Manager, or the interactive MU dialog in **Settings**. For more information, see the PowerShell Microsoft Update FAQ.

If you want to upgrade to the latest version of PowerShell 7 before it's available through Microsoft Update, you should use the same install method you used when you first installed PowerShell. Newer versions of PowerShell 7 replace existing previous versions of PowerShell 7.

Preview versions of PowerShell can be installed side-by-side with non-preview versions of PowerShell. Newer preview versions replace existing previous preview versions.

If you aren't sure how PowerShell was installed, you can check the value of the `$PSHOME` variable, which always points to the directory containing PowerShell that the current session is running.

- If the value is `$HOME\.dotnet\tools`, PowerShell was installed with the .NET Global tool.
- If the value is `$Env:ProgramFiles\PowerShell\7`, PowerShell was probably installed using the MSI package. You can verify this by looking for PowerShell in the **Programs and Features** Control Panel.
- If the value starts with `$Env:ProgramFiles\WindowsApps\`, PowerShell was installed using the MSIX package.
- If the value is anything else, it's likely that PowerShell was installed as a ZIP package.

To determine whether PowerShell may be upgraded with WinGet, run the following command:

PowerShell

```
winget list --id Microsoft.PowerShell --upgrade-available
```

If there is an available upgrade, the output indicates the latest available version. Use the following command to upgrade PowerShell using WinGet:

PowerShell

```
winget upgrade --id Microsoft.PowerShell
```

If available in the new version, WinGet uses the same package format (MSI or MSIX) that was used to install the current version of PowerShell. Alternatively, you can manually download and install the package you want.

Uninstall PowerShell 7

The process of uninstalling PowerShell 7 depends on the installation method you used.

- If you installed PowerShell using WinGet, run the following command:

  PowerShell

  ```
  winget uninstall --id Microsoft.PowerShell
  ```

- If you installed PowerShell using the MSI package, you can uninstall it from the **Programs and Features** Control Panel.
- If you installed PowerShell using the ZIP package, delete the folder where you unzipped the files.
- If you installed PowerShell from the Microsoft Store, open the **Start** menu and search for `PowerShell 7`. Select **Uninstall** from the menu of options.
- If you installed PowerShell as a .NET Global tool, run the following command:

```
PowerShell
dotnet tool uninstall --global PowerShell
```

Supported versions of Windows

Microsoft supports PowerShell until PowerShell reaches end-of-support or the version of Windows reaches end-of-support.

The Docker images for the .NET SDK contain the latest versions of PowerShell. These images are available from the Microsoft Artifact Registry. These images may not have the latest security updates. Microsoft recommends that you update the OS packages to the latest version to ensure the latest security updates are applied. These images are provided for testing purposes. If you need a Docker image for a production workload, you should build and maintain your own image.

> **Note**
>
> Support for a specific version of Windows is determined by the Microsoft Support Lifecycle policies. For more information, see:
>
> - Windows client lifecycle FAQ
> - Modern Lifecycle Policy FAQ

You can check the version that you are using by running `winver.exe`.

Supported installation methods

Microsoft supports the installation methods in this document. There may be other third-party methods of installation available from other sources. While those tools and methods may work,

Microsoft can't support those methods.

> **Note**
>
> The installation commands in this article are for the latest stable release of PowerShell. To install a different version of PowerShell, adjust the command to match the version you need. Open the Release tags page on GitHub. Select the tag for the release version you want to install. The download links for every package are found in the **Assets** section of the release. The **Assets** section may be collapsed, so you may need to click to expand it.

Last updated on 09/21/2026

PowerShell support for Linux

PowerShell can be installed on several different Linux distributions. Most Linux platforms and distributions have a major release each year, and provide a package manager that's used to install PowerShell. PowerShell can be installed on some distributions of Linux that aren't supported by Microsoft. In those cases, you may find support from the community for PowerShell on those platforms. For more information, see the PowerShell Support Lifecycle documentation.

This article lists the supported Linux distributions and package managers. All PowerShell releases remain supported until either the version of PowerShell or the version of the Linux distribution reaches end-of-support.

For the best compatibility, choose a long-term release (LTS) version.

Alpine

Microsoft supports PowerShell until PowerShell reaches end-of-support or the version of Alpine reaches end-of-life.

Support for these versions of Alpine ends on the following dates:

- Alpine 3.23 - 2027-11-01
- Alpine 3.22 - 2027-05-01
- Alpine 3.21 - 2026-11-01
- Alpine 3.20 - 2026-04-01

The Docker images for the .NET SDK contain the latest versions of PowerShell. These images are available from the Microsoft Artifact Registry.

These images are built from official operating system (OS) images provided by the OS distributor. These images may not have the latest security updates. Microsoft recommends that you update the OS packages to the latest version to ensure the latest security updates are applied.

These images are provided for testing purposes. If you need a Docker image for a production workload, you should build and maintain your own.

For more information, see Install PowerShell on Alpine.

Debian

Debian uses APT (Advanced Package Tool) as a package manager.

Microsoft supports PowerShell until PowerShell reaches end-of-support or the version of Debian reaches end-of-life.

Support for these versions of Debian ends on the following dates:

- Debian 13 - 2028-08-09
- Debian 12 - 2026-06-10

Install package files (`.deb`) are also available from https://packages.microsoft.com/.

The Docker images for the .NET SDK contain the latest versions of PowerShell. These images are available from the Microsoft Artifact Registry.

These images are built from official operating system (OS) images provide by the OS distributor. These images may not have the latest security updates. Microsoft recommends that you update the OS packages to the latest version to ensure the latest security updates are applied.

These images are provided for testing purposes. If you need a Docker image for a production workload, you should build and maintain your own.

For more information, see Install PowerShell on Debian.

Red Hat Enterprise Linux (RHEL)

RHEL 7 uses yum and RHEL 8 uses the dnf package manager.

Microsoft supports PowerShell until PowerShell reaches end-of-support or the version of RHEL reaches end-of-support.

Support for these versions of RHEL ends on the following dates:

- RHEL 10 - 2035-05-31
- RHEL 9 - 2032-05-31
- RHEL 8 - 2029-05-31

Install package files (`.rpm`) are also available from https://packages.microsoft.com/.

PowerShell is tested on Red Hat Universal Base Images (UBI). For more information, see the UBI information page.

For more information, see Install PowerShell on RHEL.

Ubuntu

Ubuntu uses APT (Advanced Package Tool) as a package manager.

Microsoft supports PowerShell until PowerShell reaches end-of-support or the version of Ubuntu reaches end-of-support.

Support for these versions of Ubuntu ends on the following dates:

- Ubuntu 24.04 - 2029-05-31
- Ubuntu 22.04 - 2027-04-01

Install package files (`.deb`) are also available from https://packages.microsoft.com/.

The Docker images for the .NET SDK contain the latest versions of PowerShell. You can download these images from the Microsoft Artifact Registry.

These images are built from official operating system (OS) images provide by the OS distributor. These images may not have the latest security updates. Microsoft recommends that you update the OS packages to the latest version to ensure the latest security updates are applied.

These images are provided for testing purposes. If you need a Docker image for a production workload, you should build and maintain your own.

> **Note**
>
> Ubuntu 25.10 (Questing Quokka) is an interim release. Microsoft doesn't test or support interim releases of Ubuntu. For more information, see **Community supported distributions**.

For more information, see Install PowerShell on Ubuntu.

Community supported distributions

PowerShell can be installed on many distributions of Linux that aren't supported by Microsoft. In those cases, you may find support from the community for PowerShell on those platforms.

To be supported by Microsoft, the Linux distribution must meet the following criteria:

- The version and architecture of the distribution is supported by .NET Core.
- The version of the distribution is supported for at least one year.
- The version of the distribution isn't an interim release or equivalent.
- The PowerShell team has tested the version of the distribution.

For more information, see Community support for PowerShell on Linux.

Alternate installation methods

There are other ways to install PowerShell on Linux, including Linux distributions that aren't officially supported. You can try to install PowerShell using the PowerShell Snap Package. You can also try deploying PowerShell binaries directly using the Linux `tar.gz` package. For more information, see Alternate ways to install PowerShell.

Last updated on 03/31/2026

Install PowerShell 7 on Alpine Linux

There are multiple package versions of PowerShell 7 that can be installed. This article focuses on installing the latest stable release package. For more information about the package versions, see the PowerShell Support Lifecycle article.

Newer versions of PowerShell 7 replace existing previous versions of PowerShell 7. Preview versions of PowerShell can be installed side-by-side with other versions of PowerShell. Newer preview versions replace existing previous preview versions. If you need to run PowerShell 7.5 side-by-side with a previous version, reinstall the previous version using the binary archive method.

Install PowerShell 7

On Alpine Linux, PowerShell is installed from the `tar.gz` package downloaded from the releases page. Select the URL of the package version you want to install.

- PowerShell 7.6 (LTS) - `https://github.com/PowerShell/PowerShell/releases/download/v7.6.5/powershell-7.6.5-linux-musl-x64.tar.gz`
- PowerShell 7.5 - `https://github.com/PowerShell/PowerShell/releases/download/v7.5.10/powershell-7.5.10-linux-musl-x64.tar.gz`
- PowerShell 7.4 (LTS) - `https://github.com/PowerShell/PowerShell/releases/download/v7.4.19/powershell-7.4.19-linux-musl-x64.tar.gz`

Use the following shell commands to install PowerShell 7:

sh

```
#!/bin/bash
# install the requirements
sudo apk add --no-cache \
    ca-certificates \
    less \
    ncurses-terminfo-base \
    krb5-libs \
    libgcc \
    libintl \
    libssl3 \
```

```
    libstdc++ \
    tzdata \
    userspace-rcu \
    zlib \
    icu-libs \
    curl

apk -X https://dl-cdn.alpinelinux.org/alpine/edge/main add --no-cache \
    lttng-ust \
    openssh-client \

# Download the powershell '.tar.gz' archive
curl -L https://github.com/PowerShell/PowerShell/releases/download/v7.6.5/powershell-
7.6.5-linux-musl-x64.tar.gz -o /tmp/powershell.tar.gz

# Create the target folder where powershell will be placed
sudo mkdir -p /opt/microsoft/powershell/7

# Expand powershell to the target folder
sudo tar zxf /tmp/powershell.tar.gz -C /opt/microsoft/powershell/7

# Set execute permissions
sudo chmod +x /opt/microsoft/powershell/7/pwsh

# Create the symbolic link that points to pwsh
sudo ln -s /opt/microsoft/powershell/7/pwsh /usr/bin/pwsh

# Start PowerShell
pwsh
```

Start PowerShell 7

After the package is installed, run `pwsh` from a terminal. If you have installed a Preview package, run `pwsh-preview`.

- The location of `$PSHOME` varies based on the package you installed.
  - For Stable and LTS packages: `/opt/microsoft/powershell/7/`
  - For Preview packages: `/opt/microsoft/powershell/7-preview/`
- The profiles scripts are stored in the following locations:
  - AllUsersAllHosts - `$PSHOME/profile.ps1`
  - AllUsersCurrentHost - `$PSHOME/Microsoft.PowerShell_profile.ps1`
  - CurrentUserAllHosts - `~/.config/powershell/profile.ps1`
  - CurrentUserCurrentHost - `~/.config/powershell/Microsoft.PowerShell_profile.ps1`
- Modules are stored in the following locations:
  - User modules - `~/.local/share/powershell/Modules`
  - Shared modules - `/usr/local/share/powershell/Modules`
  - Default modules - `$PSHOME/Modules`

- PSReadLine history is recorded in `~/.local/share/powershell/PSReadLine/ConsoleHost_history.txt`

The profiles respect PowerShell's per-host configuration, so the default host-specific profiles exists at `Microsoft.PowerShell_profile.ps1` in the same locations.

PowerShell respects the XDG Base Directory Specification on Linux.

Uninstall PowerShell 7

```sh
sudo rm -rf /usr/bin/pwsh /opt/microsoft/powershell
```

Supported OS versions

Microsoft supports PowerShell until PowerShell reaches end-of-support or the version of Alpine reaches end-of-life.

Support for these versions of Alpine ends on the following dates:

- Alpine 3.24 - 2028-06-01
- Alpine 3.23 - 2027-11-01
- Alpine 3.22 - 2027-05-01
- Alpine 3.21 - 2026-11-01

The Docker images for the .NET SDK contain the latest versions of PowerShell. These images are available from the Microsoft Artifact Registry.

These images are built from official operating system (OS) images provided by the OS distributor. These images may not have the latest security updates. Microsoft recommends that you update the OS packages to the latest version to ensure the latest security updates are applied.

These images are provided for testing purposes. If you need a Docker image for a production workload, you should build and maintain your own.

Supported installation methods

Microsoft supports the installation methods in this document. There may be other third-party methods of installation available from other sources. While those tools and methods may work, Microsoft can't support those methods.

Last updated on 08/13/2026

Install PowerShell 7 on Debian

There are multiple package versions of PowerShell 7 that can be installed. This article focuses on installing the latest stable release package. For more information about the package versions, see the PowerShell Support Lifecycle article.

Newer versions of PowerShell 7 replace existing previous versions of PowerShell 7. Preview versions of PowerShell can be installed side-by-side with other versions of PowerShell. Newer preview versions replace existing previous preview versions. If you need to run PowerShell 7.5 side-by-side with a previous version, reinstall the previous version using the binary archive method.

Choose an installation method

On Debian Linux, you can install PowerShell using the universal `.deb` package from the Microsoft package repository or by downloading a file from the GitHub releases page.

Install PowerShell 7 from the Package Repository

Microsoft builds and supports a variety of software products for Linux systems and makes them available via Linux packaging clients (apt, dnf, yum, etc). These Linux software packages are hosted on the *Linux package repository for Microsoft products*, https://packages.microsoft.com, also known as *PMC*.

Installing PowerShell from PMC is the preferred method of installation.

> **Note**
>
> This script only works for supported versions of Debian that have a package published to the Microsoft package repository. For other supported versions of Debian, use the manual installation method.

sh

```sh
#!/bin/bash
###################################
# Prerequisites

# Update the list of packages
```

```
sudo apt-get update

# Install pre-requisite packages.
sudo apt-get install -y wget

# Get the version of Debian
source /etc/os-release

# Download the Microsoft repository GPG keys
wget -q https://packages.microsoft.com/config/debian/$VERSION_ID/packages-microsoft-prod.deb

# Register the Microsoft repository GPG keys
sudo dpkg -i packages-microsoft-prod.deb

# Delete the Microsoft repository GPG keys file
rm packages-microsoft-prod.deb

# Update the list of packages after we added packages.microsoft.com
sudo apt-get update

###################################
# Install PowerShell
sudo apt-get install -y powershell

# Start PowerShell
pwsh
```

Manually download and install PowerShell 7

Download the universal package from the GitHub releases page. Choose the link for the version you want to install.

- PowerShell 7.6 (LTS) universal package for supported versions of Debian
  - `https://github.com/PowerShell/PowerShell/releases/download/v7.6.6/powershell_7.6.6-1.deb_amd64.deb`
- PowerShell 7.5 universal package for supported versions of Debian
  - `https://github.com/PowerShell/PowerShell/releases/download/v7.5.11/powershell_7.5.11-1.deb_amd64.deb`
- PowerShell 7.4 (LTS) universal package for supported versions of Debian
  - `https://github.com/PowerShell/PowerShell/releases/download/v7.4.20/powershell_7.4.20-1.deb_amd64.deb`

The following shell script downloads and installs the current release of PowerShell. You can change the URL to download the version of PowerShell that you want to install.

sh

```
#!/bin/bash
###################################
# Prerequisites

# Update the list of packages
sudo apt-get update

# Install pre-requisite packages.
sudo apt-get install -y wget

# Download the PowerShell package file
wget
https://github.com/PowerShell/PowerShell/releases/download/v7.6.6/powershell_7.6.6-
1.deb_amd64.deb

###################################
# Install the PowerShell package
sudo dpkg -i powershell_7.6.6-1.deb_amd64.deb

# Resolve missing dependencies and finish the install (if necessary)
sudo apt-get install -f

# Delete the downloaded package file
rm powershell_7.6.6-1.deb_amd64.deb

# Start PowerShell
pwsh
```

Start PowerShell 7

After the package is installed, run `pwsh` from a terminal. If you have installed a Preview package, run `pwsh-preview`.

- The location of `$PSHOME` varies based on the package you installed.
  - For Stable and LTS packages: `/opt/microsoft/powershell/7/`
  - For Preview packages: `/opt/microsoft/powershell/7-preview/`
- The profiles scripts are stored in the following locations:
  - AllUsersAllHosts - `$PSHOME/profile.ps1`
  - AllUsersCurrentHost - `$PSHOME/Microsoft.PowerShell_profile.ps1`
  - CurrentUserAllHosts - `~/.config/powershell/profile.ps1`
  - CurrentUserCurrentHost - `~/.config/powershell/Microsoft.PowerShell_profile.ps1`
- Modules are stored in the following locations:
  - User modules - `~/.local/share/powershell/Modules`
  - Shared modules - `/usr/local/share/powershell/Modules`
  - Default modules - `$PSHOME/Modules`

- PSReadLine history is recorded in `~/.local/share/powershell/PSReadLine/ConsoleHost_history.txt`

The profiles respect PowerShell's per-host configuration, so the default host-specific profiles exists at `Microsoft.PowerShell_profile.ps1` in the same locations.

PowerShell respects the XDG Base Directory Specification on Linux.

Uninstall PowerShell 7

```sh
sudo apt-get remove powershell
```

Supported OS versions

Microsoft supports PowerShell until PowerShell reaches end-of-support or the version of Debian reaches end-of-life.

Support for these versions of Debian ends on the following dates:

- Debian 13 - 2030-06-30
- Debian 12 - 2028-06-30

Install package files (`.deb`) are also available from https://packages.microsoft.com/.

The Docker images for the .NET SDK contain the latest versions of PowerShell. These images are available from the Microsoft Artifact Registry.

These images are built from official operating system (OS) images provide by the OS distributor. These images may not have the latest security updates. Microsoft recommends that you update the OS packages to the latest version to ensure the latest security updates are applied.

These images are provided for testing purposes. If you need a Docker image for a production workload, you should build and maintain your own.

Supported installation methods

Microsoft supports the installation methods in this document. There may be other methods of installation available from other third-party sources. While those tools and methods may work, Microsoft can't support those methods.

Last updated on 09/09/2026

Install PowerShell 7 on Red Hat Enterprise Linux (RHEL)

There are multiple package versions of PowerShell 7 that can be installed. This article focuses on installing the latest stable release package. For more information about the package versions, see the PowerShell Support Lifecycle article.

Newer versions of PowerShell 7 replace existing previous versions of PowerShell 7. Preview versions of PowerShell can be installed side-by-side with other versions of PowerShell. Newer preview versions replace existing previous preview versions.

Choose an installation method

On RHEL, you can install PowerShell using the universal `.rpm` package from the Microsoft package repository or by downloading file from the GitHub release page.

Install PowerShell 7 from the Package Repository

Microsoft builds and supports a variety of software products for Linux systems and makes them available via Linux packaging clients (apt, dnf, yum, etc). These Linux software packages are hosted on the *Linux package repository for Microsoft products*, https://packages.microsoft.com, also known as *PMC*.

Installing PowerShell from PMC is the preferred method of installation.

> **Note**
>
> This script only works for supported versions of RHEL that have a package published to the Microsoft package repository. For other supported versions of RHEL, use the manual installation method.

sh

```sh
#!/bin/bash
###################################
# Prerequisites

# Get version of RHEL
source /etc/os-release
```

```
majorver=${VERSION_ID%.*}

# Download the Microsoft RedHat repository package
curl -sSL -O https://packages.microsoft.com/config/rhel/$majorver/packages-microsoft-prod.rpm

# Register the Microsoft RedHat repository
sudo rpm -i packages-microsoft-prod.rpm

# Delete the downloaded package after installing
rm packages-microsoft-prod.rpm

# Update package index files
sudo dnf update
# Install PowerShell
sudo dnf install powershell -y
```

Manually download and install PowerShell 7

Download the universal package from the GitHub releases page. Select the URL of the package version you want to install.

- PowerShell 7.6 (LTS) universal package
  - `https://github.com/PowerShell/PowerShell/releases/download/v7.6.5/powershell-7.6.5-1.rh.x86_64.rpm`
- PowerShell 7.5 universal package
  - `https://github.com/PowerShell/PowerShell/releases/download/v7.5.10/powershell-7.5.10-1.rh.x86_64.rpm`
- PowerShell 7.4 (LTS) universal package
  - `https://github.com/PowerShell/PowerShell/releases/download/v7.4.19/powershell-7.4.19-1.rh.x86_64.rpm`

The following shell script downloads and installs the current release of PowerShell. You can change the URL to download the version of PowerShell that you want to install.

sh

```
sudo dnf install https://github.com/PowerShell/PowerShell/releases/download/v7.6.5/powershell-7.6.5-1.rh.x86_64.rpm
```

Start PowerShell 7

After the package is installed, run `pwsh` from a terminal. If you have installed a Preview package, run `pwsh-preview`.

- The location of `$PSHOME` varies based on the package you installed.
  - For Stable and LTS packages: `/opt/microsoft/powershell/7/`
  - For Preview packages: `/opt/microsoft/powershell/7-preview/`
- The profiles scripts are stored in the following locations:
  - AllUsersAllHosts - `$PSHOME/profile.ps1`
  - AllUsersCurrentHost - `$PSHOME/Microsoft.PowerShell_profile.ps1`
  - CurrentUserAllHosts - `~/.config/powershell/profile.ps1`
  - CurrentUserCurrentHost - `~/.config/powershell/Microsoft.PowerShell_profile.ps1`
- Modules are stored in the following locations:
  - User modules - `~/.local/share/powershell/Modules`
  - Shared modules - `/usr/local/share/powershell/Modules`
  - Default modules - `$PSHOME/Modules`
- PSReadLine history is recorded in `~/.local/share/powershell/PSReadLine/ConsoleHost_history.txt`

The profiles respect PowerShell's per-host configuration, so the default host-specific profiles exists at `Microsoft.PowerShell_profile.ps1` in the same locations.

PowerShell respects the XDG Base Directory Specification on Linux.

Uninstall PowerShell 7

sh

```
sudo dnf remove powershell
```

Supported versions of RHEL

Microsoft supports PowerShell until PowerShell reaches end-of-support or the version of RHEL reaches end-of-support.

Support for these versions of RHEL ends on the following dates:

- RHEL 10 - 2035-05-31
- RHEL 9 - 2032-05-31
- RHEL 8 - 2029-05-31

Install package files (`.rpm`) are also available from https://packages.microsoft.com/.

PowerShell is tested on Red Hat Universal Base Images (UBI). For more information, see the UBI information page.

Supported installation methods

Microsoft supports the installation methods in this document. There may be other third-party methods of installation available from other sources. While those tools and methods may work, Microsoft can't support those methods.

Last updated on 08/12/2026

Install PowerShell 7 on Ubuntu

There are multiple package versions of PowerShell 7 that can be installed. This article focuses on installing the latest stable release package. For more information about the package versions, see the PowerShell Support Lifecycle article.

Newer versions of PowerShell 7 replace existing previous versions of PowerShell 7. Preview versions of PowerShell can be installed side-by-side with other versions of PowerShell. Newer preview versions replace existing previous preview versions. If you need to run PowerShell 7.5 side-by-side with a previous version, reinstall the previous version using the binary archive method.

Choose an installation method

On Ubuntu Linux, you can install PowerShell using the universal `.deb` package from the Microsoft package repository or by downloading a file from the stable release page.

Install PowerShell 7 from the Package Repository

Microsoft builds and supports a variety of software products for Linux systems and makes them available via Linux packaging clients (apt, dnf, yum, etc). These Linux software packages are hosted on the *Linux package repository for Microsoft products*, https://packages.microsoft.com, also known as *PMC*.

Installing PowerShell from PMC is the preferred method of installation.

> **Note**
>
> This script only works for supported versions of Ubuntu that have a package published to the Microsoft package repository. For other versions of Ubuntu, use the manual installation method.

sh

```sh
#!/bin/bash
###################################
# Prerequisites

# Update the list of packages
```

```
sudo apt-get update

# Install pre-requisite packages.
sudo apt-get install -y wget apt-transport-https software-properties-common

# Get the version of Ubuntu
source /etc/os-release

# Download the Microsoft repository keys
wget -q https://packages.microsoft.com/config/ubuntu/$VERSION_ID/packages-microsoft-
prod.deb

# Register the Microsoft repository keys
sudo dpkg -i packages-microsoft-prod.deb

# Delete the Microsoft repository keys file
rm packages-microsoft-prod.deb

# Update the list of packages after we added packages.microsoft.com
sudo apt-get update

###################################
# Install PowerShell
sudo apt-get install -y powershell

# Start PowerShell
pwsh
```

> **ⓘ Important**
>
> Ubuntu comes preconfigured with a package repository that includes .NET packages, but not PowerShell. Using these instructions to install PowerShell registers the Microsoft repository as a package source. You can install PowerShell and some versions of .NET from this repository. However, the Ubuntu package repository has different versions of the .NET packages. This can cause problems when installing .NET for other purposes. For more information about these problems, see Troubleshoot .NET package mix ups on Linux.
>
> You must choose the feed you want to use to install .NET. You can set the priority of the package repositories to favor one over the other. For instructions on how to set the priorities, see My Linux distribution provides .NET packages, and I want to use them.

Manually download and install PowerShell 7

Download the universal package from the GitHub releases page. Select the URL of the package version you want to install.

- PowerShell 7.6 (LTS) universal package
  - `https://github.com/PowerShell/PowerShell/releases/download/v7.6.4/powershell_7.6.4-1.deb_amd64.deb`
- PowerShell 7.5 universal package
  - `https://github.com/PowerShell/PowerShell/releases/download/v7.5.9/powershell_7.5.9-1.deb_amd64.deb`
- PowerShell 7.4 (LTS) universal package
  - `https://github.com/PowerShell/PowerShell/releases/download/v7.4.18/powershell_7.4.18-1.deb_amd64.deb`

The following shell script downloads and installs the current release of PowerShell. You can change the URL to download the version of PowerShell that you want to install.

sh

```sh
#!/bin/bash
###################################
# Prerequisites

# Update the list of packages
sudo apt-get update

# Install pre-requisite packages.
sudo apt-get install -y wget

# Download the PowerShell package file
wget
https://github.com/PowerShell/PowerShell/releases/download/v7.6.4/powershell_7.6.4-1.deb_amd64.deb

###################################
# Install the PowerShell package
sudo dpkg -i powershell_7.6.4-1.deb_amd64.deb

# Resolve missing dependencies and finish the install (if necessary)
sudo apt-get install -f

# Delete the downloaded package file
rm powershell_7.6.4-1.deb_amd64.deb
```

Start PowerShell 7

After the package is installed, run `pwsh` from a terminal. If you have installed a Preview package, run `pwsh-preview`.

- The location of `$PSHOME` varies based on the package you installed.

    - For Stable and LTS packages: `/opt/microsoft/powershell/7/`
    - For Preview packages: `/opt/microsoft/powershell/7-preview/`
- The profiles scripts are stored in the following locations:
    - AllUsersAllHosts - `$PSHOME/profile.ps1`
    - AllUsersCurrentHost - `$PSHOME/Microsoft.PowerShell_profile.ps1`
    - CurrentUserAllHosts - `~/.config/powershell/profile.ps1`
    - CurrentUserCurrentHost - `~/.config/powershell/Microsoft.PowerShell_profile.ps1`
- Modules are stored in the following locations:
    - User modules - `~/.local/share/powershell/Modules`
    - Shared modules - `/usr/local/share/powershell/Modules`
    - Default modules - `$PSHOME/Modules`
- PSReadLine history is recorded in `~/.local/share/powershell/PSReadLine/ConsoleHost_history.txt`

The profiles respect PowerShell's per-host configuration, so the default host-specific profiles exists at `Microsoft.PowerShell_profile.ps1` in the same locations.

PowerShell respects the XDG Base Directory Specification on Linux.

Uninstall PowerShell

```sh
sudo apt-get remove powershell
```

Support for Arm processors

PowerShell 7.2 and newer supports running on Ubuntu using 32-bit Arm processors. Use the binary archive installation method of installing PowerShell that's described in Alternate ways to install PowerShell on Linux.

Supported versions

Microsoft supports PowerShell until PowerShell reaches end-of-support or the version of Ubuntu reaches end-of-support.

Support for these versions of Ubuntu ends on the following dates:

- Ubuntu 26.04 (Resolute Raccoon) - 2031-04-30
- Ubuntu 24.04 (Noble Numbat) - 2029-05-31

- Ubuntu 22.04 (Jammy Jellyfish) - 2024-09-30

Install package files (`.deb`) are also available from https://packages.microsoft.com/.

The Docker images for the .NET SDK contain the latest versions of PowerShell. You can download these images from the Microsoft Artifact Registry.

These images are built from official operating system (OS) images provide by the OS distributor. These images may not have the latest security updates. Microsoft recommends that you update the OS packages to the latest version to ensure the latest security updates are applied.

These images are provided for testing purposes. If you need a Docker image for a production workload, you should build and maintain your own.

> **Note**
>
> Ubuntu 25.10 (Questing Quokka) is an interim release. Microsoft doesn't test or support interim releases of Ubuntu. For more information, see Community supported distributions.

Supported installation methods

Microsoft supports the installation methods in this document. There may be other methods of installation available from other third-party sources. While those tools and methods may work, Microsoft can't support those methods.

Last updated on 08/11/2026

Community support for PowerShell on Linux

You can install PowerShell on some distributions of Linux that aren't supported by Microsoft. In those cases, you might find support from the community for PowerShell on those platforms.

Supported Linux distributions must meet the following criteria:

- The version and architecture of the distribution is supported by .NET Core.
- The version of the distribution is supported for at least one year.
- The version of the distribution isn't an interim release or equivalent.
- The PowerShell team has tested the version of the distribution.

For more information, see the PowerShell Support Lifecycle documentation.

The following distributions are examples of distributions supported by the community. Each distribution has its own community support mechanisms. Consult the distribution's website to find their community resources. You can also get help from these PowerShell Community resources.

Ubuntu interim releases

The documented steps to install PowerShell on Ubuntu might work on Ubuntu interim releases. However, Microsoft only supports PowerShell on the Long Term Servicing (LTS) releases of Ubuntu. Microsoft doesn't support interim releases of Ubuntu.

Arch Linux

PowerShell is available from the Arch Linux User Repository (AUR). Packages in the AUR are maintained by the Arch community. To install the latest release binary, see the Arch Linux wiki or Using PowerShell in Docker.

Kali

Installation - Kali

sh

```
# Install PowerShell package
apt update && apt -y install powershell
```

```
# Start PowerShell
pwsh
```

Uninstallation - Kali

sh

```
# Uninstall PowerShell package
apt -y remove powershell
```

Gentoo

You can install PowerShell on Gentoo Linux using packages from the Gentoo package repository. For information about installing these packages, see the PowerShell page in the Gentoo wiki.

SLES and openSUSE

You may be able to install PowerShell on SLES and openSUSE using the Snap package manager. Also, the following article provides information on how to install PowerShell on openSUSE:

- PowerShell - openSUSE Wiki

Raspberry Pi OS

Raspberry Pi OS is a free operating system based on Debian.

> **Important**
>
> .NET isn't supported on ARMv6 architecture devices, including Raspberry Pi Zero and Raspberry Pi devices released before Raspberry Pi 2.

Install on Raspberry Pi OS

Being Debian-based, you can install PowerShell on Raspberry Pi OS using the Snap package. For more information, see the Snap installation instructions.

Or you can install PowerShell on Raspberry Pi OS using the binary archives. Download the tar.gz package from the releases page onto your Raspberry Pi computer. The links to the current versions are:

- PowerShell 7.6 - latest LTS release
  - `https://github.com/PowerShell/PowerShell/releases/download/v7.6.6/powershell-7.6.6-linux-arm32.tar.gz`
  - `https://github.com/PowerShell/PowerShell/releases/download/v7.6.6/powershell-7.6.6-linux-arm64.tar.gz`
- PowerShell 7.5 - latest stable release
  - `https://github.com/PowerShell/PowerShell/releases/download/v7.5.11/powershell-7.5.11-linux-arm32.tar.gz`
  - `https://github.com/PowerShell/PowerShell/releases/download/v7.5.11/powershell-7.5.11-linux-arm64.tar.gz`
- PowerShell 7.4 - previous LTS release
  - `https://github.com/PowerShell/PowerShell/releases/download/v7.4.20/powershell-7.4.20-linux-arm32.tar.gz`
  - `https://github.com/PowerShell/PowerShell/releases/download/v7.4.20/powershell-7.4.20-linux-arm64.tar.gz`

Use the following shell commands to download and install the package. This script detects whether you're running a 32-bit or 64-bit OS and installs the latest stable version of PowerShell for that processor type.

sh

```
###################################
# Prerequisites

# Update package lists
sudo apt-get update

# Install dependencies
sudo apt-get install jq libssl1.1 libunwind8 -y

###################################
# Download and extract PowerShell

# Grab the latest tar.gz
bits=$(getconf LONG_BIT)
release=$(curl -sL https://api.github.com/repos/PowerShell/PowerShell/releases/latest)
package=$(echo -E $release | jq -r ".assets[].browser_download_url" | grep "linux-arm${bits}.tar.gz")
wget $package

# Make folder to put powershell
mkdir ~/powershell

# Unpack the tar.gz file
tar -xvf "./${package##*/}" -C ~/powershell
```

```
# Start PowerShell
~/powershell/pwsh
```

Optionally, you can create a symbolic link to start PowerShell without specifying the path to the `pwsh` binary.

sh

```
# Start PowerShell from bash with sudo to create a symbolic link
sudo ~/powershell/pwsh -Command 'New-Item -ItemType SymbolicLink -Path "/usr/bin/pwsh"
-Target "$PSHOME/pwsh" -Force'

# alternatively you can run following to create a symbolic link
# sudo ln -s ~/powershell/pwsh /usr/bin/pwsh

# Now to start PowerShell you can just run "pwsh"
```

Uninstallation - Raspberry Pi OS

sh

```
rm -rf ~/powershell
```

Last updated on 09/09/2026

Install PowerShell 7 on macOS

There are multiple package versions of PowerShell 7 that can be installed. This article focuses on installing the latest stable release package. For more information about the package versions, see the PowerShell Support Lifecycle article.

Newer versions of PowerShell 7 replace existing previous versions of PowerShell 7. Preview versions of PowerShell can be installed side-by-side with other versions of PowerShell. Newer preview versions replace existing previous preview versions. If you need to run PowerShell 7 side-by-side with a previous version, reinstall the previous version using the binary archive method.

Choose an installation method

There are several ways to install PowerShell on macOS. Choose one:

- Download and install the package file - this is the preferred method for most users
- Install as a .NET Global tool - this method is useful for developers that already have the .NET Core SDK installed
- Install from a binary archive - this method is useful for advanced users who need more control over the installation
- If you previously installed PowerShell using Homebrew, see *Install on macOS using Homebrew* in Alternate ways to install PowerShell.

Download and install the package file

Beginning with the May 2026 releases of PowerShell, the macOS PKG package is notarized and signed by Microsoft. To install the package, download the PKG file and open it.

1. Download the install package from the releases page. Select the package version you want to install.

    - PowerShell 7.7-preview
        - Arm64 processors - powershell-7.7.0-preview.4-osx-arm64.pkg
        - x64 processors - powershell-7.7.0-preview.4-osx-x64.pkg
    - PowerShell 7.6 (LTS)
        - Arm64 processors - powershell-7.6.6-osx-arm64.pkg
        - x64 processors - powershell-7.6.6-osx-x64.pkg
    - PowerShell 7.5

    - Arm64 processors - powershell-7.5.11-osx-arm64.pkg
    - x64 processors - powershell-7.5.11-osx-x64.pkg
- PowerShell 7.4 (LTS)
    - Arm64 processors - powershell-7.4.20-osx-arm64.pkg
    - x64 processors - powershell-7.4.20-osx-x64.pkg

> **Note**
>
> Beginning with macOS 27 (Golden Gate), macOS only runs on Apple Silicon (Arm64) processors.

2. Open **Finder**
3. Locate the downloaded package
4. Double-click the file

Install as a .NET Global tool

If you already have the .NET Core SDK installed, you can use the .NET Global tool to install PowerShell 7.

```sh
dotnet tool install --global PowerShell
```

The dotnet tool installer adds `~/.dotnet/tools` to your `PATH` environment variable. However, the currently running shell doesn't have the updated `PATH`. Start PowerShell from a new shell by typing `pwsh`.

Install PowerShell 7 from a binary archive

PowerShell binary `tar.gz` archives are provided for the macOS platform to enable advanced deployment scenarios. When you install using this method, you must also manually install any dependencies.

Download the install package from the releases page. Select the archive version you want to install.

- PowerShell 7.7-preview
    - Arm64 processors - powershell-7.7.0-preview.4-osx-arm64.tar.gz

    - x64 processors - powershell-7.7.0-preview.4-osx-x64.tar.gz
- PowerShell 7.6 (LTS)
    - Arm64 processors - powershell-7.6.6-osx-arm64.tar.gz
    - x64 processors - powershell-7.6.6-osx-x64.tar.gz
- PowerShell 7.5
    - Arm64 processors - powershell-7.5.11-osx-arm64.tar.gz
    - x64 processors - powershell-7.5.11-osx-x64.tar.gz
- PowerShell 7.4 (LTS)
    - Arm64 processors - powershell-7.4.20-osx-arm64.tar.gz
    - x64 processors - powershell-7.4.20-osx-x64.tar.gz

Use the following commands to install PowerShell from the binary archive. Change the download URL to match the version you want to install.

sh

```sh
# Download the powershell '.tar.gz' archive
curl -L -o /tmp/powershell.tar.gz
https://github.com/PowerShell/PowerShell/releases/download/v7.6.6/powershell-7.6.6-
osx-arm64.tar.gz

# Create the target folder where powershell is placed
sudo mkdir -p /usr/local/microsoft/powershell/7

# Expand powershell to the target folder
sudo tar zxf /tmp/powershell.tar.gz -C /usr/local/microsoft/powershell/7

# Set execute permissions
sudo chmod +x /usr/local/microsoft/powershell/7/pwsh

# Create the symbolic link that points to pwsh
sudo ln -s /usr/local/microsoft/powershell/7/pwsh /usr/local/bin/pwsh
```

Start PowerShell 7

After the package is installed, run `pwsh` from a terminal. If you have installed a Preview package, run `pwsh-preview`.

- The location of `$PSHOME` varies based on the package you installed.
    - For Stable and LTS packages: `/usr/local/microsoft/powershell/7/`
    - For Preview packages: `/usr/local/microsoft/powershell/7-preview/`
    - The macOS install package creates a symbolic link, `/usr/local/bin/pwsh` that points to `pwsh` in the `$PSHOME` location.

- User profiles are read from `~/.config/powershell/profile.ps1`
- Default profiles are read from `$PSHOME/profile.ps1`
- User modules are read from `~/.local/share/powershell/Modules`
- Shared modules are read from `/usr/local/share/powershell/Modules`
- Default modules are read from `$PSHOME/Modules`
- PSReadLine history is recorded to `~/.local/share/powershell/PSReadLine/ConsoleHost_history.txt`

PowerShell respects the XDG Base Directory Specification on macOS.

Update PowerShell 7

To update PowerShell, download the new version of the package or binary archive and install it.

Uninstall PowerShell 7

To uninstall PowerShell you need to delete the application folder and other support files. The following command removes the symbolic link and PowerShell files.

```sh
sudo rm -rf /usr/local/bin/pwsh /usr/local/microsoft/powershell
```

Use `sudo rm` to remove any other remaining PowerShell files and folders.

Supported versions of macOS

Microsoft supports PowerShell until PowerShell reaches end-of-support or the version of macOS reaches end-of-support.

The following versions of macOS are supported:

- macOS 27 (Golden Gate) Arm64
- macOS 26 (Tahoe) x64 and Arm64
- macOS 15 (Sequoia) x64 and Arm64

Apple determines the support lifecycle of macOS. For more information, see the following:

- macOS release notes
- Apple Security Updates

Supported installation methods

Microsoft supports the installation methods in this document. There may be other third-party methods of installation available from other sources. While those tools and methods may work, Microsoft can't support those methods. For more information, see Alternate ways to install PowerShell.

Last updated on 09/22/2026

Alternate ways to install PowerShell

There are other ways to install PowerShell on non-Windows platforms.

These methods may work but aren't officially supported by Microsoft. You may be able to get support from the PowerShell Community or the operating system vendor. For support options, see Community Support.

Install on macOS using Homebrew

Homebrew is a popular package manager for macOS. To install Homebrew, follow the instructions on the Homebrew website.

> **Important**
>
> The brew formula is maintained and supported by the Homebrew community. The brew formula builds PowerShell from source code rather than installing a package built by Microsoft.
>
> If you previously installed PowerShell using the Homebrew cask, you must first uninstall the cask before you can successfully install using the Homebrew formula. Use the following commands to uninstall the cask:

```sh
# Uninstall the PowerShell cask instance
brew uninstall --cask powershell
# Uninstall the PowerShell Preview cask instance
brew uninstall --cask powershell-preview
```

Run the following command to install PowerShell using the Homebrew formula:

```sh
brew install powershell
```

If you receive the message: "*Warning: PowerShell is already installed, it's just not linked.*", run the following command:

sh

```
brew link powershell
```

Update PowerShell 7

Run the following commands to update the installed version of PowerShell to the latest release.

sh

```
brew update
brew upgrade powershell
```

Uninstall PowerShell 7

If you installed PowerShell with Homebrew, use the following command to uninstall:

sh

```
brew uninstall powershell
```

If you manually installed PowerShell 7, you must manually remove it. The following command removes the symbolic link and PowerShell files.

sh

```
sudo rm -rf /usr/local/bin/pwsh /usr/local/microsoft/powershell
```

Use `sudo rm` to remove any other remaining PowerShell files and folders.

Install on Linux using a Snap package

Snaps are application packages that are easy to install if your platform supports Snap. You can find and install Snap packages from the Snap Store.

> ⓘ Note
>
> The Snap Store contains PowerShell snap packages for many Linux distributions that aren't officially supported by Microsoft.

Getting snapd

The snap daemon, known as `snapd`, is the background service that manages and maintains your snaps. It needs to be running before a snap can be installed. For instructions on how to install `snapd`, see the Snapcraft documentation.

Installation via Snap

There are two PowerShell for Linux is published to the Snap store: `powershell` and `powershell-preview`.

Use the following command to install the latest stable version of PowerShell:

```sh
# Install PowerShell
sudo snap install powershell --classic

# Start PowerShell
pwsh
```

If you don't specify the `--channel` parameter, Snap installs the latest stable version. To install the latest LTS version, use the following method:

```sh
# Install PowerShell
sudo snap install powershell --channel=lts/stable --classic

# Start PowerShell
pwsh
```

To install a preview version, use the following method:

```sh
# Install PowerShell
sudo snap install powershell-preview --classic

# Start PowerShell
pwsh-preview
```

> **Note**
>
> Microsoft only supports the `latest/stable` and `lts/stable` channels for the `powershell` package. Microsoft only supports the `latest/stable` channel for the `powershell-preview`

package. Do not install packages from the other channels.

After installation, Snap will automatically upgrade. You can trigger an upgrade using `sudo snap refresh powershell` or `sudo snap refresh powershell-preview`.

> **Important**
>
> The Snap packages are maintained and supported by Canonical. Snap packages build PowerShell from source code rather than installing a package built by Microsoft.

Uninstall using Snap

```sh
sudo snap remove powershell
```

or

```sh
sudo snap remove powershell-preview
```

Install from binary archives

PowerShell binary `tar.gz` archives are provided for Linux platforms to enable advanced deployment scenarios.

> **Note**
>
> You can use this method to install any version of PowerShell including the latest:
>
> - Stable release: https://aka.ms/powershell-release?tag=stable
> - LTS release: https://aka.ms/powershell-release?tag=lts
> - Preview release: https://aka.ms/powershell-release?tag=preview

Dependencies

PowerShell builds portable binaries for all supported Linux distributions. But, PowerShell and the .NET runtime require different dependencies on different distributions.

It's possible that when you install PowerShell, specific dependencies may not be installed, such as when manually installing from the binary archives. The following list details Linux distributions that are supported by Microsoft and have dependencies you may need to install. Check the Linux distribution page for more information:

- Alpine
- Debian
- RHEL
- SLES
- Ubuntu

To deploy PowerShell binaries on Linux distributions that aren't officially supported, you need to install the necessary dependencies for the target OS in separate steps.

> **ⓘ Important**
>
> This method can be used to install PowerShell on any version of Linux, including distributions that are not officially supported by Microsoft. Be sure to install any necessary dependencies. For support, see the list of available **Community Support** options.

The following example shows the steps for installing the x64 binary archive. You must choose the correct binary archive that matches the processor type for your platform.

- `powershell-7.6.3-linux-arm32.tar.gz`
- `powershell-7.6.3-linux-arm64.tar.gz`
- `powershell-7.6.3-linux-x64.tar.gz`

Use the following shell commands to download and install PowerShell from the `tar.gz` binary archive. Change the URL to match the version of PowerShell you want to install.

sh

```sh
# Download the powershell '.tar.gz' archive
curl -L -o /tmp/powershell.tar.gz
https://github.com/PowerShell/PowerShell/releases/download/v7.6.3/powershell-7.6.3-
linux-x64.tar.gz

# Create the target folder where powershell will be placed
sudo mkdir -p /opt/microsoft/powershell/7

# Expand powershell to the target folder
sudo tar zxf /tmp/powershell.tar.gz -C /opt/microsoft/powershell/7
```

```
# Set execute permissions
sudo chmod +x /opt/microsoft/powershell/7/pwsh

# Create the symbolic link that points to pwsh
sudo ln -s /opt/microsoft/powershell/7/pwsh /usr/bin/pwsh
```

Uninstalling binary archives

sh

```
sudo rm -rf /usr/bin/pwsh /opt/microsoft/powershell
```

Install as a .NET Global tool

If you already have the .NET Core SDK installed, it's easy to install PowerShell as a .NET Global tool.

sh

```
dotnet tool install --global PowerShell
```

The dotnet tool installer adds `~/.dotnet/tools` to your `PATH` environment variable. However, the currently running shell doesn't have the updated `PATH`. You should be able to start PowerShell from a new shell by typing `pwsh`.

The .NET team publishes Docker images containing the .NET SDK with PowerShell already installed. You can find those images on the Microsoft Container Registry.

Last updated on 06/13/2026

Install PowerShell 7 on Windows IoT and Nano Server

This article describes how to install PowerShell 7 on Windows IoT and Nano Server.

Deploy on Windows 11 IoT

Windows 11 IoT Enterprise comes with Windows PowerShell, which is used to deploy PowerShell 7.

PowerShell

```
# Replace the placeholder information for the following variables:
$deviceip = '<device ip address>'
$zipfile = 'PowerShell-7.6.3-win-arm64.zip'
$downloadfolder = 'U:\Users\Administrator\Downloads'
# The download location is local to the device.
# There should be enough space for the zip file and the unzipped contents.

# Create PowerShell session to target device
Set-Item -Path WSMan:\localhost\Client\TrustedHosts $deviceip
$S = New-PSSession -ComputerName $deviceIp -Credential Administrator
# Copy the ZIP package to the device
Copy-Item $zipfile -Destination $downloadfolder -ToSession $S

#Connect to the device and expand the archive
Enter-PSSession $S
Set-Location U:\Users\Administrator\Downloads
Expand-Archive .\PowerShell-7.6.3-win-arm64.zip

# Set up remoting to PowerShell 7
Set-Location .\PowerShell-7.6.3-win-arm64
# Be sure to use the -PowerShellHome parameter otherwise it tries to create a new
# endpoint with Windows PowerShell 5.1
.\Install-PowerShellRemoting.ps1 -PowerShellHome .
```

When you set up PowerShell Remoting you get an error message and are disconnected from the device. PowerShell has to restart WinRM. Now you can connect to PowerShell 7 endpoint on device.

PowerShell

```

# Be sure to use the -Configuration parameter. If you omit it, you connect to Windows
PowerShell 5.1
```

```
Enter-PSSession -ComputerName $deviceIp -Credential Administrator -Configuration
PowerShell.7.6.3
```

Windows 11 IoT Core adds Windows PowerShell when you include *IOT_POWERSHELL* feature. Use Windows PowerShell to deploy PowerShell 7 using the same steps as Windows 11 IoT Enterprise.

To add the latest PowerShell in the shipping image, use the Import-PSCoreRelease command to include the package in the workarea and add the *OPENSRC_POWERSHELL* feature to your image.

> **Note**
>
> For ARM64 architecture, Windows PowerShell isn't added when you include *IOT_POWERSHELL*. So the zip based install doesn't work. You need to use `Import-PSCoreRelease` command to add it in the image.

Deploying on Nano Server

These instructions assume that the Nano Server is a "headless" OS that has a version of PowerShell already running on it. For more information, see the Nano Server Image Builder documentation.

PowerShell binaries can be deployed using two different methods.

1. Offline - Mount the Nano Server VHD and unzip the contents of the zip file to your chosen location within the mounted image.
2. Online - Transfer the zip file over a PowerShell Session and unzip it in your chosen location.

In both cases, you need the Windows x64 ZIP release package. Run the commands within an "Administrator" instance of PowerShell.

Offline Deployment of PowerShell

1. Use your favorite zip utility to unzip the package to a directory within the mounted Nano Server image.
2. Unmount the image and boot it.
3. Connect to the built-in instance of Windows PowerShell.

Online Deployment of PowerShell

Deploy PowerShell to Nano Server using the following steps.

PowerShell

```
# Replace the placeholder information for the following variables:
$ipaddr = '<Nano Server IP address>'
$credential = Get-Credential # <An Administrator account on the system>
$zipfile = 'PowerShell-7.6.3-win-x64.zip'
# Connect to the built-in instance of Windows PowerShell
$session = New-PSSession -ComputerName $ipaddr -Credential $credential
# Copy the file to the Nano Server instance
Copy-Item $zipfile C:\ -ToSession $session
# Enter the interactive remote session
Enter-PSSession $session
# Extract the ZIP file
Expand-Archive -Path C:\PowerShell-7.6.3-win-x64.zip -DestinationPath 'C:\Program
Files\PowerShell 7'
```

Supported versions of Windows

Microsoft supports PowerShell until PowerShell reaches end-of-support or the version of Windows reaches end-of-support.

The Docker images for the .NET SDK contain the latest versions of PowerShell. These images are available from the Microsoft Artifact Registry. These images may not have the latest security updates. Microsoft recommends that you update the OS packages to the latest version to ensure the latest security updates are applied. These images are provided for testing purposes. If you need a Docker image for a production workload, you should build and maintain your own image.

> **Note**
>
> Support for a specific version of Windows is determined by the Microsoft Support Lifecycle policies. For more information, see:
>
> - Windows client lifecycle FAQ
> - Modern Lifecycle Policy FAQ

You can check the version that you are using by running `winver.exe`.

Supported installation methods

Microsoft supports the installation methods in this document. There may be other third-party methods of installation available from other sources. While those tools and methods may work,

Microsoft can't support those methods.

> **Note**
>
> The installation commands in this article are for the latest stable release of PowerShell. To install a different version of PowerShell, adjust the command to match the version you need. Open the Release tags page on GitHub. Select the tag for the release version you want to install. The download links for every package are found in the **Assets** section of the release. The **Assets** section may be collapsed, so you may need to click to expand it.

Last updated on 06/13/2026

PowerShell on Arm processors

Summarize this article for me

Support for the Arm processor is based on the support policy of the version of .NET that PowerShell uses. While .NET supports many more operating systems and versions, PowerShell support is limited to the versions that have been tested.

It may be possible to use Arm-based versions of PowerShell on other Linux distributions and versions, but we don't officially support it.

PowerShell 7

Arm versions of PowerShell 7 can be installed on the following platforms:

Expand table

OS	Architectures	Lifecycle
Windows 11 Client Version 22000+	Arm64	Windows
macOS	Arm64	macOS
Raspberry Pi OS (Debian 12)	Arm32/Arm64	Raspberry Pi OS and Debian
Ubuntu 22.04	Arm32/Arm64	Ubuntu

Support is based on the .NET 8.0 Supported OS Lifecycle Policy.

Installing PowerShell on Arm-based systems

For installation instructions, see the following articles:

Windows

- Windows 11 on Arm
- Windows 11 IoT

Linux - install from the binary archives

- Alternate ways to install PowerShell on Linux

macOS

- Installing PowerShell on macOS

Raspberry Pi

- Raspberry Pi OS

Last updated on 03/12/2026

Use PowerShell in Docker

Summarize this article for me

The .NET team publishes Docker images with PowerShell preinstalled. This article shows you how to get started using PowerShell in the Docker container.

Find available images

These images require Docker 17.05 or newer. Also, you must be able to run Docker without `sudo` or local administrative rights. For install instructions, see Docker's official documentation.

The .NET team publishes several Docker images designed for different development scenarios. Only the image for the .NET SDK contains PowerShell. For more information, see Official .NET Docker images.

Use PowerShell in a container

The following command downloads the image containing the latest available stable versions of the .NET SDK and PowerShell.

Console

```
docker pull mcr.microsoft.com/dotnet/sdk:9.0
```

Use the following command to start an interactive PowerShell session in the container.

Console

```
docker run -it mcr.microsoft.com/dotnet/sdk:9.0 pwsh
```

To download and run the latest Long Term Support (LTS) version of PowerShell, change the image name to `mcr.microsoft.com/dotnet/sdk:8.0`. When you use these image tags, Docker downloads the appropriate image for your host operating system. If you want an image for a specific operating system, you can specify the operating system in the image tag. See the Microsoft Artifact Registry for a list of available tags.

- For more information about tags, the Supported tag policy
- For more information about supported operating systems, see the Supported platforms policy

Support lifecycle

The .NET support policy defines how these images are supported. These images are provided for development and testing purposes only. If you need a production-ready image, you should build your own images. For more information about these Docker images, visit the dotnet-docker repository on GitHub.

The images previously published by the PowerShell team will be marked as deprecated in the Microsoft Container Registry (MCR).

Telemetry

By default, PowerShell collects limited telemetry without personal data to help aid development of future versions of PowerShell. To opt-out of sending telemetry, create an environment variable called `POWERSHELL_TELEMETRY_OPTOUT` set to a value of `1` before starting PowerShell from the installed location. The telemetry we collect falls under the Microsoft Privacy Statement.

Last updated on 03/12/2026

Microsoft Update for PowerShell FAQ

Beginning with PowerShell 7.2, when you install using the MSI package you have the option of enabling Microsoft Update support for PowerShell.

General Information

What is the Microsoft Update feature in PowerShell?

The Microsoft Update feature of PowerShell allows you to get the latest PowerShell 7 updates in your traditional Microsoft Update (MU) management flow, whether that's with Windows Update for Business, WSUS, Microsoft Endpoint Configuration Manager, or the interactive MU dialog in Settings. Microsoft Update and the related services enable you to deploy updates:

- On your schedule
- After testing for your environment
- At scale across your enterprise

How soon after release are updates advertised by Microsoft Update?

When a new version of PowerShell is released, it can take up to two weeks for that version to become available through Microsoft Update. Updates are delivered as optional software updates, even if the update contains a security fix.

If you need to deploy the update before it becomes available in Microsoft Update, download the update from the Releases page on GitHub.

Why is the latest LTS version not marked as LTS?

We mark the earliest minor version LTS until it goes out of support. For example, both PowerShell 7.2 and 7.4 are LTS releases and have a year of overlapping support. PowerShell 7.2 was marked as the latest LTS in MU until it reached end of support in November 2024.

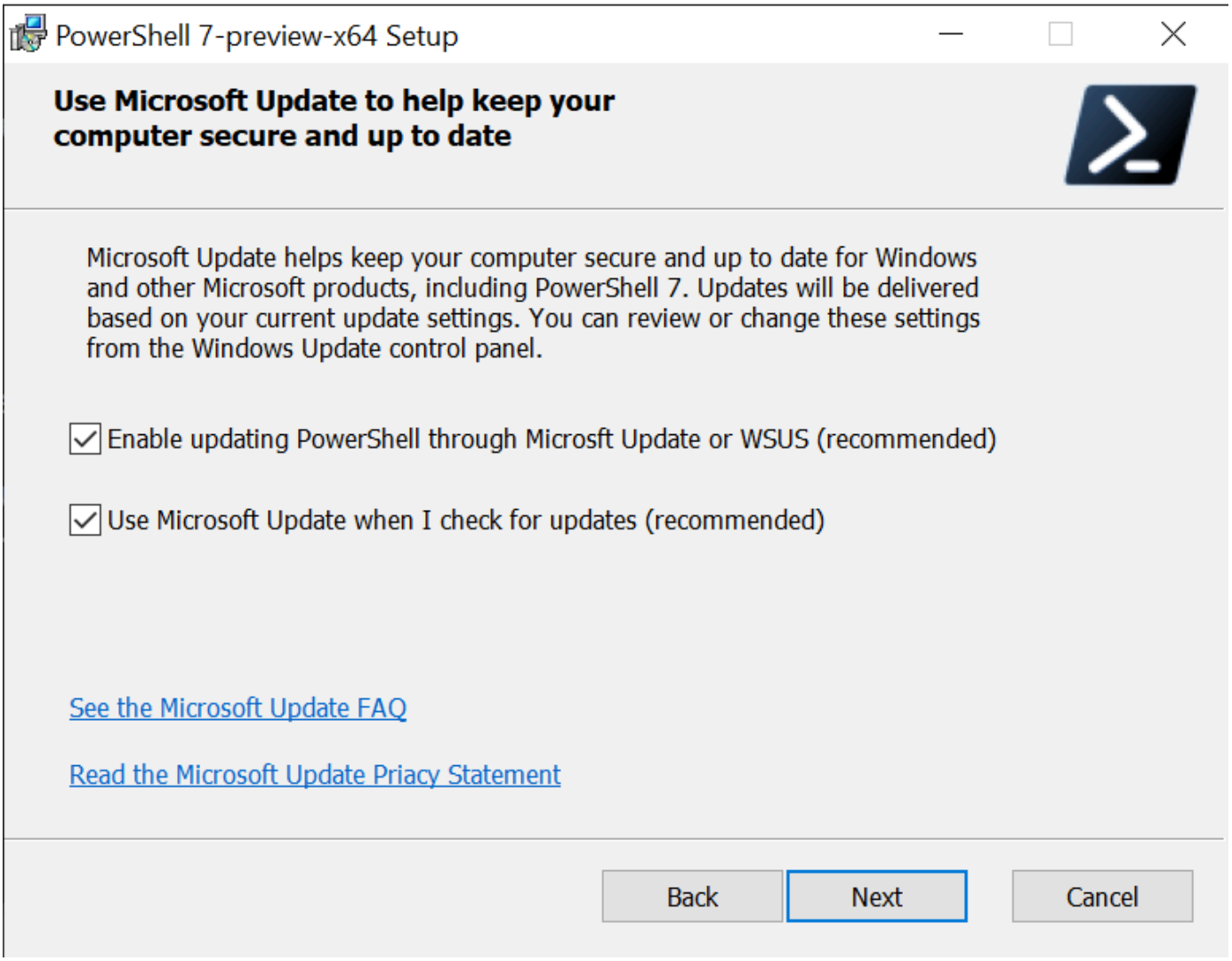
Configuration

What version of Windows is required to support the Microsoft Update feature?

You must have Windows Version 1809 or newer installed on an x64-based system. Version 1809 is the Windows 10 October 2018 Update or Windows Server 2019. Versions prior to 1809 do not support Microsoft Update for PowerShell.

Do I need to check both boxes in the setup dialog?

While the two options on the dialog are independent, in most cases, it's best to check both boxes.



What does each checkbox do?

The first checkbox enables updates for PowerShell. These updates can be delivered by Microsoft Update, a WSUS server, or SCCM. If this checkbox is unchecked, you cannot receive updates through any of these channels.

The second checkbox enables Microsoft Update on your system. This allows you to receive updates for any supported Microsoft software, not just Windows. If the box is unchecked, you will not receive the update from Microsoft Update, but you can receive updates from WSUS or SCCM.

What if I want to opt-out later?

If you want to opt-out of updates later, you can run the MSI install package and uncheck the first checkbox. Unchecking the second checkbox has no effect.

Can I enable these update options from the command line or in a script?

Yes. The MSI package includes two new MSI options for enabling the update features:

- `USE_MU` - This property has two possible values:
  - `1` (default) - Opts into updating through Microsoft Update, WSUS, or SCCM
  - `0` - Do not opt into updating through Microsoft Update, WSUS, or SCCM
- `ENABLE_MU`
  - `1` (default) - Opts into using Microsoft Update for Automatic Updates
  - `0` - Do not opt into using Microsoft Update

> ⓘ **Note**
>
> Setting `ENABLE_MU=0` does not disable Microsoft Update.

Troubleshooting

Why haven't I received an update for the new release?

There can be several reasons for not receiving the update:

- We may not have published the update yet. Our goal is to make the update available to Microsoft Update within two weeks of release, but there is no guarantee for that availability.
- There are group policy settings that control Microsoft Update. Your system administrator may have policies set that prevent you from using Microsoft Update. The checkbox in the installer cannot override the Group Policy.

- Make sure you have checked both checkboxes. When doing a repair installation, the installer doesn't show the check box options. To enable MU updates run the following command:

  PowerShell

  ```
  msiexec.exe /fmu .\PowerShell-7.4.10-win-x64.msi USE_MU=1 ENABLE_MU=1
  ```

  For more information about running `msiexec.exe` from the command line, see msiexec.

I am on PowerShell 7.x, why have I not been upgraded to 7.y?

The Microsoft Update feature for PowerShell only updates versions in the same release channel. PowerShell 7.4 is the latest long term supported (LTS) version. PowerShell 7.5 is the latest stable (non-LTS) version. Microsoft Update provides updates for the next patch level versions of either version. For example:

- If you are running 7.4, you will receive updates for 7.4.
- If you are running 7.5, you will receive updates for 7.5.

Microsoft Update will never upgrade an LTS release to a stable non-LTS release. However, a stable non-LTS release will be upgraded to the higher LTS release when support for the stable release ends.

Preview releases are never upgraded to the GA release version. However, they will be upgraded to the next available preview release. For example: Consider the scenario where you have 7.4 (LTS) installed and the 7.5-rc.1 (preview) release installed. When 7.5.0 (Stable) released, your 7.4 (LTS) installation is not upgraded to 7.5.0. Also, 7.5.0 can't upgrade 7.5-rc.1. However, 7.6-preview.2 can upgrade 7.5-rc.1.

For more information, see PowerShell Support Lifecycle.

Last updated on 03/12/2026

Introduction

Expand table



This content originally appeared in the book *PowerShell 101* by Mike F. Robbins. We thank Mike for granting us permission to reuse the content here. We edited the content for publication on this platform. You can still get the original book from Leanpub at PowerShell 101.

About this book

Before PowerShell, I began my career as an IT Pro, pointing and clicking in the GUI. I wrote this book to save IT Pros from themselves by reducing the learning curve and helping them avoid being reluctant to learn PowerShell.

Instead of a book that covers topics with fictitious scenarios, this book is a condensed version targeting the specific topics that an IT Pro needs to know to succeed with PowerShell in a real-world production environment. It's a collection of what I wish someone had told me when I started learning PowerShell. I include tips, tricks, and best practices that I learned while using PowerShell since 2007.

Each chapter includes a curated collection of links to specific help articles that expand on the information covered. These resources expand on the concepts discussed and broaden your understanding of PowerShell.

Who is this book for?

This book is for anyone wanting to learn PowerShell. Whether you're a beginner or an experienced user, this book helps you improve your PowerShell skills.

This book focuses on Windows PowerShell version 5.1 running on Windows 11 and Windows Server 2022 in a Microsoft Active Directory domain environment. However, the basic concepts apply to all versions of PowerShell running on any supported platform.

Lab environment

The examples in this book were created and tested on Windows 11 and Windows Server 2022 operating systems, using Windows PowerShell version 5.1. If you're running a different operating system or version of PowerShell, your results might vary from the ones presented in this book.

About the author

Mike F. Robbins, a former Microsoft MVP, is the lead technical writer for Azure PowerShell at Microsoft. With extensive experience in PowerShell, he is a scripting, automation, and efficiency expert. As a lifelong learner, Mike continuously strives to improve his skills and empower others by sharing his knowledge and experience. He is also a published author of several books, including:

- Author of PowerShell 101: The No-Nonsense Guide to Windows PowerShell
- Creator of The PowerShell Conference Book
- Contributing author in the PowerShell Deep Dives book

When Mike's not writing documentation for Microsoft, he shares his thoughts and insights on his blog at mikefrobbins.com and interacts with his followers on Twitter @mikefrobbins.

Where to begin

If you're new to PowerShell or want to ensure you have a solid foundation, start with Chapter 1. It covers finding and launching PowerShell, understanding versions, and setting up your environment so you can follow along with the examples throughout the rest of this book.

Last updated on 03/17/2026

Chapter 1 - Getting started with PowerShell

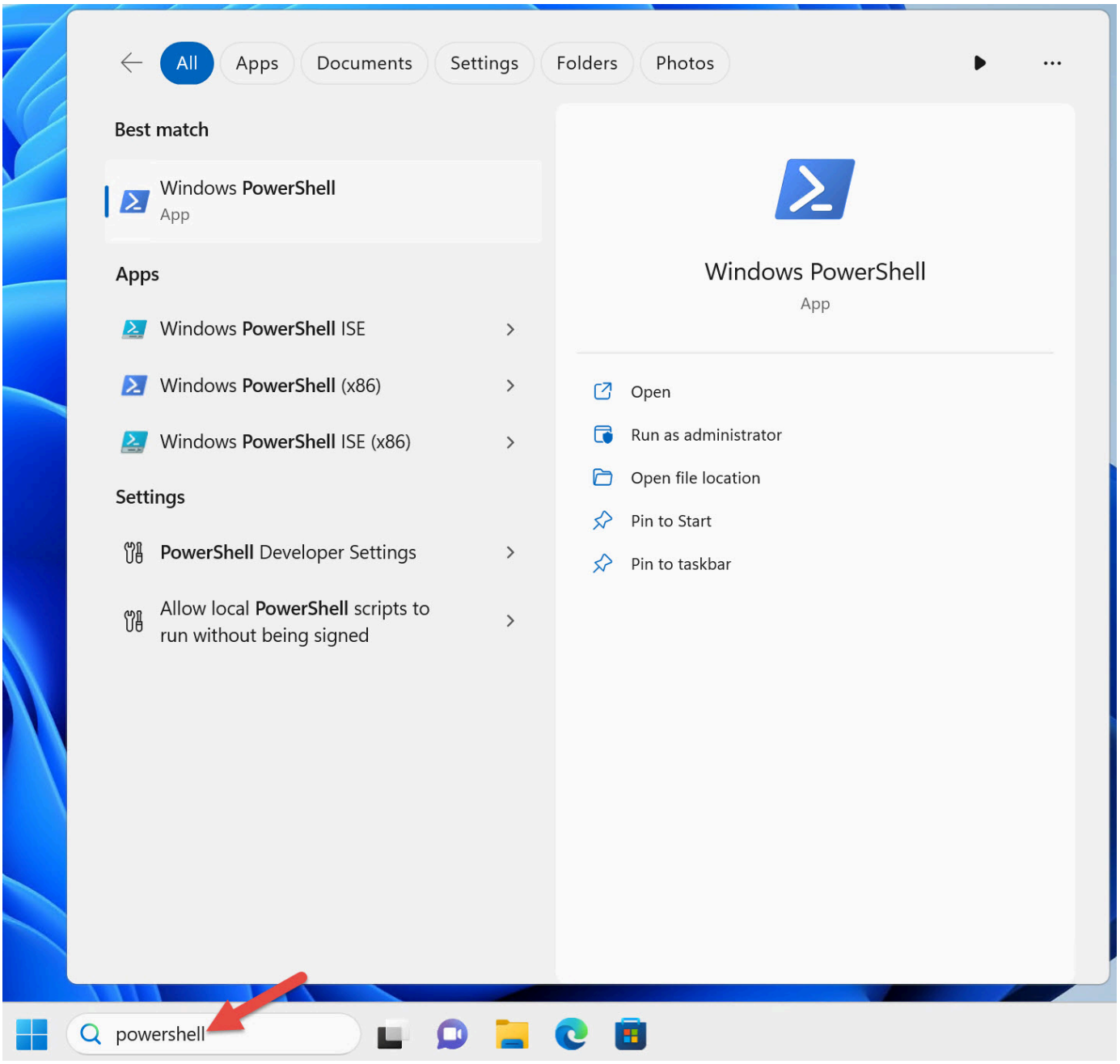
This chapter focuses on finding and launching PowerShell and solving the initial pain points that new users experience with PowerShell. Follow along and walk through the examples in this chapter on your lab environment computer.

What is PowerShell?

Windows PowerShell is an easy-to-use command-line shell and scripting environment for automating administrative tasks of Windows-based systems. Windows PowerShell is preinstalled on all modern versions of the Windows operating system.

Where to find PowerShell

The easiest way to find PowerShell on Windows 11 is to type `PowerShell` into the search bar, as shown in Figure 1-1. Notice that there are four different shortcuts for Windows PowerShell.



Windows PowerShell shortcuts on a 64-bit version of Windows:

- Windows PowerShell
- Windows PowerShell ISE
- Windows PowerShell (x86)
- Windows PowerShell ISE (x86)

On a 64-bit version of Windows, you have a 64-bit version of the Windows PowerShell console and the Windows PowerShell Integrated Scripting Environment (ISE) and a 32-bit version of each one, as indicated by the (x86) suffix on the shortcuts.

> ⓘ **Note**

Windows 11 only ships as a 64-bit operating system. There is no 32-bit version of Windows 11. However, Windows 11 includes 32-bit versions of Windows PowerShell and the Windows PowerShell ISE.

You only have two shortcuts if you're running an older 32-bit version of Windows. Those shortcuts don't have the (x86) suffix but are 32-bit versions.

I recommend using the 64-bit version of Windows PowerShell if you're running a 64-bit operating system unless you have a specific reason for using the 32-bit version.

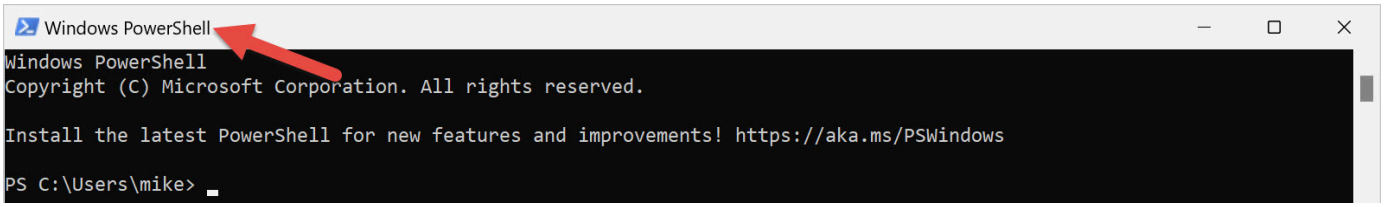
Depending on what version of Windows 11 you're running, Windows PowerShell might open in Windows Terminal.

Microsoft no longer updates the PowerShell ISE. The ISE only works with Windows PowerShell 5.1. Visual Studio Code (VS Code) with the PowerShell extension works with both versions of PowerShell. VS Code and the PowerShell extension don't ship in Windows. Install VS Code and the extension on the computer where you create PowerShell scripts. You don't need to install them on all the computers where you run PowerShell.

How to launch PowerShell

I use three different Active Directory user accounts in the production environments I support. I mirrored those accounts in the lab environment used in this book. I sign into my Windows 11 computer as a domain user without domain or local administrator rights.

Launch the PowerShell console by clicking the **Windows PowerShell** shortcut, as shown in Figure 1-1. Notice that the title bar of the console says **Windows PowerShell**, as shown in Figure 1-2.



Some commands run fine when you run PowerShell as an ordinary user. However, PowerShell doesn't participate in User Access Control (UAC). That means it's unable to prompt for elevation for tasks that require the approval of an administrator.

> ⓘ Note

UAC is a Windows security feature that helps prevent malicious code from running with elevated privileges.

When signed on as an ordinary user, PowerShell returns an error when you run a command that requires elevation. For example, stopping a Windows service:

PowerShell

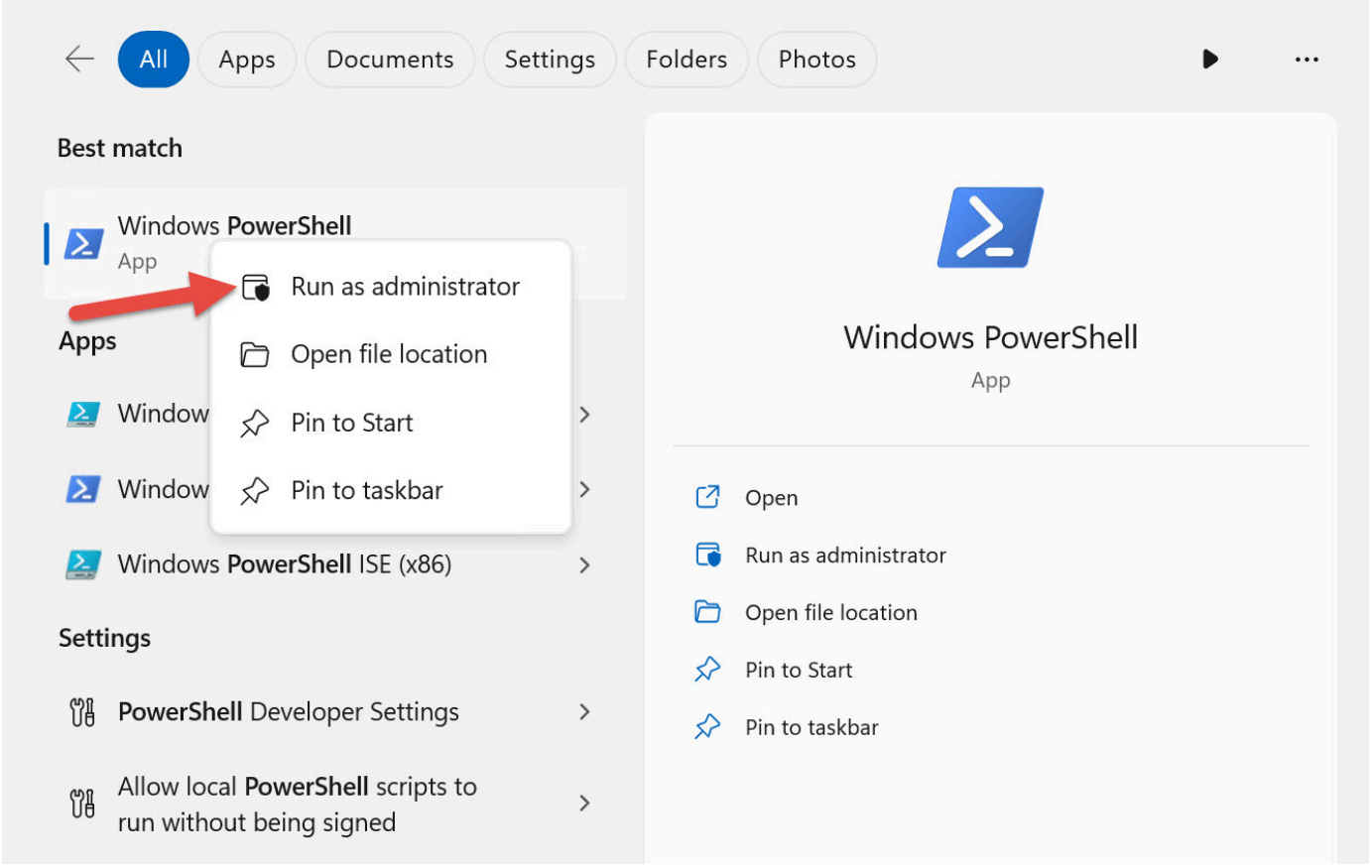
```
Stop-Service -Name W32Time
```

Output

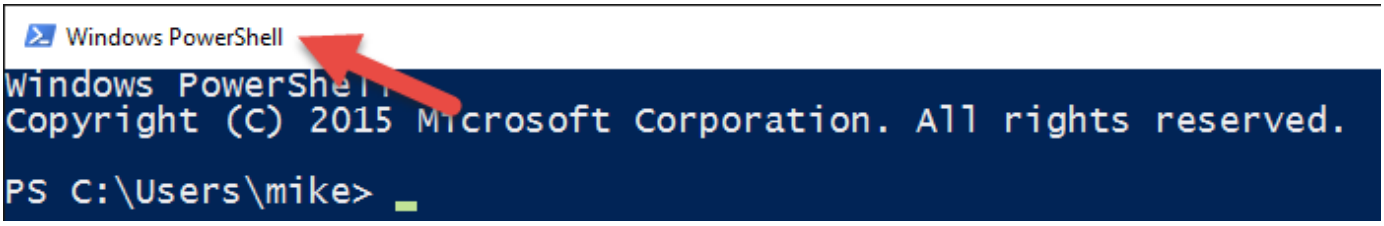
```
Stop-Service : Service 'Windows Time (W32Time)' cannot be stopped due to
the following error: Cannot open W32Time service on computer '.'.
At line:1 char:1
+ Stop-Service -Name W32Time
+ ~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : CloseError: (System.ServiceProcess.ServiceCon
   troller:ServiceController) [Stop-Service], ServiceCommandException
    + FullyQualifiedErrorId : CouldNotStopService,Microsoft.PowerShell.Comm
   ands.StopServiceCommand
```

The solution is to run PowerShell elevated as a user who is a local administrator. That's how I configured my second domain user account. Following the principle of least privilege, this account shouldn't be a domain administrator or have any elevated privileges in the domain.

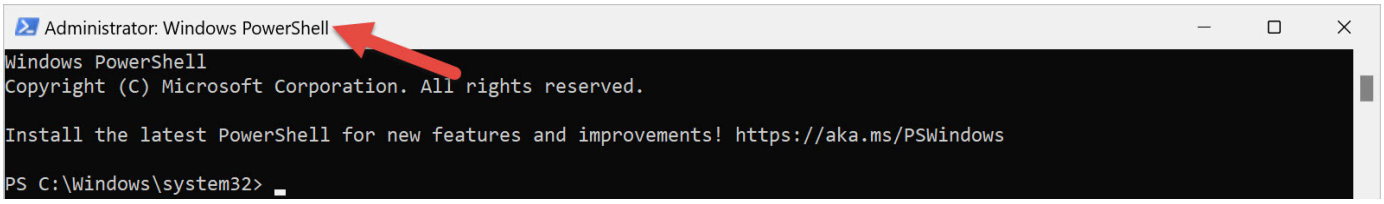
To start PowerShell with elevated rights, right-click the **Windows PowerShell** shortcut and select **Run as administrator**, as shown in Figure 1-3.



Windows prompts you for credentials because you logged into Windows as an ordinary user. Enter the credentials of your domain user who is a local administrator, as shown in Figure 1-4.



Notice that the title bar of the elevated console windows says **Administrator: Windows PowerShell**, as shown in Figure 1-5.

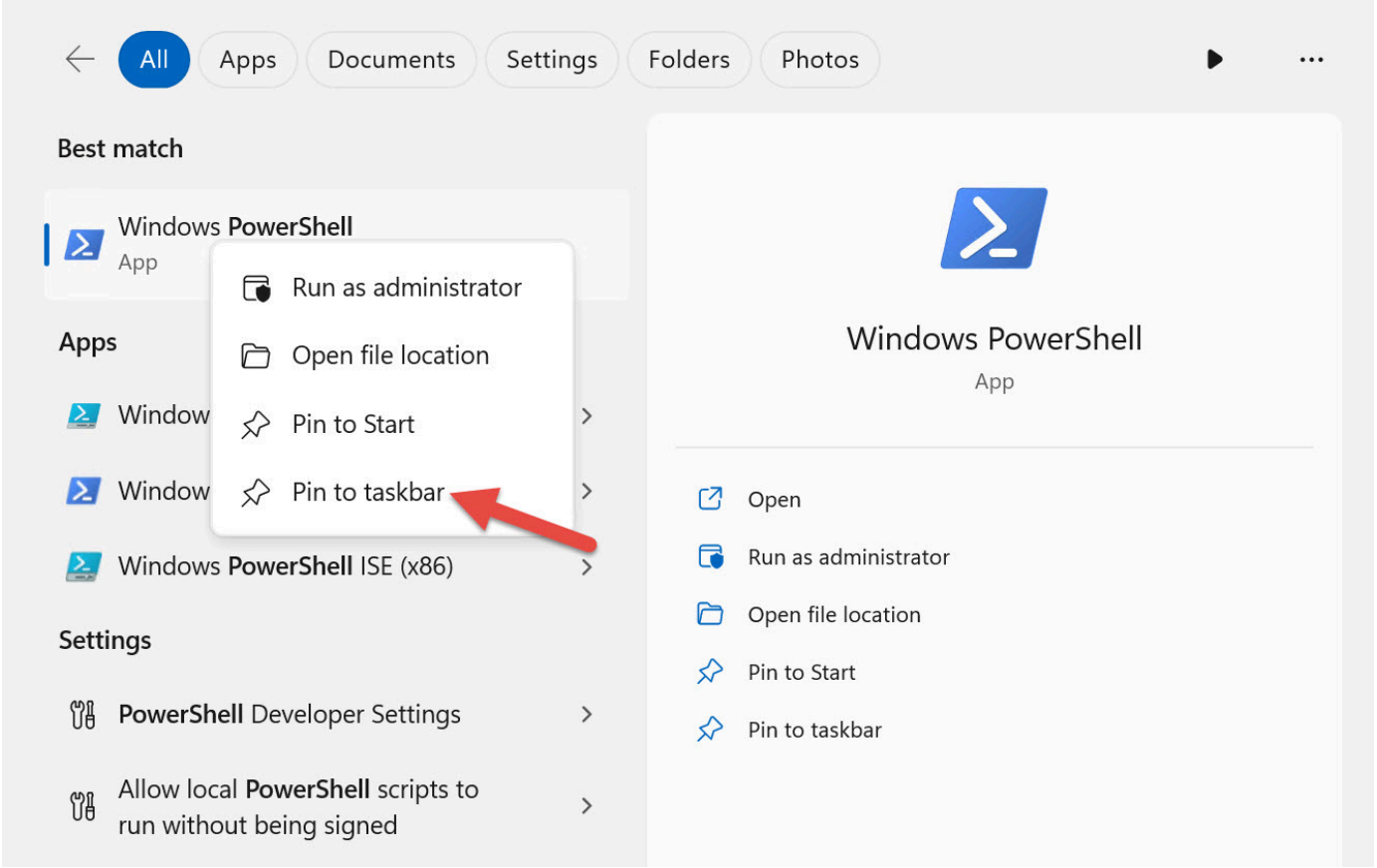


Now that you're running PowerShell elevated as an administrator, UAC is no longer a problem when you run a command that requires elevation.

> **Important**
>
> You should only run PowerShell elevated as an administrator when absolutely necessary.

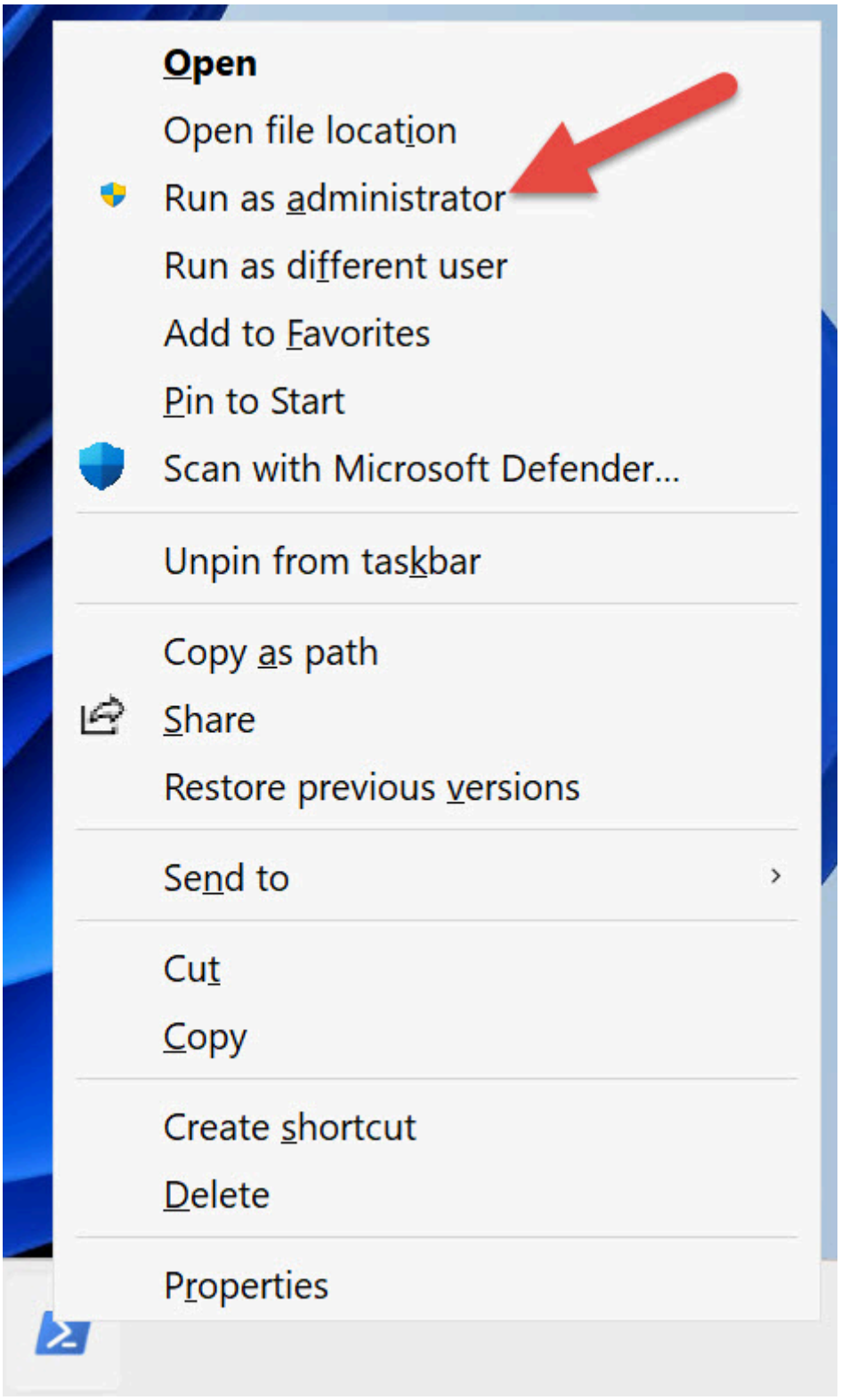
When you target remote computers, there's no need to run PowerShell elevated. Running PowerShell elevated only affects commands that run against your local computer.

You can simplify finding and launching PowerShell. Pin the PowerShell or Windows Terminal shortcut to your taskbar. Search for PowerShell again, except this time right-click on it and select **Pin to taskbar** as shown in Figure 1-6.



> **Important**
>
> The original version of this book, published in 2017, recommended pinning a shortcut to the taskbar to launch an elevated instance automatically every time you start PowerShell. However, due to potential security concerns, I no longer recommend it. Any applications you launch from an elevated instance of PowerShell also bypass UAC and run elevated. For example, if you launch a web browser from an elevated instance of PowerShell, any website you visit containing malicious code also runs elevated.

When you need to run PowerShell with elevated permissions, right-click the PowerShell shortcut pinned to your taskbar while pressing `Shift`. Select **Run as administrator**, as shown in Figure 1-7.



Determine your version of PowerShell

There are automatic variables in PowerShell that store state information. One of these variables is `$PSVersionTable`, which contains version information about your PowerShell session.

PowerShell

```
$PSVersionTable
```

Output

```
Name                           Value
----                           -----
PSVersion                      5.1.22621.2428
PSEdition                      Desktop
PSCompatibleVersions           {1.0, 2.0, 3.0, 4.0...}
BuildVersion                   10.0.22621.2428
CLRVersion                     4.0.30319.42000
WSManStackVersion              3.0
PSRemotingProtocolVersion      2.3
SerializationVersion           1.1.0.1
```

If you're running a version of Windows PowerShell older than 5.1, you should update your version of Windows. Windows PowerShell 5.1 is preinstalled on the currently supported versions of Windows.

PowerShell version 7 isn't a replacement for Windows PowerShell 5.1; it installs side-by-side with Windows PowerShell. Windows PowerShell version 5.1 and PowerShell version 7 are two different products. For more information about the differences between Windows PowerShell version 5.1 and PowerShell version 7, see Migrating from Windows PowerShell 5.1 to PowerShell 7.

> **Tip**
>
> PowerShell version 6, formerly known as PowerShell Core, is no longer supported.

Execution policy

PowerShell execution policy controls the conditions under which you can run PowerShell scripts. The execution policy in PowerShell is a safety feature designed to help prevent the unintentional execution of malicious scripts. However, it's not a security boundary because it can't stop determined users from deliberately running scripts. A determined user can bypass the execution policy in PowerShell.

You can set an execution policy for the local computer, current user, or a PowerShell session. You can also set execution policies for users and computers with Group Policy.

The following table shows the default execution policy for current Windows operating systems.

Expand table

Windows Operating System Version	Default Execution Policy
Windows Server 2022	Remote Signed
Windows Server 2019	Remote Signed
Windows Server 2016	Remote Signed
Windows 11	Restricted
Windows 10	Restricted

Regardless of the execution policy setting, you can run any PowerShell command interactively. The execution policy only affects commands running in a script. Use the `Get-ExecutionPolicy` cmdlet to determine the current execution policy setting.

Check the execution policy setting on your computer.

PowerShell

```powershell
Get-ExecutionPolicy
```

Output

```
Restricted
```

List the execution policy settings for all scopes.

PowerShell

```powershell
Get-ExecutionPolicy -List
```

Output

```
        Scope ExecutionPolicy
        ----- ---------------
MachinePolicy       Undefined
   UserPolicy       Undefined
      Process       Undefined
  CurrentUser       Undefined
 LocalMachine       Undefined
```

All Windows client operating systems have the default execution policy setting of `Restricted`. You can't run PowerShell scripts using the `Restricted` execution policy setting. To test the execution policy, save the following code as a `.ps1` file named `Get-TimeService.ps1`.

> **Tip**
>
> A PowerShell script is a plaintext file that contains the commands you want to run. PowerShell script files use the `.ps1` file extension. To create a PowerShell script, use a code editor like Visual Studio Code (VS Code) or any text editor such as Notepad.

When you run the following command interactively, it completes without error.

PowerShell

```powershell
Get-Service -Name W32Time
```

However, PowerShell returns an error when you run the same command from a script.

PowerShell

```powershell
.\Get-TimeService.ps1
```

Output

```
.\Get-TimeService.ps1 : File C:\tmp\Get-TimeService.ps1 cannot be loaded
because running scripts is disabled on this system. For more information,
see about_Execution_Policies at
https:/go.microsoft.com/fwlink/?LinkID=135170.
At line:1 char:1
+ .\Get-TimeService.ps1
+ ~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : SecurityError: (:) [], PSSecurityException
    + FullyQualifiedErrorId : UnauthorizedAccess
```

When you run a command in PowerShell that generates an error, read the error message before retrying the command. Notice the error message tells you why the command failed:

> *... running scripts is disabled on this system.*

To enable the execution of scripts, change the execution policy with the `Set-ExecutionPolicy` cmdlet. `LocalMachine` is the default scope when you don't specify the **Scope** parameter. You must run PowerShell elevated as an administrator to change the execution policy for the local machine. Unless you're signing your scripts, I recommend using the `RemoteSigned` execution policy. `RemoteSigned` prevents you from running downloaded scripts that aren't signed by a trusted publisher.

Before you change the execution policy, read the about_Execution_Policies help article to understand the security implications.

Change the execution policy setting on your computer to `RemoteSigned`.

PowerShell

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned
```

If you have successfully changed the execution policy, PowerShell displays the following warning:

Output

```
Execution Policy Change
The execution policy helps protect you from scripts that you do not trust.
Changing the execution policy might expose you to the security risks
described in the about_Execution_Policies help topic at
https:/go.microsoft.com/fwlink/?LinkID=135170. Do you want to change the
execution policy?
[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help
(default is "N"):y
```

If you're not running PowerShell elevated as an administrator, PowerShell returns the following error message:

Output

```
Set-ExecutionPolicy : Access to the registry key 'HKEY_LOCAL_MACHINE\SOFTWAR
E\Microsoft\PowerShell\1\ShellIds\Microsoft.PowerShell' is denied. To
change the execution policy for the default (LocalMachine) scope, start
Windows PowerShell with the "Run as administrator" option. To change the
execution policy for the current user, run "Set-ExecutionPolicy -Scope
CurrentUser".
At line:1 char:1
+ Set-ExecutionPolicy -ExecutionPolicy RemoteSigned
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : PermissionDenied: (:) [Set-ExecutionPolicy],
   UnauthorizedAccessException
    + FullyQualifiedErrorId : System.UnauthorizedAccessException,Microsoft.
   PowerShell.Commands.SetExecutionPolicyCommand
```

It's also possible to change the execution policy for the current user without requiring you to run PowerShell elevated as an administrator. This step is unnecessary if you successfully set the execution policy for the local machine to `RemoteSigned`.

PowerShell

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

With the execution policy set to `RemoteSigned`, the `Get-TimeService.ps1` script runs successfully.

PowerShell

```
.\Get-TimeService.ps1
```

Output

```
Status   Name               DisplayName
------   ----               -----------
Running  W32Time            Windows Time
```

Summary

In this chapter, you learned where to find and how to launch PowerShell. You also learned how to determine the version of PowerShell and the purpose of execution policies.

Review

1. How do you determine what PowerShell version a computer is running?
2. When should you launch PowerShell elevated as an administrator?
3. What's the default execution policy on Windows client computers, and what does it prevent you from doing?
4. How do you determine the current PowerShell execution policy setting?
5. How do you change the PowerShell execution policy?

For answers to the review questions, see the Appendix of this book.

References

To learn more about the concepts covered in this chapter, read the following PowerShell help articles.

- about_Automatic_Variables
- about_Execution_Policies

Next steps

In Chapter 2, you'll learn about the discoverability of commands in PowerShell. You'll also learn how to download PowerShell's help files so you can view the help in your PowerShell session.

Last updated on 02/06/2026

Chapter 2 - The Help system

In an experiment designed to assess proficiency in PowerShell, two distinct groups of IT professionals — beginners and experts — were first given a written examination without access to a computer. Surprisingly, the test scores indicated comparable skills across both groups. A subsequent test was then administered, mirroring the first but with one key difference: participants had access to an offline computer equipped with PowerShell. The results revealed a significant skills gap between the two groups this time.

What factors contributed to the outcomes observed between the two assessments?

> Experts don't always know the answers, but they know how to figure out the answers.

The outcomes observed in the results of the two tests were because experts don't memorize thousands of PowerShell commands. Instead, they excel at using the Help system within PowerShell, enabling them to discover and learn how to use commands when necessary.

> Becoming proficient with the Help system is the key to success with PowerShell.

I heard Jeffrey Snover, the creator of PowerShell, share a similar story on multiple occasions.

Discoverability

Compiled commands in PowerShell are known as cmdlets, pronounced as "*command-let*", not "*CMD-let*". The naming convention for cmdlets follows a singular **Verb-Noun** format to make them easily discoverable. For instance, `Get-Process` is the cmdlet to determine what processes are running, and `Get-Service` is the cmdlet to retrieve a list of services. Functions, also known as script cmdlets, and aliases are other types of PowerShell commands that are discussed later in this book. The term "*PowerShell command*" describes any command in PowerShell, regardless of whether it's a cmdlet, function, or alias.

You can also run operating system native commands from PowerShell, such as traditional command-line programs like `ping.exe` and `ipconfig.exe`.

The three core cmdlets in PowerShell

- `Get-Help`

- `Get-Command`
- `Get-Member` (covered in chapter 3)

I'm often asked: "*How do you figure out what the commands are in PowerShell?*". Both `Get-Help` and `Get-Command` are invaluable resources for discovering and understanding commands in PowerShell.

Get-Help

The first thing you need to know about the Help system in PowerShell is how to use the `Get-Help` cmdlet.

`Get-Help` is a multipurpose command that helps you learn how to use commands once you find them. You can also use `Get-Help` to locate commands, but in a different and more indirect way when compared to `Get-Command`.

When using `Get-Help` to locate commands, it initially performs a wildcard search for command names based on your input. If that doesn't find any matches, it conducts a comprehensive full-text search across all PowerShell help articles on your system. If that also fails to find any results, it returns an error.

Here's how to use `Get-Help` to view the help content for the `Get-Help` cmdlet.

PowerShell

```
Get-Help -Name Get-Help
```

Beginning with PowerShell version 3.0, the help content doesn't ship preinstalled with the operating system. When you run `Get-Help` for the first time, a message asks if you want to download the PowerShell help files to your computer.

Answering **Yes** by pressing `Y` executes the `Update-Help` cmdlet, downloading the help content.

Output

```
Do you want to run Update-Help?
The Update-Help cmdlet downloads the most current Help files for Windows
PowerShell modules, and installs them on your computer. For more information
about the Update-Help cmdlet, see
https:/go.microsoft.com/fwlink/?LinkId=210614.
[Y] Yes  [N] No  [S] Suspend  [?] Help (default is "Y"):
```

If you don't receive this message, run `Update-Help` from an elevated PowerShell session running as an administrator.

Once the update is complete, the help article is displayed.

Take a moment to run the example on your computer, review the output, and observe how the help system organizes the information.

- NAME
- SYNOPSIS
- SYNTAX
- DESCRIPTION
- RELATED LINKS
- REMARKS

As you review the output, keep in mind that help articles often contain a vast amount of information, and what you see by default isn't the entire help article.

Parameters

When you run a command in PowerShell, you might need to provide additional information or input to the command. Parameters allow you to specify options and arguments that change the behavior of a command. The **SYNTAX** section of each help article outlines the available parameters for the command.

`Get-Help` has several parameters that you can specify to return the entire help article or a subset for a command. To view all the available parameters for `Get-Help`, see the **SYNTAX** section of its help article, as shown in the following example.

Output

```
...
SYNTAX
    Get-Help [[-Name] <System.String>] [-Category {Alias | Cmdlet | Provider
    | General | FAQ | Glossary | HelpFile | ScriptCommand | Function |
    Filter | ExternalScript | All | DefaultHelp | Workflow | DscResource |
    Class | Configuration}] [-Component <System.String[]>] [-Full]
    [-Functionality <System.String[]>] [-Path <System.String>] [-Role
    <System.String[]>] [<CommonParameters>]

    Get-Help [[-Name] <System.String>] [-Category {Alias | Cmdlet | Provider
    | General | FAQ | Glossary | HelpFile | ScriptCommand | Function |
    Filter | ExternalScript | All | DefaultHelp | Workflow | DscResource |
    Class | Configuration}] [-Component <System.String[]>] -Detailed
    [-Functionality <System.String[]>] [-Path <System.String>] [-Role
```

```
    <System.String[]>] [<CommonParameters>]

    Get-Help [[-Name] <System.String>] [-Category {Alias | Cmdlet | Provider
    | General | FAQ | Glossary | HelpFile | ScriptCommand | Function |
    Filter | ExternalScript | All | DefaultHelp | Workflow | DscResource |
    Class | Configuration}] [-Component <System.String[]>] -Examples
    [-Functionality <System.String[]>] [-Path <System.String>] [-Role
    <System.String[]>] [<CommonParameters>]

    Get-Help [[-Name] <System.String>] [-Category {Alias | Cmdlet | Provider
    | General | FAQ | Glossary | HelpFile | ScriptCommand | Function |
    Filter | ExternalScript | All | DefaultHelp | Workflow | DscResource |
    Class | Configuration}] [-Component <System.String[]>] [-Functionality
    <System.String[]>] -Online [-Path <System.String>] [-Role
    <System.String[]>] [<CommonParameters>]

    Get-Help [[-Name] <System.String>] [-Category {Alias | Cmdlet | Provider
    | General | FAQ | Glossary | HelpFile | ScriptCommand | Function |
    Filter | ExternalScript | All | DefaultHelp | Workflow | DscResource |
    Class | Configuration}] [-Component <System.String[]>] [-Functionality
    <System.String[]>] -Parameter <System.String> [-Path <System.String>]
    [-Role <System.String[]>] [<CommonParameters>]

    Get-Help [[-Name] <System.String>] [-Category {Alias | Cmdlet | Provider
    | General | FAQ | Glossary | HelpFile | ScriptCommand | Function |
    Filter | ExternalScript | All | DefaultHelp | Workflow | DscResource |
    Class | Configuration}] [-Component <System.String[]>] [-Functionality
    <System.String[]>] [-Path <System.String>] [-Role <System.String[]>]
    -ShowWindow [<CommonParameters>]
...
```

Parameter sets

When you review the **SYNTAX** section for `Get-Help`, notice that the information appears to be repeated six times. Each of those blocks is an individual parameter set, indicating the `Get-Help` cmdlet features six distinct sets of parameters. A closer look reveals each parameter set contains at least one unique parameter, making it different from the others.

Parameter sets are mutually exclusive. Once you specify a unique parameter that only exists in one parameter set, PowerShell limits you to using the parameters contained within that parameter set. For instance, you can't use the **Full** and **Detailed** parameters of `Get-Help` together because they belong to different parameter sets.

Each of the following parameters belongs to a different parameter set for the `Get-Help` cmdlet.

- Full
- Detailed
- Examples

- Online
- Parameter
- ShowWindow

The command syntax

If you're new to PowerShell, comprehending the cryptic information — characterized by square and angle brackets — in the **SYNTAX** section might seem overwhelming. However, learning these syntax elements is essential to becoming proficient with PowerShell. The more frequently you use the PowerShell Help system, the easier it becomes to remember all the nuances.

View the syntax of the `Get-EventLog` cmdlet.

PowerShell

```powershell
Get-Help Get-EventLog
```

The following output shows the relevant portion of the help article.

Output

```
...
SYNTAX
    Get-EventLog [-LogName] <System.String> [[-InstanceId]
    <System.Int64[]>] [-After <System.DateTime>] [-AsBaseObject] [-Before
    <System.DateTime>] [-ComputerName <System.String[]>] [-EntryType {Error
    | Information | FailureAudit | SuccessAudit | Warning}] [-Index
    <System.Int32[]>] [-Message <System.String>] [-Newest <System.Int32>]
    [-Source <System.String[]>] [-UserName <System.String[]>]
    [<CommonParameters>]

    Get-EventLog [-AsString] [-ComputerName <System.String[]>] [-List]
    [<CommonParameters>]
...
```

The syntax information includes pairs of square brackets (`[]`). Depending on their usage, these square brackets serve two different purposes.

- Elements enclosed in square brackets are optional.
- An empty set of square brackets following a datatype, such as `<string[]>`, indicates that the parameter can accept multiple values passed as an array or a collection object.

Positional parameters

Some cmdlets are designed to accept positional parameters. Positional parameters allow you to provide a value without specifying the name of the parameter. When using a parameter positionally, you must specify its value in the correct position on the command line. You can find the positional information for a parameter in the **PARAMETERS** section of a command's help article. When you explicitly specify parameter names, you can use the parameters in any order.

For the `Get-EventLog` cmdlet, the first parameter in the first parameter set is **LogName**. **LogName** is enclosed in square brackets, indicating it's a positional parameter.

Syntax

```
Get-EventLog [-LogName] <System.String>
```

Since **LogName** is a positional parameter, you can specify it by either name or position. According to the angle brackets following the parameter name, the value for **LogName** must be a single string. The absence of square brackets enclosing both the parameter name and datatype indicates that **LogName** is a required parameter within this particular parameter set.

The second parameter in that parameter set is **InstanceId**. Both the parameter name and datatype are entirely enclosed in square brackets, signifying that **InstanceId** is an optional parameter.

Syntax

```
[[-InstanceId] <System.Int64[]>]
```

Furthermore, **InstanceId** has its own pair of square brackets, indicating that it's a positional parameter similar to the **LogName** parameter. Following the datatype, an empty set of square brackets implies that **InstanceId** can accept multiple values.

`[switch]` parameters

A parameter that doesn't require a value is called a `[switch]` parameter. You can easily identify `[switch]` parameters because there's no datatype following the parameter name. When you specify a `[switch]` parameter, its value is `true`. When you don't specify a `[switch]` parameter, its value is `false`.

The second parameter set includes a **List** parameter, which is a `[switch]` parameter. When you specify the **List** parameter, it returns a list of event logs on the local computer.

Syntax

```
[-List]
```

A simplified approach to syntax

There's a more user-friendly method to obtain the same information as the cryptic command syntax for some commands, except in plain English. PowerShell returns the complete help article when using `Get-Help` with the **Full** parameter, making it easier to understand a command's usage.

PowerShell

```
Get-Help -Name Get-Help -Full
```

Take a moment to run the example on your computer, review the output, and observe how the help system organizes the information.

- NAME
- SYNOPSIS
- SYNTAX
- DESCRIPTION
- PARAMETERS
- INPUTS
- OUTPUTS
- NOTES
- EXAMPLES
- RELATED LINKS

By specifying the **Full** parameter with the `Get-Help` cmdlet, the output includes several extra sections. Among these sections, **PARAMETERS** often provides a detailed explanation for each parameter. However, the extent of this information varies depending on the specific command you're investigating.

Output

```
...
    -Detailed <System.Management.Automation.SwitchParameter>
        Adds parameter descriptions and examples to the basic help display.
        This parameter is effective only when the help files are installed
        on the computer. It has no effect on displays of conceptual ( About_
        ) help.
```

```
        Required?                    true
        Position?                    named
        Default value                False
        Accept pipeline input?       False
        Accept wildcard characters?  false

    -Examples <System.Management.Automation.SwitchParameter>
        Displays only the name, synopsis, and examples. This parameter is
        effective only when the help files are installed on the computer. It
        has no effect on displays of conceptual ( About_ ) help.

        Required?                    true
        Position?                    named
        Default value                False
        Accept pipeline input?       False
        Accept wildcard characters?  false

    -Full <System.Management.Automation.SwitchParameter>
        Displays the entire help article for a cmdlet. Full includes
        parameter descriptions and attributes, examples, input and output
        object types, and additional notes.

        This parameter is effective only when the help files are installed
        on the computer. It has no effect on displays of conceptual ( About_
        ) help.

        Required?                    false
        Position?                    named
        Default value                False
        Accept pipeline input?       False
        Accept wildcard characters?  false
...
```

When you ran the previous command to display the help for the `Get-Help` command, you probably noticed the output scrolled by too quickly to read it.

If you're using the PowerShell console, Windows Terminal, or VS Code and need to view a help article, the `help` function can be useful. It pipes the output of `Get-Help` to `more.com`, displaying one page of help content at a time. I recommend using the `help` function instead of the `Get-Help` cmdlet because it provides a better user experience and it's less to type.

> ⓘ **Note**
>
> The ISE doesn't support using `more.com`, so running `help` works the same way as `Get-Help`.

Run each of the following commands in PowerShell on your computer.

PowerShell

```
Get-Help -Name Get-Help -Full
help -Name Get-Help -Full
help Get-Help -Full
```

Did you observe any variations in the output when you ran the previous commands?

In the previous example, the first line uses the `Get-Help` cmdlet, the second uses the `help` function, and the third line omits the **Name** parameter while using the `help` function. Since **Name** is a positional parameter, the third example takes advantage of its position instead of explicitly stating the parameter's name.

The difference is that the last two commands display their output one page at a time. When using the `help` function, press the `Spacebar` to display the next page of content or `Q` to quit. If you need to terminate any command running interactively in PowerShell, press `Ctrl` + `C`.

To quickly find information about a specific parameter, use the **Parameter** parameter. This approach returns content containing only the parameter-specific information, rather than the entire help article. This is the easiest way to find information about a specific parameter.

The following example uses the `help` function with the **Parameter** parameter to return information from the help article for the **Name** parameter of `Get-Help`.

PowerShell

```
help Get-Help -Parameter Name
```

The help information shows that the **Name** parameter is positional and must be specified in the first position (position zero) when used positionally.

Output

```
-Name <System.String>
    Gets help about the specified command or concept. Enter the name of a
    cmdlet, function, provider, script, or workflow, such as `Get-Member`,
    a conceptual article name, such as `about_Objects`, or an alias, such
    as `ls`. Wildcard characters are permitted in cmdlet and provider
    names, but you can't use wildcard characters to find the names of
    function help and script help articles.

    To get help for a script that isn't located in a path that's listed in
    the `$env:Path` environment variable, type the script's path and file
    name.

    If you enter the exact name of a help article, `Get-Help` displays the
```

```
    article contents.

    If you enter a word or word pattern that appears in several help
    article titles, `Get-Help` displays a list of the matching titles.

    If you enter any text that doesn't match any help article titles,
    `Get-Help` displays a list of articles that include that text in their
    contents.

    The names of conceptual articles, such as `about_Objects`, must be
    entered in English, even in non-English versions of PowerShell.

    Required?                    false
    Position?                    0
    Default value                None
    Accept pipeline input?       True (ByPropertyName)
    Accept wildcard characters?  true
```

The **Name** parameter expects a string value as identified by the `<String>` datatype next to the parameter name.

There are several other parameters you can specify with `Get-Help` to return a subset of a help article. To see how they work, run the following commands on your computer.

PowerShell

```powershell
Get-Help -Name Get-Command -Full
Get-Help -Name Get-Command -Detailed
Get-Help -Name Get-Command -Examples
Get-Help -Name Get-Command -Online
Get-Help -Name Get-Command -Parameter Noun
Get-Help -Name Get-Command -ShowWindow
```

I typically use `help <command name>` with the **Full** or **Online** parameter. If you only have an interest in the examples, use the **Examples** parameter. If you only have an interest in a specific parameter, use the **Parameter** parameter.

When you use the **ShowWindow** parameter, it displays the help content in a separate searchable window. You can move that window to a different monitor if you have multiple monitors. However, the **ShowWindow** parameter has a known bug that might prevent it from displaying the entire help article. The **ShowWindow** parameter also requires an operating system with a Graphical User Interface (GUI). It returns an error when you attempt to use it on Windows Server Core.

If you have internet access, you can use the **Online** parameter instead. The **Online** parameter opens the help article in your default web browser. The online content is the most up-to-date

content. The browser allows you to search the help content and view other related help articles.

> ⓘ Note
>
> The **Online** parameter isn't supported for **About** articles.

PowerShell

```
help Get-Command -Online
```

Finding commands with Get-Help

To find commands with `Get-Help`, specify a search term surrounded by asterisk (`*`) wildcard characters for the value of the **Name** parameter. The following example uses the **Name** parameter positionally.

PowerShell

```
help *process*
```

Output

```
Name                              Category  Module                    Synops
----                              --------  ------                    ------
Enter-PSHostProcess               Cmdlet    Microsoft.PowerShell.Core Con...
Exit-PSHostProcess                Cmdlet    Microsoft.PowerShell.Core Clo...
Get-PSHostProcessInfo             Cmdlet    Microsoft.PowerShell.Core Get...
Debug-Process                     Cmdlet    Microsoft.PowerShell.M... Deb...
Get-Process                       Cmdlet    Microsoft.PowerShell.M... Get...
Start-Process                     Cmdlet    Microsoft.PowerShell.M... Sta...
Stop-Process                      Cmdlet    Microsoft.PowerShell.M... Sto...
Wait-Process                      Cmdlet    Microsoft.PowerShell.M... Wai...
Invoke-LapsPolicyProcessing       Cmdlet    LAPS                      Inv...
ConvertTo-ProcessMitigationPolicy Cmdlet    ProcessMitigations        Con...
Get-ProcessMitigation             Cmdlet    ProcessMitigations        Get...
Set-ProcessMitigation             Cmdlet    ProcessMitigations        Set...
```

In this scenario, you aren't required to add the `*` wildcard characters. If `Get-Help` can't find a command matching the value you provided, it does a full-text search for that value. The following example produces the same results as specifying the `*` wildcard character on each end of `process`.

PowerShell

```
help process
```

When you specify a wildcard character within the value, `Get-Help` only searches for commands that match the pattern you provided. It doesn't perform a full-text search. The following command doesn't return any results.

PowerShell

```
help pr*cess
```

PowerShell generates an error if you specify a value that begins with a dash without enclosing it in quotes because it interprets it as a parameter name. No such parameter name exists for the `Get-Help` cmdlet.

PowerShell

```
help -process
```

If you're attempting to search for commands that end with `-process`, you must add an `*` to the beginning of the value.

PowerShell

```
help *-process
```

When you search for PowerShell commands with `Get-Help`, it's better to be vague rather than too specific.

When you searched for `process` earlier, the results only returned commands that included `process` in their name. But if you search for `processes`, it doesn't find any matches for command names. As previously stated, when help doesn't find any matches, it performs a comprehensive full-text search of every help article on your system and returns those results. This type of search often produces more results than expected, including information not relevant to you.

PowerShell

```
help processes
```

Output

```
Name                              Category  Module                    Synops
----                              --------  ------                    ------
Disconnect-PSSession              Cmdlet    Microsoft.PowerShell.Core Dis...
Enter-PSHostProcess               Cmdlet    Microsoft.PowerShell.Core Con...
ForEach-Object                    Cmdlet    Microsoft.PowerShell.Core Per...
Get-PSHostProcessInfo             Cmdlet    Microsoft.PowerShell.Core Get...
Get-PSSessionConfiguration        Cmdlet    Microsoft.PowerShell.Core Get...
New-PSSessionOption               Cmdlet    Microsoft.PowerShell.Core Cre...
New-PSTransportOption             Cmdlet    Microsoft.PowerShell.Core Cre...
Out-Host                          Cmdlet    Microsoft.PowerShell.Core Sen...
Start-Job                         Cmdlet    Microsoft.PowerShell.Core Sta...
Where-Object                      Cmdlet    Microsoft.PowerShell.Core Sel...
Debug-Process                     Cmdlet    Microsoft.PowerShell.M... Deb...
Get-Process                       Cmdlet    Microsoft.PowerShell.M... Get...
Get-WmiObject                     Cmdlet    Microsoft.PowerShell.M... Get...
Start-Process                     Cmdlet    Microsoft.PowerShell.M... Sta...
Stop-Process                      Cmdlet    Microsoft.PowerShell.M... Sto...
Wait-Process                      Cmdlet    Microsoft.PowerShell.M... Wai...
Clear-Variable                    Cmdlet    Microsoft.PowerShell.U... Del...
Convert-String                    Cmdlet    Microsoft.PowerShell.U... For...
ConvertFrom-Csv                   Cmdlet    Microsoft.PowerShell.U... Con...
ConvertFrom-Json                  Cmdlet    Microsoft.PowerShell.U... Con...
ConvertTo-Html                    Cmdlet    Microsoft.PowerShell.U... Con...
ConvertTo-Xml                     Cmdlet    Microsoft.PowerShell.U... Cre...
Debug-Runspace                    Cmdlet    Microsoft.PowerShell.U... Sta...
Export-Csv                        Cmdlet    Microsoft.PowerShell.U... Con...
Export-FormatData                 Cmdlet    Microsoft.PowerShell.U... Sav...
Format-List                       Cmdlet    Microsoft.PowerShell.U... For...
Format-Table                      Cmdlet    Microsoft.PowerShell.U... For...
Get-Unique                        Cmdlet    Microsoft.PowerShell.U... Ret...
Group-Object                      Cmdlet    Microsoft.PowerShell.U... Gro...
Import-Clixml                     Cmdlet    Microsoft.PowerShell.U... Imp...
Import-Csv                        Cmdlet    Microsoft.PowerShell.U... Cre...
Measure-Object                    Cmdlet    Microsoft.PowerShell.U... Cal...
Out-File                          Cmdlet    Microsoft.PowerShell.U... Sen...
Out-GridView                      Cmdlet    Microsoft.PowerShell.U... Sen...
Select-Object                     Cmdlet    Microsoft.PowerShell.U... Sel...
Set-Variable                      Cmdlet    Microsoft.PowerShell.U... Set...
Sort-Object                       Cmdlet    Microsoft.PowerShell.U... Sor...
Tee-Object                        Cmdlet    Microsoft.PowerShell.U... Sav...
Trace-Command                     Cmdlet    Microsoft.PowerShell.U... Con...
Write-Information                 Cmdlet    Microsoft.PowerShell.U... Spe...
Export-BinaryMiLog                Cmdlet    CimCmdlets                Cre...
Get-CimAssociatedInstance         Cmdlet    CimCmdlets                Ret...
Get-CimInstance                   Cmdlet    CimCmdlets                Get...
Import-BinaryMiLog                Cmdlet    CimCmdlets                Use...
Invoke-CimMethod                  Cmdlet    CimCmdlets                Inv...
New-CimInstance                   Cmdlet    CimCmdlets                Cre...
Remove-CimInstance                Cmdlet    CimCmdlets                Rem...
Set-CimInstance                   Cmdlet    CimCmdlets                Mod...
Compress-Archive                  Function  Microsoft.PowerShell.A... Cre...
Get-Counter                       Cmdlet    Microsoft.PowerShell.D... Get...
Invoke-WSManAction                Cmdlet    Microsoft.WSMan.Manage... Inv...
Remove-WSManInstance              Cmdlet    Microsoft.WSMan.Manage... Del...
```

```
Get-WSManInstance                 Cmdlet    Microsoft.WSMan.Manage... Dis...
New-WSManInstance                 Cmdlet    Microsoft.WSMan.Manage... Cre...
Set-WSManInstance                 Cmdlet    Microsoft.WSMan.Manage... Mod...
about_Arithmetic_Operators        HelpFile
about_Arrays                      HelpFile
about_Environment_Variables       HelpFile
about_Execution_Policies          HelpFile
about_Functions                   HelpFile
about_Jobs                        HelpFile
about_Logging                     HelpFile
about_Methods                     HelpFile
about_Objects                     HelpFile
about_Pipelines                   HelpFile
about_Preference_Variables        HelpFile
about_Remote                      HelpFile
about_Remote_Jobs                 HelpFile
about_Session_Configuration_Files HelpFile
about_Simplified_Syntax           HelpFile
about_Switch                      HelpFile
about_Variables                   HelpFile
about_Variable_Provider           HelpFile
about_Windows_PowerShell_5.1      HelpFile
about_WQL                         HelpFile
about_WS-Management_Cmdlets       HelpFile
about_Foreach-Parallel            HelpFile
about_Parallel                    HelpFile
about_Sequence                    HelpFile
```

When you searched for `process`, it returned 12 results. However, when searching for `processes`, it produced 78 results. If your search only finds one match, `Get-Help` displays the help content instead of listing the search results.

PowerShell

```
help *hotfix*
```

Output

```
NAME
    Get-HotFix

SYNOPSIS
    Gets the hotfixes that are installed on local or remote computers.


SYNTAX
    Get-HotFix [-ComputerName <System.String[]>] [-Credential
    <System.Management.Automation.PSCredential>] [-Description
    <System.String[]>] [<CommonParameters>]

    Get-HotFix [[-Id] <System.String[]>] [-ComputerName <System.String[]>]
```

```
    [-Credential <System.Management.Automation.PSCredential>]
    [<CommonParameters>]


DESCRIPTION
    > This cmdlet is only available on the Windows platform. The
    `Get-HotFix` cmdlet uses the Win32_QuickFixEngineering WMI class to
    list hotfixes that are installed on the local computer or specified
    remote computers.


RELATED LINKS
    Online Version: https://learn.microsoft.com/powershell/module/microsoft.
    powershell.management/get-hotfix?view=powershell-5.1&WT.mc_id=ps-gethelp
    about_Arrays
    Add-Content
    Get-ComputerRestorePoint
    Get-Credential
    Win32_QuickFixEngineering class

REMARKS
    To see the examples, type: "Get-Help Get-HotFix -Examples".
    For more information, type: "Get-Help Get-HotFix -Detailed".
    For technical information, type: "Get-Help Get-HotFix -Full".
    For online help, type: "Get-Help Get-HotFix -Online"
```

You can also find commands that lack help articles with `Get-Help`, although this capability isn't commonly known. The `more` function is one of the commands that doesn't have a help article. To confirm that you can find commands with `Get-Help` that don't include help articles, use the `help` function to find `more`.

PowerShell

```
help *more*
```

The search only found one match, so it returned the basic syntax information you see when a command doesn't have a help article.

Output

```
NAME
    more

SYNTAX
    more [[-paths] <string[]>]

ALIASES
    None
```

```
REMARKS
    None
```

The PowerShell help system also contains conceptual **About** help articles. You must update the help content on your system to get the **About** articles. For more information, see the Updating help section of this chapter.

Use the following command to return a list of all **About** help articles on your system.

PowerShell

```
help About_*
```

When you limit the results to one **About** help article, `Get-Help` displays the content of that article.

PowerShell

```
help about_Updatable_Help
```

Updating help

Earlier in this chapter, you updated the PowerShell help articles on your computer the first time you ran the `Get-Help` cmdlet. You should periodically run the `Update-Help` cmdlet on your computer to obtain any updates to the help content.

> **ⓘ Important**
>
> In Windows PowerShell 5.1, you must run `Update-Help` as an administrator in an elevated PowerShell session.

In the following example, `Update-Help` downloads the PowerShell help content for all modules installed on your computer. You should use the **Force** parameter to ensure that you download the latest version of the help content.

PowerShell

```
Update-Help -Force
```

As shown in the following results, a module returned an error. Errors aren't uncommon and usually occur when the module's author doesn't configure updatable help correctly.

Output

```
Update-Help : Failed to update Help for the module(s) 'BitsTransfer' with UI
culture(s) {en-US} : Unable to retrieve the HelpInfo XML file for UI culture
en-US. Make sure the HelpInfoUri property in the module manifest is valid or
check your network connection and then try the command again.
At line:1 char:1
+ Update-Help
+ ~~~~~~~~~~~
    + CategoryInfo          : ResourceUnavailable: (:) [Update-Help], Except
   ion
    + FullyQualifiedErrorId : UnableToRetrieveHelpInfoXml,Microsoft.PowerShe
   ll.Commands.UpdateHelpCommand
```

`Update-Help` requires internet access to download the help content. If your computer doesn't have internet access, use the `Save-Help` cmdlet on a computer with internet access to download and save the updated help content. Then, use the **SourcePath** parameter of `Update-Help` to specify the location of the saved updated help content.

Get-Command

`Get-Command` is another multipurpose command that helps you find commands. When you run `Get-Command` without any parameters, it returns a list of all PowerShell commands on your system. You can also use `Get-Command` to get command syntax similar to `Get-Help`.

How do you determine the syntax for `Get-Command`? You could use `Get-Help` to display the help article for `Get-Command`, as shown in the Get-Help section of this chapter. You can also use `Get-Command` with the **Syntax** parameter to view the syntax for any command. This shortcut helps you quickly determine how to use a command without navigating through its help content.

PowerShell

```
Get-Command -Name Get-Command -Syntax
```

Using `Get-Command` with the **Syntax** parameter provides a more concise view of the syntax that shows the parameters and their value types, without listing the specific allowable values like `Get-Help` shows.

Output

```
Get-Command [[-ArgumentList] <Object[]>] [-Verb <string[]>]
[-Noun <string[]>] [-Module <string[]>]
[-FullyQualifiedModule <ModuleSpecification[]>] [-TotalCount <int>]
[-Syntax] [-ShowCommandInfo] [-All] [-ListImported]
[-ParameterName <string[]>] [-ParameterType <PSTypeName[]>]
[<CommonParameters>]

Get-Command [[-Name] <string[]>] [[-ArgumentList] <Object[]>]
[-Module <string[]>] [-FullyQualifiedModule <ModuleSpecification[]>]
[-CommandType <CommandTypes>] [-TotalCount <int>] [-Syntax]
[-ShowCommandInfo] [-All] [-ListImported] [-ParameterName <string[]>]
[-ParameterType <PSTypeName[]>] [<CommonParameters>]
```

If you need more detailed information about how to use a command, use `Get-Help`.

PowerShell

```
help Get-Command -Full
```

The **SYNTAX** section of `Get-Help` provides a more user-friendly display by expanding enumerated values for parameters. It shows you the actual values you can use, making it easier to understand the available options.

Output

```
...
    Get-Command [[-Name] <System.String[]>] [[-ArgumentList]
    <System.Object[]>] [-All] [-CommandType {Alias | Function | Filter |
    Cmdlet | ExternalScript | Application | Script | Workflow |
    Configuration | All}] [-FullyQualifiedModule
    <Microsoft.PowerShell.Commands.ModuleSpecification[]>] [-ListImported]
    [-Module <System.String[]>] [-ParameterName <System.String[]>]
    [-ParameterType <System.Management.Automation.PSTypeName[]>]
    [-ShowCommandInfo] [-Syntax] [-TotalCount <System.Int32>]
    [<CommonParameters>]

    Get-Command [[-ArgumentList] <System.Object[]>] [-All]
    [-FullyQualifiedModule
    <Microsoft.PowerShell.Commands.ModuleSpecification[]>] [-ListImported]
    [-Module <System.String[]>] [-Noun <System.String[]>] [-ParameterName
    <System.String[]>] [-ParameterType
    <System.Management.Automation.PSTypeName[]>] [-ShowCommandInfo]
    [-Syntax] [-TotalCount <System.Int32>] [-Verb <System.String[]>]
    [<CommonParameters>]
...
```

The **PARAMETERS** section of the help for `Get-Command` reveals that the **Name**, **Noun**, and **Verb** parameters accept wildcard characters.

Output

```
...
    -Name <System.String[]>
        Specifies an array of names. This cmdlet gets only commands that
        have the specified name. Enter a name or name pattern. Wildcard
        characters are permitted.

        To get commands that have the same name, use the All parameter. When
        two commands have the same name, by default, `Get-Command` gets the
        command that runs when you type the command name.

        Required?                    false
        Position?                    0
        Default value                None
        Accept pipeline input?       True (ByPropertyName, ByValue)
        Accept wildcard characters?  true

    -Noun <System.String[]>
        Specifies an array of command nouns. This cmdlet gets commands,
        which include cmdlets, functions, and aliases, that have names that
        include the specified noun. Enter one or more nouns or noun
        patterns. Wildcard characters are permitted.

        Required?                    false
        Position?                    named
        Default value                None
        Accept pipeline input?       True (ByPropertyName)
        Accept wildcard characters?  true
    -Verb <System.String[]>
        Specifies an array of command verbs. This cmdlet gets commands,
        which include cmdlets, functions, and aliases, that have names that
        include the specified verb. Enter one or more verbs or verb
        patterns. Wildcard characters are permitted.

        Required?                    false
        Position?                    named
        Default value                None
        Accept pipeline input?       True (ByPropertyName)
        Accept wildcard characters?  true
...
```

The following example uses the `*` wildcard character with the value for the **Name** parameter of `Get-Command`.

PowerShell

```
Get-Command -Name *service*
```

When you use wildcard characters with the **Name** parameter of `Get-Command`, it returns PowerShell commands and native commands, as shown in the following results.

Output

```
CommandType     Name                                               Version
-----------     ----                                               -------
Function        Get-NetFirewallServiceFilter                       2.0.0.0
Function        Set-NetFirewallServiceFilter                       2.0.0.0
Cmdlet          Get-Service                                        3.1.0.0
Cmdlet          New-Service                                        3.1.0.0
Cmdlet          New-WebServiceProxy                                3.1.0.0
Cmdlet          Restart-Service                                    3.1.0.0
Cmdlet          Resume-Service                                     3.1.0.0
Cmdlet          Set-Service                                        3.1.0.0
Cmdlet          Start-Service                                      3.1.0.0
Cmdlet          Stop-Service                                       3.1.0.0
Cmdlet          Suspend-Service                                    3.1.0.0
Application     SecurityHealthService.exe                          10.0.2...
Application     SensorDataService.exe                              10.0.2...
Application     services.exe                                       10.0.2...
Application     services.msc                                       0.0.0.0
Application     TieringEngineService.exe                           10.0.2...
Application     Windows.WARP.JITService.exe                        10.0.2...
```

You can limit the results of `Get-Command` to PowerShell commands using the **CommandType** parameter.

PowerShell

```powershell
Get-Command -Name *service* -CommandType Cmdlet, Function, Alias, Script
```

Another option might be to use either the **Verb** or **Noun** parameter or both since only PowerShell commands have verbs and nouns.

The following example uses `Get-Command` to find commands on your computer that work with processes. Use the **Noun** parameter and specify `Process` as its value.

PowerShell

```powershell
Get-Command -Noun Process
```

Output

```
CommandType     Name                                               Version
-----------     ----                                               -------
Cmdlet          Debug-Process                                      3.1.0.0
Cmdlet          Get-Process                                        3.1.0.0
Cmdlet          Start-Process                                      3.1.0.0
```

```
Cmdlet          Stop-Process                                       3.1.0.0
Cmdlet          Wait-Process                                       3.1.0.0
```

Summary

In this chapter, you learned how to find commands with `Get-Help` and `Get-Command`. You also learned how to use the help system to understand how to use commands once you find them. In addition, you learned how to update the help system on your computer when new help content is available.

Review

1. Is the **DisplayName** parameter of `Get-Service` positional?
2. How many parameter sets does the `Get-Process` cmdlet have?
3. What PowerShell commands exist for working with event logs?
4. What's the PowerShell command for returning a list of PowerShell processes running on your computer?
5. How do you update the PowerShell help content stored on your computer?

References

To learn more about the concepts covered in this chapter, read the following PowerShell help articles.

- Get-Help
- Get-Command
- Update-Help
- Save-Help
- about_Updatable_Help
- about_Command_Syntax

Next steps

In Chapter 3, you'll learn about objects, properties, methods, and the `Get-Member` cmdlet.

Last updated on 04/08/2026

Chapter 3 - Discovering objects, properties, and methods

PowerShell is an object-oriented scripting language. It represents data and system states using structured objects derived from .NET classes defined in the .NET Framework. By leveraging the .NET Framework, PowerShell offers access to various system capabilities, including file system, registry, and Windows Management Instrumentation (WMI) classes. PowerShell also has access to the .NET Framework class library, which contains a vast collection of classes that you can use to develop robust PowerShell scripts.

In PowerShell, each item or state is an instance of an object that can be explored and manipulated. The `Get-Member` cmdlet is one of the primary tools provided by PowerShell for object discovery, which reveals an object's characteristics. This chapter explores how PowerShell leverages objects and how you can discover and manipulate these objects to streamline your scripts and manage your systems more efficiently.

Prerequisites

To follow the specific examples in this chapter, ensure that your lab environment computer is part of your lab environment Active Directory domain. You must also install the Active Directory PowerShell module bundled with the Windows Remote Server Administration Tools (RSAT). If you're using Windows 10 build 1809 or later, including Windows 11, you can install RSAT as a Windows feature.

> ⓘ **Note**
>
> Active Directory is unsupported for Windows Home editions.

- For information about installing RSAT, see Windows Management modules.
- For older versions of Windows, see RSAT for Windows.

Get-Member

`Get-Member` provides insight into the objects, properties, and methods associated with PowerShell commands. You can pipe any PowerShell command that produces object-based

output to `Get-Member`. When you pipe the output of a command to `Get-Member`, it reveals the structure of the object returned by the command, detailing its properties and methods.

- **Properties**: The attributes of an object.
- **Methods**: The actions you can perform on an object.

To illustrate this concept, consider a driver's license as an analogy. Like any object, a driver's license has properties, such as **eye color**, which typically includes `blue` and `brown` values. In contrast, methods represent actions you can perform on the object. For instance, **Revoke** is a method that the Department of Motor Vehicles can perform on a driver's license.

Properties

To retrieve details about the Windows Time service on your system using PowerShell, use the `Get-Service` cmdlet.

PowerShell

```powershell
Get-Service -Name w32time
```

The results include the **Status**, **Name**, and **DisplayName** properties. The **Status** property indicates that the service is `Running`. The value for the **Name** property is `w32time`, and the value for the **DisplayName** property is `Windows Time`.

Output

```
Status   Name               DisplayName
------   ----               -----------
Running  w32time            Windows Time
```

To list all available properties and methods for `Get-Service`, pipe it to `Get-Member`.

PowerShell

```powershell
Get-Service -Name w32time | Get-Member
```

The results show the first line contains one piece of significant information. **TypeName** identifies the type of object returned, which in this example is a **System.ServiceProcess.ServiceController** object. This name is often abbreviated to the last part of the **TypeName**, such as **ServiceController**, in this example.

Output

```
   TypeName: System.ServiceProcess.ServiceController

Name                      MemberType    Definition
----                      ----------    ----------
Name                      AliasProperty Name = ServiceName
RequiredServices          AliasProperty RequiredServices = ServicesDepend...
Disposed                  Event         System.EventHandler Disposed(Syst...
Close                     Method        void Close()
Continue                  Method        void Continue()
CreateObjRef              Method        System.Runtime.Remoting.ObjRef Cr...
Dispose                   Method        void Dispose(), void IDisposable....
Equals                    Method        bool Equals(System.Object obj)
ExecuteCommand            Method        void ExecuteCommand(int command)
GetHashCode               Method        int GetHashCode()
GetLifetimeService        Method        System.Object GetLifetimeService()
GetType                   Method        type GetType()
InitializeLifetimeService Method        System.Object InitializeLifetimeS...
Pause                     Method        void Pause()
Refresh                   Method        void Refresh()
Start                     Method        void Start(), void Start(string[]...
Stop                      Method        void Stop()
WaitForStatus             Method        void WaitForStatus(System.Service...
CanPauseAndContinue       Property      bool CanPauseAndContinue {get;}
CanShutdown               Property      bool CanShutdown {get;}
CanStop                   Property      bool CanStop {get;}
Container                 Property      System.ComponentModel.IContainer ...
DependentServices         Property      System.ServiceProcess.ServiceCont...
DisplayName               Property      string DisplayName {get;set;}
MachineName               Property      string MachineName {get;set;}
ServiceHandle             Property      System.Runtime.InteropServices.Sa...
ServiceName               Property      string ServiceName {get;set;}
ServicesDependedOn        Property      System.ServiceProcess.ServiceCont...
ServiceType               Property      System.ServiceProcess.ServiceType...
Site                      Property      System.ComponentModel.ISite Site ...
StartType                 Property      System.ServiceProcess.ServiceStar...
Status                    Property      System.ServiceProcess.ServiceCont...
ToString                  ScriptMethod  System.Object ToString();
```

Notice when you piped `Get-Service` to `Get-Member`, there are more properties than are displayed by default. Although these additional properties aren't shown by default, you can select them by piping to `Select-Object` and using the **Property** parameter. The following example selects all properties by piping the results of `Get-Service` to `Select-Object` and specifying the `*` wildcard character as the value for the **Property** parameter.

PowerShell

```powershell
Get-Service -Name w32time | Select-Object -Property *
```

By default, PowerShell returns four properties as a table and five or more as a list. However, some commands apply custom formatting to override the default number of properties displayed in a table. You can use `Format-Table` and `Format-List` to override these defaults manually.

Output

```
Name                : w32time
RequiredServices    : {}
CanPauseAndContinue : False
CanShutdown         : True
CanStop             : True
DisplayName         : Windows Time
DependentServices   : {}
MachineName         : .
ServiceName         : w32time
ServicesDependedOn  : {}
ServiceHandle       :
Status              : Running
ServiceType         : Win32OwnProcess, Win32ShareProcess
StartType           : Manual
Site                :
Container           :
```

Specific properties can also be selected using a comma-separated list as the value of the **Property** parameter.

PowerShell

```powershell
Get-Service -Name w32time |
    Select-Object -Property Status, Name, DisplayName, ServiceType
```

Output

```
 Status Name    DisplayName                          ServiceType
 ------ ----    -----------                          -----------
Running w32time Windows Time Win32OwnProcess, Win32ShareProcess
```

You can use wildcard characters when specifying property names with `Select-Object`.

In the following example, use `Can*` as one of the values for the **Property** parameter to return all the properties that start with `Can`. These include **CanPauseAndContinue**, **CanShutdown**, and **CanStop**.

PowerShell

```
Get-Service -Name w32time |
    Select-Object -Property Status, DisplayName, Can*
```

Notice there are more properties listed than are displayed by default.

Output

```
Status              : Running
DisplayName         : Windows Time
CanPauseAndContinue : False
CanShutdown         : True
CanStop             : True
```

Methods

Methods are actions you can perform on an object. Use the **MemberType** parameter to narrow down the results of `Get-Member` to display only the methods for `Get-Service`.

PowerShell

```
Get-Service -Name w32time | Get-Member -MemberType Method
```

As you can see, there are several methods.

Output

```
   TypeName: System.ServiceProcess.ServiceController

Name                      MemberType Definition
----                      ---------- ----------
Close                     Method     void Close()
Continue                  Method     void Continue()
CreateObjRef              Method     System.Runtime.Remoting.ObjRef Creat...
Dispose                   Method     void Dispose(), void IDisposable.Dis...
Equals                    Method     bool Equals(System.Object obj)
ExecuteCommand            Method     void ExecuteCommand(int command)
GetHashCode               Method     int GetHashCode()
GetLifetimeService        Method     System.Object GetLifetimeService()
GetType                   Method     type GetType()
InitializeLifetimeService Method     System.Object InitializeLifetimeServ...
Pause                     Method     void Pause()
Refresh                   Method     void Refresh()
Start                     Method     void Start(), void Start(string[] args)
Stop                      Method     void Stop()
WaitForStatus             Method     void WaitForStatus(System.ServicePro...
```

You can use the **Stop** method to stop a Windows service. You must run this command from an elevated PowerShell session.

PowerShell

```powershell
(Get-Service -Name w32time).Stop()
```

Query the status of the Windows Time service to confirm it's stopped.

PowerShell

```powershell
Get-Service -Name w32time
```

Output

```
Status   Name               DisplayName
------   ----               -----------
Stopped  w32time            Windows Time
```

You might use methods sparingly, but you should be aware of them. Sometimes, you find `Get-*` commands without a corresponding `Set-*` command. Often, you can find a method to perform a `Set-*` action in this scenario. The `Get-SqlAgentJob` cmdlet in the **SqlServer** PowerShell module is an excellent example. No corresponding `Set-*` cmdlet exists, but you can use a method to complete the same task. For more information about the **SqlServer** PowerShell module and installation instructions, see the SQL Server PowerShell overview.

Another reason to be aware of methods is some PowerShell users assume you can't make destructive changes with `Get-*` commands, but they can actually cause severe problems if misused.

A better option is to use a dedicated cmdlet if one exists to perform an action. For example, use the `Start-Service` cmdlet to start the Windows Time service.

By default, `Start-Service`, like the **Start** method of `Get-Service`, doesn't return any results. However, one of the benefits of using a cmdlet is that it often provides additional capabilities that aren't available with a method.

In the following example, use the **PassThru** parameter, which causes a cmdlet that doesn't typically produce output to generate output.

Since PowerShell doesn't participate in User Access Control (UAC), you must run commands that require elevation, such as `Start-Service`, from an elevated PowerShell session.

PowerShell

```
Get-Service -Name w32time | Start-Service -PassThru
```

Output

```
Status   Name               DisplayName
------   ----               -----------
Running  w32time            Windows Time
```

> **Note**
>
> When working with PowerShell cmdlets, it's important to avoid making assumptions about their output.

To retrieve information about the PowerShell process running on your lab environment computer, use the `Get-Process` cmdlet.

PowerShell

```
Get-Process -Name powershell
```

Output

```
Handles  NPM(K)    PM(K)      WS(K)     CPU(s)      Id  SI ProcessName
-------  ------    -----      -----     ------      --  -- -----------
    710      31    55692      70580       0.72    9436   2 powershell
```

To determine the available properties, pipe `Get-Process` to `Get-Member`.

PowerShell

```
Get-Process -Name powershell | Get-Member
```

When using the `Get-Process` command, you might notice that some properties displayed by default are missing when you view the results of `Get-Member`. This behavior is because several of the values shown by default, such as `NPM(K)`, `PM(K)`, `WS(K)`, and `CPU(s)`, are calculated properties. You must pipe commands to `Get-Member` to determine their actual property names.

Output

```
   TypeName: System.Diagnostics.Process

Name                        MemberType     Definition
----                        ----------     ----------
Handles                     AliasProperty  Handles = Handlecount
Name                        AliasProperty  Name = ProcessName
NPM                         AliasProperty  NPM = NonpagedSystemMemorySize64
PM                          AliasProperty  PM = PagedMemorySize64
SI                          AliasProperty  SI = SessionId
VM                          AliasProperty  VM = VirtualMemorySize64
WS                          AliasProperty  WS = WorkingSet64
Disposed                    Event          System.EventHandler Disposed(Sy...
ErrorDataReceived           Event          System.Diagnostics.DataReceived...
Exited                      Event          System.EventHandler Exited(Syst...
OutputDataReceived          Event          System.Diagnostics.DataReceived...
BeginErrorReadLine          Method         void BeginErrorReadLine()
BeginOutputReadLine         Method         void BeginOutputReadLine()
CancelErrorRead             Method         void CancelErrorRead()
CancelOutputRead            Method         void CancelOutputRead()
Close                       Method         void Close()
CloseMainWindow             Method         bool CloseMainWindow()
CreateObjRef                Method         System.Runtime.Remoting.ObjRef ...
Dispose                     Method         void Dispose(), void IDisposabl...
Equals                      Method         bool Equals(System.Object obj)
GetHashCode                 Method         int GetHashCode()
GetLifetimeService          Method         System.Object GetLifetimeService()
GetType                     Method         type GetType()
InitializeLifetimeService   Method         System.Object InitializeLifetim...
Kill                        Method         void Kill()
Refresh                     Method         void Refresh()
Start                       Method         bool Start()
ToString                    Method         string ToString()
WaitForExit                 Method         bool WaitForExit(int millisecon...
WaitForInputIdle            Method         bool WaitForInputIdle(int milli...
__NounName                  NoteProperty   string __NounName=Process
BasePriority                Property       int BasePriority {get;}
Container                   Property       System.ComponentModel.IContaine...
EnableRaisingEvents         Property       bool EnableRaisingEvents {get;s...
ExitCode                    Property       int ExitCode {get;}
ExitTime                    Property       datetime ExitTime {get;}
Handle                      Property       System.IntPtr Handle {get;}
HandleCount                 Property       int HandleCount {get;}
HasExited                   Property       bool HasExited {get;}
Id                          Property       int Id {get;}
MachineName                 Property       string MachineName {get;}
MainModule                  Property       System.Diagnostics.ProcessModul...
MainWindowHandle            Property       System.IntPtr MainWindowHandle ...
MainWindowTitle             Property       string MainWindowTitle {get;}
MaxWorkingSet               Property       System.IntPtr MaxWorkingSet {ge...
MinWorkingSet               Property       System.IntPtr MinWorkingSet {ge...
Modules                     Property       System.Diagnostics.ProcessModul...
NonpagedSystemMemorySize    Property       int NonpagedSystemMemorySize {g...
NonpagedSystemMemorySize64  Property       long NonpagedSystemMemorySize64...
PagedMemorySize             Property       int PagedMemorySize {get;}
```

```
PagedMemorySize64           Property       long PagedMemorySize64 {get;}
PagedSystemMemorySize       Property       int PagedSystemMemorySize {get;}
PagedSystemMemorySize64     Property       long PagedSystemMemorySize64 {g...
PeakPagedMemorySize         Property       int PeakPagedMemorySize {get;}
PeakPagedMemorySize64       Property       long PeakPagedMemorySize64 {get;}
PeakVirtualMemorySize       Property       int PeakVirtualMemorySize {get;}
PeakVirtualMemorySize64     Property       long PeakVirtualMemorySize64 {g...
PeakWorkingSet              Property       int PeakWorkingSet {get;}
PeakWorkingSet64            Property       long PeakWorkingSet64 {get;}
PriorityBoostEnabled        Property       bool PriorityBoostEnabled {get;...
PriorityClass               Property       System.Diagnostics.ProcessPrior...
PrivateMemorySize           Property       int PrivateMemorySize {get;}
PrivateMemorySize64         Property       long PrivateMemorySize64 {get;}
PrivilegedProcessorTime     Property       timespan PrivilegedProcessorTim...
ProcessName                 Property       string ProcessName {get;}
ProcessorAffinity           Property       System.IntPtr ProcessorAffinity...
Responding                  Property       bool Responding {get;}
SafeHandle                  Property       Microsoft.Win32.SafeHandles.Saf...
SessionId                   Property       int SessionId {get;}
Site                        Property       System.ComponentModel.ISite Sit...
StandardError               Property       System.IO.StreamReader Standard...
StandardInput               Property       System.IO.StreamWriter Standard...
StandardOutput              Property       System.IO.StreamReader Standard...
StartInfo                   Property       System.Diagnostics.ProcessStart...
StartTime                   Property       datetime StartTime {get;}
SynchronizingObject         Property       System.ComponentModel.ISynchron...
Threads                     Property       System.Diagnostics.ProcessThrea...
TotalProcessorTime          Property       timespan TotalProcessorTime {get;}
UserProcessorTime           Property       timespan UserProcessorTime {get;}
VirtualMemorySize           Property       int VirtualMemorySize {get;}
VirtualMemorySize64         Property       long VirtualMemorySize64 {get;}
WorkingSet                  Property       int WorkingSet {get;}
WorkingSet64                Property       long WorkingSet64 {get;}
PSConfiguration             PropertySet    PSConfiguration {Name, Id, Prio...
PSResources                 PropertySet    PSResources {Name, Id, Handleco...
Company                     ScriptProperty System.Object Company {get=$thi...
CPU                         ScriptProperty System.Object CPU {get=$this.To...
Description                 ScriptProperty System.Object Description {get=...
FileVersion                 ScriptProperty System.Object FileVersion {get=...
Path                        ScriptProperty System.Object Path {get=$this.M...
Product                     ScriptProperty System.Object Product {get=$thi...
ProductVersion              ScriptProperty System.Object ProductVersion {g...
```

You can't pipe a command to `Get-Member` that doesn't generate output. Because `Start-Service` doesn't produce output by default, attempting to pipe it to `Get-Member` results in an error.

PowerShell

```powershell
Start-Service -Name w32time | Get-Member
```

> **Note**

To be piped to `Get-Member`, a command must produce object-based output.

Output

```
Get-Member : You must specify an object for the Get-Member cmdlet.
At line:1 char:31
+ Start-Service -Name w32time | Get-Member
+                               ~~~~~~~~~~
    + CategoryInfo          : CloseError: (:) [Get-Member], InvalidOperation
   Exception
    + FullyQualifiedErrorId : NoObjectInGetMember,Microsoft.PowerShell.Comma
   nds.GetMemberCommand
```

To avoid this error, specify the **PassThru** parameter with `Start-Service`. As previously mentioned, adding the **PassThru** parameter causes a cmdlet that doesn't usually produce output to generate output.

PowerShell

```powershell
Start-Service -Name w32time -PassThru | Get-Member
```

Output

```
   TypeName: System.ServiceProcess.ServiceController

Name                      MemberType    Definition
----                      ----------    ----------
Name                      AliasProperty Name = ServiceName
RequiredServices          AliasProperty RequiredServices = ServicesDepend...
Disposed                  Event         System.EventHandler Disposed(Syst...
Close                     Method        void Close()
Continue                  Method        void Continue()
CreateObjRef              Method        System.Runtime.Remoting.ObjRef Cr...
Dispose                   Method        void Dispose(), void IDisposable....
Equals                    Method        bool Equals(System.Object obj)
ExecuteCommand            Method        void ExecuteCommand(int command)
GetHashCode               Method        int GetHashCode()
GetLifetimeService        Method        System.Object GetLifetimeService()
GetType                   Method        type GetType()
InitializeLifetimeService Method        System.Object InitializeLifetimeS...
Pause                     Method        void Pause()
Refresh                   Method        void Refresh()
Start                     Method        void Start(), void Start(string[]...
Stop                      Method        void Stop()
WaitForStatus             Method        void WaitForStatus(System.Service...
CanPauseAndContinue       Property      bool CanPauseAndContinue {get;}
CanShutdown               Property      bool CanShutdown {get;}
CanStop                   Property      bool CanStop {get;}
Container                 Property      System.ComponentModel.IContainer ...
```

```
DependentServices         Property      System.ServiceProcess.ServiceCont...
DisplayName               Property      string DisplayName {get;set;}
MachineName               Property      string MachineName {get;set;}
ServiceHandle             Property      System.Runtime.InteropServices.Sa...
ServiceName               Property      string ServiceName {get;set;}
ServicesDependedOn        Property      System.ServiceProcess.ServiceCont...
ServiceType               Property      System.ServiceProcess.ServiceType...
Site                      Property      System.ComponentModel.ISite Site ...
StartType                 Property      System.ServiceProcess.ServiceStar...
Status                    Property      System.ServiceProcess.ServiceCont...
ToString                  ScriptMethod  System.Object ToString();
```

`Out-Host` is designed to show output directly in the PowerShell host and doesn't produce object-based output. As a result, you can't pipe its output to `Get-Member`, which requires object-based input.

PowerShell

```powershell
Get-Service -Name w32time | Out-Host | Get-Member
```

Output

```
Status   Name               DisplayName
------   ----               -----------
Running  w32time            Windows Time

Get-Member : You must specify an object for the Get-Member cmdlet.
At line:1 char:40
+ Get-Service -Name w32time | Out-Host | Get-Member
+                                        ~~~~~~~~~~
    + CategoryInfo          : CloseError: (:) [Get-Member], InvalidOperation
   Exception
    + FullyQualifiedErrorId : NoObjectInGetMember,Microsoft.PowerShell.Comma
   nds.GetMemberCommand
```

Get-Command

Knowing the type of object a command produces allows you to search for commands that accept that type of object as input.

PowerShell

```powershell
Get-Command -ParameterType ServiceController
```

The following commands accept a **ServiceController** object via pipeline or parameter input.

Output

```
CommandType     Name                                               Version
-----------     ----                                               -------
Cmdlet          Get-Service                                        3.1.0.0
Cmdlet          Restart-Service                                    3.1.0.0
Cmdlet          Resume-Service                                     3.1.0.0
Cmdlet          Set-Service                                        3.1.0.0
Cmdlet          Start-Service                                      3.1.0.0
Cmdlet          Stop-Service                                       3.1.0.0
Cmdlet          Suspend-Service                                    3.1.0.0
```

Active Directory

> **Note**
>
> As mentioned in the chapter prerequisites, ensure you have RSAT installed for this section. Additionally, your lab environment computer must be a member of your lab environment Active Directory domain.

To identify the commands added to the **ActiveDirectory** PowerShell module after you install RSAT, use `Get-Command` combined with the **Module** parameter. The following example lists all the commands available in the **ActiveDirectory** module.

PowerShell

```powershell
Get-Command -Module ActiveDirectory
```

The **ActiveDirectory** PowerShell module added a total of 147 commands.

Have you observed the naming convention of these commands? The nouns in the command names are prefixed with *AD* to avoid potential naming conflicts with commands in other modules. This prefixing is a common practice among PowerShell modules.

Output

```
CommandType     Name                                               Version
-----------     ----                                               -------
Cmdlet          Add-ADCentralAccessPolicyMember                    1.0.1.0
Cmdlet          Add-ADComputerServiceAccount                       1.0.1.0
Cmdlet          Add-ADDomainControllerPasswordReplicationPolicy    1.0.1.0
Cmdlet          Add-ADFineGrainedPasswordPolicySubject             1.0.1.0
Cmdlet          Add-ADGroupMember                                  1.0.1.0
Cmdlet          Add-ADPrincipalGroupMembership                     1.0.1.0
Cmdlet          Add-ADResourcePropertyListMember                   1.0.1.0
```

```
Cmdlet          Clear-ADAccountExpiration                          1.0.1.0
Cmdlet          Clear-ADClaimTransformLink                         1.0.1.0
Cmdlet          Disable-ADAccount                                  1.0.1.0
...
```

By default, the `Get-ADUser` cmdlet retrieves a limited set of properties for user objects and limits its output to the first 1,000 users. This constraint is a performance optimization designed to avoid overwhelming Active Directory with excessive data retrieval.

PowerShell

```powershell
Get-ADUser -Identity mike | Get-Member -MemberType Properties
```

Even if you only have a basic understanding of Active Directory, you might recognize that a user account has more properties than those shown in the example.

Output

```
   TypeName: Microsoft.ActiveDirectory.Management.ADUser

Name              MemberType Definition
----              ---------- ----------
DistinguishedName Property   System.String DistinguishedName {get;set;}
Enabled           Property   System.Boolean Enabled {get;set;}
GivenName         Property   System.String GivenName {get;set;}
Name              Property   System.String Name {get;}
ObjectClass       Property   System.String ObjectClass {get;set;}
ObjectGUID        Property   System.Nullable`1[[System.Guid, mscorlib, Ve...
SamAccountName    Property   System.String SamAccountName {get;set;}
SID               Property   System.Security.Principal.SecurityIdentifier...
Surname           Property   System.String Surname {get;set;}
UserPrincipalName Property   System.String UserPrincipalName {get;set;}
```

The `Get-ADUser` cmdlet includes a **Properties** parameter to specify additional properties beyond the defaults you want to retrieve. To return all properties, use the `*` wildcard character as the parameter value.

PowerShell

```powershell
Get-ADUser -Identity mike -Properties * | Get-Member -MemberType Properties
```

Output

```
   TypeName: Microsoft.ActiveDirectory.Management.ADUser

Name                                MemberType Definition
```

```
----                                ---------- ----------
AccountExpirationDate               Property   System.DateTime AccountEx...
accountExpires                      Property   System.Int64 accountExpir...
AccountLockoutTime                  Property   System.DateTime AccountLo...
AccountNotDelegated                 Property   System.Boolean AccountNot...
AllowReversiblePasswordEncryption   Property   System.Boolean AllowRever...
AuthenticationPolicy                Property   Microsoft.ActiveDirectory...
AuthenticationPolicySilo            Property   Microsoft.ActiveDirectory...
BadLogonCount                       Property   System.Int32 BadLogonCoun...
badPasswordTime                     Property   System.Int64 badPasswordT...
badPwdCount                         Property   System.Int32 badPwdCount ...
CannotChangePassword                Property   System.Boolean CannotChan...
CanonicalName                       Property   System.String CanonicalNa...
Certificates                        Property   Microsoft.ActiveDirectory...
City                                Property   System.String City {get;s...
CN                                  Property   System.String CN {get;}
codePage                            Property   System.Int32 codePage {ge...
Company                             Property   System.String Company {ge...
CompoundIdentitySupported           Property   Microsoft.ActiveDirectory...
Country                             Property   System.String Country {ge...
countryCode                         Property   System.Int32 countryCode ...
Created                             Property   System.DateTime Created {...
createTimeStamp                     Property   System.DateTime createTim...
Deleted                             Property   System.Boolean Deleted {g...
Department                          Property   System.String Department ...
Description                         Property   System.String Description...
DisplayName                         Property   System.String DisplayName...
DistinguishedName                   Property   System.String Distinguish...
Division                            Property   System.String Division {g...
DoesNotRequirePreAuth               Property   System.Boolean DoesNotReq...
dSCorePropagationData               Property   Microsoft.ActiveDirectory...
EmailAddress                        Property   System.String EmailAddres...
EmployeeID                          Property   System.String EmployeeID ...
EmployeeNumber                      Property   System.String EmployeeNum...
Enabled                             Property   System.Boolean Enabled {g...
Fax                                 Property   System.String Fax {get;set;}
GivenName                           Property   System.String GivenName {...
HomeDirectory                       Property   System.String HomeDirecto...
HomedirRequired                     Property   System.Boolean HomedirReq...
HomeDrive                           Property   System.String HomeDrive {...
HomePage                            Property   System.String HomePage {g...
HomePhone                           Property   System.String HomePhone {...
Initials                            Property   System.String Initials {g...
instanceType                        Property   System.Int32 instanceType...
isDeleted                           Property   System.Boolean isDeleted ...
KerberosEncryptionType              Property   Microsoft.ActiveDirectory...
LastBadPasswordAttempt              Property   System.DateTime LastBadPa...
LastKnownParent                     Property   System.String LastKnownPa...
lastLogoff                          Property   System.Int64 lastLogoff {...
lastLogon                           Property   System.Int64 lastLogon {g...
LastLogonDate                       Property   System.DateTime LastLogon...
lastLogonTimestamp                  Property   System.Int64 lastLogonTim...
LockedOut                           Property   System.Boolean LockedOut ...
logonCount                          Property   System.Int32 logonCount {...
LogonWorkstations                   Property   System.String LogonWorkst...
```

```
Manager                                 Property   System.String Manager {ge...
MemberOf                                Property   Microsoft.ActiveDirectory...
MNSLogonAccount                         Property   System.Boolean MNSLogonAc...
MobilePhone                             Property   System.String MobilePhone...
Modified                                Property   System.DateTime Modified ...
modifyTimeStamp                         Property   System.DateTime modifyTim...
msDS-User-Account-Control-Computed      Property   System.Int32 msDS-User-Ac...
Name                                    Property   System.String Name {get;}
nTSecurityDescriptor                    Property   System.DirectoryServices....
ObjectCategory                          Property   System.String ObjectCateg...
ObjectClass                             Property   System.String ObjectClass...
ObjectGUID                              Property   System.Nullable`1[[System...
objectSid                               Property   System.Security.Principal...
Office                                  Property   System.String Office {get...
OfficePhone                             Property   System.String OfficePhone...
Organization                            Property   System.String Organizatio...
OtherName                               Property   System.String OtherName {...
PasswordExpired                         Property   System.Boolean PasswordEx...
PasswordLastSet                         Property   System.DateTime PasswordL...
PasswordNeverExpires                    Property   System.Boolean PasswordNe...
PasswordNotRequired                     Property   System.Boolean PasswordNo...
POBox                                   Property   System.String POBox {get;...
PostalCode                              Property   System.String PostalCode ...
PrimaryGroup                            Property   System.String PrimaryGrou...
primaryGroupID                          Property   System.Int32 primaryGroup...
PrincipalsAllowedToDelegateToAccount    Property   Microsoft.ActiveDirectory...
ProfilePath                             Property   System.String ProfilePath...
ProtectedFromAccidentalDeletion         Property   System.Boolean ProtectedF...
pwdLastSet                              Property   System.Int64 pwdLastSet {...
SamAccountName                          Property   System.String SamAccountN...
sAMAccountType                          Property   System.Int32 sAMAccountTy...
ScriptPath                              Property   System.String ScriptPath ...
sDRightsEffective                       Property   System.Int32 sDRightsEffe...
ServicePrincipalNames                   Property   Microsoft.ActiveDirectory...
SID                                     Property   System.Security.Principal...
SIDHistory                              Property   Microsoft.ActiveDirectory...
SmartcardLogonRequired                  Property   System.Boolean SmartcardL...
sn                                      Property   System.String sn {get;set;}
State                                   Property   System.String State {get;...
StreetAddress                           Property   System.String StreetAddre...
Surname                                 Property   System.String Surname {ge...
Title                                   Property   System.String Title {get;...
TrustedForDelegation                    Property   System.Boolean TrustedFor...
TrustedToAuthForDelegation              Property   System.Boolean TrustedToA...
UseDESKeyOnly                           Property   System.Boolean UseDESKeyO...
userAccountControl                      Property   System.Int32 userAccountC...
userCertificate                         Property   Microsoft.ActiveDirectory...
UserPrincipalName                       Property   System.String UserPrincip...
uSNChanged                              Property   System.Int64 uSNChanged {...
uSNCreated                              Property   System.Int64 uSNCreated {...
whenChanged                             Property   System.DateTime whenChang...
whenCreated                             Property   System.DateTime whenCreat...
```

The default configuration for retrieving Active Directory user account properties is intentionally limited to avoid performance issues. Trying to return every property for every user account in your production Active Directory environment could severely degrade the performance of your domain controllers and network. Usually, you only need specific properties for certain users. However, returning all properties for a single user is reasonable when identifying the available properties.

It's not uncommon to run a command multiple times when prototyping it. If you anticipate running a resource-intensive query when prototyping a command, consider executing it once and storing the results in a variable. Then, you can work with the variable's contents more efficiently than repeatedly executing a resource-intensive query.

For example, the following command retrieves all properties for a user account and stores the results in a variable named `$Users`. Work with the contents of the `$Users` variable instead of running the `Get-ADUser` command multiple times. Remember, the variable's contents don't update automatically when a user's information changes in Active Directory.

PowerShell

```
$Users = Get-ADUser -Identity mike -Properties *
```

You can explore the available properties by piping the `$Users` variable to `Get-Member`.

PowerShell

```
$Users | Get-Member -MemberType Properties
```

To view specific properties such as **Name**, **LastLogonDate**, and **LastBadPasswordAttempt**, pipe the `$Users` variable to `Select-Object`. This method displays the desired properties and their values based on the contents of the `$Users` variable, eliminating the need for multiple queries to Active Directory. It's a more resource-efficient approach than repeatedly executing the `Get-ADUser` command.

PowerShell

```
$Users | Select-Object -Property Name, LastLogonDate, LastBadPasswordAttempt
```

When you query Active Directory, filter the data at the source using the **Properties** parameter of `Get-ADUser` to return only the necessary properties.

PowerShell

```
Get-ADUser -Identity mike -Properties LastLogonDate, LastBadPasswordAttempt
```

Output

```
DistinguishedName      : CN=Mike F. Robbins,CN=Users,DC=mikefrobbins,DC=com
Enabled                : True
GivenName              : Mike
LastBadPasswordAttempt :
LastLogonDate          : 11/14/2023 5:10:16 AM
Name                   : Mike F. Robbins
ObjectClass            : user
ObjectGUID             : 11c7b61f-46c3-4399-9ed0-ff4e453bc2a2
SamAccountName         : mike
SID                    : S-1-5-21-611971124-518002951-3581791498-1105
Surname                : Robbins
UserPrincipalName      : µ@mikefrobbins.com
```

Summary

In this chapter, you learned how to determine what type of object a command produces, what properties and methods are available for a command, and how to work with commands that limit the properties returned by default.

Review

1. What type of object does the `Get-Process` cmdlet produce?
2. How do you determine what the available properties are for a command?
3. What should you check for if a command exists to get something but not to set the same thing?
4. How can some commands that don't return output by default be made to generate output?
5. What should you consider doing when prototyping a command that produces a large amount of output?

References

- Get-Member
- Viewing Object Structure (Get-Member)
- about_Objects
- about_Properties
- about_Methods

- No PowerShell Cmdlet to Start or Stop Something? Don't Forget to Check for Methods on the Get Cmdlets

Next steps

In Chapter 4, you'll learn about one-liners and the pipeline.

Last updated on 02/06/2026

Chapter 4 - One-Liners and the pipeline

When I started learning PowerShell, I initially relied on the Graphical User Interface (GUI) for tasks that seemed too complex for simple PowerShell commands. However, as I continued to learn, I improved my skills and moved from basic one-liners to creating scripts, functions, and modules. It's important to remember that feeling overwhelmed by advanced examples online is normal. No one starts as an expert in PowerShell; we all start as beginners.

For those who primarily use the GUI for administrative tasks, install the management tools on your administrative workstation to remotely manage your servers. Whether your server uses a GUI or the Server Core OS installation, this approach is beneficial. It's a practical way to familiarize yourself with remote server management in preparation for performing administrative tasks with PowerShell.

As with the previous chapters, try these concepts in your lab environment.

One-Liners

A PowerShell one-liner is one continuous pipeline. It's a common misconception that a command on one physical line is a PowerShell one-liner, but this isn't always true.

For instance, consider the following example: the command extends over multiple physical lines, yet it's a PowerShell one-liner because it forms a continuous pipeline. Line-breaking a lengthy one-liner at the pipe symbol, a natural breaking point in PowerShell, is recommended to enhance readability and clarity. This strategic use of line breaks improves readability without disrupting the flow of the pipeline.

PowerShell

```powershell
Get-Service |
    Where-Object CanPauseAndContinue -EQ $true |
    Select-Object -Property *
```

Output

```
Name                : LanmanWorkstation
RequiredServices    : {NSI, MRxSmb20, Bowser}
CanPauseAndContinue : True
CanShutdown         : False
CanStop             : True
DisplayName         : Workstation
```

```
DependentServices   : {SessionEnv, Netlogon}
MachineName         : .
ServiceName         : LanmanWorkstation
ServicesDependedOn  : {NSI, MRxSmb20, Bowser}
ServiceHandle       :
Status              : Running
ServiceType         : Win32OwnProcess, Win32ShareProcess
StartType           : Automatic
Site                :
Container           :

Name                : Netlogon
RequiredServices    : {LanmanWorkstation}
CanPauseAndContinue : True
CanShutdown         : False
CanStop             : True
DisplayName         : Netlogon
DependentServices   : {}
MachineName         : .
ServiceName         : Netlogon
ServicesDependedOn  : {LanmanWorkstation}
ServiceHandle       :
Status              : Running
ServiceType         : Win32ShareProcess
StartType           : Automatic
Site                :
Container           :

Name                : vmicheartbeat
RequiredServices    : {}
CanPauseAndContinue : True
CanShutdown         : False
CanStop             : True
DisplayName         : Hyper-V Heartbeat Service
DependentServices   : {}
MachineName         : .
ServiceName         : vmicheartbeat
ServicesDependedOn  : {}
ServiceHandle       :
Status              : Running
ServiceType         : Win32OwnProcess, Win32ShareProcess
StartType           : Manual
Site                :
Container           :

Name                : vmickvpexchange
RequiredServices    : {}
CanPauseAndContinue : True
CanShutdown         : False
CanStop             : True
DisplayName         : Hyper-V Data Exchange Service
DependentServices   : {}
MachineName         : .
ServiceName         : vmickvpexchange
ServicesDependedOn  : {}
```

```
ServiceHandle       :
Status              : Running
ServiceType         : Win32OwnProcess, Win32ShareProcess
StartType           : Manual
Site                :
Container           :

Name                : vmicrdv
RequiredServices    : {}
CanPauseAndContinue : True
CanShutdown         : False
CanStop             : True
DisplayName         : Hyper-V Remote Desktop Virtualization Service
DependentServices   : {}
MachineName         : .
ServiceName         : vmicrdv
ServicesDependedOn  : {}
ServiceHandle       :
Status              : Running
ServiceType         : Win32OwnProcess, Win32ShareProcess
StartType           : Manual
Site                :
Container           :

Name                : vmicshutdown
RequiredServices    : {}
CanPauseAndContinue : True
CanShutdown         : False
CanStop             : True
DisplayName         : Hyper-V Guest Shutdown Service
DependentServices   : {}
MachineName         : .
ServiceName         : vmicshutdown
ServicesDependedOn  : {}
ServiceHandle       :
Status              : Running
ServiceType         : Win32OwnProcess, Win32ShareProcess
StartType           : Manual
Site                :
Container           :

Name                : vmicvss
RequiredServices    : {}
CanPauseAndContinue : True
CanShutdown         : False
CanStop             : True
DisplayName         : Hyper-V Volume Shadow Copy Requestor
DependentServices   : {}
MachineName         : .
ServiceName         : vmicvss
ServicesDependedOn  : {}
ServiceHandle       :
Status              : Running
ServiceType         : Win32OwnProcess, Win32ShareProcess
StartType           : Manual
```

```
Site                :
Container           :

Name                : webthreatdefsvc
RequiredServices    : {RpcSs, wtd}
CanPauseAndContinue : True
CanShutdown         : True
CanStop             : True
DisplayName         : Web Threat Defense Service
DependentServices   : {}
MachineName         : .
ServiceName         : webthreatdefsvc
ServicesDependedOn  : {RpcSs, wtd}
ServiceHandle       :
Status              : Running
ServiceType         : Win32OwnProcess, Win32ShareProcess
StartType           : Manual
Site                :
Container           :

Name                : webthreatdefusersvc_644de
RequiredServices    : {}
CanPauseAndContinue : True
CanShutdown         : True
CanStop             : True
DisplayName         : Web Threat Defense User Service_644de
DependentServices   : {}
MachineName         : .
ServiceName         : webthreatdefusersvc_644de
ServicesDependedOn  : {}
ServiceHandle       :
Status              : Running
ServiceType         : 240
StartType           : Automatic
Site                :
Container           :

Name                : Winmgmt
RequiredServices    : {RPCSS}
CanPauseAndContinue : True
CanShutdown         : True
CanStop             : True
DisplayName         : Windows Management Instrumentation
DependentServices   : {}
MachineName         : .
ServiceName         : Winmgmt
ServicesDependedOn  : {RPCSS}
ServiceHandle       :
Status              : Running
ServiceType         : Win32OwnProcess, Win32ShareProcess
StartType           : Automatic
Site                :
Container           :
```

Natural line breaks can occur at commonly used characters, including comma (`,`) and opening brackets (`[`), braces (`{`), and parenthesis (`(`). Others that aren't so common include the semicolon (`;`), equals sign (`=`), and both opening single and double quotes (`'`,`"`).

Using the backtick (`` ` ``) or grave accent character as a line continuation is controversial. It's best to avoid it if possible. Using a backtick following a natural line break character is a common mistake. This redundancy is unnecessary and can clutter the code.

The commands in the following example execute correctly from the PowerShell console. However, attempting to run them in the console pane of the PowerShell Integrated Scripting Environment (ISE) results in an error. This difference occurs because, unlike the PowerShell console, the console pane of the ISE doesn't automatically anticipate the continuation of a command onto the next line. To prevent this issue, press `Shift` + `Enter` in the console pane of the ISE instead of `Enter` when you need to extend a command across multiple lines. This key combination signals to the ISE that the command is continuing on the following line, preventing the execution that leads to errors.

PowerShell

```powershell
Get-Service -Name w32time |
    Select-Object -Property *
```

Output

```
Name                : w32time
RequiredServices    : {}
CanPauseAndContinue : False
CanShutdown         : True
CanStop             : True
DisplayName         : Windows Time
DependentServices   : {}
MachineName         : .
ServiceName         : w32time
ServicesDependedOn  : {}
ServiceHandle       :
Status              : Running
ServiceType         : Win32OwnProcess, Win32ShareProcess
StartType           : Manual
Site                :
Container           :
```

This next example doesn't qualify as a PowerShell one-liner because it's not one continuous pipeline. Instead, it's two separate commands placed on a single line, separated by a semicolon. This semicolon indicates the end of one command and the beginning of another.

PowerShell

```powershell
$Service = 'w32time'; Get-Service -Name $Service
```

Output

```
Status   Name               DisplayName
------   ----               -----------
Running  w32time            Windows Time
```

Many programming and scripting languages require a semicolon at the end of each line. However, in PowerShell, semicolons at the end of lines are unnecessary and not recommended. You should avoid them for cleaner and more readable code.

Filter Left

This chapter demonstrates how to filter the results of various commands.

It's a best practice in PowerShell to filter the results as early as possible in the pipeline. Achieving this involves applying filters using parameters on the initial command, usually at the beginning of the pipeline. This is commonly referred to as *filtering left*.

To illustrate this concept, consider the following example: Use the **Name** parameter of `Get-Service` to filter the results at the beginning of the pipeline, returning only the details for the Windows Time service. This method demonstrates efficient data retrieval, ensuring you only return the necessary and relevant information.

PowerShell

```powershell
Get-Service -Name w32time
```

Output

```
Status   Name               DisplayName
------   ----               -----------
Running  w32time            Windows Time
```

It's common to see online examples of a PowerShell command being piped to the `Where-Object` cmdlet to filter its results. This technique is inefficient if an earlier command in the pipeline has a parameter to perform the filtering.

PowerShell

```powershell
Get-Service | Where-Object Name -EQ w32time
```

Output

```
Status   Name               DisplayName
------   ----               -----------
Running  W32Time            Windows Time
```

The first example demonstrates filtering directly at the source, returning results specifically for the Windows Time service. In contrast, the second example retrieves all services and then uses another command to filter the results. This might seem insignificant in small-scale scenarios, but consider a situation involving a large dataset, like Active Directory. It's inefficient to retrieve details for thousands of user accounts only to narrow them down to a small subset. Practice *filtering left* — applying filters as early as possible in the command sequence — even in seemingly trivial cases. This habit ensures efficiency in more complex scenarios where it becomes more important.

Command sequencing for effective filtering

There's a misconception that the order of commands in PowerShell is inconsequential, but this is a misunderstanding. The sequence in which you arrange commands, particularly when filtering, is important. For example, suppose you're using `Select-Object` to choose specific properties and `Where-Object` to filter. In that case, it's essential to apply the filtering first. Failing to do so means the necessary properties might not be available in the pipeline for filtering, leading to ineffective or erroneous results.

The following example fails to produce results because the **CanPauseAndContinue** property is absent when `Select-Object` is piped to `Where-Object`. This is because the **CanPauseAndContinue** property wasn't included in the selection made by `Select-Object`. Effectively, it's excluded or filtered out.

PowerShell

```powershell
Get-Service |
    Select-Object -Property DisplayName, Running, Status |
    Where-Object CanPauseAndContinue
```

Reversing the order of `Select-Object` and `Where-Object` produces the desired results.

PowerShell

```powershell
Get-Service |
    Where-Object CanPauseAndContinue |
    Select-Object -Property DisplayName, Status
```

Output

```
DisplayName                                    Status
-----------                                    ------
Workstation                                   Running
Netlogon                                      Running
Hyper-V Heartbeat Service                     Running
Hyper-V Data Exchange Service                 Running
Hyper-V Remote Desktop Virtualization Service Running
Hyper-V Guest Shutdown Service                Running
Hyper-V Volume Shadow Copy Requestor          Running
Web Threat Defense Service                    Running
Web Threat Defense User Service_644de         Running
Windows Management Instrumentation            Running
```

The Pipeline

As seen in many examples throughout this book, you can often use the output of one command as input for another command. In Chapter 3, `Get-Member` was used to determine what type of object a command produces.

Chapter 3 also showed using the **ParameterType** parameter of `Get-Command` to determine what commands accepted that type of input. Depending on how thorough help for a command is, it might include an **INPUTS** and **OUTPUTS** section.

The **INPUTS** section indicates that you can pipe a **ServiceController** or a **String** object to the `Stop-Service` cmdlet.

PowerShell

```powershell
help Stop-Service -Full
```

The following output is abbreviated to show the relevant portion of the help.

Output

```
...
INPUTS
    System.ServiceProcess.ServiceController
        You can pipe a service object to this cmdlet.


```

```
    System.String
        You can pipe a string that contains the name of a service to this
        cmdlet.


OUTPUTS
    None
        By default, this cmdlet returns no output.

    System.ServiceProcess.ServiceController
        When you use the PassThru parameter, this cmdlet returns a
        ServiceController object representing the service.
...
```

However, it doesn't specify which parameters accept this type of input. You can determine that information by checking the different parameters in the full version of the help for the `Stop-Service` cmdlet.

PowerShell

```
help Stop-Service -Full
```

Once again, only the relevant help is shown in the following results. Notice that the **DisplayName** parameter doesn't accept pipeline input. The **InputObject** parameter accepts pipeline input by value for **ServiceController** objects. The Name parameter accepts pipeline input by value for **String** objects and pipeline input **by property name**.

Output

```
...
-DisplayName <System.String[]>
    Specifies the display names of the services to stop. Wildcard
    characters are permitted.

    Required?                    true
    Position?                    named
    Default value                None
    Accept pipeline input?       False
    Accept wildcard characters?  true

-InputObject <System.ServiceProcess.ServiceController[]>
    Specifies ServiceController objects that represent the services to
    stop. Enter a variable that contains the objects, or type a command
    or expression that gets the objects.

    Required?                    true
    Position?                    0
    Default value                None
    Accept pipeline input?       True (ByValue)
```

```
    Accept wildcard characters?  false

-Name <System.String[]>
    Specifies the service names of the services to stop. Wildcard
    characters are permitted.

    Required?                    true
    Position?                    0
    Default value                None
    Accept pipeline input?       True (ByPropertyName, ByValue)
    Accept wildcard characters?  true
...
```

When handling pipeline input, a parameter that accepts pipeline input both **by property name** and **by value** prioritizes **by value** binding first. If this method fails, it attempts to process pipeline input **by property name**. However, the term **by value** can be misleading. A more accurate description is **by type**.

For instance, if you pipe the output of a command that generates a **ServiceController** object to `Stop-Service`, this output is bound to the **InputObject** parameter. If the piped command produces a **String** object, it associates the output with the **Name** parameter. If you pipe output from a command that doesn't produce a **ServiceController** or **String** object, but does include a property named **Name**, `Stop-Service` binds the value of the **Name** property to its **Name** parameter.

Determine what type of output the `Get-Service` command produces.

PowerShell

```powershell
Get-Service -Name w32time | Get-Member
```

`Get-Service` produces a **ServiceController** object type.

Output

```
   TypeName: System.ServiceProcess.ServiceController
```

As shown in the help for `Stop-Service` cmdlet, the **InputObject** parameter accepts **ServiceController** objects through the pipeline **by value**. This implies that when you pipe the output of the `Get-Service` cmdlet to `Stop-Service`, the **ServiceController** objects produced by `Get-Service` bind to the **InputObject** parameter of `Stop-Service`.

PowerShell

```
Get-Service -Name w32time | Stop-Service
```

Now try string input. Pipe `w32time` to `Get-Member` to confirm that it's a string.

PowerShell

```
'w32time' | Get-Member
```

Output

```
   TypeName: System.String
```

The PowerShell help documentation illustrates that when you pipe a string to `Stop-Service`, it binds to the **Name** parameter **by value**. Conduct a practical test to see this in action: pipe the string `w32time` to `Stop-Service`. This example demonstrates how `Stop-Service` processes the string `w32time` as the name of the service to stop. Execute the following command to observe this binding and command execution in action.

Notice that `w32time` is enclosed in single quotes. In PowerShell, it's a best practice to use single quotes for static strings, reserving double quotes for situations where the string contains variables that require expansion. Single quotes tell PowerShell to treat the content literally without parsing for variables. This approach not only ensures accuracy in how your script interprets the string but also enhances performance, as PowerShell expends less processing effort on strings within single quotes.

PowerShell

```
'w32time' | Stop-Service
```

Create a custom object to test pipeline input by property name for the **Name** parameter of `Stop-Service`.

PowerShell

```
$customObject = [pscustomobject]@{
    Name = 'w32time'
}
```

The contents of the **CustomObject** variable is a **PSCustomObject** object type and it contains a property named **Name**.

PowerShell

```
$customObject | Get-Member
```

Output

```
   TypeName: System.Management.Automation.PSCustomObject

Name        MemberType   Definition
----        ----------   ----------
Equals      Method       bool Equals(System.Object obj)
GetHashCode Method       int GetHashCode()
GetType     Method       type GetType()
ToString    Method       string ToString()
Name        NoteProperty string Name=w32time
```

When working with variables in PowerShell, such as `$customObject` in this example, it's important to use double quotes if you need to enclose the variable in quotes. Double quotes allow for variable expansion — PowerShell evaluates the variable and uses its value. For example, if you enclose `$customObject` in double quotes and pipe it to `Get-Member`, PowerShell processes the value of `$customObject`. In contrast, using single quotes would result in piping the literal string `$customObject` to `Get-Member`, not the value of the variable. This distinction is important for scenarios where you need to evaluate the value of variables.

When piping the contents of the `$customObject` variable to the `Stop-Service` cmdlet, the binding to the **Name** parameter occurs by **property name** rather than **by value**. This is because `$customObject` is an object that contains a property named **Name**. In this scenario, PowerShell identifies the **Name** property within `$customObject` and uses its value for the **Name** parameter of `Stop-Service`.

Create another custom object using a different property name, such as **Service**.

PowerShell

```
$customObject = [pscustomobject]@{
    Service = 'w32time'
}
```

An error occurs while trying to stop the `w32time` service by piping `$customObject` to `Stop-Service`. The pipeline binding fails because `$customObject` doesn't produce a **ServiceController** or **String** object and doesn't contain a **Name** property.

PowerShell

```
$customObject | Stop-Service
```

Output

```
Stop-Service : Cannot find any service with service name
'@{Service=w32time}'.
At line:1 char:17
+ $customObject | Stop-Service
+                 ~~~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (@{Service=w32time}:String) [
   Stop-Service], ServiceCommandException
    + FullyQualifiedErrorId : NoServiceFoundForGivenName,Microsoft.PowerShe
   ll.Commands.StopServiceCommand
```

When the output property names of one command don't match the pipeline input requirements of another command, you can use `Select-Object` to rename the property names so they line up correctly.

In the following example, use `Select-Object` to rename the **Service** property to a property named **Name**.

At first glance, the syntax of this example might appear complex. However, it's essential to understand that more than copying and pasting code is required to learn the syntax. Instead, take the time to type out the code manually. This hands-on practice helps you remember the syntax, and it becomes more intuitive with repeated effort. Utilizing multiple monitors or split screen can also aid in the learning process. Display the example code on one screen while actively typing and experimenting with it on another. This setup makes it easier to follow along and enhances your understanding and retention of the syntax.

PowerShell

```
$customObject |
    Select-Object -Property @{Name='Name';Expression={$_.Service}} |
    Stop-Service
```

There are instances where you might need to use a parameter that doesn't accept pipeline input. In such cases, you can still use the output of one command as the input for another. First, capture and save the display names of a few specific Windows services into a text file. This step allows you to use the saved data as input for another command.

PowerShell

```
'Background Intelligent Transfer Service', 'Windows Time' |
    Out-File -FilePath $env:TEMP\services.txt
```

You can use parentheses to pass the output of one command as input for a parameter to another command.

PowerShell

```
Stop-Service -DisplayName (Get-Content -Path $env:TEMP\services.txt)
```

This concept is like the order of operations in Algebra. Just as mathematical operations within parentheses are computed first, the command enclosed in parentheses is executed before the outer command.

PowerShellGet

PowerShellGet, a module included with PowerShell version 5.0 and higher, provides commands to discover, install, update, and publish PowerShell modules and other items in a NuGet repository. For those using PowerShell version 3.0 and above, PowerShellGet is also available as a separate download.

The PowerShell Gallery is an online repository hosted by Microsoft, designed as a central hub for sharing PowerShell modules, scripts, and other resources. While Microsoft hosts the PowerShell Gallery, the PowerShell community contributes most of the available modules and scripts. Given the source of these modules and scripts, exercise caution before integrating any code from the PowerShell Gallery into your environment. Review and test downloads from the PowerShell Gallery in an isolated test environment. This process ensures the code is secure and reliable, works as expected, and safeguards your environment from potential issues or vulnerabilities arising from unvetted code.

Many organizations opt to establish their own internal, private NuGet repository. This repository serves a dual purpose. First, it acts as a secure location for storing modules developed in-house, intended solely for internal use. Secondly, it provides a vetted collection of modules sourced externally, including those from public repositories. Companies typically undertake a thorough validation process before adding these external modules to the internal repository. This process is important to ensure the modules are free from malicious content and align with the security and operational standards of the company.

Use the `Find-Module` cmdlet that's part of the **PowerShellGet** module to find a module in the PowerShell Gallery that I wrote named **MrToolkit**.

PowerShell

```
Find-Module -Name MrToolkit
```

Output

```
NuGet provider is required to continue
PowerShellGet requires NuGet provider version '2.8.5.201' or newer to
interact with NuGet-based repositories. The NuGet provider must be available
 in 'C:\Program Files\PackageManagement\ProviderAssemblies' or
'C:\Users\mikefrobbins\AppData\Local\PackageManagement\ProviderAssemblies'.
You can also install the NuGet provider by running 'Install-PackageProvider
-Name NuGet -MinimumVersion 2.8.5.201 -Force'. Do you want PowerShellGet to
install and import the NuGet provider now?
[Y] Yes  [N] No  [S] Suspend  [?] Help (default is "Y"):

Version    Name                        Repository        Description
-------    ----                        ----------        -----------
1.3        MrToolkit                   PSGallery         Misc PowerShell Tools
```

The first time you use one of the commands from the **PowerShellGet** module, you're prompted to install the NuGet provider.

To install the **MrToolkit** module, pipe the previous command to `Install-Module`.

PowerShell

```
Find-Module -Name MrToolkit | Install-Module -Scope CurrentUser
```

Output

```
Untrusted repository
You are installing the modules from an untrusted repository. If you trust
this repository, change its InstallationPolicy value by running the
Set-PSRepository cmdlet. Are you sure you want to install the modules from
'https://www.powershellgallery.com/api/v2'?
[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help
(default is "N"):y
```

Since the PowerShell Gallery is an untrusted repository, it prompts you to approve the installation of the module.

Finding pipeline input the easy way

The **MrToolkit** module includes a function named `Get-MrPipelineInput`. This cmdlet is designed to provide users with a convenient method for identifying the parameters of a

command capable of accepting pipeline input. Specifically, it reveals three key aspects:

- Which parameters of a command can receive pipeline input
- The type of object each parameter accepts
- Whether they accept pipeline input **by value** or **by property name**

This capability dramatically simplifies the process of understanding and utilizing the pipeline capabilities of PowerShell commands.

The information previously obtained by analyzing the help documentation can be determined using this function.

PowerShell

```powershell
Get-MrPipelineInput -Name Stop-Service | Format-List
```

Output

```
ParameterName                   : InputObject
ParameterType                   : System.ServiceProcess.ServiceController[]
ValueFromPipeline               : True
ValueFromPipelineByPropertyName : False

ParameterName                   : Name
ParameterType                   : System.String[]
ValueFromPipeline               : True
ValueFromPipelineByPropertyName : True
```

Summary

In this chapter, you learned about the intricacies of PowerShell one-liners. You also learned that the physical line count of a command is irrelevant to its classification as a PowerShell one-liner. Additionally, you learned about key concepts such as filtering left, the pipeline, and **PowerShellGet**.

Review

1. What's a PowerShell one-liner?
2. What are some characters where natural line breaks can occur in PowerShell?
3. Why should you filter left?
4. What are the two ways that a PowerShell command can accept pipeline input?
5. Why shouldn't you trust commands found in the PowerShell Gallery?

References

- about_Pipelines
- about_Command_Syntax
- about_Parameters
- PowerShellGet: The BIG EASY way to discover, install, and update PowerShell modules

Next steps

In Chapter 5, you'll learn about formatting, aliases, providers, and comparison operators.

Last updated on 02/06/2026

Chapter 5 - Formatting, aliases, providers, comparison

Prerequisites

The **SqlServer** PowerShell module is required by some examples shown in this chapter. For more information about the **SqlServer** PowerShell module and installation instructions, see SQL Server PowerShell overview. It's also used in subsequent chapters. Download and install it on your Windows lab environment computer.

Format Right

In Chapter 4, you learned to filter as far to the left as possible. The rule for manually formatting a command's output is similar to that rule, except it needs to occur as far to the right as possible.

The most common format commands are `Format-Table` and `Format-List`. `Format-Wide` and `Format-Custom` can also be used, but are less common.

As mentioned in Chapter 3, a command that returns more than four properties defaults to a list unless custom formatting is used.

PowerShell

```
Get-Service -Name w32time |
    Select-Object -Property Status, DisplayName, Can*
```

Output

```
Status              : Running
DisplayName         : Windows Time
CanPauseAndContinue : False
CanShutdown         : True
CanStop             : True
```

Use the `Format-Table` cmdlet to manually override the formatting and show the output in a table instead of a list.

PowerShell

```powershell
Get-Service -Name w32time |
    Select-Object -Property Status, DisplayName, Can* |
    Format-Table
```

Output

```
Status  DisplayName  CanPauseAndContinue CanShutdown CanStop
------  -----------  ------------------- ----------- -------
Running Windows Time               False        True    True
```

The default output for `Get-Service` is three properties in a table.

PowerShell

```powershell
Get-Service -Name w32time
```

Output

```
Status   Name               DisplayName
------   ----               -----------
Running  w32time            Windows Time
```

Use the `Format-List` cmdlet to override the default formatting and return the results in a list.

PowerShell

```powershell
Get-Service -Name w32time | Format-List
```

Notice that simply piping `Get-Service` to `Format-List` made it return additional properties. This doesn't occur with every command because of how the format for that particular command is set up behind the scenes.

Output

```
Name                : w32time
DisplayName         : Windows Time
Status              : Running
DependentServices   : {}
ServicesDependedOn  : {}
CanPauseAndContinue : False
CanShutdown         : True
CanStop             : True
ServiceType         : Win32OwnProcess, Win32ShareProcess
```

The number one thing to be aware of with the format cmdlets is they produce format objects that are different than normal objects in PowerShell.

PowerShell

```powershell
Get-Service -Name w32time | Format-List | Get-Member
```

Output

```
   TypeName: Microsoft.PowerShell.Commands.Internal.Format.FormatStartData

Name                                   MemberType Definition
----                                   ---------- ----------
Equals                                 Method     bool Equals(System.Obj...
GetHashCode                            Method     int GetHashCode()
GetType                                Method     type GetType()
ToString                               Method     string ToString()
autosizeInfo                           Property   Microsoft.PowerShell.C...
ClassId2e4f51ef21dd47e99d3c952918aff9cd Property   string ClassId2e4f51ef...
groupingEntry                          Property   Microsoft.PowerShell.C...
pageFooterEntry                        Property   Microsoft.PowerShell.C...
pageHeaderEntry                        Property   Microsoft.PowerShell.C...
shapeInfo                              Property   Microsoft.PowerShell.C...


   TypeName: Microsoft.PowerShell.Commands.Internal.Format.GroupStartData

Name                                   MemberType Definition
----                                   ---------- ----------
Equals                                 Method     bool Equals(System.Obj...
GetHashCode                            Method     int GetHashCode()
GetType                                Method     type GetType()
ToString                               Method     string ToString()
ClassId2e4f51ef21dd47e99d3c952918aff9cd Property   string ClassId2e4f51ef...
groupingEntry                          Property   Microsoft.PowerShell.C...
shapeInfo                              Property   Microsoft.PowerShell.C...


   TypeName: Microsoft.PowerShell.Commands.Internal.Format.FormatEntryData

Name                                   MemberType Definition
----                                   ---------- ----------
Equals                                 Method     bool Equals(System.Obj...
GetHashCode                            Method     int GetHashCode()
GetType                                Method     type GetType()
ToString                               Method     string ToString()
ClassId2e4f51ef21dd47e99d3c952918aff9cd Property   string ClassId2e4f51ef...
formatEntryInfo                        Property   Microsoft.PowerShell.C...
outOfBand                              Property   bool outOfBand {get;set;}
writeStream                            Property   Microsoft.PowerShell.C...
```

```
   TypeName: Microsoft.PowerShell.Commands.Internal.Format.GroupEndData

Name                                    MemberType Definition
----                                    ---------- ----------
Equals                                  Method     bool Equals(System.Obj...
GetHashCode                             Method     int GetHashCode()
GetType                                 Method     type GetType()
ToString                                Method     string ToString()
ClassId2e4f51ef21dd47e99d3c952918aff9cd Property   string ClassId2e4f51ef...
groupingEntry                           Property   Microsoft.PowerShell.C...


   TypeName: Microsoft.PowerShell.Commands.Internal.Format.FormatEndData

Name                                    MemberType Definition
----                                    ---------- ----------
Equals                                  Method     bool Equals(System.Obj...
GetHashCode                             Method     int GetHashCode()
GetType                                 Method     type GetType()
ToString                                Method     string ToString()
ClassId2e4f51ef21dd47e99d3c952918aff9cd Property   string ClassId2e4f51ef...
groupingEntry                           Property   Microsoft.PowerShell.C...
```

What this means is format commands can't be piped to most other commands. They can be piped to some of the `Out-*` commands, but that's about it. This is why you want to perform any formatting at the very end of the line (format right).

Aliases

An alias in PowerShell is a shorter name for a command. PowerShell includes a set of built-in aliases and you can also define your own aliases.

The `Get-Alias` cmdlet is used to find aliases. If you already know the alias for a command, the **Name** parameter is used to determine what command the alias is associated with.

PowerShell

```powershell
Get-Alias -Name gcm
```

Output

```
CommandType     Name                                               Version
-----------     ----                                               -------
Alias           gcm -> Get-Command
```

Multiple aliases can be specified for the value of the **Name** parameter.

PowerShell

```
Get-Alias -Name gcm, gm
```

Output

```
CommandType     Name                                               Version
-----------     ----                                               -------
Alias           gcm -> Get-Command
Alias           gm -> Get-Member
```

You often see the **Name** parameter omitted since it's a positional parameter.

PowerShell

```
Get-Alias gm
```

Output

```
CommandType     Name                                               Version
-----------     ----                                               -------
Alias           gm -> Get-Member
```

If you want to find aliases for a command, you need to use the **Definition** parameter.

PowerShell

```
Get-Alias -Definition Get-Command, Get-Member
```

Output

```
CommandType     Name                                               Version
-----------     ----                                               -------
Alias           gcm -> Get-Command
Alias           gm -> Get-Member
```

The **Definition** parameter can't be used positionally, so it must be specified.

Aliases can save you a few keystrokes, and they're fine when you type commands into the console. They shouldn't be used in scripts or any code that you're saving or sharing with others. As mentioned earlier in this book, using full cmdlet and parameter names is self-documenting and easier to understand.

Use caution when creating your own aliases because they only exist in your current PowerShell session on your computer.

Providers

A provider in PowerShell is an interface that allows file system-like access to a data store. There are several built-in providers in PowerShell.

PowerShell

```
Get-PSProvider
```

As you can see in the following results, there are built-in providers for the registry, aliases, environment variables, the file system, functions, variables, certificates, and WSMan.

Output

```
Name                 Capabilities                 Drives
----                 ------------                 ------
Registry             ShouldProcess, Transactions  {HKLM, HKCU}
Alias                ShouldProcess                {Alias}
Environment          ShouldProcess                {Env}
FileSystem           Filter, ShouldProcess, Cr... {C, D}
Function             ShouldProcess                {Function}
Variable             ShouldProcess                {Variable}
```

The actual drives that these providers use to expose their data store can be determined with the `Get-PSDrive` cmdlet. The `Get-PSDrive` cmdlet not only displays drives exposed by providers but also displays Windows logical drives, including drives mapped to network shares.

PowerShell

```
Get-PSDrive
```

Output

```
Name           Used (GB)     Free (GB) Provider      Root
----           ---------     --------- --------      ----
Alias                                  Alias
C                  18.56        107.62 FileSystem    C:\
Cert                                   Certificate   \
D                                      FileSystem    D:\
Env                                    Environment
Function                               Function
HKCU                                   Registry      HKEY_CURRENT_USER
```

```
HKLM                                Registry      HKEY_LOCAL_MACHINE
Variable                            Variable
WSMan                               WSMan
```

Third-party modules such as the **ActiveDirectory** PowerShell module and the **SqlServer** PowerShell module both add their own PowerShell provider and PSDrive.

Import the **ActiveDirectory** and **SqlServer** PowerShell modules.

PowerShell

```
Import-Module -Name ActiveDirectory, SQLServer
```

Check to see if any additional PowerShell providers were added.

PowerShell

```
Get-PSProvider
```

Notice that in the following set of results, two new PowerShell providers now exist, one for Active Directory and another one for SQL Server.

Output

```
Name                Capabilities                       Drives
----                ------------                       ------
Registry            ShouldProcess, Transactions        {HKLM, HKCU}
Alias               ShouldProcess                      {Alias}
Environment         ShouldProcess                      {Env}
FileSystem          Filter, ShouldProcess, Credentials {C, A, D}
Function            ShouldProcess                      {Function}
Variable            ShouldProcess                      {Variable}
ActiveDirectory     Include, Exclude, Filter, Shoul... {AD}
SqlServer           Credentials                        {SQLSERVER}
```

A PSDrive for each of those modules was also added.

PowerShell

```
Get-PSDrive
```

Output

```
Name           Used (GB)     Free (GB) Provider      Root
----           ---------     --------- --------      ----
A                                      FileSystem    A:\
```

```
AD                                    ActiveDire... //RootDSE/
Alias                                 Alias
C                   19.38       107.13 FileSystem    C:\
Cert                                  Certificate    \
D                                     FileSystem     D:\
Env                                   Environment
Function                              Function
HKCU                                  Registry       HKEY_CURRENT_USER
HKLM                                  Registry       HKEY_LOCAL_MACHINE
SQLSERVER                             SqlServer      SQLSERVER:\
Variable                              Variable
WSMan                                 WSMan
```

PSDrives can be accessed just like a traditional file system.

PowerShell

```powershell
Get-ChildItem -Path Cert:\LocalMachine\CA
```

Output

```
   PSParentPath: Microsoft.PowerShell.Security\Certificate::LocalMachine\CA

Thumbprint                                Subject
----------                                -------
FEE449EE0E3965A5246F000E87FDE2A065FD89D4  CN=Root Agency
D559A586669B08F46A30A133F8A9ED3D038E2EA8  OU=www.verisign.com/CPS Incorp....
109F1CAED645BB78B3EA2B94C0697C740733031C  CN=Microsoft Windows Hardware C...
```

Comparison Operators

PowerShell contains various comparison operators that are used to compare values or find values that match certain patterns. The following table contains a list of comparison operators in PowerShell.

All the operators listed in the table are case-insensitive. To make them case-sensitive, place a `c` in front of the operator. For example, `-ceq` is the case-sensitive version of the equals (`-eq`) comparison operator.

Expand table

Operator	Definition
`-eq`	Equal to
`-ne`	Not equal to

Operator	Definition
`-gt`	Greater than
`-ge`	Greater than or equal to
`-lt`	Less than
`-le`	Less than or equal to
`-like`	Match using the `*` wildcard character
`-notlike`	Doesn't match using the `*` wildcard character
`-match`	Matches the specified regular expression
`-notmatch`	Doesn't match the specified regular expression
`-contains`	Determines if a collection contains a specified value
`-notcontains`	Determines if a collection doesn't contain a specific value
`-in`	Determines if a specified value is in a collection
`-notin`	Determines if a specified value isn't in a collection
`-replace`	Replaces the specified value

Proper case "PowerShell" is equal to lower case "powershell" using the equals comparison operator.

PowerShell

```
'PowerShell' -eq 'powershell'
```

Output

```
True
```

It's not equal using the case-sensitive version of the equals comparison operator.

PowerShell

```
'PowerShell' -ceq 'powershell'
```

Output

```
False
```

The not equal comparison operator reverses the condition.

PowerShell

```
'PowerShell' -ne 'powershell'
```

Output

```
False
```

Greater than, greater than or equal to, less than, and less than or equal all work with string or numeric values.

PowerShell

```
5 -gt 5
```

Output

```
False
```

Using greater than or equal to instead of greater than with the previous example returns the **Boolean** true since five is equal to five.

PowerShell

```
5 -ge 5
```

Output

```
True
```

Based on the results from the previous two examples, you can probably guess how both less than and less than or equal to work.

PowerShell

```
5 -lt 10
```

Output

```
True
```

The `-like` and `-match` operators can be confusing, even for experienced PowerShell users. `-like` is used with the wildcard characters `*` and `?` to perform "like" matches.

PowerShell

```powershell
'PowerShell' -like '*shell'
```

Output

```
True
```

The `-match` operator uses a regular expression to perform the matching.

PowerShell

```powershell
'PowerShell' -match '^.*shell$'
```

Output

```
True
```

Use the range operator to store the numbers 1 through 10 in a variable.

PowerShell

```powershell
$Numbers = 1..10
```

Determine if the `$Numbers` variable includes 15.

PowerShell

```powershell
$Numbers -contains 15
```

Output

```
False
```

Determine if it includes the number 10.

PowerShell

```
$Numbers -contains 10
```

Output

```
True
```

The `-notcontains` operator reverses the logic to see if the `$Numbers` variable doesn't contain a value.

PowerShell

```
$Numbers -notcontains 15
```

The previous example returns the **Boolean** true because it's true that the `$Numbers` variable doesn't contain 15.

Output

```
True
```

It does, however, contain the number 10, so it's false when tested.

PowerShell

```
$Numbers -notcontains 10
```

Output

```
False
```

The `-in` comparison operator was first introduced in PowerShell version 3.0. It's used to determine if a value is *in* an array. The `$Numbers` variable is an array since it contains multiple values.

PowerShell

```
15 -in $Numbers
```

Output

```
False
```

In other words, `-in` performs the same test as the contains comparison operator except from the opposite direction.

PowerShell

```
10 -in $Numbers
```

Output

```
True
```

Fifteen isn't in the `$Numbers` array, so false is returned in the following example.

PowerShell

```
15 -in $Numbers
```

Output

```
False
```

Just like the `-contains` operator, `not` reverses the logic for the `-in` operator.

PowerShell

```
10 -notin $Numbers
```

The previous example returns false because the `$Numbers` array does include 10 and the condition tests to determine if it doesn't contain 10.

Output

```
False
```

Determine if fifteen isn't in the `$Numbers` array.

PowerShell

```
15 -notin $Numbers
```

15 is "not in" the `$Numbers` array so it returns the **Boolean** true.

Output

```
True
```

The `-replace` operator does just what you would think. It's used to replace something. Specifying one value replaces that value with nothing. In the following example, you replace "Shell" with nothing.

PowerShell

```
'PowerShell' -replace 'Shell'
```

Output

```
Power
```

If you want to replace a value with a different one, specify the new one after the pattern you want to replace. SQL Saturday in Baton Rouge is an event I try to speak at every year. In the following example, the word "Saturday" is replaced with the abbreviation "Sat".

PowerShell

```
'SQL Saturday - Baton Rouge' -replace 'saturday','Sat'
```

Output

```
SQL Sat - Baton Rouge
```

There are also methods like **Replace()** that can be used to replace things similar to how the replace operator works. However, the `-replace` operator is case-insensitive by default, and the **Replace()** method is case-sensitive.

PowerShell

```
'SQL Saturday - Baton Rouge'.Replace('saturday','Sat')
```

Notice that the word "Saturday" isn't replaced. This is because it's specified in a different case than the original.

Output

```
SQL Saturday - Baton Rouge
```

When the word "Saturday" is specified in the same case as the original, the **Replace()** method performs the replacement as expected.

PowerShell

```
'SQL Saturday - Baton Rouge'.Replace('Saturday','Sat')
```

Output

```
SQL Sat - Baton Rouge
```

Be careful when using methods to transform data because you can encounter unforeseen problems, such as failing the *Turkey Test*. For an example, see my blog article, Using Pester to Test PowerShell Code with Other Cultures. I recommend using operators instead of methods whenever possible to avoid these types of problems.

While the comparison operators can be used, as shown in the previous examples, I typically use them with the `Where-Object` cmdlet to perform filtering.

Summary

You learned several topics in this chapter, including Formatting Right, Aliases, Providers, and Comparison Operators.

Review

1. Why is it necessary to perform formatting as far to the right as possible?
2. How do you determine what the actual cmdlet is for the `%` alias?
3. Why shouldn't you use aliases in scripts you save or code you share with others?
4. Perform a directory listing on the drives that are associated with the Registry provider.
5. What's one of the main benefits of using the replace operator instead of the replace method?

References

- Format-Table

- Format-List
- Format-Wide
- about_Aliases
- about_Providers
- about_Comparison_Operators
- about_Arrays

Next steps

In Chapter 6, you'll learn about flow control, scripting, loops, and conditional logic.

Last updated on 02/06/2026

Chapter 6 - Flow control

Scripting

When you move from writing PowerShell one-liners to writing scripts, it sounds more complicated than it is. A script is nothing more than the same or similar commands you run interactively in the PowerShell console, except you save them as a `.ps1` file. There are some scripting constructs that you might use, such as a `foreach` loop instead of the `ForEach-Object` cmdlet. The differences can be confusing for beginners when considering that `foreach` is both a language keyword and an alias for the `ForEach-Object` cmdlet.

Looping

One of the best aspects of PowerShell is its scalability. Once you learn how to perform a task for a single item, applying the same action to hundreds of items is almost as straightforward. Loop through the items using one of the different types of loops in PowerShell.

ForEach-Object

`ForEach-Object` is a cmdlet for iterating through items in a pipeline, such as with PowerShell one-liners. `ForEach-Object` streams the objects through the pipeline.

Although the **Module** parameter of `Get-Command` accepts multiple string values, it only accepts them via pipeline input by property name. In the following scenario, if you want to pipe two string values to `Get-Command` for use with the **Module** parameter, you need to use the `ForEach-Object` cmdlet.

PowerShell

```powershell
'ActiveDirectory', 'SQLServer' |
    ForEach-Object {Get-Command -Module $_} |
    Group-Object -Property ModuleName -NoElement |
    Sort-Object -Property Count -Descending
```

Output

```
Count Name
----- ----
```

```
  147 ActiveDirectory
   82 SqlServer
```

In the previous example, `$_` is the current object. Beginning with PowerShell version 3.0, `$PSItem` can be used instead of `$_`. Most experienced PowerShell users prefer using `$_` since it's backward compatible and less to type.

When using the `foreach` keyword, you must store the items in memory before iterating through them, which could be difficult if you don't know how many items you're working with.

PowerShell

```powershell
$ComputerName = 'DC01', 'WEB01'
foreach ($Computer in $ComputerName) {
    Get-ADComputer -Identity $Computer
}
```

Output

```
DistinguishedName : CN=DC01,OU=Domain Controllers,DC=mikefrobbins,DC=com
DNSHostName       : dc01.mikefrobbins.com
Enabled           : True
Name              : DC01
ObjectClass       : computer
ObjectGUID        : c38da20c-a484-469d-ba4c-bab3fb71ae8e
SamAccountName    : DC01$
SID               : S-1-5-21-2989741381-570885089-3319121794-1001
UserPrincipalName :

DistinguishedName : CN=WEB01,CN=Computers,DC=mikefrobbins,DC=com
DNSHostName       : web01.mikefrobbins.com
Enabled           : True
Name              : WEB01
ObjectClass       : computer
ObjectGUID        : 33aa530e-1e31-40d8-8c78-76a18b673c33
SamAccountName    : WEB01$
SID               : S-1-5-21-2989741381-570885089-3319121794-1107
UserPrincipalName :
```

Many times a loop such as `foreach` or `ForEach-Object` is necessary. Otherwise you receive an error message.

PowerShell

```powershell
Get-ADComputer -Identity 'DC01', 'WEB01'
```

Output

```
Get-ADComputer : Cannot convert 'System.Object[]' to the type
'Microsoft.ActiveDirectory.Management.ADComputer' required by parameter
'Identity'. Specified method is not supported.
At line:1 char:26
+ Get-ADComputer -Identity 'DC01', 'WEB01'
+                          ~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidArgument: (:) [Get-ADComputer], Parame
   terBindingException
    + FullyQualifiedErrorId : CannotConvertArgument,Microsoft.ActiveDirecto
   ry.Management.Commands.GetADComputer
```

Other times, you can get the same results while eliminating the loop. Consult the cmdlet help to understand your options.

PowerShell

```powershell
'DC01', 'WEB01' | Get-ADComputer
```

Output

```
DistinguishedName : CN=DC01,OU=Domain Controllers,DC=mikefrobbins,DC=com
DNSHostName       : dc01.mikefrobbins.com
Enabled           : True
Name              : DC01
ObjectClass       : computer
ObjectGUID        : c38da20c-a484-469d-ba4c-bab3fb71ae8e
SamAccountName    : DC01$
SID               : S-1-5-21-2989741381-570885089-3319121794-1001
UserPrincipalName :

DistinguishedName : CN=WEB01,CN=Computers,DC=mikefrobbins,DC=com
DNSHostName       : web01.mikefrobbins.com
Enabled           : True
Name              : WEB01
ObjectClass       : computer
ObjectGUID        : 33aa530e-1e31-40d8-8c78-76a18b673c33
SamAccountName    : WEB01$
SID               : S-1-5-21-2989741381-570885089-3319121794-1107
UserPrincipalName :
```

As you can see in the previous examples, the **Identity** parameter for `Get-ADComputer` only accepts a single value when provided via parameter input. However, by using the pipeline, you can send multiple values to the command because the values are processed one at a time.

For

A `for` loop iterates while a specified condition is true. I don't use the `for` loop often, but it has uses.

PowerShell

```powershell
for ($i = 1; $i -lt 5; $i++) {
    Write-Output "Sleeping for $i seconds"
    Start-Sleep -Seconds $i
}
```

Output

```
Sleeping for 1 seconds
Sleeping for 2 seconds
Sleeping for 3 seconds
Sleeping for 4 seconds
```

In the previous example, the loop iterates four times by starting with the number one and continuing as long as the counter variable `$i` is less than 5. It sleeps for a total of 10 seconds.

Do

There are two different `do` loops in PowerShell: `do until` and `do while`. `do until` runs while the specified condition is false.

The following example is a numbers game that continues until the value you guess equals the same number that the `Get-Random` cmdlet generated.

PowerShell

```powershell
$number = Get-Random -Minimum 1 -Maximum 10
do {
    $guess = Read-Host -Prompt "What's your guess?"
    if ($guess -lt $number) {
        Write-Output 'Too low!'
    } elseif ($guess -gt $number) {
        Write-Output 'Too high!'
    }
}
until ($guess -eq $number)
```

Output

```
What's your guess?: 1
Too low!
What's your guess?: 2
```

```
Too low!
What's your guess?: 3
```

`Do While` is the opposite. It runs as long as the specified condition is evaluated as true.

PowerShell

```powershell
$number = Get-Random -Minimum 1 -Maximum 10
do {
    $guess = Read-Host -Prompt "What's your guess?"
    if ($guess -lt $number) {
        Write-Output 'Too low!'
    } elseif ($guess -gt $number) {
        Write-Output 'Too high!'
    }
}
while ($guess -ne $number)
```

Output

```
What's your guess?: 1
Too low!
What's your guess?: 2
Too low!
What's your guess?: 3
Too low!
What's your guess?: 4
```

The same results are achieved with a `Do While` loop by reversing the test condition to not equals.

`do` loops always run at least once because the condition is evaluated at the end of the loop.

While

Like the `do while` loop, a `while` loop runs as long as the specified condition is true. The difference, however, is that a `while` loop evaluates the condition at the top of the loop before any code is run. So, it doesn't run if the condition is evaluated as false.

The following example calculates what day Thanksgiving Day is on in the United States. It's always on the fourth Thursday of November. The loop starts with the 22nd day of November and adds a day, while the day of the week isn't equal to Thursday. If the 22nd is a Thursday, the loop doesn't run at all.

PowerShell

```powershell
$date = Get-Date -Date 'November 22'
while ($date.DayOfWeek -ne 'Thursday') {
    $date = $date.AddDays(1)
}
Write-Output $date
```

Output

```
Thursday, November 23, 2017 12:00:00 AM
```

break, continue, and return

The `break` keyword is designed to exit a loop and is often used with the `switch` statement. In the following example, `break` causes the loop to end after the first iteration.

PowerShell

```powershell
for ($i = 1; $i -lt 5; $i++) {
    Write-Output "Sleeping for $i seconds"
    Start-Sleep -Seconds $i
    break
}
```

Output

```
Sleeping for 1 seconds
```

The `continue` keyword is designed to skip to the next iteration of a loop.

The following example outputs the numbers 1, 2, 4, and 5. It skips number 3 and continues with the next iteration of the loop. Like `break`, `continue` breaks out of the loop except only for the current iteration. Execution continues with the next iteration instead of breaking out of the loop altogether and stopping.

PowerShell

```powershell
while ($i -lt 5) {
    $i += 1
    if ($i -eq 3) {
        continue
    }
    Write-Output $i
}
```

Output

```
1
2
4
5
```

The `return` keyword is designed to exit out of the existing scope.

Notice in the following example that `return` outputs the first result and then exits out of the loop.

PowerShell

```powershell
$number = 1..10
foreach ($n in $number) {
    if ($n -ge 4) {
        return $n
    }
}
```

Output

```
4
```

A more thorough explanation of the result statement can be found in one of my blog articles: The PowerShell return keyword.

Summary

In this chapter, you learned about the different types of loops that exist in PowerShell.

Review

1. What's the difference between the `ForEach-Object` cmdlet and the `foreach` statement?
2. What's the primary advantage of using a `while` loop instead of a `do while` or `do until` loop?
3. How do the `break` and `continue` statements differ?

References

- ForEach-Object
- about_Foreach

- about_For
- about_Do
- about_While
- about_Break
- about_Continue
- about_Return

Next steps

In Chapter 7, you'll learn how to retrieve and work with system information using Windows Management Instrumentation (WMI) and the Common Information Model (CIM). You'll explore modern CIM-based cmdlets, understand how they differ from legacy WMI commands, and use them to query local and remote systems efficiently.

Last updated on 03/30/2026

Chapter 7 - Working with WMI

WMI and CIM

Windows PowerShell ships by default with cmdlets for working with other technologies, such as Windows Management Instrumentation (WMI). The WMI cmdlets are deprecated and aren't available in PowerShell 6+, but are covered here as you might encounter them in older scripts running on Windows PowerShell. For new development, use the CIM cmdlets instead.

Several native WMI cmdlets exist in PowerShell without you having to install any other software or modules. `Get-Command` can be used to determine what WMI cmdlets exist in Windows PowerShell. The following results are from a Windows 11 system running PowerShell version 5.1. Your results might differ depending on the PowerShell version you're running.

PowerShell

```powershell
Get-Command -Noun WMI*
```

Output

```
CommandType     Name                                               Version
-----------     ----                                               -------
Cmdlet          Get-WmiObject                                      3.1.0.0
Cmdlet          Invoke-WmiMethod                                   3.1.0.0
Cmdlet          Register-WmiEvent                                  3.1.0.0
Cmdlet          Remove-WmiObject                                   3.1.0.0
Cmdlet          Set-WmiInstance                                    3.1.0.0
```

The Common Information Model (CIM) cmdlets were introduced in PowerShell 3.0 and are grouped within a dedicated module. To list all available CIM cmdlets, use the `Get-Command` cmdlet with the **Module** parameter, as shown in the following example.

PowerShell

```powershell
Get-Command -Module CimCmdlets
```

Output

```
CommandType     Name                                               Version
-----------     ----                                               -------
Cmdlet          Export-BinaryMiLog                                 1.0.0.0
Cmdlet          Get-CimAssociatedInstance                          1.0.0.0
Cmdlet          Get-CimClass                                       1.0.0.0
Cmdlet          Get-CimInstance                                    1.0.0.0
Cmdlet          Get-CimSession                                     1.0.0.0
```

```
Cmdlet          Import-BinaryMiLog                              1.0.0.0
Cmdlet          Invoke-CimMethod                                1.0.0.0
Cmdlet          New-CimInstance                                 1.0.0.0
Cmdlet          New-CimSession                                  1.0.0.0
Cmdlet          New-CimSessionOption                            1.0.0.0
Cmdlet          Register-CimIndicationEvent                     1.0.0.0
Cmdlet          Remove-CimInstance                              1.0.0.0
Cmdlet          Remove-CimSession                               1.0.0.0
Cmdlet          Set-CimInstance                                 1.0.0.0
```

The CIM cmdlets still allow you to work with WMI, so don't be confused when someone states: "*When I query WMI with the PowerShell CIM cmdlets*".

As previously mentioned, WMI is a separate technology from PowerShell, and you're just using the CIM cmdlets to access WMI. You might find an old VBScript that uses WMI Query Language (WQL) to query WMI, such as in the following example.

VB

```
strComputer = "."
Set objWMIService = GetObject("winmgmts:" _
    & "{impersonationLevel=impersonate}!\\" & strComputer & "\root\CIMV2")

Set colBIOS = objWMIService.ExecQuery _
    ("Select * from Win32_BIOS")

For each objBIOS in colBIOS
    Wscript.Echo "Manufacturer: " & objBIOS.Manufacturer
    Wscript.Echo "Name: " & objBIOS.Name
    Wscript.Echo "Serial Number: " & objBIOS.SerialNumber
    Wscript.Echo "SMBIOS Version: " & objBIOS.SMBIOSBIOSVersion
    Wscript.Echo "Version: " & objBIOS.Version
Next
```

You can take the WQL query from the VBScript and use it with the `Get-CimInstance` cmdlet without any modifications.

PowerShell

```
Get-CimInstance -Query 'Select * from Win32_BIOS'
```

Output

```
SMBIOSBIOSVersion : 090006
Manufacturer      : American Megatrends Inc.
Name              : Intel(R) Xeon(R) CPU E3-1505M v5 @ 2.80GHz
SerialNumber      : 3810-1995-1654-4615-2295-2755-89
Version           : VRTUAL - 4001628
```

The previous example isn't how I typically query WMI with PowerShell. But it works and allows you to easily migrate existing Visual Basic scripts to PowerShell. When writing a one-liner to query WMI, I use the following syntax.

PowerShell

```
Get-CimInstance -ClassName Win32_BIOS
```

Output

```
SMBIOSBIOSVersion : 090006
Manufacturer      : American Megatrends Inc.
Name              : Intel(R) Xeon(R) CPU E3-1505M v5 @ 2.80GHz
SerialNumber      : 3810-1995-1654-4615-2295-2755-89
Version           : VRTUAL - 4001628
```

If you only want the serial number, pipe the output to `Select-Object` and specify only the **SerialNumber** property.

PowerShell

```
Get-CimInstance -ClassName Win32_BIOS |
    Select-Object -Property SerialNumber
```

Output

```
SerialNumber
------------
3810-1995-1654-4615-2295-2755-89
```

By default, when querying WMI, several properties that are never used are retrieved behind the scenes. It doesn't matter much when querying WMI on the local computer. But once you start querying remote computers, it's not only extra processing time to return that information but also more unnecessary information to send across the network. `Get-CimInstance` has a **Property** parameter that limits the information retrieved, making the WMI query more efficient.

PowerShell

```
Get-CimInstance -ClassName Win32_BIOS -Property SerialNumber |
    Select-Object -Property SerialNumber
```

Output

```
SerialNumber
------------
```

```
3810-1995-1654-4615-2295-2755-89
```

The previous results returned an object. To return a string, use the **ExpandProperty** parameter.

PowerShell

```
Get-CimInstance -ClassName Win32_BIOS -Property SerialNumber |
    Select-Object -ExpandProperty SerialNumber
```

Output

```
3810-1995-1654-4615-2295-2755-89
```

You could also use the dotted syntax style to return a string, eliminating the need to pipe to `Select-Object`.

PowerShell

```
(Get-CimInstance -ClassName Win32_BIOS -Property SerialNumber).SerialNumber
```

Output

```
3810-1995-1654-4615-2295-2755-89
```

Query Remote Computers with the CIM cmdlets

You should still be running PowerShell as a local admin and domain user. When you try to query information from a remote computer using the `Get-CimInstance` cmdlet, you receive an access denied error message.

PowerShell

```
Get-CimInstance -ComputerName dc01 -ClassName Win32_BIOS
```

Output

```
Get-CimInstance : Access is denied.
At line:1 char:1
+ Get-CimInstance -ComputerName dc01 -ClassName Win32_BIOS
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : PermissionDenied: (root\cimv2:Win32_BIOS:Stri
   ng) [Get-CimInstance], CimException
    + FullyQualifiedErrorId : HRESULT 0x80070005,Microsoft.Management.Infra
   structure.CimCmdlets.GetCimInstanceCommand
    + PSComputerName        : dc01
```

Many people have security concerns regarding PowerShell, but you have the same permissions in PowerShell as in the GUI. No more and no less. The problem in the previous example is that the user running PowerShell doesn't have rights to query WMI information from the DC01 server. You could relaunch PowerShell as a domain administrator since `Get-CimInstance` doesn't have a **Credential** parameter. But that isn't a good idea because anything you run from PowerShell would run as a domain admin. Depending on the situation, that scenario could be dangerous from a security standpoint.

Using the principle of least privilege, elevate to your domain admin account on a per-command basis using the **Credential** parameter if a command has one. `Get-CimInstance` doesn't have a **Credential** parameter, so the solution in this scenario is to create a **CimSession** first. Then, use the **CimSession** instead of a computer name to query WMI on the remote computer.

PowerShell

```
$CimSession = New-CimSession -ComputerName dc01 -Credential (Get-Credential)
```

Output

```
cmdlet Get-Credential at command pipeline position 1
Supply values for the following parameters:
Credential
```

The CIM session was stored in a variable named `$CimSession`. Notice that you also specify the `Get-Credential` cmdlet in parentheses so that it executes first, prompting for alternate credentials, before creating the new session. I show you another more efficient way to specify alternate credentials later in this chapter, but it's important to understand this basic concept before making it more complicated.

You can now use the CIM session created in the previous example with the `Get-CimInstance` cmdlet to query the BIOS information from WMI on the remote computer.

PowerShell

```
Get-CimInstance -CimSession $CimSession -ClassName Win32_BIOS
```

Output

```
SMBIOSBIOSVersion : 090006
Manufacturer      : American Megatrends Inc.
Name              : Intel(R) Xeon(R) CPU E3-1505M v5 @ 2.80GHz
SerialNumber      : 0986-6980-3916-0512-6608-8243-13
```

```
Version           : VRTUAL - 4001628
PSComputerName    : dc01
```

There are several other benefits to using CIM sessions instead of just specifying a computer name. When you run multiple queries to the same computer, using a CIM session is more efficient than using the computer name for each query. Creating a CIM session only sets up the connection once. Then, multiple queries use that same session to retrieve information. Using the computer name requires the cmdlets to set up and tear down the connection with each query.

The `Get-CimInstance` cmdlet uses the WSMan protocol by default, which means the remote computer needs PowerShell version 3.0 or higher to connect. It's actually not the PowerShell version that matters, it's the stack version. The stack version can be determined using the `Test-WSMan` cmdlet. It needs to be version 3.0, which you find with PowerShell version 3.0 and higher.

PowerShell

```
Test-WSMan -ComputerName dc01
```

Output

```
wsmid           : http://schemas.dmtf.org/wbem/wsman/identity/1/wsmanidentit
                  y.xsd
ProtocolVersion : http://schemas.dmtf.org/wbem/wsman/1/wsman.xsd
ProductVendor   : Microsoft Corporation
ProductVersion  : OS: 0.0.0 SP: 0.0 Stack: 3.0
```

The older WMI cmdlets use the DCOM protocol, which is compatible with older versions of Windows. However, the firewall typically blocks DCOM on newer versions of Windows. The `New-CimSessionOption` cmdlet allows you to create a DCOM protocol connection for use with `New-CimSession`. This option allows the `Get-CimInstance` cmdlet to communicate with versions of Windows as old as Windows Server 2000. This ability also means that PowerShell isn't required on the remote computer when using the `Get-CimInstance` cmdlet with a CimSession configured to use the DCOM protocol.

Create the DCOM protocol option using the `New-CimSessionOption` cmdlet and store it in a variable.

PowerShell

```
$DCOM = New-CimSessionOption -Protocol Dcom
```

For efficiency, you can store your domain administrator or elevated credentials in a variable so you don't have to constantly enter them for each command.

PowerShell

```
$Cred = Get-Credential
```

Output

```
cmdlet Get-Credential at command pipeline position 1
Supply values for the following parameters:
Credential
```

I have a server named SQL03 that runs Windows Server 2008 (non-R2). It's the newest Windows Server operating system that doesn't have PowerShell installed by default.

Create a **CimSession** to SQL03 using the DCOM protocol.

PowerShell

```
$CimSession = New-CimSession -ComputerName sql03 -SessionOption $DCOM -Credential
$Cred
```

Notice in the previous command that you specify the variable named `$Cred` as the value for the **Credential** parameter instead of manually entering your credentials again.

The output of the query is the same regardless of the underlying protocol.

PowerShell

```
Get-CimInstance -CimSession $CimSession -ClassName Win32_BIOS
```

Output

```
SMBIOSBIOSVersion : 090006
Manufacturer      : American Megatrends Inc.
Name              : Intel(R) Xeon(R) CPU E3-1505M v5 @ 2.80GHz
SerialNumber      : 7237-7483-8873-8926-7271-5004-86
Version           : VRTUAL - 4001628
PSComputerName    : sql03
```

The `Get-CimSession` cmdlet is used to see what CimSessions are currently connected and what protocols they use.

PowerShell

```
Get-CimSession
```

Output

```
Id           : 1
Name         : CimSession1
InstanceId   : 80742787-e38e-41b1-a7d7-fa1369cf1402
ComputerName : dc01
Protocol     : WSMAN

Id           : 2
Name         : CimSession2
InstanceId   : 8fcabd81-43cf-4682-bd53-ccce1e24aecb
ComputerName : sql03
Protocol     : DCOM
```

Retrieve and store the previously created CimSessions in a variable named `$CimSession`.

PowerShell

```
$CimSession = Get-CimSession
```

Query both computers with one command, one using the WSMan protocol and the other with DCOM.

PowerShell

```
Get-CimInstance -CimSession $CimSession -ClassName Win32_BIOS
```

Output

```
SMBIOSBIOSVersion : 090006
Manufacturer      : American Megatrends Inc.
Name              : Intel(R) Xeon(R) CPU E3-1505M v5 @ 2.80GHz
SerialNumber      : 0986-6980-3916-0512-6608-8243-13
Version           : VRTUAL - 4001628
PSComputerName    : dc01

SMBIOSBIOSVersion : 090006
Manufacturer      : American Megatrends Inc.
Name              : Intel(R) Xeon(R) CPU E3-1505M v5 @ 2.80GHz
SerialNumber      : 7237-7483-8873-8926-7271-5004-86
Version           : VRTUAL - 4001628
PSComputerName    : sql03
```

One of my blog articles on WMI and CIM cmdlets features a PowerShell function that automatically detects whether to use WSMan or DCOM and then sets up the appropriate CIM session for you. For more information, see PowerShell Function to Create CimSessions to Remote Computers with Fallback to Dcom.

When you finish with the CIM sessions, remove them with the `Remove-CimSession` cmdlet. To remove all CIM sessions, pipe `Get-CimSession` to `Remove-CimSession`.

PowerShell

```
Get-CimSession | Remove-CimSession
```

Summary

In this chapter, you learned about using PowerShell to work with WMI on local and remote computers. You also learned how to use the CIM cmdlets to work with remote computers using the WSMan and DCOM protocols.

Review

1. What's the difference in the WMI and CIM cmdlets?
2. By default, what protocol does the `Get-CimInstance` cmdlet use?
3. What are some benefits of using a CIM session instead of specifying a computer name with `Get-CimInstance`?
4. How do you specify an alternate protocol other than the default one for use with `Get-CimInstance`?
5. How do you close or remove CIM sessions?

References

- about_WMI
- about_WMI_Cmdlets
- about_WQL
- CimCmdlets Module
- Video: Using CIM Cmdlets and CIM Sessions

Next steps

In Chapter 8, you'll learn about PowerShell remoting. You'll explore how to run commands on remote systems, understand one-to-one and one-to-many remoting scenarios, and learn how remoting enables scalable automation across multiple computers.

Last updated on 02/06/2026

Chapter 8 - PowerShell remoting

PowerShell offers several ways to run commands against remote computers. In the last chapter, you explored how to remotely query WMI using the CIM cmdlets. PowerShell also includes several cmdlets that feature a built-in **ComputerName** parameter.

As shown in the following example, you can use `Get-Command` with the **ParameterName** parameter to identify cmdlets that include a **ComputerName** parameter.

PowerShell

```
Get-Command -ParameterName ComputerName
```

Output

```
CommandType Name               Version Source
----------- ----               ------- ------
Cmdlet      Add-Computer       3.1.0.0 Microsoft.PowerShell.Management
Cmdlet      Clear-EventLog     3.1.0.0 Microsoft.PowerShell.Management
Cmdlet      Connect-PSSession  3.0.0.0 Microsoft.PowerShell.Core
Cmdlet      Enter-PSSession    3.0.0.0 Microsoft.PowerShell.Core
Cmdlet      Get-EventLog       3.1.0.0 Microsoft.PowerShell.Management
Cmdlet      Get-HotFix         3.1.0.0 Microsoft.PowerShell.Management
Cmdlet      Get-Process        3.1.0.0 Microsoft.PowerShell.Management
Cmdlet      Get-PSSession      3.0.0.0 Microsoft.PowerShell.Core
Cmdlet      Get-Service        3.1.0.0 Microsoft.PowerShell.Management
Cmdlet      Get-WmiObject      3.1.0.0 Microsoft.PowerShell.Management
Cmdlet      Invoke-Command     3.0.0.0 Microsoft.PowerShell.Core
Cmdlet      Invoke-WmiMethod   3.1.0.0 Microsoft.PowerShell.Management
Cmdlet      Limit-EventLog     3.1.0.0 Microsoft.PowerShell.Management
Cmdlet      New-EventLog       3.1.0.0 Microsoft.PowerShell.Management
Cmdlet      New-PSSession      3.0.0.0 Microsoft.PowerShell.Core
Cmdlet      Receive-Job        3.0.0.0 Microsoft.PowerShell.Core
Cmdlet      Receive-PSSession  3.0.0.0 Microsoft.PowerShell.Core
Cmdlet      Register-WmiEvent  3.1.0.0 Microsoft.PowerShell.Management
Cmdlet      Remove-Computer    3.1.0.0 Microsoft.PowerShell.Management
Cmdlet      Remove-EventLog    3.1.0.0 Microsoft.PowerShell.Management
Cmdlet      Remove-PSSession   3.0.0.0 Microsoft.PowerShell.Core
Cmdlet      Remove-WmiObject   3.1.0.0 Microsoft.PowerShell.Management
Cmdlet      Rename-Computer    3.1.0.0 Microsoft.PowerShell.Management
Cmdlet      Restart-Computer   3.1.0.0 Microsoft.PowerShell.Management
Cmdlet      Send-MailMessage   3.1.0.0 Microsoft.PowerShell.Utility
Cmdlet      Set-Service        3.1.0.0 Microsoft.PowerShell.Management
Cmdlet      Set-WmiInstance    3.1.0.0 Microsoft.PowerShell.Management
Cmdlet      Show-EventLog      3.1.0.0 Microsoft.PowerShell.Management
Cmdlet      Stop-Computer      3.1.0.0 Microsoft.PowerShell.Management
Cmdlet      Test-Connection    3.1.0.0 Microsoft.PowerShell.Management
Cmdlet      Write-EventLog     3.1.0.0 Microsoft.PowerShell.Management
```

Commands such as `Get-Process` and `Get-HotFix` include a **ComputerName** parameter, but this approach isn't the long-term direction Microsoft recommends for running commands against remote systems. Even when you find a command with a **ComputerName** parameter, it often lacks a **Credential** parameter, making it difficult to specify alternate credentials. Running PowerShell from an elevated session doesn't guarantee success, as a network firewall can block the request between your system and the remote computer.

To use the PowerShell remoting commands demonstrated in this chapter, PowerShell remoting must be enabled on the remote computer. You can enable it by running the `Enable-PSRemoting` cmdlet.

PowerShell

```
Enable-PSRemoting
```

Output

```
WinRM has been updated to receive requests.
WinRM service type changed successfully.
WinRM service started.

WinRM has been updated for remote management.
WinRM firewall exception enabled.
```

One-to-one remoting

If you want an interactive remote session, one-to-one remoting is what you want. This type of remoting is provided via the `Enter-PSSession` cmdlet.

Store your domain admin credentials in the `$Cred` variable. This approach allows you to enter your credentials once and reuse them on a per-command basis as long as your current PowerShell session remains active.

PowerShell

```
$Cred = Get-Credential
```

Establish a one-to-one PowerShell remoting session to the domain controller named dc01.

PowerShell

```
Enter-PSSession -ComputerName dc01 -Credential $Cred
```

Notice the PowerShell prompt is preceded by `[dc01]`. This prefix indicates you're in an interactive session with the remote computer named dc01. Any commands you run now execute on dc01, not your local machine.

Output

```
[dc01]: PS C:\Users\Administrator\Documents>
```

Remember that you can only access the PowerShell commands and modules installed on the remote computer. If you installed other modules locally, they aren't available in the remote session.

When connected via a one-to-one interactive remoting session, it's as if you're sitting directly at the remote machine.

PowerShell

```
[dc01]: Get-Process | Get-Member
```

Output

```
   TypeName: System.Diagnostics.Process

Name                       MemberType     Definition
----                       ----------     ----------
Handles                    AliasProperty  Handles = Handlecount
Name                       AliasProperty  Name = ProcessName
NPM                        AliasProperty  NPM = NonpagedSystemMemorySize64
PM                         AliasProperty  PM = PagedMemorySize64
SI                         AliasProperty  SI = SessionId
VM                         AliasProperty  VM = VirtualMemorySize64
WS                         AliasProperty  WS = WorkingSet64
Disposed                   Event          System.EventHandler Disposed(Sy...
ErrorDataReceived          Event          System.Diagnostics.DataReceived...
Exited                     Event          System.EventHandler Exited(Syst...
OutputDataReceived         Event          System.Diagnostics.DataReceived...
BeginErrorReadLine         Method         void BeginErrorReadLine()
BeginOutputReadLine        Method         void BeginOutputReadLine()
CancelErrorRead            Method         void CancelErrorRead()
CancelOutputRead           Method         void CancelOutputRead()
Close                      Method         void Close()
CloseMainWindow            Method         bool CloseMainWindow()
CreateObjRef               Method         System.Runtime.Remoting.ObjRef ...
Dispose                    Method         void Dispose(), void IDisposabl...
Equals                     Method         bool Equals(System.Object obj)
GetHashCode                Method         int GetHashCode()
GetLifetimeService         Method         System.Object GetLifetimeService()
GetType                    Method         type GetType()
InitializeLifetimeService  Method         System.Object InitializeLifetim...
Kill                       Method         void Kill()
```

```
Refresh                    Method         void Refresh()
Start                      Method         bool Start()
ToString                   Method         string ToString()
WaitForExit                Method         bool WaitForExit(int millisecon...
WaitForInputIdle           Method         bool WaitForInputIdle(int milli...
__NounName                 NoteProperty   string __NounName=Process
BasePriority               Property       int BasePriority {get;}
Container                  Property       System.ComponentModel.IContaine...
EnableRaisingEvents        Property       bool EnableRaisingEvents {get;s...
ExitCode                   Property       int ExitCode {get;}
ExitTime                   Property       datetime ExitTime {get;}
Handle                     Property       System.IntPtr Handle {get;}
HandleCount                Property       int HandleCount {get;}
HasExited                  Property       bool HasExited {get;}
Id                         Property       int Id {get;}
MachineName                Property       string MachineName {get;}
MainModule                 Property       System.Diagnostics.ProcessModul...
MainWindowHandle           Property       System.IntPtr MainWindowHandle ...
MainWindowTitle            Property       string MainWindowTitle {get;}
MaxWorkingSet              Property       System.IntPtr MaxWorkingSet {ge...
MinWorkingSet              Property       System.IntPtr MinWorkingSet {ge...
Modules                    Property       System.Diagnostics.ProcessModul...
NonpagedSystemMemorySize   Property       int NonpagedSystemMemorySize {g...
NonpagedSystemMemorySize64 Property       long NonpagedSystemMemorySize64...
PagedMemorySize            Property       int PagedMemorySize {get;}
PagedMemorySize64          Property       long PagedMemorySize64 {get;}
PagedSystemMemorySize      Property       int PagedSystemMemorySize {get;}
PagedSystemMemorySize64    Property       long PagedSystemMemorySize64 {g...
PeakPagedMemorySize        Property       int PeakPagedMemorySize {get;}
PeakPagedMemorySize64      Property       long PeakPagedMemorySize64 {get;}
PeakVirtualMemorySize      Property       int PeakVirtualMemorySize {get;}
PeakVirtualMemorySize64    Property       long PeakVirtualMemorySize64 {g...
PeakWorkingSet             Property       int PeakWorkingSet {get;}
PeakWorkingSet64           Property       long PeakWorkingSet64 {get;}
PriorityBoostEnabled       Property       bool PriorityBoostEnabled {get;...
PriorityClass              Property       System.Diagnostics.ProcessPrior...
PrivateMemorySize          Property       int PrivateMemorySize {get;}
PrivateMemorySize64        Property       long PrivateMemorySize64 {get;}
PrivilegedProcessorTime    Property       timespan PrivilegedProcessorTim...
ProcessName                Property       string ProcessName {get;}
ProcessorAffinity          Property       System.IntPtr ProcessorAffinity...
Responding                 Property       bool Responding {get;}
SafeHandle                 Property       Microsoft.Win32.SafeHandles.Saf...
SessionId                  Property       int SessionId {get;}
Site                       Property       System.ComponentModel.ISite Sit...
StandardError              Property       System.IO.StreamReader Standard...
StandardInput              Property       System.IO.StreamWriter Standard...
StandardOutput             Property       System.IO.StreamReader Standard...
StartInfo                  Property       System.Diagnostics.ProcessStart...
StartTime                  Property       datetime StartTime {get;}
SynchronizingObject        Property       System.ComponentModel.ISynchron...
Threads                    Property       System.Diagnostics.ProcessThrea...
TotalProcessorTime         Property       timespan TotalProcessorTime {get;}
UserProcessorTime          Property       timespan UserProcessorTime {get;}
VirtualMemorySize          Property       int VirtualMemorySize {get;}
```

```
VirtualMemorySize64        Property       long VirtualMemorySize64 {get;}
WorkingSet                 Property       int WorkingSet {get;}
WorkingSet64               Property       long WorkingSet64 {get;}
PSConfiguration            PropertySet    PSConfiguration {Name, Id, Prio...
PSResources                PropertySet    PSResources {Name, Id, Handleco...
Company                    ScriptProperty System.Object Company {get=$thi...
CPU                        ScriptProperty System.Object CPU {get=$this.To...
Description                ScriptProperty System.Object Description {get=...
FileVersion                ScriptProperty System.Object FileVersion {get=...
Path                       ScriptProperty System.Object Path {get=$this.M...
Product                    ScriptProperty System.Object Product {get=$thi...
ProductVersion             ScriptProperty System.Object ProductVersion {g...
```

When you finish working with the remote computer, run the `Exit-PSSession` cmdlet to end the remote session.

PowerShell

```
[dc01]:  Exit-PSSession
```

One-to-many remoting

While you might occasionally need to perform tasks interactively on a remote computer, PowerShell remoting becomes more powerful when you simultaneously execute commands across multiple remote systems. Use the `Invoke-Command` cmdlet to run commands on one or more remote computers at the same time.

In the following example, you query three servers for the status of the Windows Time service. The `Get-Service` cmdlet is placed inside the script block of `Invoke-Command`, meaning it executes on each remote computer.

PowerShell

```
Invoke-Command -ComputerName dc01, sql02, web01 {
    Get-Service -Name W32time
} -Credential $Cred
```

The results are returned to your local session as deserialized objects.

Output

```
Status   Name        DisplayName       PSComputerName
------   ----        -----------       --------------
Running  W32time     Windows Time      web01
Start... W32time     Windows Time      dc01
Running  W32time     Windows Time      sql02
```

To confirm the returned objects are deserialized, pipe the output to `Get-Member`.

PowerShell

```
Invoke-Command -ComputerName dc01, sql02, web01 {
    Get-Service -Name W32time
} -Credential $Cred | Get-Member
```

Output

```
   TypeName: Deserialized.System.ServiceProcess.ServiceController

Name                  MemberType   Definition
----                  ----------   ----------
GetType               Method       type GetType()
ToString              Method       string ToString(), string ToString(strin...
Name                  NoteProperty string Name=W32time
PSComputerName        NoteProperty string PSComputerName=dc01
PSShowComputerName    NoteProperty bool PSShowComputerName=True
RequiredServices      NoteProperty Deserialized.System.ServiceProcess.Servi...
RunspaceId            NoteProperty guid RunspaceId=5ed06925-8037-43ef-9072-...
CanPauseAndContinue   Property     System.Boolean {get;set;}
CanShutdown           Property     System.Boolean {get;set;}
CanStop               Property     System.Boolean {get;set;}
Container             Property      {get;set;}
DependentServices     Property     Deserialized.System.ServiceProcess.Servi...
DisplayName           Property     System.String {get;set;}
MachineName           Property     System.String {get;set;}
ServiceHandle         Property     System.String {get;set;}
ServiceName           Property     System.String {get;set;}
ServicesDependedOn    Property     Deserialized.System.ServiceProcess.Servi...
ServiceType           Property     System.String {get;set;}
Site                  Property      {get;set;}
StartType             Property     System.String {get;set;}
Status                Property     System.String {get;set;}
```

Notice that most methods are missing from deserialized objects. The methods are missing because these objects aren't live. They're inert snapshots of the object's state when you execute the command against the remote computer. For example, you can't start or stop a service using a deserialized object since it no longer has access to the required methods.

However, this doesn't mean you can't use methods like `Stop()` with `Invoke-Command`. The key is that you must call the method within the remote session.

To demonstrate, stop the Windows Time service on all three remote servers by invoking the `Stop()` method remotely.

PowerShell

```
Invoke-Command -ComputerName dc01, sql02, web01 {
    (Get-Service -Name W32time).Stop()
} -Credential $Cred

Invoke-Command -ComputerName dc01, sql02, web01 {
    Get-Service -Name W32time
} -Credential $Cred
```

Output

```
Status   Name        DisplayName       PSComputerName
------   ----        -----------       --------------
Stopped  W32time     Windows Time      web01
Stopped  W32time     Windows Time      dc01
Stopped  W32time     Windows Time      sql02
```

As mentioned in an earlier chapter, if there's a cmdlet available to accomplish a task, it's preferable to use it rather than calling a method directly. For example, use the `Stop-Service` cmdlet instead of the `Stop()` method to stop a service.

In the previous example, the `Stop()` method is used to make a point. Some people mistakenly believe that you can't use methods with PowerShell remoting. While it's true that you can't call methods on deserialized objects returned to your local session, you can, however, invoke them within the remote session.

PowerShell sessions

In the final example from the previous section, you ran two commands using the `Invoke-Command` cmdlet. This scenario resulted in two separate sessions being established and torn down. One for each command.

Like CIM sessions, a persistent PowerShell session allows you to run multiple commands against a remote computer without the overhead of creating a new session for each command.

Create a PowerShell session to each of the three computers you're working with in this chapter, DC01, SQL02, and WEB01.

PowerShell

```
$Session = New-PSSession -ComputerName dc01, sql02, web01 -Credential $Cred
```

Now, use the `$Session` variable to start the Windows Time service by calling its method and then verify the service status.

PowerShell

```
Invoke-Command -Session $Session {(Get-Service -Name W32time).Start()}
Invoke-Command -Session $Session {Get-Service -Name W32time}
```

Output

```
Status   Name        DisplayName       PSComputerName
------   ----        -----------       --------------
Running  W32time     Windows Time      web01
Start... W32time     Windows Time      dc01
Running  W32time     Windows Time      sql02
```

Once you create the session with alternate credentials, you don't need to specify those credentials again for each command.

Be sure to remove the sessions when you finish using them.

PowerShell

```
Get-PSSession | Remove-PSSession
```

Summary

In this chapter, you learned the fundamentals of PowerShell remoting, including running commands interactively on a single remote computer and executing commands across multiple systems using one-to-many remoting. You also explored the advantages of using persistent PowerShell sessions when running multiple commands against the same remote computer.

Review

1. How do you enable PowerShell remoting?
2. What PowerShell command do you use to start an interactive session with a remote computer?
3. What's one benefit of using a PowerShell remoting session instead of specifying the computer name with each command?
4. Can you use a PowerShell session in a one-to-one interactive remoting scenario?
5. What's the difference between the objects returned by cmdlets run locally and objects returned when the same cmdlets are executed on remote computers using `Invoke-Command`?

References

- about_Remote
- about_Remote_Output
- about_Remote_Requirements
- about_Remote_Troubleshooting
- about_Remote_Variables
- PowerShell Remoting FAQ

Next steps

In Chapter 9, you'll learn how to write reusable PowerShell functions. You'll explore function design, parameters, pipeline input, error handling, and best practices for turning one-liners and scripts into reliable tools.

Last updated on 02/06/2026

Chapter 9 - Functions

PowerShell one-liners and scripts that have to be modified often are good candidates to turn into reusable functions.

Write functions whenever possible because they're more tool-oriented. You can add the functions to a script module, put that module in a location defined in the `$env:PSModulePath`, and call the functions without needing to locate where you saved the functions. Using the **PowerShellGet** module, it's easy to share your PowerShell modules in a NuGet repository. **PowerShellGet** ships with PowerShell version 5.0 and higher. It's also available as a separate download for PowerShell version 3.0 and higher.

Don't overcomplicate things. Keep it simple and use the most straightforward way to accomplish a task. Avoid aliases and positional parameters in any code that you reuse. Format your code for readability. Don't hardcode values; use parameters and variables. Don't write unnecessary code even if it doesn't hurt anything. It adds unnecessary complexity. Attention to detail goes a long way when writing any PowerShell code.

Naming

When naming your functions in PowerShell, use a Pascal case name with an approved verb and a singular noun. To obtain a list of approved verbs in PowerShell, run `Get-Verb`. The following example sorts the results of `Get-Verb` by the **Verb** property.

PowerShell

```powershell
Get-Verb | Sort-Object -Property Verb
```

The **Group** property gives you an idea of how the verbs are meant to be used.

Output

```
Verb        Group
----        -----
Add         Common
Approve     Lifecycle
Assert      Lifecycle
Backup      Data
Block       Security
Checkpoint  Data
Clear       Common
```

```
Close       Common
Compare     Data
Complete    Lifecycle
Compress    Data
Confirm     Lifecycle
Connect     Communications
Convert     Data
ConvertFrom Data
ConvertTo   Data
Copy        Common
Debug       Diagnostic
Deny        Lifecycle
Disable     Lifecycle
Disconnect  Communications
Dismount    Data
Edit        Data
Enable      Lifecycle
Enter       Common
Exit        Common
Expand      Data
Export      Data
Find        Common
Format      Common
Get         Common
Grant       Security
Group       Data
Hide        Common
Import      Data
Initialize  Data
Install     Lifecycle
Invoke      Lifecycle
Join        Common
Limit       Data
Lock        Common
Measure     Diagnostic
Merge       Data
Mount       Data
Move        Common
New         Common
Open        Common
Optimize    Common
Out         Data
Ping        Diagnostic
Pop         Common
Protect     Security
Publish     Data
Push        Common
Read        Communications
Receive     Communications
Redo        Common
Register    Lifecycle
Remove      Common
Rename      Common
Repair      Diagnostic
Request     Lifecycle
```

```
Reset       Common
Resize      Common
Resolve     Diagnostic
Restart     Lifecycle
Restore     Data
Resume      Lifecycle
Revoke      Security
Save        Data
Search      Common
Select      Common
Send        Communications
Set         Common
Show        Common
Skip        Common
Split       Common
Start       Lifecycle
Step        Common
Stop        Lifecycle
Submit      Lifecycle
Suspend     Lifecycle
Switch      Common
Sync        Data
Test        Diagnostic
Trace       Diagnostic
Unblock     Security
Undo        Common
Uninstall   Lifecycle
Unlock      Common
Unprotect   Security
Unpublish   Data
Unregister  Lifecycle
Update      Data
Use         Other
Wait        Lifecycle
Watch       Common
Write       Communications
```

It's important to use an approved verb for your PowerShell functions. Modules that contain functions with unapproved verbs generate a warning message when they're imported into a PowerShell session. That warning message makes your functions look unprofessional. Unapproved verbs also limit the discoverability of your functions.

A simple function

A function in PowerShell is declared with the function keyword followed by the function name and then an opening and closing curly brace (`{ }`). The code executed by the function is contained within those curly braces.

PowerShell

```
function Get-Version {
    $PSVersionTable.PSVersion
}
```

The function shown in the following example is a simple example that returns the version of PowerShell.

PowerShell

```
Get-Version
```

Output

```
Major  Minor  Build  Revision
-----  -----  -----  --------
5      1      14393  693
```

When you use a generic name for your functions, such as `Get-Version`, it could cause naming conflicts. Default commands added in the future or commands that others might write could conflict with them. Prefix the noun portion of your function names to help prevent naming conflicts. For example: `<ApprovedVerb>-<Prefix><SingularNoun>`.

The following example uses the prefix `PS`.

PowerShell

```
function Get-PSVersion {
    $PSVersionTable.PSVersion
}
```

Other than the name, this function is identical to the previous one.

PowerShell

```
Get-PSVersion
```

Output

```
Major  Minor  Build  Revision
-----  -----  -----  --------
5      1      14393  693
```

You can still have a name conflict even when you add a prefix to the noun. I like to prefix my function nouns with my initials. Develop a standard and stick to it.

PowerShell

```powershell
function Get-MrPSVersion {
    $PSVersionTable.PSVersion
}
```

This function is no different than the previous two, except for using a more unique name to try to prevent naming conflicts with other PowerShell commands.

PowerShell

```powershell
Get-MrPSVersion
```

Output

```
Major  Minor  Build  Revision
-----  -----  -----  --------
5      1      14393  693
```

Once loaded into memory, you can see functions on the **Function** PSDrive.

PowerShell

```powershell
Get-ChildItem -Path Function:\Get-*Version
```

Output

```
CommandType     Name                                               Version
-----------     ----                                               -------
Function        Get-Version
Function        Get-PSVersion
Function        Get-MrPSVersion
```

If you want to remove these functions from your current session, remove them from the **Function** PSDrive or close and reopen PowerShell.

PowerShell

```powershell
Get-ChildItem -Path Function:\Get-*Version | Remove-Item
```

Verify that the functions were indeed removed.

PowerShell

```
Get-ChildItem -Path Function:\Get-*Version
```

If the functions were loaded as part of a module, you can unload the module to remove them.

PowerShell

```
Remove-Module -Name <ModuleName>
```

The `Remove-Module` cmdlet removes PowerShell modules from memory in your current PowerShell session. It doesn't remove them from your system or disk.

Parameters

Don't statically assign values. Use parameters and variables instead. When naming your parameters, use the same name as the default cmdlets for your parameter names whenever possible.

In the following function, notice that I used **ComputerName** and not **Computer**, **ServerName**, or **Host** for the parameter name. Using **ComputerName** standardizes the parameter name to match the parameter name and case like the default cmdlets.

PowerShell

```
function Test-MrParameter {

    param (
        $ComputerName
    )

    Write-Output $ComputerName

}
```

The following function queries all commands on your system and returns the number with specific parameter names.

PowerShell

```
function Get-MrParameterCount {
    param (
        [string[]]$ParameterName
    )
```

```
    foreach ($Parameter in $ParameterName) {
        $Results = Get-Command -ParameterName $Parameter -ErrorAction
SilentlyContinue

        [pscustomobject]@{
            ParameterName   = $Parameter
            NumberOfCmdlets = $Results.Count
        }
    }
}
```

As you can see in the following results, 39 commands that have a **ComputerName** parameter. There aren't any commands with parameters such as **Computer**, **ServerName**, **Host**, or **Machine**.

PowerShell

```
Get-MrParameterCount -ParameterName ComputerName, Computer, ServerName,
    Host, Machine
```

Output

```
ParameterName NumberOfCmdlets
------------- ---------------
ComputerName               39
Computer                    0
ServerName                  0
Host                        0
Machine                     0
```

Use the same case for your parameter names as the default cmdlets. For example, use `ComputerName`, not `computername`. This naming scheme helps people familiar with PowerShell discover your functions and look and feel like the default cmdlets.

The `param` statement allows you to define one or more parameters. A comma (`,`) separates the parameter definitions. For more information, see about_Functions_Advanced_Parameters.

Advanced functions

Turning a function into an advanced function in PowerShell is simple. One of the differences between a function and an advanced function is that advanced functions have common parameters that are automatically added. Common parameters include parameters such as **Verbose** and **Debug**.

Start with the `Test-MrParameter` function that was used in the previous section.

PowerShell

```powershell
function Test-MrParameter {

    param (
        $ComputerName
    )

    Write-Output $ComputerName

}
```

There are a couple of different ways to see the common parameters. One is by viewing the syntax with `Get-Command`.

PowerShell

```powershell
Get-Command -Name Test-MrParameter -Syntax
```

Notice the `Test-MrParameter` function doesn't have any common parameters.

Output

```
Test-MrParameter [[-ComputerName] <Object>]
```

Another is to drill down into the parameters property of `Get-Command`.

PowerShell

```powershell
(Get-Command -Name Test-MrParameter).Parameters.Keys
```

Output

```
ComputerName
```

Add the `CmdletBinding` attribute to turn the function into an advanced function.

PowerShell

```powershell
function Test-MrCmdletBinding {

    [CmdletBinding()] # Turns a regular function into an advanced function
    param (
        $ComputerName
    )
```

```
    Write-Output $ComputerName

}
```

When you specify `CmdletBinding`, the common parameters are added automatically. `CmdletBinding` requires a `param` block, but the `param` block can be empty.

PowerShell

```
Get-Command -Name Test-MrCmdletBinding -Syntax
```

Output

```
Test-MrCmdletBinding [[-ComputerName] <Object>] [<CommonParameters>]
```

Drilling down into the parameters property of `Get-Command` shows the actual parameter names, including the common ones.

PowerShell

```
(Get-Command -Name Test-MrCmdletBinding).Parameters.Keys
```

Output

```
ComputerName
Verbose
Debug
ErrorAction
WarningAction
InformationAction
ErrorVariable
WarningVariable
InformationVariable
OutVariable
OutBuffer
PipelineVariable
```

SupportsShouldProcess

The `SupportsShouldProcess` attribute adds the **WhatIf** and **Confirm** risk mitigation parameters. These parameters are only needed for commands that make changes.

PowerShell

```
function Test-MrSupportsShouldProcess {

    [CmdletBinding(SupportsShouldProcess)]
    param (
        $ComputerName
    )

    Write-Output $ComputerName

}
```

Notice that there are now **WhatIf** and **Confirm** parameters.

PowerShell

```
Get-Command -Name Test-MrSupportsShouldProcess -Syntax
```

Output

```
Test-MrSupportsShouldProcess [[-ComputerName] <Object>] [-WhatIf] [-Confirm]
[<CommonParameters>]
```

Once again, you can also use `Get-Command` to return a list of the actual parameter names, including the common, ones along with **WhatIf** and **Confirm**.

PowerShell

```
(Get-Command -Name Test-MrSupportsShouldProcess).Parameters.Keys
```

Output

```
ComputerName
Verbose
Debug
ErrorAction
WarningAction
InformationAction
ErrorVariable
WarningVariable
InformationVariable
OutVariable
OutBuffer
PipelineVariable
WhatIf
Confirm
```

Parameter validation

Validate input early on. Don't allow your code to continue on a path when it can't complete without valid input.

Always specify a datatype for the variables used for parameters. In the following example, **String** is specified as the datatype for the **ComputerName** parameter. This validation limits it to only allow a single computer name to be specified for the **ComputerName** parameter.

PowerShell

```
function Test-MrParameterValidation {

    [CmdletBinding()]
    param (
        [string]$ComputerName
    )

    Write-Output $ComputerName

}
```

An error is generated if more than one computer name is specified.

PowerShell

```
Test-MrParameterValidation -ComputerName Server01, Server02
```

Output

```
Test-MrParameterValidation : Cannot process argument transformation on
parameter 'ComputerName'. Cannot convert value to type System.String.
At line:1 char:42
+ Test-MrParameterValidation -ComputerName Server01, Server02
+                                          ~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidData: (:) [Test-MrParameterValidation]
   , ParameterBindingArgumentTransformationException
    + FullyQualifiedErrorId : ParameterArgumentTransformationError,Test-MrP
   arameterValidation
```

The problem with the current definition is that it's valid to omit the value of the **ComputerName** parameter, but a value is required for the function to complete successfully. This scenario is where the `Mandatory` parameter attribute is beneficial.

The syntax used in the following example is compatible with PowerShell version 3.0 and higher. `[Parameter(Mandatory=$true)]` could be specified to make the function compatible with PowerShell version 2.0 or higher.

PowerShell

```
function Test-MrParameterValidation {

    [CmdletBinding()]
    param (
        [Parameter(Mandatory)]
        [string]$ComputerName
    )

    Write-Output $ComputerName

}
```

Now that the **ComputerName** is required, if one isn't specified, the function prompts for one.

PowerShell

```
Test-MrParameterValidation
```

Output

```
cmdlet Test-MrParameterValidation at command pipeline position 1
Supply values for the following parameters:
ComputerName:
```

If you want to allow more than one value for the **ComputerName** parameter, use the **String** datatype but add square brackets (`[]`) to the datatype to allow an array of strings.

PowerShell

```
function Test-MrParameterValidation {

    [CmdletBinding()]
    param (
        [Parameter(Mandatory)]
        [string[]]$ComputerName
    )

    Write-Output $ComputerName

}
```

Maybe you want to specify a default value for the **ComputerName** parameter if one isn't specified. The problem is that default values can't be used with mandatory parameters. Instead, use the `ValidateNotNullOrEmpty` parameter validation attribute with a default value.

Even when setting a default value, try not to use static values. In the following example, `$env:COMPUTERNAME` is used as the default value, which is automatically translated to the local computer name if a value isn't provided.

PowerShell

```
function Test-MrParameterValidation {

    [CmdletBinding()]
    param (
        [ValidateNotNullOrEmpty()]
        [string[]]$ComputerName = $env:COMPUTERNAME
    )

    Write-Output $ComputerName

}
```

Verbose output

Inline comments are useful if you're writing complex code, but users don't see them unless they look at the code.

The function in the following example has an inline comment in the `foreach` loop. While this particular comment might not be difficult to locate, imagine if the function contained hundreds of lines of code.

PowerShell

```
function Test-MrVerboseOutput {

    [CmdletBinding()]
    param (
        [ValidateNotNullOrEmpty()]
        [string[]]$ComputerName = $env:COMPUTERNAME
    )

    foreach ($Computer in $ComputerName) {
        #Attempting to perform an action on $Computer <<-- Don't use
        #inline comments like this, use write verbose instead.
        Write-Output $Computer
    }
```

```
}
```

A better option is to use `Write-Verbose` instead of inline comments.

PowerShell

```powershell
function Test-MrVerboseOutput {

    [CmdletBinding()]
    param (
        [ValidateNotNullOrEmpty()]
        [string[]]$ComputerName = $env:COMPUTERNAME
    )

    foreach ($Computer in $ComputerName) {
        Write-Verbose -Message "Attempting to perform an action on $Computer"
        Write-Output $Computer
    }

}
```

The verbose output isn't displayed when the function is called without the **Verbose** parameter.

PowerShell

```powershell
Test-MrVerboseOutput -ComputerName Server01, Server02
```

The verbose output is displayed when the function is called with the **Verbose** parameter.

PowerShell

```powershell
Test-MrVerboseOutput -ComputerName Server01, Server02 -Verbose
```

Pipeline input

Extra code is necessary when you want your function to accept pipeline input. As mentioned earlier in this book, commands can accept pipeline input **by value** (by type) or **by property name**. You can write your functions like the native commands so they accept either one or both of these input types.

To accept pipeline input **by value**, specify the `ValueFromPipeline` parameter attribute for that particular parameter. You can only accept pipeline input **by value** from one parameter of each datatype. If you have two parameters that accept string input, only one of them can accept pipeline input **by value**. If you specified **by value** for both of the string parameters, the input

wouldn't know which parameter to bind to. This scenario is another reason I call this type of pipeline input *by type* instead of **by value**.

Pipeline input is received one item at a time, similar to how items are handled in a `foreach` loop. A `process` block is required to process each item if your function accepts an array as input. If your function only accepts a single value as input, a `process` block isn't necessary but is recommended for consistency.

PowerShell

```
function Test-MrPipelineInput {

    [CmdletBinding()]
    param (
        [Parameter(Mandatory,
                   ValueFromPipeline)]
        [string[]]$ComputerName
    )

    process {
        Write-Output $ComputerName
    }

}
```

Accepting pipeline input **by property name** is similar, except you specify it with the `ValueFromPipelineByPropertyName` parameter attribute, and it can be specified for any number of parameters regardless of datatype. The key is the output of the command being piped in must have a property name that matches the name of the parameter or a parameter alias of your function.

PowerShell

```
function Test-MrPipelineInput {

    [CmdletBinding()]
    param (
        [Parameter(Mandatory,
                   ValueFromPipelineByPropertyName)]
        [string[]]$ComputerName
    )

    process {
            Write-Output $ComputerName
    }

}
```

`begin` and `end` blocks are optional. `begin` is specified before the `process` block and is used to perform any initial work before the items are received from the pipeline. Values that are piped in aren't accessible in the `begin` block. The `end` block is specified after the `process` block and is used for cleanup after all items piped in are processed.

Error handling

The function shown in the following example generates an unhandled exception when a computer can't be contacted.

PowerShell

```
function Test-MrErrorHandling {

    [CmdletBinding()]
    param (
        [Parameter(Mandatory,
                   ValueFromPipeline,
                   ValueFromPipelineByPropertyName)]
        [string[]]$ComputerName
    )

    process {
        foreach ($Computer in $ComputerName) {
            Test-WSMan -ComputerName $Computer
        }
    }

}
```

There are a couple of different ways to handle errors in PowerShell. `Try/Catch` is the more modern way to handle errors.

PowerShell

```
function Test-MrErrorHandling {

    [CmdletBinding()]
    param (
        [Parameter(Mandatory,
                   ValueFromPipeline,
                   ValueFromPipelineByPropertyName)]
        [string[]]$ComputerName
    )

    process {
        foreach ($Computer in $ComputerName) {
            try {
```

```
                Test-WSMan -ComputerName $Computer
            }
            catch {
                Write-Warning -Message "Unable to connect to Computer: $Computer"
            }
        }
    }

}
```

Although the function shown in the previous example uses error handling, it generates an unhandled exception because the command doesn't generate a terminating error. Only terminating errors are caught. Specify the **ErrorAction** parameter with **Stop** as its value to turn a nonterminating error into a terminating one.

PowerShell

```
function Test-MrErrorHandling {

    [CmdletBinding()]
    param (
        [Parameter(Mandatory,
                   ValueFromPipeline,
                   ValueFromPipelineByPropertyName)]
        [string[]]$ComputerName
    )

    process {
        foreach ($Computer in $ComputerName) {
            try {
                Test-WSMan -ComputerName $Computer -ErrorAction Stop
            }
            catch {
                Write-Warning -Message "Unable to connect to Computer: $Computer"
            }
        }
    }

}
```

Don't modify the global `$ErrorActionPreference` variable unless absolutely necessary. If you change it in a local scope, it reverts to the previous value when you exit that scope.

If you're using something like .NET directly from within your PowerShell function, you can't specify the **ErrorAction** parameter on the command itself. You can change the `$ErrorActionPreference` variable just before you call the .NET method.

Comment-based help

Adding help to your functions is considered a best practice. Help allows people you share them with to know how to use them.

PowerShell

```powershell
function Get-MrAutoStoppedService {

<#
.SYNOPSIS
    Returns a list of services that are set to start automatically, are not
    currently running, excluding the services that are set to delayed start.

.DESCRIPTION
    Get-MrAutoStoppedService is a function that returns a list of services
    from the specified remote computer(s) that are set to start
    automatically, are not currently running, and it excludes the services
    that are set to start automatically with a delayed startup.

.PARAMETER ComputerName
    The remote computer(s) to check the status of the services on.

.PARAMETER Credential
    Specifies a user account that has permission to perform this action. The
    default is the current user.

.EXAMPLE
     Get-MrAutoStoppedService -ComputerName 'Server1', 'Server2'

.EXAMPLE
     'Server1', 'Server2' | Get-MrAutoStoppedService

.EXAMPLE
     Get-MrAutoStoppedService -ComputerName 'Server1' -Credential (Get-Credential)

.INPUTS
    String

.OUTPUTS
    PSCustomObject

.NOTES
    Author:  Mike F. Robbins
    Website: https://mikefrobbins.com
    Twitter: @mikefrobbins
#>

    [CmdletBinding()]
    param (

    )

    #Function Body
```

```
}
```

When you add comment-based help to your functions, help can be retrieved for them like the default built-in commands.

All the syntax for writing a function in PowerShell can seem overwhelming for someone getting started. If you can't remember the syntax for something, open a second instance of the PowerShell Integrated Scripting Environment (ISE) on a separate monitor and view the "Cmdlet (advanced function) - Complete" snippet while typing in the code for your functions. Snippets can be accessed in the PowerShell ISE using the `Ctrl` + `J` key combination.

Summary

In this chapter, you learned the basics of writing functions in PowerShell, including how to:

- Create advanced functions
- Use parameter validation
- Use verbose output
- Support pipeline input
- Handle errors
- Create comment-based help

Review

1. How do you obtain a list of approved verbs in PowerShell?
2. How do you turn a PowerShell function into an advanced function?
3. When should **WhatIf** and **Confirm** parameters be added to your PowerShell functions?
4. How do you turn a nonterminating error into a terminating one?
5. Why should you add comment-based help to your functions?

References

- about_Functions
- about_Functions_Advanced_Parameters
- about_CommonParameters
- about_Functions_CmdletBindingAttribute
- about_Functions_Advanced
- about_Try_Catch_Finally

- about_Comment_Based_Help
- Video: PowerShell Toolmaking with Advanced Functions and Script Modules

Next steps

In Chapter 10, you'll learn how to package functions into script modules. You'll explore module structure, manifests, exporting public commands, and best practices for organizing, sharing, and maintaining reusable PowerShell tooling.

Last updated on 02/06/2026

Chapter 10 - Script modules

If you find yourself using the same PowerShell one-liners or scripts often, turning them into reusable tools is even more important. Packaging your functions in a script module gives them a more professional feel and makes them easier to support and share with others.

Dot-sourcing functions

One thing we didn't cover in the previous chapter is dot-sourcing functions. When you define a function in a script but not part of a module, the only way to load it into memory is by dot-sourcing its `.ps1` file.

For example, save the following function in a file named `Get-MrPSVersion.ps1`.

PowerShell

```
function Get-MrPSVersion {
    $PSVersionTable
}
```

When you run the script, it appears that nothing happens.

PowerShell

```
.\Get-MrPSVersion.ps1
```

Attempting to call the function results in an error because it isn't loaded into memory.

PowerShell

```
Get-MrPSVersion
```

Output

```
Get-MrPSVersion : The term 'Get-MrPSVersion' is not recognized as the name
of a cmdlet, function, script file, or operable program. Check the spelling
of the name, or if a path was included, verify that the path is correct and
try again.
At line:1 char:1
+ Get-MrPSVersion
+ ~~~~~~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (Get-MrPSVersion:String) [],
```

```
    CommandNotFoundException
     + FullyQualifiedErrorId : CommandNotFoundException
```

You can confirm whether functions are loaded into memory by verifying their existence on the **Function:** PSDrive.

PowerShell

```powershell
Get-ChildItem -Path Function:\Get-MrPSVersion
```

Output

```
Get-ChildItem : Cannot find path 'Get-MrPSVersion' because it does not
exist.
At line:1 char:1
+ Get-ChildItem -Path Function:\Get-MrPSVersion
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (Get-MrPSVersion:String) [Get
   -ChildItem], ItemNotFoundException
    + FullyQualifiedErrorId : PathNotFound,Microsoft.PowerShell.Commands.Ge
   tChildItemCommand
```

The issue with running the script that defines the function is that it loads it into the **Script** scope. Once the script finishes executing, PowerShell discards that scope along with the function.

To keep the function available after the script runs, it needs to be loaded into the **Global** scope. You can accomplish this by dot-sourcing the script file. You can use a relative path for this purpose.

PowerShell

```powershell
. .\Get-MrPSVersion.ps1
```

You can also use the full path to the script when dot-sourcing it.

PowerShell

```powershell
. C:\Demo\Get-MrPSVersion.ps1
```

If part of the path is stored in a variable, you can combine it with the rest of the path. There's no need to use string concatenation to do this.

PowerShell

```
$Path = 'C:\'
. $Path\Get-MrPSVersion.ps1
```

Now, if you check the **Function** PSDrive, you see the `Get-MrPSVersion` function is available.

PowerShell

```
Get-ChildItem -Path Function:\Get-MrPSVersion
```

Output

```
CommandType     Name                                               Version
-----------     ----                                               -------
Function        Get-MrPSVersion
```

Script modules

In PowerShell, a script module is simply a `.psm1` file that contains one or more functions, just like a regular script, but with a different file extension.

How do you create a script module? You might assume with a command named something like `New-Module`. That assumption is a reasonable guess, but that command actually creates a dynamic module, not a script module.

This scenario is a good reminder to always read the help documentation, even when a command name looks exactly like what you need.

PowerShell

```
help New-Module
```

Output

```
NAME
    New-Module

SYNOPSIS
    Creates a new dynamic module that exists only in memory.


SYNTAX
    New-Module [-Name] <System.String> [-ScriptBlock]
    <System.Management.Automation.ScriptBlock> [-ArgumentList
    <System.Object[]>] [-AsCustomObject] [-Cmdlet <System.String[]>]
    [-Function <System.String[]>] [-ReturnResult] [<CommonParameters>]
```

```
DESCRIPTION
    The `New-Module` cmdlet creates a dynamic module from a script block.
    The members of the dynamic module, such as functions and variables, are
    immediately available in the session and remain available until you
    close the session.

    Like static modules, by default, the cmdlets and functions in a dynamic
    module are exported and the variables and aliases are not. However, you
    can use the Export-ModuleMember cmdlet and the parameters of
    `New-Module` to override the defaults.

    You can also use the **AsCustomObject** parameter of `New-Module` to return
    the dynamic module as a custom object. The members of the modules, such
    as functions, are implemented as script methods of the custom object
    instead of being imported into the session.

    Dynamic modules exist only in memory, not on disk. Like all modules,
    the members of dynamic modules run in a private module scope that is a
    child of the global scope. Get-Module cannot get a dynamic module, but
    Get-Command can get the exported members.

    To make a dynamic module available to `Get-Module`, pipe a `New-Module`
    command to Import-Module, or pipe the module object that `New-Module`
    returns to `Import-Module`. This action adds the dynamic module to the
    `Get-Module` list, but it does not save the module to disk or make it
    persistent.


RELATED LINKS
    Online Version: https://learn.microsoft.com/powershell/module/microsoft.
    powershell.core/new-module?view=powershell-5.1&WT.mc_id=ps-gethelp
    Export-ModuleMember
    Get-Module
    Import-Module
    Remove-Module
    about_Modules

REMARKS
    To see the examples, type: "Get-Help New-Module -Examples".
    For more information, type: "Get-Help New-Module -Detailed".
    For technical information, type: "Get-Help New-Module -Full".
    For online help, type: "Get-Help New-Module -Online"
```

The previous chapter mentioned that functions should use approved verbs. Otherwise, PowerShell generates a warning when the module is imported.

The following example uses the `New-Module` cmdlet to create a dynamic module in memory, specifically to demonstrate what happens when you don't use an approved verb.

PowerShell

```
New-Module -Name MyModule -ScriptBlock {

    function Return-MrOsVersion {
        Get-CimInstance -ClassName Win32_OperatingSystem |
        Select-Object -Property @{Label='OperatingSystem';Expression={$_.Caption}}
    }

    Export-ModuleMember -Function Return-MrOsVersion

} | Import-Module
```

Output

```
WARNING: The names of some imported commands from the module 'MyModule' include
unapproved verbs that might make them less discoverable. To find the commands with
unapproved verbs, run the Import-Module command again with the Verbose parameter.
For a
list of approved verbs, type Get-Verb.
```

Although you used the `New-Module` cmdlet in the previous example, as mentioned before, it's not the command for creating script modules in PowerShell.

To create a script module, save your functions in a `.psm1` file. For example, save the following two functions in a file named `MyScriptModule.psm1`.

PowerShell

```
function Get-MrPSVersion {
    $PSVersionTable
}

function Get-MrComputerName {
    $env:COMPUTERNAME
}
```

Try to run one of the functions.

PowerShell

```
Get-MrComputerName
```

When you call the function, you receive an error saying PowerShell can't find it. Like before, checking the **Function:** PSDrive confirms that it isn't loaded into memory.

Output

```
Get-MrComputerName : The term 'Get-MrComputerName' is not recognized as the
name of a cmdlet, function, script file, or operable program. Check the
spelling of the name, or if a path was included, verify that the path is
correct and try again.
At line:1 char:1
+ Get-MrComputerName
+ ~~~~~~~~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (Get-MrComputerName:String) [
   ], CommandNotFoundException
    + FullyQualifiedErrorId : CommandNotFoundException
```

To make the function available, you can manually import the `MyScriptModule.psm1` file using the `Import-Module` cmdlet.

PowerShell

```
Import-Module C:\MyScriptModule.psm1
```

PowerShell introduced module autoloading in version 3. To take advantage of this feature, the script module must be saved in a folder with the same base name as the `.psm1` file. That folder must be located in one of the directories specified in the `$env:PSModulePath` environment variable.

PowerShell

```
$env:PSModulePath
```

The output of `$env:PSModulePath` is difficult to read.

Output

```
C:\Users\mike-ladm\Documents\WindowsPowerShell\Modules;C:\Program Files\Wind
owsPowerShell\Modules;C:\Windows\system32\WindowsPowerShell\v1.0\Modules;C:\
Program Files (x86)\Microsoft SQL Server\130\Tools\PowerShell\Modules\
```

To make the results more readable, split the paths on the semicolon path separator so each one appears on its own line.

PowerShell

```
$env:PSModulePath -split ';'
```

The first three paths in the list are the default module locations. SQL Server Management Studio added the last path when you installed it.

Output

```
C:\Users\mike-ladm\Documents\WindowsPowerShell\Modules
C:\Program Files\WindowsPowerShell\Modules
C:\Windows\system32\WindowsPowerShell\v1.0\Modules
C:\Program Files (x86)\Microsoft SQL Server\130\Tools\PowerShell\Modules\
```

For module autoloading to work, you must place the `MyScriptModule.psm1` file must in a folder named `MyScriptModule`, and that folder must reside directly inside one of the paths listed in `$env:PSModulePath`.

Not all those paths are equally useful. For example, the current user path on my system isn't the first one in the list. That's because I sign in to Windows with a different account than the one I use to run PowerShell. So, it doesn't point to my user's documents folder.

The second path is the **AllUsers** path, which is where I store all of my modules.

The third path points to `C:\Windows\System32`, a protected system location. Only Microsoft should be placing modules there, as it falls under the operating system's directory structure.

Once you place the `.psm1` file in an appropriate folder within one of these paths, PowerShell automatically loads the module the first time you call one of its commands.

Module manifests

Every module should include a module manifest, which is a `.psd1` file containing metadata about the module. While the `.psd1` extension is used for manifests, not all `.psd1` files are module manifests. You can also use them for other purposes, such as defining environment data in a DSC
configuration.

You can create a module manifest using the `New-ModuleManifest` cmdlet. The only required parameter is **Path**, but for the module to work correctly, you must also specify the **RootModule** parameter.

It's a best practice to include values like **Author** and **Description**, especially if you plan to publish your module to a NuGet repository using **PowerShellGet**. These fields are required in that scenario.

One quick way to tell if a module lacks a manifest is to check its version.

PowerShell

```
Get-Module -Name MyScriptModule
```

A version number of `0.0` is a clear sign that the module lacks a manifest.

Output

```
ModuleType Version    Name                                ExportedCommands
---------- -------    ----                                ----------------
Script     0.0        MyScriptModule                      {Get-MrComputer...
```

You should include all recommended details when creating a module manifest to ensure your module is well-documented and ready for sharing or publishing.

PowerShell

```
$moduleManifestParams = @{
    Path =
"$env:ProgramFiles\WindowsPowerShell\Modules\MyScriptModule\MyScriptModule.psd1"
    RootModule = 'MyScriptModule'
    Author = 'Mike F. Robbins'
    Description = 'MyScriptModule'
    CompanyName = 'mikefrobbins.com'
}

New-ModuleManifest @moduleManifestParams
```

If you omit any values when initially creating the module manifest, you can add or update it later using the `Update-ModuleManifest` cmdlet. Avoid recreating the manifest with `New-ModuleManifest` once you create it, as doing so generates a new GUID.

Defining public and private functions

Sometimes, your module might include helper functions you don't want to expose to users. These private functions are used internally by other functions in the module but aren't exposed to users. There are a few ways to handle this scenario.

If you're not following best practices and only have a `.psm1` file without a module manifest, your only option is to control visibility using the `Export-ModuleMember` cmdlet. This option lets you explicitly define which functions should be exposed directly from within the `.psm1` script module file, keeping everything else private by default.

In the following example, only the `Get-MrPSVersion` function is exposed to users of your module, while the `Get-MrComputerName` function remains accessible internally to other functions

within the module.

PowerShell

```powershell
function Get-MrPSVersion {
    $PSVersionTable
}

function Get-MrComputerName {
    $env:COMPUTERNAME
}

Export-ModuleMember -Function Get-MrPSVersion
```

Determine what commands are available publicly in the **MyScriptModule** module.

PowerShell

```powershell
Get-Command -Module MyScriptModule
```

Output

```
CommandType     Name                                               Version
-----------     ----                                               -------
Function        Get-MrPSVersion                                    1.0
```

If you add a module manifest to your module, it's a best practice to explicitly list the functions you want to export in the **FunctionsToExport** section. This option gives you control over what you expose to users from the `.psd1` module manifest file.

PowerShell

```powershell
FunctionsToExport = 'Get-MrPSVersion'
```

You don't need to use both `Export-ModuleMember` in the `.psm1` file and the `FunctionsToExport` section in the module manifest. Either approach is enough on its own.

Summary

In this chapter, you learned how to turn your functions into a script module in PowerShell. You also explored best practices for creating script modules, including the importance of adding a module manifest to define metadata and manage exported commands.

Review

1. How do you create a script module in PowerShell?
2. Why is it important to use approved verbs for your function names?
3. How do you create a module manifest in PowerShell?
4. What are the two ways to export only specific functions from a module?
5. What conditions must be met for a module to autoload when you run one of its commands?

References

- How to Create PowerShell Script Modules and Module Manifests
- about_Modules
- New-ModuleManifest
- Export-ModuleMember

Last updated on 11/06/2025

Appendix - Answers to Review Questions

This appendix provides concise answers to the review questions found at the end of each chapter. Use it to validate your understanding and reinforce key concepts.

Chapter 1 - Getting started with PowerShell

1. Use the `$PSVersionTable` automatic variable.
2. Only when you need to bypass User Account Control (UAC) for tasks that require elevation on the local computer.
3. The default execution policy on Windows client systems is `Restricted`, which prevents running scripts.
4. Use `Get-ExecutionPolicy` to determine the current execution policy.
5. Use `Set-ExecutionPolicy` (for example, `Set-ExecutionPolicy -ExecutionPolicy RemoteSigned`).

Chapter 2 - The Help system

1. No. The **DisplayName** parameter of `Get-Service` is named, not positional.
2. `Get-Process` has six parameter sets.
3. Use `Get-Command -Noun EventLog`.
4. Use `Get-Process -Name powershell`.
5. Run `Update-Help` (elevated as an administrator in Windows PowerShell) to download and install the latest help content.

Chapter 3 - Discovering objects, properties, and methods

1. `Get-Process` produces a `System.Diagnostics.Process` object.
2. Pipe the command to `Get-Member`.
3. Check whether the object has a method that performs the action.
4. Use the command's `PassThru` parameter, if it has one.
5. Run the command once and store the results in a variable to avoid repeatedly generating large amounts of output while prototyping.

Chapter 4 - One-Liners and the pipeline

1. A PowerShell one-liner is a single continuous pipeline, regardless of how many physical lines it spans.
2. Natural line breaks can occur at characters such as the pipe (`|`), comma (`,`), opening brackets (`[ ]`), braces (`{ }`), and parentheses (`( )`).
3. You should filter left to improve performance and efficiency by reducing the amount of data passed through the pipeline.
4. A command can accept pipeline input by value (by type) or by property name.
5. Because most content in the PowerShell Gallery is community-contributed and may not be vetted, it should be reviewed and tested before use.

Chapter 5 - Formatting, aliases, providers, comparison

1. Because formatting cmdlets produce format objects, which break the pipeline and can't be used by most other commands.
2. Use `Get-Alias -Name %` to determine the actual cmdlet.
3. Because aliases reduce readability and portability, making scripts more difficult for others to understand.
4. Use `Get-ChildItem -Path HKLM:\, HKCU:\` to list registry keys in both hives.
5. The `-replace` operator is case-insensitive by default, whereas the `.Replace()` method is case-sensitive.

Chapter 6 - Flow control

1. `ForEach-Object` processes items one at a time from the pipeline (streaming), while the `foreach` statement processes items from a collection that's already loaded into memory.
2. A `while` loop evaluates its condition before running, so it may not run at all if the condition is false, unlike `do while` and `do until`, which run at least once.
3. `break` exits the loop entirely, while `continue` skips the current iteration and proceeds to the next one.

Chapter 7 - Working with WMI

1. WMI cmdlets (for example, `Get-WmiObject`) are older and use DCOM, while CIM cmdlets (for example, `Get-CimInstance`) are newer and use WSMan by default.
2. WSMan (Windows Remote Management).
3. CIM sessions allow reuse of connections, support alternate credentials, improve performance, and simplify managing multiple remote connections.

4. Create a session option with `New-CimSessionOption`, for example, to use DCOM, and pass it to `New-CimSession`, then use that session with `Get-CimInstance`.
5. Use `Remove-CimSession`.

Chapter 8 - PowerShell remoting

1. Use `Enable-PSRemoting`.
2. Use `Enter-PSSession`.
3. It allows you to use a persistent session instead of specifying the computer name and credentials with each command.
4. Yes, you can use a PowerShell session (PSSession) in a one-to-one interactive remoting scenario.
5. Locally run cmdlets return live objects with methods, while remote commands return deserialized objects without methods.

Chapter 9 - Functions

1. Use `Get-Verb`.
2. Add the `[CmdletBinding()]` attribute to the function.
3. When the function makes changes to system state or performs potentially impactful actions.
4. Specify `-ErrorAction Stop`.
5. To document how to use the function so you and others can easily understand it and access help with `Get-Help`.

Chapter 10 - Script modules

1. Create a `.psm1` file and place your functions in it.
2. Using approved verbs ensures consistency, avoids warnings, and improves discoverability.
3. Use `New-ModuleManifest`.
4. Use `Export-ModuleMember` in the `.psm1` file or specify functions in the `FunctionsToExport` field of the `.psd1` file.
5. The module must be in a folder named the same as the module, located in a path listed in `$env:PSModulePath`, and contain the appropriate module file (`.psm1` or manifest).

Final Notes

- These answers are intentionally concise to reinforce key concepts.
- Revisit the chapters for deeper understanding.

Last updated on 05/04/2026

Optimizing your shell experience

PowerShell is a command-line shell and a scripting language used for automation.

Wikipedia includes the following description of a shell:

> A shell manages the user-system interaction by prompting users for input, interpreting their input, and then handling output from the underlying operating system (much like a read-eval-print loop or REPL).

Similar to other shells like `bash` or `cmd.exe`, PowerShell allows you to run any command available on your system, not just PowerShell commands.

PowerShell commands are known as *cmdlets* (pronounced command-lets). Cmdlets are PowerShell commands, not stand-alone executables. PowerShell commands can't be run in other shells without running PowerShell first.

Features of the PowerShell command-line interface

PowerShell is a modern command shell that includes the best features of other popular shells. Unlike most shells that only accept and return text, PowerShell accepts and returns .NET objects. The shell has several features that you can use to optimize your interactive user experience.

- Robust command-line history
- Tab completion and command prediction
- Supports command and parameter aliases
- Pipeline for chaining commands
- In-console help system, similar to Unix `man` pages

Last updated on 11/21/2025

Running commands in the shell

PowerShell is a command-line shell and a scripting language used for automation. Similar to other shells, like `bash` on Linux or the Windows Command Shell (`cmd.exe`), PowerShell lets you run any command available on your system, not just PowerShell commands.

Types of commands

For any shell in any operating system there are three types of commands:

- **Shell language keywords** are part of the shell's scripting language.
  - Examples of `bash` keywords include: `if`, `then`, `else`, `elif`, and `fi`.
  - Examples of `cmd.exe` keywords include: `dir`, `copy`, `move`, `if`, and `echo`.
  - Examples of PowerShell keywords include: `for`, `foreach`, `try`, `catch`, and `trap`.

  Shell language keywords can only be used within the runtime environment of the shell. There's no executable file, external to the shell, that provides the keyword's functionality.

- **OS-native commands** are executable files installed in the operating system. The executables can be run from any command-line shell, like PowerShell. This includes script files that require other shells to work properly. For example, if you run a Windows batch script (`.cmd` file) in PowerShell, PowerShell runs `cmd.exe` and passes in the batch file for execution.

- **Shell environment-specific commands** are commands defined in external files that can only be used within the runtime environment of the shell. This include scripts, functions, and modules that add commands to the shell runtime. In PowerShell, these commands added by a module are known as *cmdlets* (pronounced "command-lets").

Running native commands

Any native command can be run from the PowerShell command line. Usually you run the command exactly as you would in `bash` or `cmd.exe`. The following example shows running the `grep` command in `bash` on Ubuntu Linux.

Bash

```
sdwheeler@circumflex:~$ grep sdwheeler /etc/passwd
sdwheeler:x:1000:1000:,,,:/home/sdwheeler:/bin/bash
sdwheeler@circumflex:~$ pwsh
PowerShell 7.2.6
```

```
Copyright (c) Microsoft Corporation.

https://aka.ms/powershell
Type 'help' to get help.
```

After starting PowerShell on Ubuntu, you can run the same command from the PowerShell command line:

PowerShell

```
PS /home/sdwheeler> grep sdwheeler /etc/passwd
sdwheeler:x:1000:1000:,,,:/home/sdwheeler:/bin/bash
```

Passing arguments to native commands

Most shells include features for using variables, evaluating expressions, and handling strings. But each shell does these things differently. In PowerShell, all parameters start with a hyphen (`-`) character. In `cmd.exe`, most command parameters use a forward slash (`/`) character. Other command-line tools might use spaces, hyphens, double-dash (`--`).

Each shell has its own way of handling and evaluating strings on the command line. When running native commands in PowerShell that expect strings to be quoted in a specific way, you might need to adjust how you pass those strings.

For more information, see the following articles:

- about_Parsing
- about_Quoting_Rules

PowerShell 7.2 introduced the experimental feature `PSNativeCommandArgumentPassing` that improved native command handling. PowerShell 7.3 made this feature mainstream. For more information, see $PSNativeCommandArgumentPassing.

Handling output and errors

PowerShell also has several more output streams than other shells. The `bash` and `cmd.exe` shells have **stdout** and **stderr**. PowerShell has six output streams. For more information, see about_Redirection and about_Output_Streams.

In general, the output sent to **stdout** by a native command is sent to the **Success** stream in PowerShell. Output sent to **stderr** by a native command is sent to the **Error** stream in PowerShell.

When a native command has a nonzero exit code, `$?` is set to `$false`. If the exit code is zero, `$?` is set to `$true`.

However, PowerShell 7.2 changed this behavior. Error records redirected from native commands, like when using redirection operators (`2>&1`), aren't written to PowerShell's `$Error` variable and the preference variable `$ErrorActionPreference` doesn't affect the redirected output.

Many native commands write to **stderr** as an alternative stream for additional information. This behavior can cause confusion in PowerShell when looking through errors and the additional output information can be lost if `$ErrorActionPreference` is set to a state that mutes the output.

PowerShell 7.3 added a new experimental feature `PSNativeCommandErrorActionPreference` that allows you to control how you handle non-zero exit codes from native commands. For more information, see $PSNativeCommandUseErrorActionPreference.

Running PowerShell commands

As previously noted, PowerShell commands are known as cmdlets. Cmdlets are collected into PowerShell modules that can be loaded on demand. Cmdlets can be written in any compiled .NET language or using the PowerShell scripting language itself.

PowerShell commands that run other commands

The PowerShell **call operator** (`&`) lets you run commands that are stored in variables and represented by strings or script blocks. You can use the operator to run any native command or PowerShell command. This is useful in a script when you need to dynamically construct the command-line parameters for a native command. For more information, see the call operator.

The `Start-Process` cmdlet can be used to run native commands, but should only be used when you need to control how the command is executed. The cmdlet has parameters to support the following scenarios:

- Run a command using different credentials
- Hide the console window created by the new process
- Redirect **stdin**, **stdout**, and **stderr** streams
- Use a different working directory for the command

The following example runs the native command `sort.exe` with redirected input and output streams.

```
```

PowerShell

```
$processOptions = @{
    FilePath = "sort.exe"
    RedirectStandardInput = "TestSort.txt"
    RedirectStandardOutput = "Sorted.txt"
    RedirectStandardError = "SortError.txt"
    UseNewEnvironment = $true
}
Start-Process @processOptions
```

For more information, see Start-Process.

On Windows, the `Invoke-Item` cmdlet performs the default action for the specified item. For example, it runs an executable file or opens a document file using the application associated with the document file type. The default action depends on the type of item and the PowerShell provider that provides access to the item.

The following example opens the PowerShell source code repository in your default web browser.

PowerShell

```
Invoke-Item https://github.com/PowerShell/PowerShell
```

For more information, see Invoke-Item.

Last updated on 06/29/2026

Using tab-completion in the shell

Summarize this article for me

PowerShell provides completions on input to provide hints, enable discovery, and speed up input entry. Command names, parameter names, argument values, and file paths can all be completed by pressing the `Tab` key.

The `Tab` key is the default key binding on Windows. **PSReadLine** also provides a `MenuComplete` function bound to `Ctrl` + `Space`. The `MenuComplete` function displays a list of matching completions below the command line.

The keybindings can be changed using **PSReadLine** cmdlets or the application that's hosting PowerShell. Keybindings can be different on non-Windows platforms. For more information, see about_PSReadLine_Functions.

Built-in tab completion features

PowerShell enables tab completion for many aspects of the command line experience.

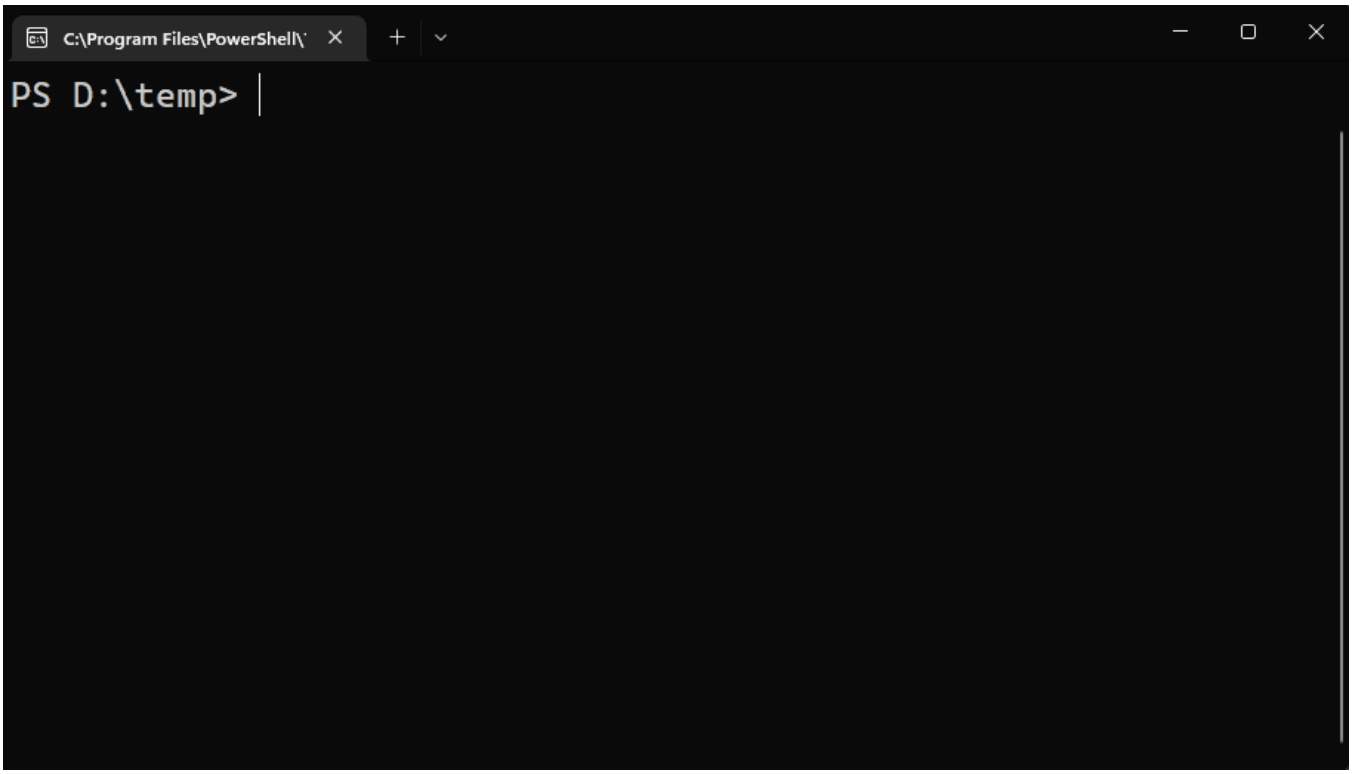
Filename completion

To fill in a filename or path from the available choices automatically, type part of the name and press the `Tab` key. PowerShell automatically expands the name to the first match that it finds. Pressing the `Tab` key again cycles through all the available choices with each key press.

Command and parameter name completion

The tab expansion of cmdlet names is slightly different. To use tab expansion on a cmdlet name, type the entire first part of the name (the verb) and the hyphen that follows it. You can fill in more of the name for a partial match. For example, if you type `get-co` and then press the `Tab` key, PowerShell automatically expands what you typed to the `Get-Command` cmdlet (notice that it also changes the case of letters to their standard form). If you press `Tab` key again, PowerShell replaces it with the next matching cmdlet, `Get-Content`. Tab completion also works to resolve PowerShell aliases and native executables.

The following graphic shows examples of tab and menu completion.



Other tab completion enhancements

Each new version of PowerShell includes improvements to tab completion that fix bugs and improve usability.

PowerShell 7.0

- Tab completion resolves variable assignments that are enums or are type constrained
- Tab completion expands abbreviated cmdlets and functions. For example, `i-psdf<tab>` returns `Import-PowerShellDataFile`

PowerShell 7.2

- Fix tab completion for unlocalized `about*` articles
- Fix splatting being treated as positional parameter in completions
- Add completions for Comment-based Help keywords
- Add completion for `#Requires` statements
- Add tab completion for **View** parameter of `Format-*` cmdlets
- Add support for class-based argument completers

PowerShell 7.3

- Fix tab completion within the script block specified for the `ValidateScriptAttribute`
- Added tab completion for loop labels after `break` and `continue`
- Improve Hashtable completion in multiple scenarios
  - Parameter splatting

    - **Arguments** parameter for `Invoke-CimMethod`
    - **FilterHashtable** parameter for `Get-WinEvent`
    - **Property** parameter for the CIM cmdlets
    - Removes duplicates from member completion scenarios
- Support forward slashes in network share (UNC path) completion
- Improve member auto completion
- Prioritize `ValidateSet` completions over enums for parameters
- Add type inference support for generic methods with type parameters
- Improve type inference and completions
    - Allows methods to be shown in completion results for `ForEach-Object -MemberName`
    - Prevents completion on expressions that return **void** like (`[void]""`)
    - Allows nondefault Class constructors to show up when class completion is based on the AST

Other ways to enhance tab completion of command parameters

Built-in tab expansion is controlled by the internal function **TabExpansion** or **TabExpansion2**. It's possible to create functions or modules that replace the default behavior of these functions. You can find examples in the PowerShell Gallery by searching for the TabExpansion keyword.

Using the `ValidateSet` or `ArgumentCompletions` attributes with parameters

The `ArgumentCompletions` attribute allows you to add tab completion values to a specific parameter. The `ArgumentCompletions` attribute is similar to `ValidateSet`. Both attributes take a list of values to be presented when the user presses `Tab` after the parameter name. However, unlike `ValidateSet`, the values aren't validated.

For more information, see:

- ArgumentCompletions
- ValidateSet

Using the `ArgumentCompleter` attribute or `Register-ArgumentCompleter` with parameters

An argument completer is a script block or function that provides dynamic tab completion for parameter values.

The `ArgumentCompleter` attribute allows you to register a function that provides tab completion values to for the parameter. The argument completer function must be available to the function containing the parameter with the `ArgumentCompleter` attribute. Usually, the function is defined in the same script or module. For more information, see ArgumentCompleter.

The `Register-ArgumentCompleter` cmdlet registers a script block as an argument completer function at run time for any command you specify. Using `Register-ArgumentCompleter` allows you to define argument completers outside of the script or module or for native commands. For more information, see Register-ArgumentCompleter.

Predictive IntelliSense in PSReadLine

PSReadLine 2.1.0 introduced the **Predictive IntelliSense** feature. Predictive IntelliSense provides suggestions for full commands based on items from your **PSReadLine** history.

PSReadLine 2.2.2 extends the power of Predictive IntelliSense by adding support for plug-in modules that use advanced logic to provide suggestions for full commands. The **Az.Tools.Predictor** module was the first plug-in for Predictive IntelliSense. It uses Machine Learning to predict what Azure PowerShell command you want to run and the parameters you want to use.

Prediction suggestions aren't tab-completion suggestions. They're generated from a different source. By default, prediction suggestions appear in gray text following the cursor. Use the `RightArrow` key to accept the suggestion. If you hit the `Tab` key instead, the suggestion will be ignored and the next tab-completion option is offered.

For more information, see Using predictors.

Last updated on 03/04/2026

Using predictors in PSReadLine

PSReadLine 2.1.0 introduced the **Predictive IntelliSense** feature. Predictive IntelliSense provides suggestions for full commands based on items from your **PSReadLine** history. **PSReadLine** 2.2.2 extends the power of Predictive IntelliSense by adding support for plug-in modules that use advanced logic to provide suggestions for full commands. The latest version, **PSReadLine** 2.2.6, enables predictions by default.

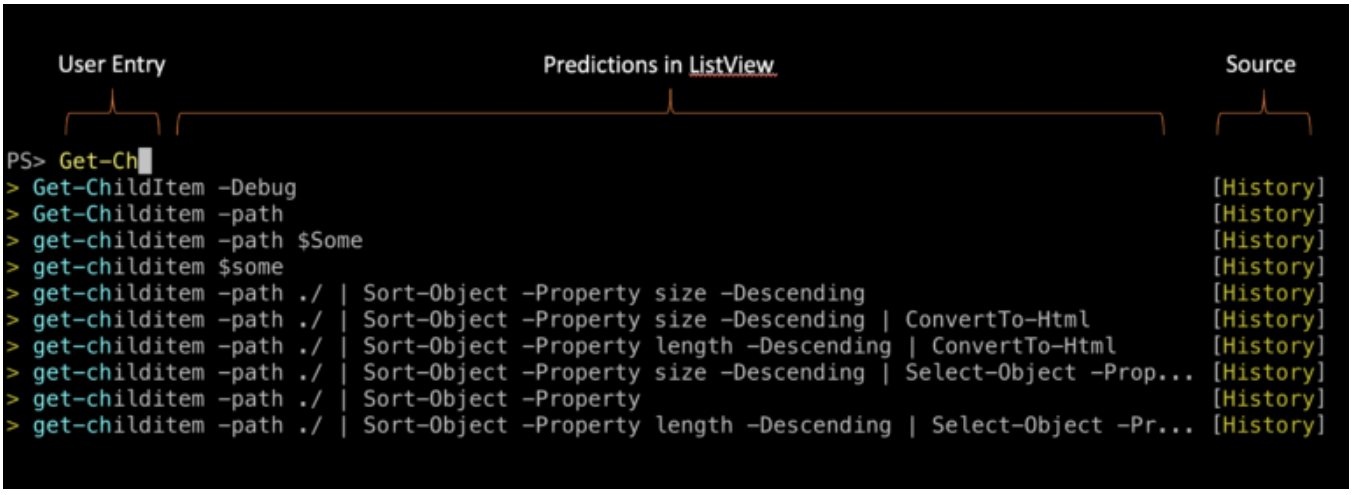
Using Predictive IntelliSense

When Predictive IntelliSense is enabled, the prediction suggestion appears as colored text following the user's cursor. The suggestions from Predictive IntelliSense help new and experienced users of PowerShell discover, edit, and execute full commands based on matching predictions. Suggestions can come from the user's history and other domain-specific plugins.



The previous image shows the default `InlineView` of the suggestion. Pressing `RightArrow` key accepts an inline suggestion. After accepting the suggestion, you can edit the command line before hitting `Enter` to run the command.

PSReadLine also offers a `ListView` presentation of the suggestions.



When in the list view, you can use the arrow keys to scroll through the available suggestions. List view also shows the source of the prediction.

PSReadLine defaults to `InlineView`. You can switch between `InlineView` and `ListView` by pressing the `F2` key. You can also use the **PredictionViewStyle** parameter of `Set-PSReadLineOption` to change the view.

Managing Predictive IntelliSense

To use Predictive IntelliSense, you must have a newer version of **PSReadLine** installed. For best results, install the latest version of the module.

Install **PSReadLine** using the Microsoft.PowerShell.PSResourceGet module:

```PowerShell
Install-PSResource -Name PSReadLine
```

PSReadLine can be installed in Windows PowerShell 5.1 or in PowerShell 7 or higher. To use predictor plug-ins, you must be running in PowerShell 7.2 or higher. Windows PowerShell 5.1 can use the history-based predictor.

In **PSReadLine** 2.2.6, Predictive IntelliSense is enabled by default depending on the following conditions:

- If Virtual Terminal (VT) is supported and **PSReadLine** running in PowerShell 7.2 or higher, **PredictionSource** is set to `HistoryAndPlugin`
- If VT is supported and **PSReadLine** running in PowerShell older than 7.2, **PredictionSource** is set to `History`
- If VT isn't supported, **PredictionSource** is set to `None`.

Use the following command to see the current setting:

```PowerShell
Get-PSReadLineOption | Select-Object -Property PredictionSource
```

You can change the prediction source using the `Set-PSReadLineOption` cmdlet with the **PredictionSource** parameter. The **PredictionSource** can be set to:

- `None`
- `History`
- `Plugin`
- `HistoryAndPlugin`

> **Note**
>
> History-based predictions come from the history maintained by **PSReadLine**. That history is more comprehensive than the session-based history you can see using `Get-History`. For more information, see **Command history** section of about_PSReadLine.

Setting the prediction color

By default, predictions appear in light grey text on the same line the user is typing. To support accessibility needs, you can customize the prediction color. Colors are defined using ANSI escape sequences. You can use `$PSStyle` to compose ANSI escape sequences.

PowerShell

```
Set-PSReadLineOption -Colors @{ InlinePrediction = $PSStyle.Background.Blue }
```

Or you can create your own. The default light-grey prediction text color can be restored using the following ANSI escape sequence.

PowerShell

```
Set-PSReadLineOption -Colors @{ InlinePrediction = "`e[38;5;238m" }
```

For more information about setting prediction color and other PSReadLine settings, see Set-PSReadLineOption.

Changing keybindings

PSReadLine contains functions to navigate and accept predictions. For example:

- `AcceptSuggestion` - Accept the current inline suggestion
- `AcceptNextSuggestionWord` - Accept the next word of the inline suggestion
- `AcceptSuggestion` is built within `ForwardChar`, which is bound to `RightArrow` by default
- `AcceptNextSuggestionWord` is built within the function `ForwardWord`, which can be bound to `Ctrl` + `f`

You can use the `Set-PSReadLineKeyHandler` cmdlet to change key bindings.

PowerShell

```
Set-PSReadLineKeyHandler -Chord "Ctrl+f" -Function ForwardWord
```

With this binding, pressing `Ctrl` + `f` accepts the next word of an inline suggestion when the cursor is at the end of current editing line. You can bind other keys to `AcceptSuggestion` and `AcceptNextSuggestionWord` for similar functionalities. For example, you might want to make `RightArrow` accept the next word of the inline suggestion, instead of the whole suggestion line.

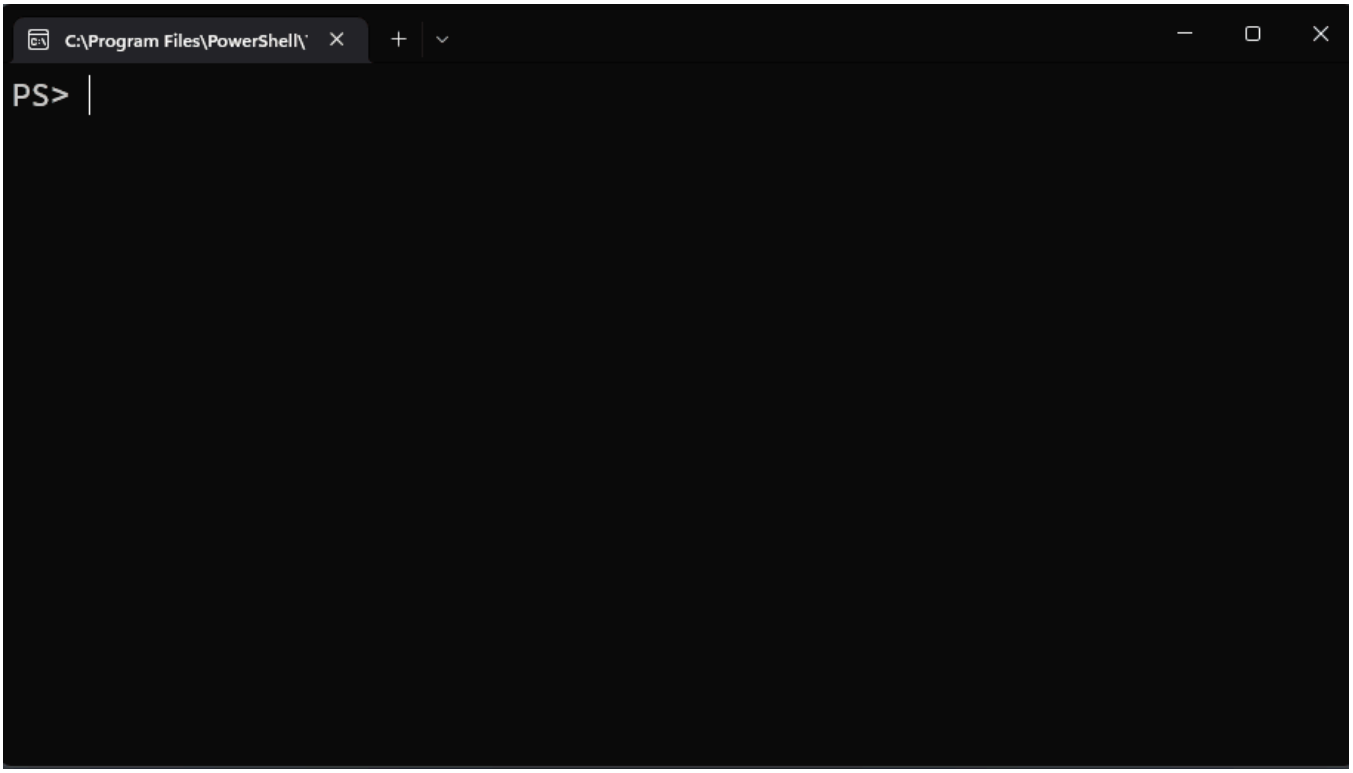
PowerShell

```
Set-PSReadLineKeyHandler -Chord "RightArrow" -Function ForwardWord
```

Using other predictor plug-ins

The **Az.Tools.Predictor** module was the first plug-in for Predictive IntelliSense. It uses Machine Learning to predict what Azure PowerShell command you want to run and the parameters you want to use. For more information and installation instructions, see Announcing General Availability of Az.Tools.Predictor.

The **CompletionPredictor** module adds an IntelliSense experience for anything that can be tab-completed in PowerShell. With **PSReadLine** set to `InlineView`, you get the normal tab completion experience. When you switch to `ListView`, you get the IntelliSense experience. You can install the CompletionPredictor module from the PowerShell Gallery.



As previously noted, `ListView` shows you the source of the prediction. If you have multiple plug-ins installed, the predictions are grouped by source with **History** listed first followed by each plug-in in the order that they were loaded.

Creating your own predictor module

You can write your own predictor using C# to create a compiled PowerShell module. The module must implement the **System.Management.Automation.Subsystem.Prediction.ICommandPredictor** interface. This interface declares the methods used to query for prediction results and provide feedback.

For more information, see How to create a command-line predictor.

Last updated on 04/07/2026

Using dynamic help

Dynamic Help provides just-in-time help that allows you to stay focused on your work without losing your place typing on the command line.

Getting cmdlet help

Dynamic Help provides a view of full cmdlet help shown in an alternative screen buffer. **PSReadLine** maps the function `ShowCommandHelp` to the `F1` key.

- When the cursor is at the end of a fully expanded cmdlet name, pressing `F1` displays the help for that cmdlet.
- When the cursor is at the end of a fully expanded parameter name, pressing `F1` displays the help for the cmdlet beginning at the parameter.



The pager in **PSReadLine** allows you to scroll the displayed help using the up and down arrow keys. Pressing `Q` exits the alternative screen buffer and returns to the current cursor position on the command line on the primary screen.

Getting focused parameter help

Pressing `Alt` + `h` provides dynamic help for parameters. The help is shown below the current command line similar to MenuComplete. The cursor must be at the end of the fully expanded parameter name when you press the `Alt` + `h` key.

```
PS>
PS> Get-ChildItem -path

-Path <System.String[]>

DESC: Specifies a path to one or more locations. Wildcards are accepted. The default
 location is the current directory (`.`).
Required: false, Position: 0, Default Value: Current directory, Pipeline Input: True
 (ByPropertyName, ByValue), WildCard: true
```

Select arguments on the command line

To quickly select and edit the arguments of a cmdlet without disturbing your syntax, use `Alt` + `a`. It searches from the current cursor position and stops when it finds an argument on the command line. It selects the full argument, making it simple to replace or edit the argument.

```
PS>
PS> get-childitem -path ./ | Sort-Object -Property length -Descending | format-table
 -AutoSize
```

Choosing keybindings

Not all keybindings work for all operating systems and terminal applications. For example, keybindings for the `Alt` key don't work on macOS by default. On Linux, `Ctrl` + `[` is the same as `Escape`. And `Ctrl` + `Spacebar` generates a `Control` + `2` key sequence instead of the `Control` + `Spacebar` sequence expected.

To work around these quirks, map the PSReadLine function to an available key combination. For example:

PowerShell

```
Set-PSReadLineKeyHandler -Chord 'Ctrl+l' -Function ShowParameterHelp
Set-PSReadLineKeyHandler -Chord 'Ctrl+k' -Function SelectCommandArgument
```

For more information about keybindings and workarounds, see Using PSReadLine key handlers.

Last updated on 11/21/2025

Using aliases

An alias is an alternate name or shorthand name for a cmdlet or for a command element, such as a function, script, file, or executable file. You can run the command using the alias instead of the executable name.

Managing command aliases

PowerShell provides cmdlets for managing command aliases. The following command shows the cmdlets that manage aliases.

PowerShell

```
Get-Command -Noun Alias
```

Output

```
CommandType Name         Version Source
----------- ----         ------- ------
Cmdlet      Export-Alias 7.0.0.0 Microsoft.PowerShell.Utility
Cmdlet      Get-Alias    7.0.0.0 Microsoft.PowerShell.Utility
Cmdlet      Import-Alias 7.0.0.0 Microsoft.PowerShell.Utility
Cmdlet      New-Alias    7.0.0.0 Microsoft.PowerShell.Utility
Cmdlet      Remove-Alias 7.0.0.0 Microsoft.PowerShell.Utility
Cmdlet      Set-Alias    7.0.0.0 Microsoft.PowerShell.Utility
```

For more information, see about_Aliases.

Use the Get-Alias cmdlet to list the aliases available in your environment. To list the aliases for a single cmdlet, use the **Definition** parameter and specify the executable name.

PowerShell

```
Get-Alias -Definition Get-ChildItem
```

Output

```
CommandType     Name
-----------     ----
Alias           dir -> Get-ChildItem
Alias           gci -> Get-ChildItem
Alias           ls -> Get-ChildItem
```

To get the definition of a single alias, use the **Name** parameter.

PowerShell

```
Get-Alias -Name gci
```

Output

```
CommandType     Name
-----------     ----
Alias           gci -> Get-ChildItem
```

To create an alias, use the `Set-Alias` command. You can create aliases for cmdlets, functions, scripts, and native executables files.

PowerShell

```
Set-Alias -Name np -Value Notepad.exe
Set-Alias -Name cmpo  -Value Compare-Object
```

Compatibility aliases in Windows

PowerShell has several aliases that allow **Unix** and `cmd.exe` users to use familiar commands in Windows. The following table shows common commands, the related PowerShell cmdlet, and the PowerShell alias:

Expand table

Windows Command Shell	Unix command	PowerShell cmdlet	PowerShell alias
`cd`, `chdir`	`cd`	`Set-Location`	`sl`, `cd`, `chdir`
`cls`	`clear`	`Clear-Host`	`cls` `clear`
`copy`	`cp`	`Copy-Item`	`cpi`, `cp`, `copy`
`del`, `erase`, `rd`, `rmdir`	`rm`	`Remove-Item`	`ri`, `del`, `erase`, `rd`, `rm`, `rmdir`
`dir`	`ls`	`Get-ChildItem`	`gci`, `dir`, `ls`
`echo`	`echo`	`Write-Output`	`write` `echo`
`md`	`mkdir`	`New-Item`	`ni`
`move`	`mv`	`Move-Item`	`mi`, `move`, `mv`
`popd`	`popd`	`Pop-Location`	`popd`
	`pwd`	`Get-Location`	`gl`, `pwd`, `$PWD`

Windows Command Shell	Unix command	PowerShell cmdlet	PowerShell alias
`pushd`	`pushd`	`Push-Location`	`pushd`
`ren`	`mv`	`Rename-Item`	`rni`, `ren`
`type`	`cat`	`Get-Content`	`gc`, `cat`, `type`

> **Note**
>
> The aliases in this table are Windows-specific. Some aliases aren't available on other platforms to allow an existing native command to work in a PowerShell session. For example, `ls` isn't defined as a PowerShell alias on macOS or Linux so that PowerShell runs the native command instead of `Get-ChildItem`.

Creating alternate names for commands with parameters

You can assign an alias to a cmdlet, script, function, or executable file. Unlike some Unix shells, the definition of the alias can't include parameters. For example, you can assign an alias to the `Get-Eventlog` cmdlet, but you can't assign an alias to the `Get-Eventlog -LogName System` command. You must create a function that contains the command with parameters.

For more information, see about_Aliases.

Parameter aliases and shorthand names

PowerShell also provides ways to create shorthand names for parameters. Parameter aliases are defined using the `Alias` attribute when you declare the parameter. Parameter aliases can't be defined using the `*-Alias` cmdlets.

For more information, see the Alias attribute documentation.

In addition to parameter aliases, PowerShell lets you specify the parameter name using the fewest characters needed to uniquely identify the parameter. For example, the `Get-ChildItem` cmdlet has the **Filter** and **Force** parameters. Using `-F` is ambiguous because both parameters start with the letter `F`. If you try to use `-F`, PowerShell returns an error:

PowerShell

```
PS> Get-ChildItem -f
```

Output

```
Get-ChildItem : Parameter cannot be processed because the parameter name 'f' is
ambiguous. Possible
matches include: -Filter -Force.
```

To uniquely identify the **Filter** parameter, you need to use `-Fi`.

> **Note**
>
> The `Get-ChildItem` cmdlet also has a dynamic parameter, `-File`. Using `-f`, `-fi` and `-fil` doesn't create ambiguity with `-File` because `-File` is a dynamic `FileSystem` provider parameter. PowerShell binds dynamic parameters after binding static parameters. The only way to specify the parameter is to use the full name, `-File`.

Don't use aliases in scripts

Aliases are a convenience feature to be used interactively in the shell. You should always use the full command and parameter names in your scripts.

- You can delete or redefine aliases in a profile script
- Aliases you define in your profile aren't be available to other users
- Aliases make your code harder to read and maintain

Last updated on 01/13/2026

Customizing your shell environment

A PowerShell profile is a script that runs when PowerShell starts. You can use the profile to customize the environment. You can:

- Add aliases, functions, and variables
- Load modules
- Create PowerShell drives
- Run arbitrary commands
- Change preference settings

Putting these settings in your profile ensures that they're available whenever you start PowerShell on your system.

> **Note**
>
> To run scripts in Windows, the PowerShell execution policy needs to be set to `RemoteSigned` at a minimum. Execution policies don't apply to macOS and Linux. For more information, see about_Execution_Policy.

The $PROFILE variable

The `$PROFILE` automatic variable stores the paths to the PowerShell profiles that are available in the current session.

There are four possible profiles available to support different user scopes and different PowerShell hosts. The fully qualified paths for each profile script are stored in the following member properties of `$PROFILE`.

- **AllUsersAllHosts**
- **AllUsersCurrentHost**
- **CurrentUserAllHosts**
- **CurrentUserCurrentHost**

You can create profile scripts that run for all users or just one user, the **CurrentUser**. **CurrentUser** profiles are stored under the user's home directory path. The location varies depending on the operating system and the version of PowerShell you use.

By default, referencing the `$PROFILE` variable returns the path to the "Current User, Current Host" profile. The other profiles path can be accessed through the properties of the `$PROFILE` variable. The following command shows the default profile locations on Windows.

PowerShell

```
PS> $PROFILE | Select-Object *
AllUsersAllHosts       : C:\Program Files\PowerShell\7\profile.ps1
AllUsersCurrentHost    : C:\Program
Files\PowerShell\7\Microsoft.PowerShell_profile.ps1
CurrentUserAllHosts    : C:\Users\username\Documents\PowerShell\profile.ps1
CurrentUserCurrentHost :
C:\Users\username\Documents\PowerShell\Microsoft.PowerShell_profile.ps1
Length                 : 69
```

The following command shows the default profile locations on Ubuntu Linux.

PowerShell

```
$PROFILE | Select-Object *

AllUsersAllHosts       : /opt/microsoft/powershell/7/profile.ps1
AllUsersCurrentHost    :
/opt/microsoft/powershell/7/Microsoft.PowerShell_profile.ps1
CurrentUserAllHosts    : /home/username/.config/powershell/profile.ps1
CurrentUserCurrentHost :
/home/username/.config/powershell/Microsoft.PowerShell_profile.ps1
Length                 : 67
```

There are also profiles that run for all PowerShell hosts or specific hosts. The profile script for each PowerShell host has a name unique for that host. For example, the filename for the standard Console Host on Windows or the default terminal application on other platforms is `Microsoft.PowerShell_profile.ps1`. For Visual Studio Code (VS Code), the filename is `Microsoft.VSCode_profile.ps1`.

For more information, see about_Profiles.

How to create your personal profile

When you first install PowerShell on a system, the profile script files and the directories they belong to don't exist. The following command creates the "Current User, Current Host" profile script file if it doesn't exist.

PowerShell

```
if (!(Test-Path -Path $PROFILE)) {
  New-Item -ItemType File -Path $PROFILE -Force
}
```

The **Force** parameter of `New-Item` cmdlet creates the necessary folders when they don't exist. After you create the script file, you can use your favorite editor to customize your shell

environment.

Adding customizations to your profile

The previous articles talked about using tab completion, command predictors, and aliases. These articles showed the commands used to load the required modules, create custom completers, define key bindings, and other settings. These customizations are the kind that you want to have available in every PowerShell interactive session. The profile script is the place for these settings.

The simplest way to edit your profile script is to open the file in your favorite code editor. For example, the following command opens the profile in VS Code.

PowerShell

```
code $PROFILE
```

You could also use `notepad.exe` on Windows, `vi` on Linux, or any other text editor.

The following profile script has examples for many of the customizations mentioned in the previous articles. You can use any of these settings in your own profile.

PowerShell

```
## Map PSDrives to other registry hives
if (!(Test-Path HKCR:)) {
    $null = New-PSDrive -Name HKCR -PSProvider Registry -Root HKEY_CLASSES_ROOT
    $null = New-PSDrive -Name HKU -PSProvider Registry -Root HKEY_USERS
}

## Customize the prompt
function prompt {
    $identity = [Security.Principal.WindowsIdentity]::GetCurrent()
    $principal = [Security.Principal.WindowsPrincipal] $identity
    $adminRole = [Security.Principal.WindowsBuiltInRole]::Administrator

    $prefix = if (Test-Path Variable:/PSDebugContext) { '[DBG]: ' } else { '' }
    if ($principal.IsInRole($adminRole)) {
        $prefix = "[ADMIN]:$prefix"
    }
    $body = 'PS ' + $PWD.path
    $suffix = $(if ($NestedPromptLevel -ge 1) { '>>' }) + '> '
    "${prefix}${body}${suffix}"
}

## Create $PSStyle if running on a version older than 7.2
## - Add other ANSI color definitions as needed
```

```
if ($PSVersionTable.PSVersion.ToString() -lt '7.2.0') {
    # define escape char since "`e" may not be supported
    $esc = [char]0x1b
    $PSStyle = [pscustomobject]@{
        Foreground = @{
            Magenta = "${esc}[35m"
            BrightYellow = "${esc}[93m"
        }
        Background = @{
            BrightBlack = "${esc}[100m"
        }
    }
}

## Set PSReadLine options and keybindings
$PSROptions = @{
    ContinuationPrompt = '  '
    Colors             = @{
        Operator         = $PSStyle.Foreground.Magenta
        Parameter        = $PSStyle.Foreground.Magenta
        Selection        = $PSStyle.Background.BrightBlack
        InLinePrediction = $PSStyle.Foreground.BrightYellow +
$PSStyle.Background.BrightBlack
    }
}
Set-PSReadLineOption @PSROptions
Set-PSReadLineKeyHandler -Chord 'Ctrl+f' -Function ForwardWord
Set-PSReadLineKeyHandler -Chord 'Enter' -Function ValidateAndAcceptLine

## Add argument completer for the dotnet CLI tool
$scriptblock = {
    param($wordToComplete, $commandAst, $cursorPosition)
    dotnet complete --position $cursorPosition $commandAst.ToString() |
        ForEach-Object {
            [System.Management.Automation.CompletionResult]::new($_, $_,
'ParameterValue', $_)
        }
}
Register-ArgumentCompleter -Native -CommandName dotnet -ScriptBlock $scriptblock
```

This profile script provides examples for the following customization:

- Adds two new PSDrives for the other root registry hives.
- Creates a customized prompt that changes if you're running in an elevated session.
- Configures **PSReadLine** and adds key binding. The color settings use the $PSStyle feature to define the ANSI color settings.
- Adds tab completion for the dotnet CLI tool. The tool provides parameters to help resolve the command-line arguments. The script block for Register-ArgumentCompleter uses that feature to provide the tab completion.

Last updated on 11/21/2025

Using PSReadLine key handlers

The **PSReadLine** module provides key handlers that map **PSReadLine** functions to keyboard chords. Keyboard chords are a sequence of one or more keystrokes that are pressed at the same time. For example, the chord `Ctrl`+`Spacebar` is the combination of the `Ctrl` and `Spacebar` keys pressed at the same time. A **PSReadLine** function is a predefined action that can be performed on a command line. For example, the `MenuComplete` function allows you to choose from a list of options from a menu complete the input on the command line.

PSReadLine has several predefined key handlers that are bound by default. You can also define your own custom key handlers. Run the following command to list the key handlers that are currently defined.

PowerShell

```
Get-PSReadLineKeyHandler
```

You can also get a list of all unbound **PSReadLine** functions that are available to be bound to a key chord.

PowerShell

```
Get-PSReadLineKeyHandler -Unbound
```

You can use the `Set-PSReadLineKeyHandler` cmdlet to bind a function to a key handler. The following command binds the `MenuComplete` function to the chord `Ctrl`+`Spacebar`.

PowerShell

```
Set-PSReadLineKeyHandler -Chord 'Ctrl+Spacebar' -Function MenuComplete
```

Finding key names and chord bindings

The names of the keys in the chord match the names in the `[System.ConsoleKey]` enumeration. For more information, see System.ConsoleKey documentation. For example, the name of the `2` key in `[System.ConsoleKey]` is `D2`, whereas the name of the `2` key on the numeric keypad is `NumPad2`. You can use the `[System.Console]::ReadKey()` method to find the name of the key you pressed.

PowerShell

```
[System.Console]::ReadKey()
```

The following output shows the information returned by the `ReadKey()` method for the `Ctrl` + `2` key chord.

Output

```
KeyChar Key Modifiers
------- --- ---------
        D2   Control
```

For the **PSReadLine** key handler cmdlets, this chord is represented as `Ctrl+D2`. The following example binds this chord to a function.

PowerShell

```
Set-PSReadLineKeyHandler -Chord 'Ctrl+D2' -Function MenuComplete
```

You can bind multiple cords to a single function. By default, the `BackwardDeleteChar` function is bound to two chords.

PowerShell

```
Get-PSReadLineKeyHandler -Chord Backspace, Ctrl+h
```

Output

```
Key       Function           Description
---       --------           -----------
Backspace BackwardDeleteChar Delete the character before the cursor
Ctrl+h    BackwardDeleteChar Delete the character before the cursor
```

> ⓘ **Note**
>
> The **Chord** parameter is **case-sensitive**. Meaning, you can create different bindings for `Ctrl` + `X` and `Ctrl` + `x`.

On Windows, you can also use the `Alt` + `?` key chord to show the function bound to the next key chord you enter. When you type `Alt` + `?`, you see the following prompt:

Output

```
what-is-key:
```

When you hit the `Backspace` key, you get the following response:

Output

```
Backspace: BackwardDeleteChar - Delete the character before the cursor
```

Key handlers on non-Windows computers

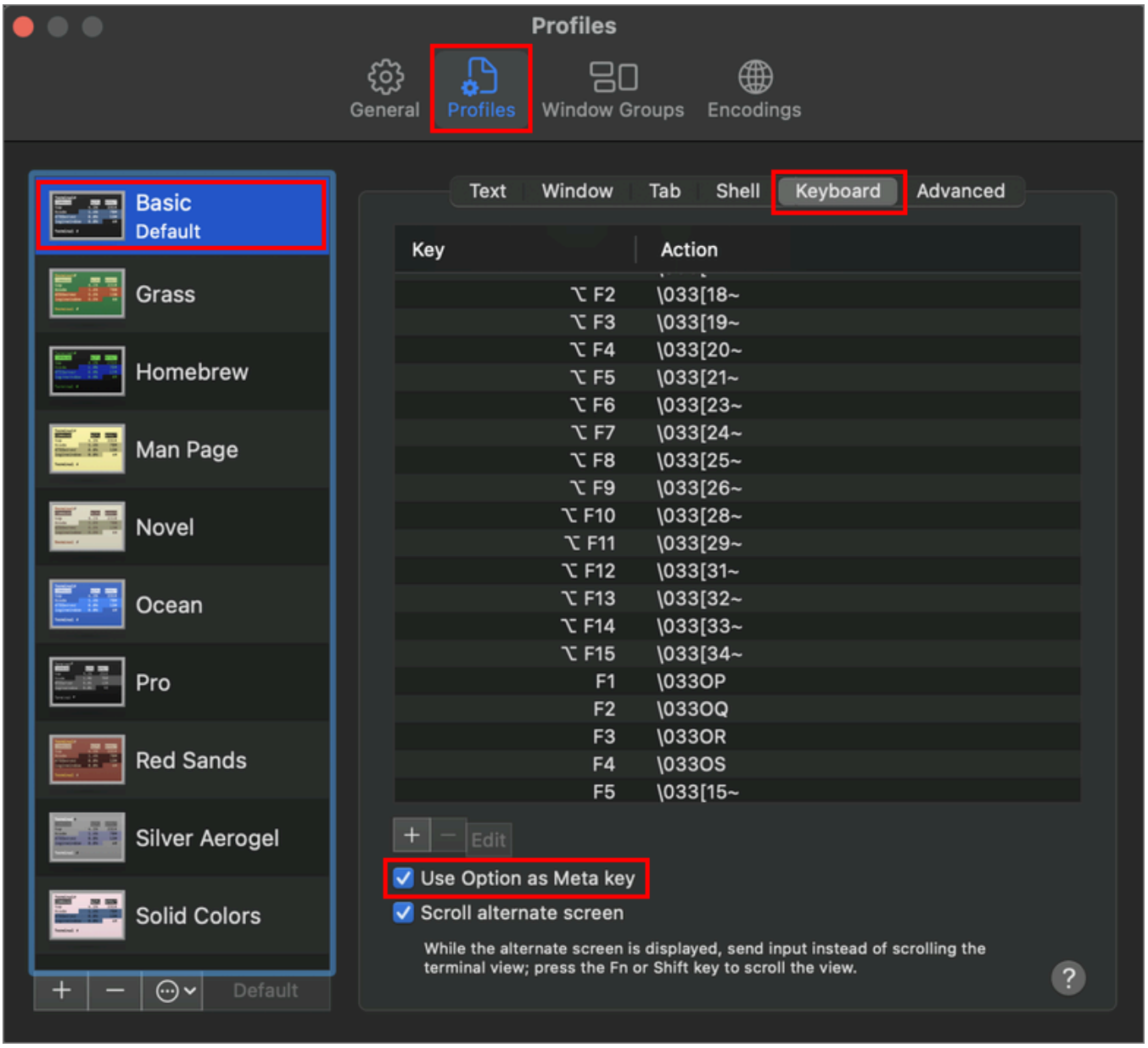
The key codes generated by your keyboard can be different depending on the operating system and terminal application you're using.

macOS

The Macintosh keyboard doesn't have an `Alt` key like Windows and Linux systems. Instead, it has the `⌥ Option` key. macOS uses this key differently than the `Alt` key on other systems. However, you can configure the terminal and iTerm2 applications on macOS to treat it as an `Alt` key.

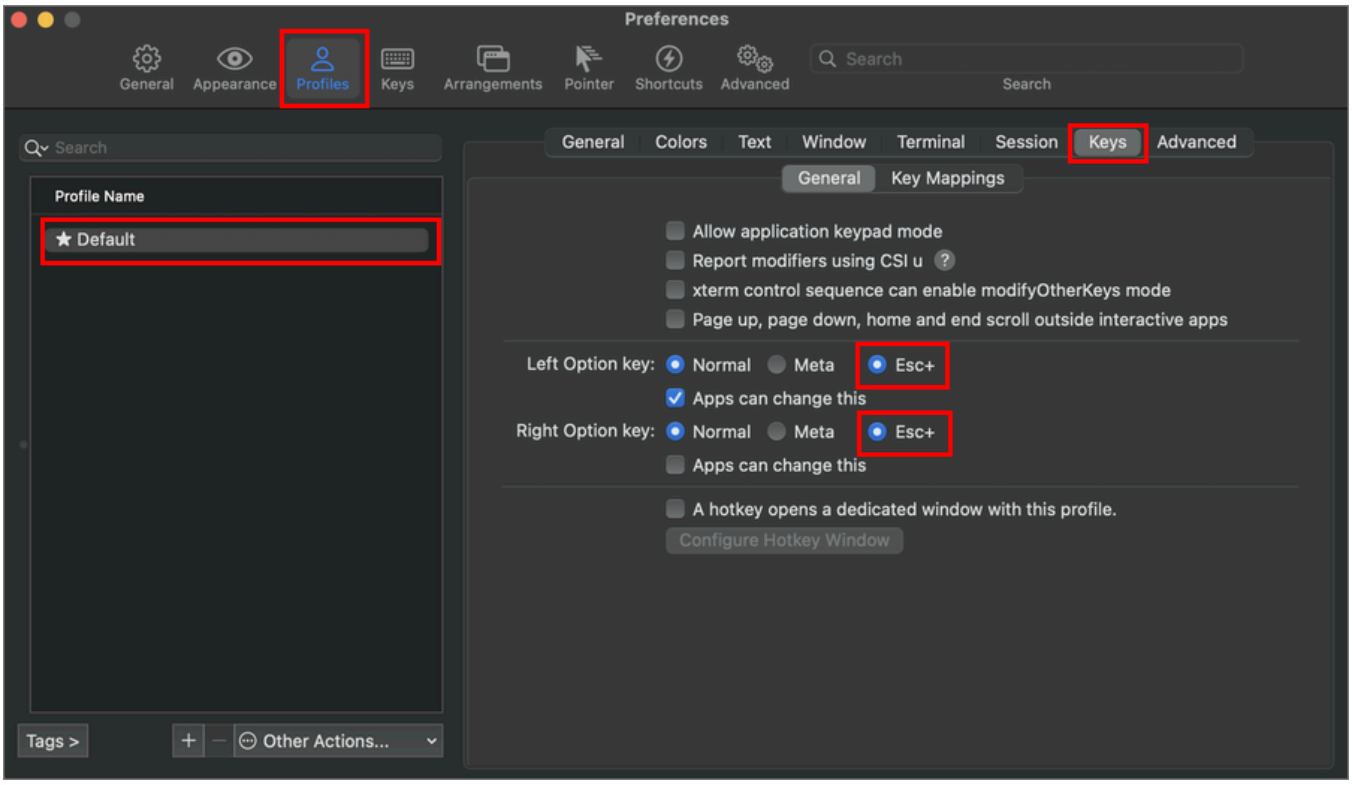
Configuring the Terminal application

Open the **Settings** window from the App bar in Terminal.app. Select **Profiles** and choose the profile you want to configure. Select the **Keyboard** tab of the configuration options. Below the list of keys, select the **Use Option as Meta Key** setting. This setting allows the `⌥ Option` key to act as `Alt` in the Terminal application.



Configuring the iTerm2 application

Open the **Settings** window from the App Bar in iTerm.app. Select **Profiles** and choose the profile you want to configure. Select the **Keys** tab of the configuration options. Select the **Esc+** option for both the **Left Option Key** and **Right Option Key** settings. This setting allows the `⌥ Option` key to act as `Alt` in the iTerm application.



> **Note**
>
> The exact steps vary depending on the versions of macOS and the terminal applications. These examples were captured on macOS Ventura 13.2.1 and iTerm2 v3.4.16.

Linux

On Linux platforms, the key code generated can be different than other systems. For example:

- `Ctrl` + `[` is the same as `Escape`
- `Ctrl` + `Spacebar` generates the key codes for `Ctrl` + `D2`. If you want to map a function to `Ctrl` + `Spacebar`, you must use the chord `Ctrl+D2`.

  PowerShell

  ```
  Set-PSReadLineKeyHandler -Chord 'Ctrl+D2' -Function MenuComplete
  ```

Use the `ReadKey()` method to verify the key codes generated by your keyboard.

Commonly used key handlers

Here are a few commonly used key handlers that are bound by default on Windows. Keybindings can be different on non-Windows platforms.

MenuComplete

Complete the input by selecting from a menu of possible completion values.

Default chord: `Ctrl+Spacebar`

The following example shows the menu of possible completions for commands beginning with `select`.

Output

```
PS C:\> select<Ctrl+Spacebar>
select                  Select-Object           Select-PSFPropertyValue  Select-
Xml
Select-AzContext        Select-PSFConfig        Select-PSMDBuildProject
Select-AzSubscription   Select-PSFObject        Select-String

Select-Object
```

Use the arrow keys to select the completion you want. Press the `Enter` key to complete the input. As you move through the selections, help for the selected command is displayed below the menu.

ClearScreen

This function clears the screen similar to the `cls` or `clear` commands.

Default chord: `Ctrl+l`

SelectCommandArgument

Selects the next argument on the command line.

Default chord: `Alt+a`

You might have command in your history that you want to run again with different parameter values. You can use the chord to cycle through each parameter and change the value as needed.

```
New-AzVM -ResourceGroupName myRGName -Location eastus -Name myVM
```

Pressing `Alt` + `a` selects the next parameter argument in turn: `myRGName`, `eastus`, `myVM`.

GotoBrace

Moves the cursor to the matching brace.

Default chord: `Ctrl+]`

The `GotoBrace` function moves your cursor to the closing brace that matches the brace at the current cursor position on the command line. The function works for brackets (`[]`), braces (`{}`), and parentheses (`()`).

DigitArgument

Start or accumulate a numeric argument that's used to repeat a keystroke the specified number of times.

Default chord: `Alt+0` through `Alt+9`

For example, typing `Alt` + `4` + `#` enters `####` on the command line.

See also

- Get-PSReadLineKeyHandler
- Set-PSReadLineKeyHandler

Last updated on 11/21/2025

Configuring a light colored theme

The default colors for both PowerShell and **PSReadLine** are selected for a dark background terminal. However, some users might choose to use a light background with dark text. Since most of the default colors don't set the background, using light foreground colors on a light background produces unreadable text.

Beginning in PowerShell 7.2, PowerShell adds colorized output to the default console experience. The `$PSStyle` feature is not natively available in Windows PowerShell. However, using the PSStyle module from the PowerShell Gallery, you can set color values using these same techniques described in this article.

The colors used are defined in the `$PSStyle` variable and are designed for a dark background. You can change these colors to work better for a light background terminal.

PSReadLine allows you to define colors for 18 different syntax elements. You can view the current settings using the `Get-PSReadLineOption` cmdlet.

Output

```
EditMode                                : Windows
AddToHistoryHandler                     : System.Func`2[System.String,System.Object]
HistoryNoDuplicates                     : True
HistorySavePath                         :
C:\Users\user1\AppData\Roaming\Microsoft\Wind...
HistorySaveStyle                        : SaveIncrementally
HistorySearchCaseSensitive              : False
HistorySearchCursorMovesToEnd           : False
MaximumHistoryCount                     : 4096
ContinuationPrompt                      : >>
ExtraPromptLineCount                    : 0
PromptText                              : {> }
BellStyle                               : Audible
DingDuration                            : 50
DingTone                                : 1221
CommandsToValidateScriptBlockArguments : {ForEach-Object, %, Invoke-Command, icm...}
CommandValidationHandler                :
CompletionQueryItems                    : 100
MaximumKillRingCount                    : 10
ShowToolTips                            : True
ViModeIndicator                         : None
WordDelimiters                          : ;:,.[]{}()/\|^&*-=+'"–—
AnsiEscapeTimeout                       : 100
PredictionSource                        : HistoryAndPlugin
PredictionViewStyle                     : InlineView
CommandColor                            : "`e[93m"
CommentColor                            : "`e[32m"
ContinuationPromptColor                 : "`e[37m"
DefaultTokenColor                       : "`e[37m"
EmphasisColor                           : "`e[96m"
```

```
ErrorColor                          : "`e[91m"
InlinePredictionColor               : "`e[38;5;238m"
KeywordColor                        : "`e[92m"
ListPredictionColor                 : "`e[33m"
ListPredictionSelectedColor         : "`e[48;5;238m"
MemberColor                         : "`e[97m"
NumberColor                         : "`e[97m"
OperatorColor                       : "`e[90m"
ParameterColor                      : "`e[90m"
SelectionColor                      : "`e[30;47m"
StringColor                         : "`e[36m"
TypeColor                           : "`e[37m"
VariableColor                       : "`e[92m"
```

The color settings are stored as strings containing ANSI escape sequences that change the color in your terminal. Using the `Set-PSReadLineOption` cmdlet you can change the colors to values that work better for a light-colored background.

Defining colors for a light theme

The PowerShell ISE can be configured to use a light theme for both the editor and console panes. You can also view and change the colors that the ISE uses for various syntax and output types. You can use these color choices to define a similar theme for **PSReadLine**.

The following hashtable defines colors for **PSReadLine** that mimic the colors in the PowerShell ISE.

PowerShell

```
$ISETheme = @{
    Command                 = $PSStyle.Foreground.FromRGB(0x0000FF)
    Comment                 = $PSStyle.Foreground.FromRGB(0x006400)
    ContinuationPrompt      = $PSStyle.Foreground.FromRGB(0x0000FF)
    Default                 = $PSStyle.Foreground.FromRGB(0x0000FF)
    Emphasis                = $PSStyle.Foreground.FromRGB(0x287BF0)
    Error                   = $PSStyle.Foreground.FromRGB(0xE50000)
    InlinePrediction        = $PSStyle.Foreground.FromRGB(0x93A1A1)
    Keyword                 = $PSStyle.Foreground.FromRGB(0x00008b)
    ListPrediction          = $PSStyle.Foreground.FromRGB(0x06DE00)
    Member                  = $PSStyle.Foreground.FromRGB(0x000000)
    Number                  = $PSStyle.Foreground.FromRGB(0x800080)
    Operator                = $PSStyle.Foreground.FromRGB(0x757575)
    Parameter               = $PSStyle.Foreground.FromRGB(0x000080)
    String                  = $PSStyle.Foreground.FromRGB(0x8b0000)
    Type                    = $PSStyle.Foreground.FromRGB(0x008080)
    Variable                = $PSStyle.Foreground.FromRGB(0xff4500)
    ListPredictionSelected  = $PSStyle.Background.FromRGB(0x93A1A1)
    Selection               = $PSStyle.Background.FromRGB(0x00BFFF)
}
```

> **Note**
>
> You can use the `FromRGB()` method to create the ANSI escape sequences for the colors you want. For more information about `$PSStyle`, see about_ANSI_Terminals. For more information about ANSI escape sequences, see the ANSI escape code article in Wikipedia.

Setting the color theme in your profile

To have the color settings you want in every PowerShell session, you must add the configuration settings to your PowerShell profile script. For an example, see Customizing your shell environment

Add the `$ISETheme` variable and the following `Set-PSReadLineOption` command to your profile.

PowerShell

```
Set-PSReadLineOption -Colors $ISETheme
```

The following settings work better for a light background terminal.

PowerShell

```
$PSStyle.Formatting.FormatAccent      = $PSStyle.Foreground.Green
$PSStyle.Formatting.TableHeader       = $PSStyle.Foreground.Green
$PSStyle.Formatting.ErrorAccent       = $PSStyle.Foreground.Cyan
$PSStyle.Formatting.Error             = $PSStyle.Foreground.Red
$PSStyle.Formatting.Warning           = $PSStyle.Foreground.Yellow
$PSStyle.Formatting.Verbose           = $PSStyle.Foreground.Yellow
$PSStyle.Formatting.Debug             = $PSStyle.Foreground.Yellow
$PSStyle.Progress.Style               = $PSStyle.Foreground.Yellow
$PSStyle.FileInfo.Directory           = $PSStyle.Background.FromRgb(0x2f6aff) +
                                        $PSStyle.Foreground.BrightWhite
$PSStyle.FileInfo.SymbolicLink        = $PSStyle.Foreground.Cyan
$PSStyle.FileInfo.Executable          = $PSStyle.Foreground.BrightMagenta
$PSStyle.FileInfo.Extension['.ps1']   = $PSStyle.Foreground.Cyan
$PSStyle.FileInfo.Extension['.ps1xml'] = $PSStyle.Foreground.Cyan
$PSStyle.FileInfo.Extension['.psd1']  = $PSStyle.Foreground.Cyan
$PSStyle.FileInfo.Extension['.psm1']  = $PSStyle.Foreground.Cyan
```

Choosing colors for accessibility

The ISE color theme might not work for users with color-blindness or other conditions that limit their ability to see colors.

The World Wide Web Consortium (W3C) has recommendations for using colors for accessibility. The Web Content Accessibility Guidelines (WCAG) 2.1 recommends that "visual presentation of text and images of text has a contrast ratio of at least 4.5:1." For more information, see Success Criterion 1.4.3 Contrast (Minimum).

The Contrast Ratio website provides a tool that lets you pick foreground and background colors and measure the contrast. You can use this tool to find color combinations that work best for you.

Last updated on 12/09/2025

Improve the accessibility of output in PowerShell

Most terminal environments only display raw text. Users that rely on screen readers are faced with tedious narration when consuming large amounts of raw text because the raw output doesn't have the accessibility metadata to characterize the format of the content.

There are two ways to improve the accessibility of the output in PowerShell:

- Output the data in a way that it can be viewed in another tool that supports screen reading technologies.
- Reduce the amount of output displayed in the terminal by filtering and selecting the data you want and output the text in a more readable format.

Display the data in a tool outside of the terminal

For large amounts of data, rather than output to the host, consider writing output in a format that can be viewed in another tool that supports screen reading technologies. You might need to save the data to a file in a format that can be opened in another application.

Out-GridView command on Windows

For small to moderate size output, use the `Out-GridView` command. The output is rendered using Windows Presentation Foundation (WPF) in tabular form, much like a spreadsheet. The GridView control allows you to sort, filter, and search the data, which reduces the amount of data that needs to be read. The GridView control is also accessible to screen readers. The **Narrator** tool built into Windows is able to read the GridView details, including column names and row count.

The following example shows how to display a list of services in a GridView control.

PowerShell

```
Get-Service | Out-GridView
```

The `Out-GridView` command is only available in PowerShell on Windows.

Character Separated Value (CSV) format

Spreadsheet applications such as **Microsoft Excel** support CSV files. The following example shows how to save the output of a command to a CSV file.

PowerShell

```
Get-Service | Export-Csv -Path .\myFile.csv
Invoke-Item .\myFile.csv
```

The `Invoke-Item` command opens the file in the default application for CSV files, which is usually Microsoft Excel.

HyperText Markup Language (HTML) format

HTML files can be viewed by web browsers such as **Microsoft Edge**. The following example shows how to save the output of a command to an HTML file.

PowerShell

```
Get-Service | ConvertTo-Html | Out-File .\myFile.html
Invoke-Item .\myFile.html
```

The `Invoke-Item` command opens the file in your default web browser.

Reduce the amount of output

One way to improve the accessibility of the output is to reduce the amount of output displayed in the terminal. PowerShell has several commands that can help you filter and select the data you want.

Select and filter data

Rather than returning a large mount of data, use commands such as `Select-Object`, `Sort-Object`, and `Where-Object` to reduce the amount of output. The following example gets the list of services on the computer.

Each of the following commands improves the output in a different way:

- The `-ErrorAction SilentlyContinue` parameter suppresses error messages that might be generated if the user doesn't have permission to view some services.
- The `Where-Object` command reduces the number of items returned by filtering the list to only show services that are running and have `event` in the description.
- The `Select-Object` command selects only the service name and display name.
- The `Format-List` command displays the output in list format, which provides a better narration experience for screen readers.

PowerShell

```powershell
Get-Service -ErrorAction SilentlyContinue |
    Where-Object {$_.Status -eq 'Running' -and $_.Description -match 'event'} |
    Select-Object Name, DisplayName |
    Format-List
```

Reformat the output with calculated properties

The default property names of .NET objects output by PowerShell can be verbose and confusing. You can use calculated properties to change the property names and values to something easier to understood when read by a narrator technology.

The following example shows how to get the top five processes by memory usage and display the process name and memory usage in megabytes.

PowerShell

```powershell
Get-Process |
    Sort-Object WorkingSet -Descending |
    Select-Object -First 5 -Property ProcessName,
        @{n="MemoryMB"; e={'{0:N}' -f ($_.WorkingSet/1Mb)}} |
    Format-List
```

By default, `Get-Process` displays the **WorkingSet** as the number of bytes of memory used. Without formatting, it can be difficult to understand the magnitude of the number. The calculated property converts the number of bytes to megabytes and formats the number with commas and limits the value to two decimal places.

Output

```
ProcessName : vmmemWSL
MemoryMB    : 1,217.69

ProcessName : Memory Compression
MemoryMB    : 780.45

ProcessName : Code
MemoryMB    : 726.43

ProcessName : OUTLOOK
MemoryMB    : 460.16

ProcessName : msedgewebview2
MemoryMB    : 428.94
```

Additional reading

- Out-GridView
- Export-Csv
- ConvertTo-Html
- about_Calculated_Properties

Last updated on 11/21/2025

Deep dive articles

Article • 11/17/2022

The articles in this section are designed to be an in-depth look into PowerShell topics. These articles don't replace the reference articles, but provide diverse examples, illustrate edge cases, and warn about pitfalls and common mistakes.

This collection is also a showcase for community contributions. The inaugural set of articles come from @KevinMarquette and were originally published at PowerShellExplained.com.

How to contribute content

If you're interested in contributing content to this collection, please read the Contributor Guide. When you are ready to propose a contribution, submit an issue in the GitHub repository using the Document Idea template and include a link to the existing content you want to share.

Everything you wanted to know about arrays

Arrays are a fundamental language feature of most programming languages. They're a collection of values or objects that are difficult to avoid. Let's take a close look at arrays and everything they have to offer.

> **Note**
>
> The original version of this article appeared on the blog written by @KevinMarquette. The PowerShell team thanks Kevin for sharing this content with us. Please check out his blog at PowerShellExplained.com.

What is an array?

I'm going to start with a basic technical description of what arrays are and how they are used by most programming languages before I shift into the other ways PowerShell makes use of them.

An array is a data structure that serves as a collection of multiple items. You can iterate over the array or access individual items using an index. The array is created as a sequential chunk of memory where each value is stored right next to the other.

I'll touch on each of those details as we go.

Basic usage

Because arrays are such a basic feature of PowerShell, there is a simple syntax for working with them in PowerShell.

Create an array

An empty array can be created by using `@()`

```
PowerShell
```

```
PS> $data = @()
PS> $data.Count
0
```

We can create an array and seed it with values just by placing them in the `@()` parentheses.

PowerShell

```
PS> $data = @('Zero','One','Two','Three')
PS> $data.Count
4

PS> $data
Zero
One
Two
Three
```

This array has 4 items. When we call the `$data` variable, we see the list of our items. If it's an array of strings, then we get one line per string.

We can declare an array on multiple lines. The comma is optional in this case and generally left out.

PowerShell

```
$data = @(
    'Zero'
    'One'
    'Two'
    'Three'
)
```

I prefer to declare my arrays on multiple lines like that. Not only does it get easier to read when you have multiple items, it also makes it easier to compare to previous versions when using source control.

Other syntax

It's commonly understood that `@()` is the syntax for creating an array, but comma-separated lists work most of the time.

PowerShell

```
$data = 'Zero','One','Two','Three'
```

Write-Output to create arrays

One cool little trick worth mentioning is that you can use `Write-Output` to quickly create strings at the console.

PowerShell

```
$data = Write-Output Zero One Two Three
```

This is handy because you don't have to put quotes around the strings when the parameter accepts strings. I would never do this in a script but it's fair game in the console.

Accessing items

Now that you have an array with items in it, you may want to access and update those items.

Offset

To access individual items, we use the brackets `[]` with an offset value starting at 0. This is how we get the first item in our array:

PowerShell

```
PS> $data = 'Zero','One','Two','Three'
PS> $data[0]
Zero
```

The reason why we use zero here is because the first item is at the beginning of the list so we use an offset of 0 items to get to it. To get to the second item, we would need to use an offset of 1 to skip the first item.

PowerShell

```
PS> $data[1]
One
```

This would mean that the last item is at offset 3.

PowerShell

```
PS> $data[3]
Three
```

Index

Now you can see why I picked the values that I did for this example. I introduced this as an offset because that is what it really is, but this offset is more commonly referred to as an index. An index that starts at `0`. For the rest of this article I will call the offset an index.

Special index tricks

In most languages, you can only specify a single number as the index and you get a single item back. PowerShell is much more flexible. You can use multiple indexes at once. By providing a list of indexes, we can select several items.

PowerShell

```
PS> $data[0,2,3]
Zero
Two
Three
```

The items are returned based on the order of the indexes provided. If you duplicate an index, you get that item both times.

PowerShell

```
PS> $data[3,0,3]
Three
Zero
Three
```

We can specify a sequence of numbers with the built-in `..` operator.

PowerShell

```
PS> $data[1..3]
One
Two
Three
```

This works in reverse too.

PowerShell

```
PS> $data[3..1]
Three
```

```
Two
One
```

You can use negative index values to offset from the end. So if you need the last item in the list, you can use `-1`.

PowerShell

```
PS> $data[-1]
Three
```

One word of caution here with the `..` operator. The sequence `0..-1` and `-1..0` evaluate to the values `0,-1` and `-1,0`. It's easy to see `$data[0..-1]` and think it would enumerate all items if you forget this detail. `$data[0..-1]` gives you the same value as `$data[0,-1]` by giving you the first and last item in the array (and none of the other values). Here is a larger example:

PowerShell

```
PS> $a = 1,2,3,4,5,6,7,8
PS> $a[2..-1]
3
2
1
8
```

This is the same as:

PowerShell

```
PS> $a[2,1,0,-1]
3
2
1
8
```

Out of bounds

In most languages, if you try to access an index of an item that is past the end of the array, you would get some type of error or an exception. PowerShell silently returns nothing.

PowerShell

```
PS> $null -eq $data[9000]
True
```

Cannot index into a null array

If your variable is `$null` and you try to index it like an array, you get a `System.Management.Automation.RuntimeException` exception with the message `Cannot index into a null array`.

PowerShell

```
PS> $empty = $null
PS> $empty[0]
Error: Cannot index into a null array.
```

So make sure your arrays are not `$null` before you try to access elements inside them.

Count

Arrays and other collections have a `Count` property that tells you how many items are in the array.

PowerShell

```
PS> $data.Count
4
```

PowerShell 3.0 added a `Count` property to most objects. you can have a single object and it should give you a count of `1`.

PowerShell

```
PS> $date = Get-Date
PS> $date.Count
1
```

Even `$null` has a `Count` property except it returns `0`.

PowerShell

```
PS> $null.Count
0
```

There are some traps here that I will revisit when I cover checking for `$null` or empty arrays later on in this article.

Off-by-one errors

A common programming error is created because arrays start at index 0. Off-by-one errors can be introduced in two ways.

The first is by mentally thinking you want the second item and using an index of `2` and really getting the third item. Or by thinking that you have four items and you want last item, so you use the count to access the last item.

PowerShell

```
$data[ $data.Count ]
```

PowerShell is perfectly happy to let you do that and give you exactly what item exists at index 4: `$null`. You should be using `$data.Count - 1` or the `-1` that we learned about above.

PowerShell

```
PS> $data[ $data.Count - 1 ]
Three
```

This is where you can use the `-1` index to get the last element.

PowerShell

```
PS> $data[ -1 ]
Three
```

Lee Dailey also pointed out to me that we can use `$data.GetUpperBound(0)` to get the max index number.

PowerShell

```
PS> $data.GetUpperBound(0)
3
PS> $data[ $data.GetUpperBound(0) ]
Three
```

The second most common way is when iterating the list and not stopping at the right time. I'll revisit this when we talk about using the `for` loop.

Updating items

We can use the same index to update existing items in the array. This gives us direct access to update individual items.

PowerShell

```
$data[2] = 'dos'
$data[3] = 'tres'
```

If we try to update an item that is past the last element, then we get an `Index was outside the bounds of the array.` error.

PowerShell

```
PS> $data[4] = 'four'
Index was outside the bounds of the array.
At line:1 char:1
+ $data[4] = 'four'
+ ~~~~~~~~~~~~~~
+ CategoryInfo          : OperationStopped: (:) [], IndexOutOfRangeException
+ FullyQualifiedErrorId : System.IndexOutOfRangeException
```

I'll revisit this later when I talk about how to make an array larger.

Iteration

At some point, you might need to walk or iterate the entire list and perform some action for each item in the array.

Pipeline

Arrays and the PowerShell pipeline are meant for each other. This is one of the simplest ways to process over those values. When you pass an array to a pipeline, each item inside the array is processed individually.

PowerShell

```
PS> $data = 'Zero','One','Two','Three'
PS> $data | ForEach-Object {"Item: [$PSItem]"}
Item: [Zero]
Item: [One]
Item: [Two]
Item: [Three]
```

If you have not seen `$PSItem` before, just know that it's the same thing as `$_`. You can use either one because they both represent the current object in the pipeline.

ForEach loop

The `foreach` loop works well with collections. Using the syntax: `foreach ( <variable> in <collection> )`

PowerShell

```
foreach ( $node in $data )
{
    "Item: [$node]"
}
```

ForEach method

I tend to forget about this one but it works well for simple operations. PowerShell allows you to call `ForEach()` on a collection.

PowerShell

```
PS> $data.ForEach({"Item [$PSItem]"})
Item [Zero]
Item [One]
Item [Two]
Item [Three]
```

The `ForEach()` takes a parameter that is a script block. You can drop the parentheses and just provide the script block.

PowerShell

```
$data.ForEach{"Item [$PSItem]"}
```

This is a lesser known syntax but it works just the same. This `ForEach` method was added in PowerShell 4.0.

For loop

The `for` loop is used heavily in most other languages but you don't see it much in PowerShell. When you do see it, it's often in the context of walking an array.

PowerShell

```
for ( $index = 0; $index -lt $data.Count; $index++)
{
    "Item: [{0}]" -f $data[$index]
}
```

The first thing we do is initialize an `$index` to `0`. Then we add the condition that `$index` must be less than `$data.Count`. Finally, we specify that every time we loop that we must increase the index by `1`. In this case `$index++` is short for `$index = $index + 1`. The format operator (`-f`) is used to insert the value of `$data[$index]` in the output string.

Whenever you're using a `for` loop, pay special attention to the condition. I used `$index -lt $data.Count` here. It's easy to get the condition slightly wrong to get an off-by-one error in your logic. Using `$index -le $data.Count` or `$index -lt ($data.Count - 1)` are ever so slightly wrong. That would cause your result to process too many or too few items. This is the classic off-by-one error.

Switch loop

This is one that is easy to overlook. If you provide an array to a switch statement, it checks each item in the array.

PowerShell

```
$data = 'Zero','One','Two','Three'
switch( $data )
{
    'One'
    {
        'Tock'
    }
    'Three'
    {
        'Tock'
    }
    Default
    {
        'Tick'
    }
}
```

Output

```
Tick
Tock
Tick
Tock
```

There are a lot of cool things that we can do with the switch statement. I have another article dedicated to this.

- Everything you ever wanted to know about the switch statement

Updating values

When your array is a collection of string or integers (value types), sometimes you may want to update the values in the array as you loop over them. Most of the loops above use a variable in the loop that holds a copy of the value. If you update that variable, the original value in the array is not updated.

The exception to that statement is the `for` loop. If you want to walk an array and update values inside it, then the `for` loop is what you're looking for.

PowerShell

```powershell
for ( $index = 0; $index -lt $data.Count; $index++ )
{
    $data[$index] = "Item: [{0}]" -f $data[$index]
}
```

This example takes a value by index, makes a few changes, and then uses that same index to assign it back.

Arrays of Objects

So far, the only thing we've placed in an array is a value type, but arrays can also contain objects.

PowerShell

```powershell
$data = @(
    [pscustomobject]@{FirstName='Kevin';LastName='Marquette'}
    [pscustomobject]@{FirstName='John'; LastName='Doe'}
)
```

Many cmdlets return collections of objects as arrays when you assign them to a variable.

PowerShell

```powershell
$processList = Get-Process
```

All of the basic features we already talked about still apply to arrays of objects with a few details worth pointing out.

Accessing properties

We can use an index to access an individual item in a collection just like with value types.

PowerShell

```
PS> $data[0]

FirstName LastName
-----     ----
Kevin     Marquette
```

We can access and update properties directly.

PowerShell

```
PS> $data[0].FirstName

Kevin

PS> $data[0].FirstName = 'Jay'
PS> $data[0]

FirstName LastName
-----     ----
Jay       Marquette
```

Array properties

Normally you would have to enumerate the whole list like this to access all the properties:

PowerShell

```
PS> $data | ForEach-Object {$_.LastName}

Marquette
Doe
```

Or by using the `Select-Object -ExpandProperty` cmdlet.

PowerShell

```
PS> $data | Select-Object -ExpandProperty LastName

Marquette
Doe
```

But PowerShell offers us the ability to request `LastName` directly. PowerShell enumerates them all for us and returns a clean list.

PowerShell

```
PS> $data.LastName

Marquette
Doe
```

The enumeration still happens but we don't see the complexity behind it.

Where-Object filtering

This is where `Where-Object` comes in so we can filter and select what we want out of the array based on the properties of the object.

PowerShell

```
PS> $data | Where-Object {$_.FirstName -eq 'Kevin'}

FirstName LastName
-----     ----
Kevin     Marquette
```

We can write that same query to get the `FirstName` we are looking for.

PowerShell

```
$data | where FirstName -EQ Kevin
```

Where()

Arrays have a `Where()` method on them that allows you to specify a `scriptblock` for the filter.

PowerShell

```
$data.Where({$_.FirstName -eq 'Kevin'})
```

This feature was added in PowerShell 4.0.

Updating objects in loops

With value types, the only way to update the array is to use a for loop because we need to know the index to replace the value. We have more options with objects because they are reference types. Here is a quick example:

PowerShell

```
foreach($person in $data)
{
    $person.FirstName = 'Kevin'
}
```

This loop is walking every object in the `$data` array. Because objects are reference types, the `$person` variable references the exact same object that is in the array. So updates to its properties do update the original.

You still can't replace the whole object this way. If you try to assign a new object to the `$person` variable, you're updating the variable reference to something else that no longer points to the original object in the array. This doesn't work like you would expect:

PowerShell

```
foreach($person in $data)
{
    $person = [pscustomobject]@{
        FirstName='Kevin'
        LastName='Marquette'
    }
}
```

Operators

The operators in PowerShell also work on arrays. Some of them work slightly differently.

-join

The `-join` operator is the most obvious one so let's look at it first. I like the `-join` operator and use it often. It joins all elements in the array with the character or string that you specify.

PowerShell

```
PS> $data = @(1,2,3,4)
PS> $data -join '-'
1-2-3-4
PS> $data -join ','
1,2,3,4
```

One of the features that I like about the `-join` operator is that it handles single items.

PowerShell

```
PS> 1 -join '-'
1
```

I use this inside logging and verbose messages.

PowerShell

```
PS> $data = @(1,2,3,4)
PS> "Data is $($data -join ',')."
Data is 1,2,3,4.
```

-join $array

Here is a clever trick that Lee Dailey pointed out to me. If you ever want to join everything without a delimiter, instead of doing this:

PowerShell

```
PS> $data = @(1,2,3,4)
PS> $data -join $null
1234
```

You can use `-join` with the array as the parameter with no prefix. Take a look at this example to see that I'm talking about.

PowerShell

```
PS> $data = @(1,2,3,4)
PS> -join $data
1234
```

-replace and -split

The other operators like `-replace` and `-split` execute on each item in the array. I can't say that I have ever used them this way but here is an example.

PowerShell

```
PS> $data = @('ATX-SQL-01','ATX-SQL-02','ATX-SQL-03')
PS> $data -replace 'ATX','LAX'
LAX-SQL-01
LAX-SQL-02
LAX-SQL-03
```

-contains

The `-contains` operator allows you to check an array of values to see if it contains a specified value.

PowerShell

```
PS> $data = @('red','green','blue')
PS> $data -contains 'green'
True
```

-in

When you have a single value that you would like to verify matches one of several values, you can use the `-in` operator. The value would be on the left and the array on the right-hand side of the operator.

PowerShell

```
PS> $data = @('red','green','blue')
PS> 'green' -in $data
True
```

This can get expensive if the list is large. I often use a regex pattern if I'm checking more than a few values.

PowerShell

```
PS> $data = @('red','green','blue')
PS> $pattern = "^({0})$" -f ($data -join '|')
PS> $pattern
^(red|green|blue)$

PS> 'green' -match $pattern
True
```

-eq and -ne

Equality and arrays can get complicated. When the array is on the left side, every item gets compared. Instead of returning `True`, it returns the object that matches.

PowerShell

```
PS> $data = @('red','green','blue')
PS> $data -eq 'green'
green
```

When you use the `-ne` operator, we get all the values that are not equal to our value.

PowerShell

```
PS> $data = @('red','green','blue')
PS> $data -ne 'green'
red
blue
```

When you use this in an `if()` statement, a value that is returned is a `True` value. If no value is returned, then it's a `False` value. Both of these next statements evaluate to `True`.

PowerShell

```
$data = @('red','green','blue')
if ( $data -eq 'green' )
{
    'Green was found'
}
if ( $data -ne 'green' )
{
    'And green was not found'
}
```

I'll revisit this in a moment when we talk about testing for `$null`.

-match

The `-match` operator tries to match each item in the collection.

PowerShell

```
PS> $servers = @(
    'LAX-SQL-01'
    'LAX-API-01'
    'ATX-SQL-01'
    'ATX-API-01'
)
PS> $servers -match 'SQL'
LAX-SQL-01
ATX-SQL-01
```

When you use `-match` with a single value, a special variable `$Matches` gets populated with match info. This isn't the case when an array is processed this way.

We can take the same approach with `Select-String`.

PowerShell

```
$servers | Select-String SQL
```

I take a closer look at `Select-String`, `-match` and the `$Matches` variable in another post called The many ways to use regex.

$null or empty

Testing for `$null` or empty arrays can be tricky. Here are the common traps with arrays.

At a glance, this statement looks like it should work.

PowerShell

```
if ( $array -eq $null)
{
    'Array is $null'
}
```

But I just went over how `-eq` checks each item in the array. So we can have an array of several items with a single $null value and it would evaluate to `$true`

PowerShell

```
$array = @('one',$null,'three')
if ( $array -eq $null)
{
    'I think Array is $null, but I would be wrong'
}
```

This is why it's a best practice to place the `$null` on the left side of the operator. This makes this scenario a non-issue.

PowerShell

```
if ( $null -eq $array )
{
    'Array actually is $null'
}
```

A `$null` array isn't the same thing as an empty array. If you know you have an array, check the count of objects in it. If the array is `$null`, the count is `0`.

PowerShell

```powershell
if ( $array.Count -gt 0 )
{
    "Array isn't empty"
}
```

There is one more trap to watch out for here. You can use the `Count` even if you have a single object, unless that object is a `PSCustomObject`. This is a bug that is fixed in PowerShell 6.1. That's good news, but a lot of people are still on 5.1 and need to watch out for it.

PowerShell

```powershell
PS> $object = [pscustomobject]@{Name='TestObject'}
PS> $object.Count
$null
```

If you're still on PowerShell 5.1, you can wrap the object in an array before checking the count to get an accurate count.

PowerShell

```powershell
if ( @($array).Count -gt 0 )
{
    "Array isn't empty"
}
```

To fully play it safe, check for `$null`, then check the count.

PowerShell

```powershell
if ( $null -ne $array -and @($array).Count -gt 0 )
{
    "Array isn't empty"
}
```

All -eq

I recently saw someone on Reddit ask how to verify that every value in an array matches a given value. Reddit user `u/bis` had this clever solution that checks for any incorrect values and then flips the result.

PowerShell

```
$results = Test-Something
if ( -not ( $results -ne 'Passed') )
{
    'All results a Passed'
}
```

Adding to arrays

At this point, you're starting to wonder how to add items to an array. The quick answer is that you can't. An array is a fixed size in memory. If you need to grow it or add a single item to it, then you need to create a new array and copy all the values over from the old array. This sounds like a lot of work, however, PowerShell hides the complexity of creating the new array. PowerShell implements the addition operator (`+`) for arrays.

> ⓘ **Note**
>
> PowerShell does not implement a subtraction operation. If you want a flexible alternative to an array, you need to use a generic List object.

Array addition

We can use the addition operator with arrays to create a new array. So given these two arrays:

PowerShell

```
$first = @(
    'Zero'
    'One'
)
$second = @(
    'Two'
    'Three'
)
```

We can add them together to get a new array.

PowerShell

```
PS> $first + $second

Zero
One
```

```
Two
Three
```

Plus equals +=

We can create a new array in place and add an item to it like this:

PowerShell

```powershell
$data = @(
    'Zero'
    'One'
    'Two'
    'Three'
)
$data += 'four'
```

Just remember that every time you use `+=` that you're duplicating and creating a new array. This is a not an issue for small datasets but it scales extremely poorly.

Pipeline assignment

You can assign the results of any pipeline into a variable. It's an array if it contains multiple items.

PowerShell

```powershell
$array = 1..5 | ForEach-Object {
    "ATX-SQL-$PSItem"
}
```

Normally when we think of using the pipeline, we think of the typical PowerShell one-liners. We can leverage the pipeline with `foreach()` statements and other loops. So instead of adding items to an array in a loop, we can drop items onto the pipeline.

PowerShell

```powershell
$array = foreach ( $node in (1..5))
{
    "ATX-SQL-$node"
}
```

Array Types

By default, an array in PowerShell is created as a `[psobject[]]` type. This allows it to contain any type of object or value. This works because everything is inherited from the `PSObject` type.

Strongly typed arrays

You can create an array of any type using a similar syntax. When you create a strongly typed array, it can only contain values or objects the specified type.

PowerShell

```
PS> [int[]] $numbers = 1,2,3
PS> [int[]] $numbers2 = 'one','two','three'
ERROR: Cannot convert value "one" to type "System.Int32". Input string was not in a
correct format."

PS> [string[]] $strings = 'one','two','three'
```

ArrayList

Adding items to an array is one of its biggest limitations, but there are a few other collections that we can turn to that solve this problem.

The `ArrayList` is commonly one of the first things that we think of when we need an array that is faster to work with. It acts like an object array every place that we need it, but it handles adding items quickly.

Here is how we create an `ArrayList` and add items to it.

PowerShell

```
$myarray = [System.Collections.ArrayList]::new()
[void]$myArray.Add('Value')
```

We are calling into .NET to get this type. In this case, we are using the default constructor to create it. Then we call the `Add` method to add an item to it.

The reason I'm using `[void]` at the beginning of the line is to suppress the return code. Some .NET calls do this and can create unexpected output.

If the only data that you have in your array is strings, then also take a look at using StringBuilder. It's almost the same thing but has some methods that are just for dealing with strings. The `StringBuilder` is specially designed for performance.

It's common to see people move to `ArrayList` from arrays. But it comes from a time where C# didn't have generic support. The `ArrayList` is deprecated in support for the generic `List[]`

Generic List

A generic type is a special type in C# that defines a generalized class and the user specifies the data types it uses when created. So if you want a list of numbers or strings, you would define that you want list of `int` or `string` types.

Here is how you create a List for strings.

PowerShell

```
$mylist = [System.Collections.Generic.List[string]]::new()
```

Or a list for numbers.

PowerShell

```
$mylist = [System.Collections.Generic.List[int]]::new()
```

We can cast an existing array to a list like this without creating the object first:

PowerShell

```
$mylist = [System.Collections.Generic.List[int]]@(1,2,3)
```

We can shorten the syntax with the `using namespace` statement in PowerShell 5 and newer. The `using` statement needs to be the first line of your script. By declaring a namespace, PowerShell lets you leave it off of the data types when you reference them.

PowerShell

```
using namespace System.Collections.Generic
$myList = [List[int]]@(1,2,3)
```

This makes the `List` much more usable.

You have a similar `Add` method available to you. Unlike the ArrayList, there is no return value on the `Add` method so we don't have to `void` it.

PowerShell

```
$myList.Add(10)
```

And we can still access the elements like other arrays.

PowerShell

```
PS> $myList[-1]
10
```

List[psobject]

You can have a list of any type, but when you don't know the type of objects, you can use `[List[psobject]]` to contain them.

PowerShell

```
$list = [List[psobject]]::new()
```

Remove()

The `ArrayList` and the generic `List[]` both support removing items from the collection.

PowerShell

```
using namespace System.Collections.Generic
$myList = [List[string]]@('Zero','One','Two','Three')
[void]$myList.Remove("Two")
Zero
One
Three
```

When working with value types, it removes the first one from the list. You can call it over and over again to keep removing that value. If you have reference types, you have to provide the object that you want removed.

PowerShell

```
[List[System.Management.Automation.PSDriveInfo]]$drives = Get-PSDrive
$drives.Remove($drives[2])
```

PowerShell

```
$delete = $drives[2]
```

```
$drives.Remove($delete)
```

The remove method returns `true` if it was able to find and remove the item from the collection.

More collections

There are many other collections that can be used but these are the good generic array replacements. If you're interested in learning about more of these options, take a look at this Gist that Mark Kraus put together.

Other nuances

Now that I have covered all the major functionality, here are a few more things that I wanted to mention before I wrap this up.

Pre-sized arrays

I mentioned that you can't change the size of an array once it's created. We can create an array of a pre-determined size by calling it with the `new($size)` constructor.

PowerShell

```
$data = [Object[]]::new(4)
$data.Count
4
```

Multiplying arrays

An interesting little trick is that you can multiply an array by an integer.

PowerShell

```
PS> $data = @('red','green','blue')
PS> $data * 3
red
green
blue
red
green
blue
red
green
blue
```

Initialize with 0

A common scenario is that you want to create an array with all zeros. If you're only going to have integers, a strongly typed array of integers defaults to all zeros.

PowerShell

```
PS> [int[]]::new(4)
0
0
0
0
```

We can use the multiplying trick to do this too.

PowerShell

```
PS> $data = @(0) * 4
PS> $data
0
0
0
0
```

The nice thing about the multiplying trick is that you can use any value. So if you would rather have `255` as your default value, this would be a good way to do it.

PowerShell

```
PS> $data = @(255) * 4
PS> $data
255
255
255
255
```

Nested arrays

An array inside an array is called a nested array. I don't use these much in PowerShell but I have used them more in other languages. Consider using an array of arrays when your data fits in a grid like pattern.

Here are two ways we can create a two-dimensional array.

PowerShell

```
$data = @(@(1,2,3),@(4,5,6),@(7,8,9))

$data2 = @(
    @(1,2,3),
    @(4,5,6),
    @(7,8,9)
)
```

The comma is very important in those examples. I gave an earlier example of a normal array on multiple lines where the comma was optional. That isn't the case with a multi-dimensional array.

The way we use the index notation changes slightly now that we've a nested array. Using the `$data` above, this is how we would access the value 3.

PowerShell

```
PS> $outside = 0
PS> $inside = 2
PS> $data[$outside][$inside]
3
```

Add a set of bracket for each level of array nesting. The first set of brackets is for the outer most array and then you work your way in from there.

Write-Output -NoEnumerate

PowerShell likes to unwrap or enumerate arrays. This is a core aspect of the way PowerShell uses the pipeline but there are times that you don't want that to happen.

I commonly pipe objects to `Get-Member` to learn more about them. When I pipe an array to it, it gets unwrapped and Get-Member sees the members of the array and not the actual array.

PowerShell

```
PS> $data = @('red','green','blue')
PS> $data | Get-Member
TypeName: System.String
...
```

To prevent that unwrap of the array, you can use `Write-Output -NoEnumerate`.

PowerShell

```
PS> Write-Output -NoEnumerate $data | Get-Member
TypeName: System.Object[]
...
```

I have a second way that's more of a hack (and I try to avoid hacks like this). You can place a comma in front of the array before you pipe it. This wraps `$data` into another array where it is the only element, so after the unwrapping the outer array we get back `$data` unwrapped.

PowerShell

```
PS> ,$data | Get-Member
TypeName: System.Object[]
...
```

Return an array

This unwrapping of arrays also happens when you output or return values from a function. You can still get an array if you assign the output to a variable so this isn't commonly an issue.

The catch is that you have a new array. If that is ever a problem, you can use `Write-Output -NoEnumerate $array` or `return ,$array` to work around it.

Anything else?

I know this is all a lot to take in. My hope is that you learn something from this article every time you read it and that it turns out to be a good reference for you for a long time to come. If you found this to be helpful, please share it with others you think may get value out of it.

From here, I would recommend you check out a similar post that I wrote about hashtables.

Last updated on 03/24/2025

Everything you wanted to know about hashtables

I want to take a step back and talk about hashtables. I use them all the time now. I was teaching someone about them after our user group meeting last night and I realized I had the same confusion about them as he had. Hashtables are really important in PowerShell so it's good to have a solid understanding of them.

> **Note**
>
> The original version of this article appeared on the blog written by @KevinMarquette. The PowerShell team thanks Kevin for sharing this content with us. Please check out his blog at PowerShellExplained.com.

Hashtable as a collection of things

I want you to first see a **Hashtable** as a collection in the traditional definition of a hashtable. This definition gives you a fundamental understanding of how they work when they get used for more advanced stuff later. Skipping this understanding is often a source of confusion.

What is an array?

Before I jump into what a **Hashtable** is, I need to mention arrays first. For the purpose of this discussion, an array is a list or collection of values or objects.

PowerShell

```
$array = @(1,2,3,5,7,11)
```

Once you have your items into an array, you can either use `foreach` to iterate over the list or use an index to access individual elements in the array.

PowerShell

```
foreach($item in $array)
{
    Write-Output $item
}
```

```
Write-Output $array[3]
```

You can also update values using an index in the same way.

PowerShell

```
$array[2] = 13
```

I just scratched the surface on arrays but that should put them into the right context as I move onto hashtables.

What is a hashtable?

I'm going to start with a basic technical description of what hashtables are, in the general sense, before I shift into the other ways PowerShell uses them.

A hashtable is a data structure, much like an array, except you store each value (object) using a key. It's a basic key/value store. First, we create an empty hashtable.

PowerShell

```
$ageList = @{}
```

Notice that braces, instead of parentheses, are used to define a hashtable. Then we add an item using a key like this:

PowerShell

```
$key = 'Kevin'
$value = 36
$ageList.Add( $key, $value )

$ageList.Add( 'Alex', 9 )
```

The person's name is the key and their age is the value that I want to save.

Using the brackets for access

Once you add your values to the hashtable, you can pull them back out using that same key (instead of using a numeric index like you would have for an array).

PowerShell

```
$ageList['Kevin']
$ageList['Alex']
```

When I want Kevin's age, I use his name to access it. We can use this approach to add or update values into the hashtable too. This is just like using the `Add()` method above.

PowerShell

```
$ageList = @{}

$key = 'Kevin'
$value = 36
$ageList[$key] = $value

$ageList['Alex'] = 9
```

There's another syntax you can use for accessing and updating values that I'll cover in a later section. If you're coming to PowerShell from another language, these examples should fit in with how you may have used hashtables before.

Creating hashtables with values

So far I've created an empty hashtable for these examples. You can pre-populate the keys and values when you create them.

PowerShell

```
$ageList = @{
    Kevin = 36
    Alex  = 9
}
```

As a lookup table

The real value of this type of a hashtable is that you can use them as a lookup table. Here is a simple example.

PowerShell

```
$environments = @{
    Prod = 'SrvProd05'
    QA   = 'SrvQA02'
    Dev  = 'SrvDev12'
}
```

```
$server = $environments[$env]
```

In this example, you specify an environment for the `$env` variable and it will pick the correct server. You could use a `switch($env){...}` for a selection like this but a hashtable is a nice option.

This gets even better when you dynamically build the lookup table to use it later. So think about using this approach when you need to cross reference something. I think we would see this even more if PowerShell wasn't so good at filtering on the pipe with `Where-Object`. If you're ever in a situation where performance matters, this approach needs to be considered.

I won't say that it's faster, but it does fit into the rule of If performance matters, test it.

Multiselection

Generally, you think of a hashtable as a key/value pair, where you provide one key and get one value. PowerShell allows you to provide an array of keys to get multiple values.

PowerShell

```
$environments[@('QA','DEV')]
$environments[('QA','DEV')]
$environments['QA','DEV']
```

In this example, I use the same lookup hashtable from above and provide three different array styles to get the matches. This is a hidden gem in PowerShell that most people aren't aware of.

Iterating hashtables

Because a hashtable is a collection of key/value pairs, you iterate over it differently than you do for an array or a normal list of items.

The first thing to notice is that if you pipe your hashtable, the pipe treats it like one object.

PowerShell

```
PS> $ageList | Measure-Object
count : 1
```

Even though the `Count` property tells you how many values it contains.

PowerShell

```
PS> $ageList.Count
2
```

You get around this issue by using the `Values` property if all you need is just the values.

PowerShell

```
PS> $ageList.Values | Measure-Object -Average
Count   : 2
Average : 22.5
```

It's often more useful to enumerate the keys and use them to access the values.

PowerShell

```
PS> $ageList.Keys | ForEach-Object{
    $message = '{0} is {1} years old!' -f $_, $ageList[$_]
    Write-Output $message
}
Kevin is 36 years old
Alex is 9 years old
```

Here is the same example with a `foreach(){...}` loop.

PowerShell

```
foreach($key in $ageList.Keys)
{
    $message = '{0} is {1} years old' -f $key, $ageList[$key]
    Write-Output $message
}
```

We are walking each key in the hashtable and then using it to access the value. This is a common pattern when working with hashtables as a collection.

GetEnumerator()

That brings us to `GetEnumerator()` for iterating over our hashtable.

PowerShell

```
$ageList.GetEnumerator() | ForEach-Object{
    $message = '{0} is {1} years old!' -f $_.Key, $_.Value
    Write-Output $message
}
```

The enumerator gives you each key/value pair one after another. It was designed specifically for this use case. Thank you to Mark Kraus for reminding me of this one.

BadEnumeration

One important detail is that you can't modify a hashtable while it's being enumerated. If we start with our basic `$environments` example:

PowerShell

```
$environments = @{
    Prod = 'SrvProd05'
    QA   = 'SrvQA02'
    Dev  = 'SrvDev12'
}
```

And trying to set every key to the same server value fails.

PowerShell

```
$environments.Keys | ForEach-Object {
    $environments[$_] = 'SrvDev03'
}

An error occurred while enumerating through a collection: Collection was modified;
enumeration operation may not execute.
+ CategoryInfo          : InvalidOperation: tableEnumerator:HashtableEnumerator) [],
 RuntimeException
+ FullyQualifiedErrorId : BadEnumeration
```

This will also fail even though it looks like it should also be fine:

PowerShell

```
foreach($key in $environments.Keys) {
    $environments[$key] = 'SrvDev03'
}

Collection was modified; enumeration operation may not execute.
    + CategoryInfo          : OperationStopped: (:) [], InvalidOperationException
    + FullyQualifiedErrorId : System.InvalidOperationException
```

The trick to this situation is to clone the keys before doing the enumeration.

PowerShell

```
$environments.Keys.Clone() | ForEach-Object {
    $environments[$_] = 'SrvDev03'
}
```

> ⓘ Note
>
> You can't clone a hashtable containing a single key. PowerShell throws an error. Instead, you convert the **Keys** property to an array, then iterate over the array.

PowerShell

```
@($environments.Keys) | ForEach-Object {
    $environments[$_] = 'SrvDev03'
}
```

Hashtable as a collection of properties

So far the type of objects we placed in our hashtable were all the same type of object. I used ages in all those examples and the key was the person's name. This is a great way to look at it when your collection of objects each have a name. Another common way to use hashtables in PowerShell is to hold a collection of properties where the key is the name of the property. I'll step into that idea in this next example.

Property-based access

The use of property-based access changes the dynamics of hashtables and how you can use them in PowerShell. Here is our usual example from above treating the keys as properties.

PowerShell

```
$ageList = @{}
$ageList.Kevin = 35
$ageList.Alex = 9
```

Just like the examples above, this example adds those keys if they don't exist in the hashtable already. Depending on how you defined your keys and what your values are, this is either a little strange or a perfect fit. The age list example has worked great up until this point. We need a new example for this to feel right going forward.

PowerShell

```
$person = @{
    name = 'Kevin'
    age  = 36
}
```

And we can add and access attributes on the `$person` like this.

PowerShell

```
$person.city = 'Austin'
$person.state = 'TX'
```

All of a sudden this hashtable starts to feel and act like an object. It's still a collection of things, so all the examples above still apply. We just approach it from a different point of view.

Checking for keys and values

In most cases, you can just test for the value with something like this:

PowerShell

```
if( $person.age ){...}
```

It's simple but has been the source of many bugs for me because I was overlooking one important detail in my logic. I started to use it to test if a key was present. When the value was `$false` or zero, that statement would return `$false` unexpectedly.

PowerShell

```
if( $null -ne $person.age ){...}
```

This works around that issue for zero values but not $null vs non-existent keys. Most of the time you don't need to make that distinction but there are methods for when you do.

PowerShell

```
if( $person.ContainsKey('age') ){...}
```

We also have a `ContainsValue()` for the situation where you need to test for a value without knowing the key or iterating the whole collection.

Removing and clearing keys

You can remove keys with the `Remove()` method.

PowerShell

```
$person.Remove('age')
```

Assigning them a `$null` value just leaves you with a key that has a `$null` value.

A common way to clear a hashtable is to just initialize it to an empty hashtable.

PowerShell

```
$person = @{}
```

While that does work, try to use the `Clear()` method instead.

PowerShell

```
$person.Clear()
```

This is one of those instances where using the method creates self-documenting code and it makes the intentions of the code very clean.

All the fun stuff

Ordered hashtables

By default, hashtables aren't ordered (or sorted). In the traditional context, the order doesn't matter when you always use a key to access values. You may find that you want the properties to stay in the order that you define them. Thankfully, there's a way to do that with the `ordered` keyword.

PowerShell

```
$person = [ordered]@{
    name = 'Kevin'
    age  = 36
}
```

Now when you enumerate the keys and values, they stay in that order.

Inline hashtables

When you're defining a hashtable on one line, you can separate the key/value pairs with a semicolon.

PowerShell

```
$person = @{ name = 'kevin'; age = 36; }
```

This will come in handy if you're creating them on the pipe.

Custom expressions in common pipeline commands

There are a few cmdlets that support the use of hashtables to create custom or calculated properties. You commonly see this with `Select-Object` and `Format-Table`. The hashtables have a special syntax that looks like this when fully expanded.

PowerShell

```
$property = @{
    Name = 'TotalSpaceGB'
    Expression = { ($_.Used + $_.Free) / 1GB }
}
```

The `Name` is what the cmdlet would label that column. The `Expression` is a script block that is executed where `$_` is the value of the object on the pipe. Here is that script in action:

PowerShell

```
$drives = Get-PSDrive | where Used
$drives | Select-Object -Property Name, $property

Name     TotalSpaceGB
----     ------------
C    238.472652435303
```

I placed that in a variable but it could easily be defined inline and you can shorten `Name` to `n` and `Expression` to `e` while you're at it.

PowerShell

```
$drives | Select-Object -Property Name, @{n='TotalSpaceGB';e={($_.Used + $_.Free) /
1GB}}
```

I personally don't like how long that makes commands and it often promotes some bad behaviors that I won't get into. I'm more likely to create a new hashtable or `pscustomobject` with

all the fields and properties that I want instead of using this approach in scripts. But there's a lot of code out there that does this so I wanted you to be aware of it. I talk about creating a `pscustomobject` later on.

Custom sort expression

It's easy to sort a collection if the objects have the data that you want to sort on. You can either add the data to the object before you sort it or create a custom expression for `Sort-Object`.

PowerShell

```
Get-ADUser | Sort-Object -Property @{ e={ Get-TotalSales $_.Name } }
```

In this example I'm taking a list of users and using some custom cmdlet to get additional information just for the sort.

Sort a list of Hashtables

If you have a list of hashtables that you want to sort, you'll find that the `Sort-Object` doesn't treat your keys as properties. We can get a round that by using a custom sort expression.

PowerShell

```
$data = @(
    @{name='a'}
    @{name='c'}
    @{name='e'}
    @{name='f'}
    @{name='d'}
    @{name='b'}
)

$data | Sort-Object -Property @{e={$_.name}}
```

Splatting hashtables at cmdlets

This is one of my favorite things about hashtables that many people don't discover early on. The idea is that instead of providing all the properties to a cmdlet on one line, you can instead pack them into a hashtable first. Then you can give the hashtable to the function in a special way. Here is an example of creating a DHCP scope the normal way.

PowerShell

```
Add-DhcpServerV4Scope -Name 'TestNetwork' -StartRange '10.0.0.2' -EndRange
'10.0.0.254' -SubnetMask '255.255.255.0' -Description 'Network for testlab A' -
LeaseDuration (New-TimeSpan -Days 8) -Type "Both"
```

Without using splatting, all those things need to be defined on a single line. It either scrolls off the screen or will wrap where ever it feels like. Now compare that to a command that uses splatting.

PowerShell

```
$DHCPScope = @{
    Name          = 'TestNetwork'
    StartRange    = '10.0.0.2'
    EndRange      = '10.0.0.254'
    SubnetMask    = '255.255.255.0'
    Description   = 'Network for testlab A'
    LeaseDuration = (New-TimeSpan -Days 8)
    Type          = "Both"
}
Add-DhcpServerV4Scope @DHCPScope
```

The use of the `@` sign instead of the `$` is what invokes the splat operation.

Just take a moment to appreciate how easy that example is to read. They are the exact same command with all the same values. The second one is easier to understand and maintain going forward.

I use splatting anytime the command gets too long. I define too long as causing my window to scroll right. If I hit three properties for a function, odds are that I'll rewrite it using a splatted hashtable.

Splatting for optional parameters

One of the most common ways I use splatting is to deal with optional parameters that come from some place else in my script. Let's say I have a function that wraps a `Get-CimInstance` call that has an optional `$Credential` argument.

PowerShell

```
$CIMParams = @{
    ClassName = 'Win32_BIOS'
    ComputerName = $ComputerName
}

if($Credential)
```

```
{
    $CIMParams.Credential = $Credential
}

Get-CimInstance @CIMParams
```

I start by creating my hashtable with common parameters. Then I add the `$Credential` if it exists. Because I'm using splatting here, I only need to have the call to `Get-CimInstance` in my code once. This design pattern is very clean and can handle lots of optional parameters easily.

To be fair, you could write your commands to allow `$null` values for parameters. You just don't always have control over the other commands you're calling.

Multiple splats

You can splat multiple hashtables to the same cmdlet. If we revisit our original splatting example:

PowerShell

```
$Common = @{
    SubnetMask  = '255.255.255.0'
    LeaseDuration = (New-TimeSpan -Days 8)
    Type = "Both"
}

$DHCPScope = @{
    Name        = 'TestNetwork'
    StartRange  = '10.0.0.2'
    EndRange    = '10.0.0.254'
    Description = 'Network for testlab A'
}

Add-DhcpServerv4Scope @DHCPScope @Common
```

I'll use this method when I have a common set of parameters that I'm passing to lots of commands.

Splatting for clean code

There's nothing wrong with splatting a single parameter if makes you code cleaner.

PowerShell

```
$log = @{Path = '.\logfile.log'}
Add-Content "logging this command" @log
```

Splatting executables

Splatting also works on some executables that use a `/param:value` syntax. `Robocopy.exe`, for example, has some parameters like this.

PowerShell

```
$robo = @{R=1;W=1;MT=8}
robocopy source destination @robo
```

I don't know that this is all that useful, but I found it interesting.

Adding hashtables

Hashtables support the addition operator to combine two hashtables.

PowerShell

```
$person += @{Zip = '78701'}
```

This only works if the two hashtables don't share a key.

Nested hashtables

We can use hashtables as values inside a hashtable.

PowerShell

```
$person = @{
    name = 'Kevin'
    age  = 36
}
$person.location = @{}
$person.location.city = 'Austin'
$person.location.state = 'TX'
```

I started with a basic hashtable containing two keys. I added a key called `location` with an empty hashtable. Then I added the last two items to that `location` hashtable. We can do this all inline too.

PowerShell

```
$person = @{
    name = 'Kevin'
```

```
    age  = 36
    location = @{
        city  = 'Austin'
        state = 'TX'
    }
}
```

This creates the same hashtable that we saw above and can access the properties the same way.

PowerShell

```
$person.location.city
Austin
```

There are many ways to approach the structure of your objects. Here is a second way to look at a nested hashtable.

PowerShell

```
$people = @{
    Kevin = @{
        age  = 36
        city = 'Austin'
    }
    Alex = @{
        age  = 9
        city = 'Austin'
    }
}
```

This mixes the concept of using hashtables as a collection of objects and a collection of properties. The values are still easy to access even when they're nested using whatever approach you prefer.

PowerShell

```
PS> $people.kevin.age
36
PS> $people.kevin['city']
Austin
PS> $people['Alex'].age
9
PS> $people['Alex']['City']
Austin
```

I tend to use the dot property when I'm treating it like a property. Those are generally things I've defined statically in my code and I know them off the top of my head. If I need to walk the list or

programmatically access the keys, I use the brackets to provide the key name.

PowerShell

```powershell
foreach($name in $people.Keys)
{
    $person = $people[$name]
    '{0}, age {1}, is in {2}' -f $name, $person.age, $person.city
}
```

Having the ability to nest hashtables gives you a lot of flexibility and options.

Looking at nested hashtables

As soon as you start nesting hashtables, you're going to need an easy way to look at them from the console. If I take that last hashtable, I get an output that looks like this and it only goes so deep:

PowerShell

```
PS> $people
Name                           Value
----                           -----
Kevin                          {age, city}
Alex                           {age, city}
```

My go to command for looking at these things is `ConvertTo-Json` because it's very clean and I frequently use JSON on other things.

PowerShell

```powershell
PS> $people | ConvertTo-Json
{
    "Kevin":  {
                  "age":  36,
                  "city":  "Austin"
              },
    "Alex":  {
                  "age":  9,
                  "city":  "Austin"
              }
}
```

Even if you don't know JSON, you should be able to see what you're looking for. There's a `Format-Custom` command for structured data like this but I still like the JSON view better.

Creating objects

Sometimes you just need to have an object and using a hashtable to hold properties just isn't getting the job done. Most commonly you want to see the keys as column names. A `pscustomobject` makes that easy.

PowerShell

```
$person = [pscustomobject]@{
    name = 'Kevin'
    age  = 36
}

$person

name  age
----  ---
Kevin  36
```

Even if you don't create it as a `pscustomobject` initially, you can always cast it later when needed.

PowerShell

```
$person = @{
    name = 'Kevin'
    age  = 36
}

[pscustomobject]$person

name  age
----  ---
Kevin  36
```

I already have detailed write-up for pscustomobject that you should go read after this one. It builds on a lot of the things learned here.

Reading and writing hashtables to file

Saving to CSV

Struggling with getting a hashtable to save to a CSV is one of the difficulties that I was referring to above. Convert your hashtable to a `pscustomobject` and it will save correctly to CSV. It helps if you start with a `pscustomobject` so the column order is preserved. But you can cast it to a `pscustomobject` inline if needed.

PowerShell

```
$person | ForEach-Object{ [pscustomobject]$_ } | Export-Csv -Path $path
```

Again, check out my write-up on using a pscustomobject.

Saving a nested hashtable to file

If I need to save a nested hashtable to a file and then read it back in again, I use the JSON cmdlets to do it.

PowerShell

```
$people | ConvertTo-Json | Set-Content -Path $path
$people = Get-Content -Path $path -Raw | ConvertFrom-Json
```

There are two important points about this method. First is that the JSON is written out multiline so I need to use the `-Raw` option to read it back into a single string. The Second is that the imported object is no longer a `[hashtable]`. It's now a `[pscustomobject]` and that can cause issues if you don't expect it.

Watch for deeply-nested hashtables. When you convert it to JSON you might not get the results you expect.

PowerShell

```
@{ a = @{ b = @{ c = @{ d = "e" }}}} | ConvertTo-Json

{
  "a": {
    "b": {
      "c": "System.Collections.Hashtable"
    }
  }
}
```

Use **Depth** parameter to ensure that you have expanded all the nested hashtables.

PowerShell

```
@{ a = @{ b = @{ c = @{ d = "e" }}}} | ConvertTo-Json -Depth 3

{
  "a": {
    "b": {
      "c": {
```

```
        "d": "e"
      }
    }
  }
}
```

If you need it to be a `[hashtable]` on import, then you need to use the `Export-CliXml` and `Import-CliXml` commands.

Converting JSON to Hashtable

If you need to convert JSON to a `[hashtable]`, there's one way that I know of to do it with the JavaScriptSerializer in .NET.

PowerShell

```powershell
[Reflection.Assembly]::LoadWithPartialName("System.Web.Script.Serialization")
$JSSerializer = [System.Web.Script.Serialization.JavaScriptSerializer]::new()
$JSSerializer.Deserialize($json,'Hashtable')
```

Beginning in PowerShell v6, JSON support uses the NewtonSoft JSON.NET and adds hashtable support.

PowerShell

```powershell
'{ "a": "b" }' | ConvertFrom-Json -AsHashtable

Name      Value
----      -----
a         b
```

PowerShell 6.2 added the **Depth** parameter to `ConvertFrom-Json`. The default **Depth** is 1024.

Reading directly from a file

If you have a file that contains a hashtable using PowerShell syntax, there's a way to import it directly.

PowerShell

```powershell
$content = Get-Content -Path $Path -Raw -ErrorAction Stop
$scriptBlock = [scriptblock]::Create( $content )
$scriptBlock.CheckRestrictedLanguage( $allowedCommands, $allowedVariables, $true )
$hashtable = ( & $scriptBlock )
```

It imports the contents of the file into a `scriptblock`, then checks to make sure it doesn't have any other PowerShell commands in it before it executes it.

On that note, did you know that a module manifest (the `.psd1` file) is just a hashtable?

Keys can be any object

Most of the time, the keys are just strings. So we can put quotes around anything and make it a key.

PowerShell

```
$person = @{
    'full name' = 'Kevin Marquette'
    '#' = 3978
}
$person['full name']
```

You can do some odd things that you may not have realized you could do.

PowerShell

```
$person.'full name'

$key = 'full name'
$person.$key
```

Just because you can do something, it doesn't mean that you should. That last one just looks like a bug waiting to happen and would be easily misunderstood by anyone reading your code.

Technically your key doesn't have to be a string but they're easier to think about if you only use strings. However, indexing doesn't work well with the complex keys.

PowerShell

```
$ht = @{ @(1,2,3) = "a" }
$ht

Name                           Value
----                           -----
{1, 2, 3}                      a
```

Accessing a value in the hashtable by its key doesn't always work. For example:

PowerShell

```
$key = $ht.Keys[0]
$ht.$($key)
a
$ht[$key]
a
```

When the key is an array, you must wrap the `$key` variable in a subexpression so that it can be used with member access (`.`) notation. Or, you can use array index (`[]`) notation.

Use in automatic variables

$PSBoundParameters

`$PSBoundParameters` is an automatic variable that only exists inside the context of a function. It contains all the parameters that the function was called with. This isn't exactly a hashtable but close enough that you can treat it like one.

That includes removing keys and splatting it to other functions. If you find yourself writing proxy functions, take a closer look at this one.

See about_Automatic_Variables for more details.

PSBoundParameters gotcha

One important thing to remember is that this only includes the values that are passed in as parameters. If you also have parameters with default values but aren't passed in by the caller, `$PSBoundParameters` doesn't contain those values. This is commonly overlooked.

$PSDefaultParameterValues

This automatic variable lets you assign default values to any cmdlet without changing the cmdlet. Take a look at this example.

PowerShell

```
$PSDefaultParameterValues["Out-File:Encoding"] = "UTF8"
```

This adds an entry to the `$PSDefaultParameterValues` hashtable that sets `UTF8` as the default value for the `Out-File -Encoding` parameter. This is session-specific so you should place it in your `$PROFILE`.

I use this often to pre-assign values that I type quite often.

PowerShell

```
$PSDefaultParameterValues[ "Connect-VIServer:Server" ] = 'VCENTER01.contoso.local'
```

This also accepts wildcards so you can set values in bulk. Here are some ways you can use that:

PowerShell

```
$PSDefaultParameterValues[ "Get-*:Verbose" ] = $true
$PSDefaultParameterValues[ "*:Credential" ] = Get-Credential
```

For a more in-depth breakdown, see this great article on Automatic Defaults by Michael Sorens.

Regex $Matches

When you use the `-match` operator, an automatic variable called `$Matches` is created with the results of the match. If you have any sub expressions in your regex, those sub matches are also listed.

PowerShell

```
$message = 'My SSN is 123-45-6789.'

$message -match 'My SSN is (.+)\.'
$Matches[0]
$Matches[1]
```

Named matches

This is one of my favorite features that most people don't know about. If you use a named regex match, then you can access that match by name on the matches.

PowerShell

```
$message = 'My Name is Kevin and my SSN is 123-45-6789.'

if($message -match 'My Name is (?<Name>.+) and my SSN is (?<SSN>.+)\.')
{
    $Matches.Name
    $Matches.SSN
}
```

In the example above, the `(?<Name>.*)` is a named sub expression. This value is then placed in the `$Matches.Name` property.

Group-Object -AsHashtable

One little known feature of `Group-Object` is that it can turn some datasets into a hashtable for you.

PowerShell

```
Import-Csv $Path | Group-Object -AsHashtable -Property Email
```

This will add each row into a hashtable and use the specified property as the key to access it.

Copying Hashtables

One important thing to know is that hashtables are objects. And each variable is just a reference to an object. This means that it takes more work to make a valid copy of a hashtable.

Assigning reference types

When you have one hashtable and assign it to a second variable, both variables point to the same hashtable.

PowerShell

```
PS> $orig = @{name='orig'}
PS> $copy = $orig
PS> $copy.name = 'copy'
PS> 'Copy: [{0}]' -f $copy.name
PS> 'Orig: [{0}]' -f $orig.name

Copy: [copy]
Orig: [copy]
```

This highlights that they're the same because altering the values in one will also alter the values in the other. This also applies when passing hashtables into other functions. If those functions make changes to that hashtable, your original is also altered.

Shallow copies, single level

If we have a simple hashtable like our example above, we can use `Clone()` to make a shallow copy.

PowerShell

```
PS> $orig = @{name='orig'}
PS> $copy = $orig.Clone()
PS> $copy.name = 'copy'
PS> 'Copy: [{0}]' -f $copy.name
PS> 'Orig: [{0}]' -f $orig.name

Copy: [copy]
Orig: [orig]
```

This will allow us to make some basic changes to one that don't impact the other.

Shallow copies, nested

The reason why it's called a shallow copy is because it only copies the base level properties. If one of those properties is a reference type (like another hashtable), then those nested objects will still point to each other.

PowerShell

```
PS> $orig = @{
        person=@{
            name='orig'
        }
    }
PS> $copy = $orig.Clone()
PS> $copy.person.name = 'copy'
PS> 'Copy: [{0}]' -f $copy.person.name
PS> 'Orig: [{0}]' -f $orig.person.name

Copy: [copy]
Orig: [copy]
```

So you can see that even though I cloned the hashtable, the reference to `person` wasn't cloned. We need to make a deep copy to truly have a second hashtable that isn't linked to the first.

Deep copies

There are a couple of ways to make a deep copy of a hashtable (and keep it as a hashtable). Here's a function using PowerShell to recursively create a deep copy:

PowerShell

```
function Get-DeepClone
{
    [CmdletBinding()]
```

```
    param(
        $InputObject
    )
    process
    {
        if($InputObject -is [hashtable]) {
            $clone = @{}
            foreach($key in $InputObject.Keys)
            {
                $clone[$key] = Get-DeepClone $InputObject[$key]
            }
            return $clone
        } else {
            return $InputObject
        }
    }
}
```

It doesn't handle any other reference types or arrays, but it's a good starting point.

Another way is to use .NET to deserialize it using **CliXml** like in this function:

PowerShell

```
function Get-DeepClone
{
    param(
        $InputObject
    )
    $TempCliXmlString = [System.Management.Automation.PSSerializer]::Serialize(
        $InputObject, [int32]::MaxValue)
    return [System.Management.Automation.PSSerializer]::Deserialize($TempCliXmlString)
}
```

For extremely large hashtables, the deserializing function is faster as it scales out. However, there are some things to consider when using this method. Since it uses **CliXml**, it's memory intensive and if you are cloning huge hashtables, that might be a problem. Another limitation of the **CliXml** is there is a depth limitation of 48. Meaning, if you have a hashtable with 48 layers of nested hashtables, the cloning will fail and no hashtable will be output at all.

Anything else?

I covered a lot of ground quickly. My hope is that you walk away leaning something new or understanding it better every time you read this. Because I covered the full spectrum of this feature, there are aspects that just may not apply to you right now. That is perfectly OK and is kind of expected depending on how much you work with PowerShell.

Last updated on 07/13/2026

Everything you wanted to know about PSCustomObject

`PSCustomObject` is a great tool to add into your PowerShell tool belt. Let's start with the basics and work our way into the more advanced features. The idea behind using a `PSCustomObject` is to have a simple way to create structured data. Take a look at the first example and you'll have a better idea of what that means.

> **Note**
>
> The original version of this article appeared on the blog written by @KevinMarquette. The PowerShell team thanks Kevin for sharing this content with us. Please check out his blog at PowerShellExplained.com.

Creating a PSCustomObject

I love using `[pscustomobject]` in PowerShell. Creating a usable object has never been easier. Because of that, I'm going to skip over all the other ways you can create an object but I need to mention that most of these examples are PowerShell v3.0 and newer.

PowerShell

```
$myObject = [pscustomobject]@{
    Name     = 'Kevin'
    Language = 'PowerShell'
    State    = 'Texas'
}
```

This method works well for me because I use hashtables for just about everything. But there are times when I would like PowerShell to treat hashtables more like an object. The first place you notice the difference is when you want to use `Format-Table` or `Export-Csv` and you realize that a hashtable is just a collection of key/value pairs.

You can then access and use the values like you would a normal object.

PowerShell

```
$myObject.Name
```

Converting a hashtable

While I am on the topic, did you know you could do this:

PowerShell

```
$myHashtable = @{
    Name     = 'Kevin'
    Language = 'PowerShell'
    State    = 'Texas'
}
$myObject = [pscustomobject]$myHashtable
```

I do prefer to create the object from the start but there are times you have to work with a hashtable first. This example works because the constructor takes a hashtable for the object properties. One important note is that while this method works, it isn't an exact equivalent. The biggest difference is that the order of the properties isn't preserved.

If you want to preserve the order, see Ordered hashtables.

Legacy approach

You may have seen people use `New-Object` to create custom objects.

PowerShell

```
$myHashtable = @{
    Name     = 'Kevin'
    Language = 'PowerShell'
    State    = 'Texas'
}

$myObject = New-Object -TypeName psobject -Property $myHashtable
```

This way is quite a bit slower but it may be your best option on early versions of PowerShell.

Saving to a file

I find the best way to save a hashtable to a file is to save it as JSON. You can import it back into a `[pscustomobject]`

PowerShell

```
$myObject | ConvertTo-Json -Depth 1 | Set-Content -Path $Path
$myObject = Get-Content -Path $Path | ConvertFrom-Json
```

I cover more ways to save objects to a file in my article on The many ways to read and write to files.

Working with properties

Adding properties

You can still add new properties to your `PSCustomObject` with `Add-Member`.

PowerShell

```
$myObject | Add-Member -MemberType NoteProperty -Name 'ID' -Value 'KevinMarquette'

$myObject.ID
```

Remove properties

You can also remove properties off of an object.

PowerShell

```
$myObject.psobject.Properties.Remove('ID')
```

The `.psobject` is an intrinsic member that gives you access to base object metadata. For more information about intrinsic members, see about_Intrinsic_Members.

Enumerating property names

Sometimes you need a list of all the property names on an object.

PowerShell

```
$myObject | Get-Member -MemberType NoteProperty | select -ExpandProperty Name
```

We can get this same list off of the `psobject` property too.

PowerShell

```
$myobject.psobject.Properties.Name
```

Note

`Get-Member` returns the properties in alphabetical order. Using the member-access operator to enumerate the property names returns the properties in the order they were defined on the object.

Dynamically accessing properties

I already mentioned that you can access property values directly.

PowerShell

```
$myObject.Name
```

You can use a string for the property name and it will still work.

PowerShell

```
$myObject.'Name'
```

We can take this one more step and use a variable for the property name.

PowerShell

```
$property = 'Name'
$myObject.$property
```

I know that looks strange, but it works.

Convert PSCustomObject into a hashtable

To continue on from the last section, you can dynamically walk the properties and create a hashtable from them.

PowerShell

```
$hashtable = @{}
foreach( $property in $myobject.psobject.Properties.Name )
{
    $hashtable[$property] = $myObject.$property
}
```

Testing for properties

If you need to know if a property exists, you could just check for that property to have a value.

PowerShell

```powershell
if( $null -ne $myObject.ID )
```

But if the value could be `$null` you can check to see if it exists by checking the `psobject.Properties` for it.

PowerShell

```powershell
if( $myobject.psobject.Properties.Match('ID').Count )
```

Adding object methods

If you need to add a script method to an object, you can do it with `Add-Member` and a `ScriptBlock`. You have to use the `this` automatic variable reference the current object. Here is a `scriptblock` to turn an object into a hashtable. (same code form the last example)

PowerShell

```powershell
$ScriptBlock = {
    $hashtable = @{}
    foreach( $property in $this.psobject.Properties.Name )
    {
        $hashtable[$property] = $this.$property
    }
    return $hashtable
}
```

Then we add it to our object as a script property.

PowerShell

```powershell
$memberParam = @{
    MemberType = "ScriptMethod"
    InputObject = $myobject
    Name = "ToHashtable"
    Value = $scriptBlock
}
Add-Member @memberParam
```

Then we can call our function like this:

PowerShell

```
$myObject.ToHashtable()
```

Objects vs Value types

Objects and value types don't handle variable assignments the same way. If you assign value types to each other, only the value get copied to the new variable.

PowerShell

```
$first = 1
$second = $first
$second = 2
```

In this case, `$first` is 1 and `$second` is 2.

Object variables hold a reference to the actual object. When you assign one object to a new variable, they still reference the same object.

PowerShell

```
$third = [pscustomobject]@{Key=3}
$fourth = $third
$fourth.Key = 4
```

Because `$third` and `$fourth` reference the same instance of an object, both `$third.key` and `$fourth.Key` are 4.

psobject.Copy()

If you need a true copy of an object, you can clone it.

PowerShell

```
$third = [pscustomobject]@{Key=3}
$fourth = $third.psobject.Copy()
$fourth.Key = 4
```

Clone creates a shallow copy of the object. They have different instances now and `$third.key` is 3 and `$fourth.Key` is 4 in this example.

I call this a shallow copy because if you have nested objects (objects with properties contain other objects), only the top-level values are copied. The child objects will reference each other.

PSTypeName for custom object types

Now that we have an object, there are a few more things we can do with it that may not be nearly as obvious. First thing we need to do is give it a `PSTypeName`. This is the most common way I see people do it:

PowerShell

```
$myObject.psobject.TypeNames.Insert(0,"My.Object")
```

I recently discovered another way to do this from Redditor `u/markekraus`. He talks about this approach that allows you to define it inline.

PowerShell

```
$myObject = [pscustomobject]@{
    PSTypeName = 'My.Object'
    Name       = 'Kevin'
    Language   = 'PowerShell'
    State      = 'Texas'
}
```

I love how nicely this just fits into the language. Now that we have an object with a proper type name, we can do some more things.

> ⓘ **Note**
>
> You can also create custom PowerShell types using PowerShell classes. For more information, see PowerShell Class Overview.

Using DefaultPropertySet (the long way)

PowerShell decides for us what properties to display by default. A lot of the native commands have a `.ps1xml` formatting file that does all the heavy lifting. From this post by Boe Prox, there's another way for us to do this on our custom object using just PowerShell. We can give it a `MemberSet` for it to use.

PowerShell

```
$defaultDisplaySet = 'Name','Language'
$defaultDisplayPropertySet = New-Object
System.Management.Automation.PSPropertySet('DefaultDisplayPropertySet',
```

```
[string[]]$defaultDisplaySet)
$PSStandardMembers =
[System.Management.Automation.PSMemberInfo[]]@($defaultDisplayPropertySet)
$MyObject | Add-Member MemberSet PSStandardMembers $PSStandardMembers
```

Now when my object just falls to the shell, it will only show those properties by default.

Update-TypeData with DefaultPropertySet

This is nice but I recently saw a better way using Update-TypeData to specify the default properties.

PowerShell

```
$TypeData = @{
    TypeName = 'My.Object'
    DefaultDisplayPropertySet = 'Name','Language'
}
Update-TypeData @TypeData
```

That is simple enough that I could almost remember it if I didn't have this post as a quick reference. Now I can easily create objects with lots of properties and still give it a nice clean view when looking at it from the shell. If I need to access or see those other properties, they're still there.

PowerShell

```
$myObject | Format-List *
```

Update-TypeData with ScriptProperty

Something else I got out of that video was creating script properties for your objects. This would be a good time to point out that this works for existing objects too.

PowerShell

```
$TypeData = @{
    TypeName = 'My.Object'
    MemberType = 'ScriptProperty'
    MemberName = 'UpperCaseName'
    Value = {$this.Name.ToUpper()}
}
Update-TypeData @TypeData
```

You can do this before your object is created or after and it will still work. This is what makes this different than using `Add-Member` with a script property. When you use `Add-Member` the way I referenced earlier, it only exists on that specific instance of the object. This one applies to all objects with this `TypeName`.

Function parameters

You can now use these custom types for parameters in your functions and scripts. You can have one function create these custom objects and then pass them into other functions.

PowerShell

```
param( [PSTypeName('My.Object')]$Data )
```

PowerShell requires that the object is the type you specified. It throws a validation error if the type doesn't match automatically to save you the step of testing for it in your code. A great example of letting PowerShell do what it does best.

Function OutputType

You can also define an `OutputType` for your advanced functions.

PowerShell

```
function Get-MyObject
{
    [OutputType('My.Object')]
    [CmdletBinding()]
        param
        (
            ...
```

The **OutputType** attribute value is only a documentation note. It isn't derived from the function code or compared to the actual function output.

The main reason you would use an output type is so that meta information about your function reflects your intentions. Things like `Get-Command` and `Get-Help` that your development environment can take advantage of. If you want more information, then take a look at the help for it: about_Functions_OutputTypeAttribute.

With that said, if you're using Pester to unit test your functions then it would be a good idea to validate the output objects match your **OutputType**. This could catch variables that just fall to

the pipe when they shouldn't.

Closing thoughts

The context of this was all about `[pscustomobject]`, but a lot of this information applies to objects in general.

I have seen most of these features in passing before but never saw them presented as a collection of information on `PSCustomObject`. Just this last week I stumbled upon another one and was surprised that I had not seen it before. I wanted to pull all these ideas together so you can hopefully see the bigger picture and be aware of them when you have an opportunity to use them. I hope you learned something and can find a way to work this into your scripts.

Last updated on 03/24/2025

Everything you wanted to know about variable substitution in strings

There are many ways to use variables in strings. I'm calling this variable substitution but I'm referring to any time you want to format a string to include values from variables. This is something that I often find myself explaining to new scripters.

> **Note**
>
> The original version of this article appeared on the blog written by @KevinMarquette. The PowerShell team thanks Kevin for sharing this content with us. Please check out his blog at PowerShellExplained.com.

Concatenation

The first class of methods can be referred to as concatenation. It's basically taking several strings and joining them together. There's a long history of using concatenation to build formatted strings.

PowerShell

```
$name = 'Kevin Marquette'
$message = 'Hello, ' + $name
```

Concatenation works out OK when there are only a few values to add. But this can get complicated quickly.

PowerShell

```
$first = 'Kevin'
$last = 'Marquette'
```

PowerShell

```
$message = 'Hello, ' + $first + ' ' + $last + '.'
```

This simple example is already getting harder to read.

Variable substitution

PowerShell has another option that is easier. You can specify your variables directly in the strings.

PowerShell

```
$message = "Hello, $first $last."
```

The type of quotes you use around the string makes a difference. A double quoted string allows the substitution but a single quoted string doesn't. There are times you want one or the other so you have an option.

Command substitution

Things get a little tricky when you start trying to get the values of properties into a string. This is where many new people get tripped up. First let me show you what they think should work (and at face value almost looks like it should).

PowerShell

```
$directory = Get-Item 'C:\windows'
$message = "Time: $directory.CreationTime"
```

You would be expecting to get the `CreationTime` off of the `$directory`, but instead you get this `Time: C:\windows.CreationTime` as your value. The reason is that this type of substitution only sees the base variable. It considers the period as part of the string so it stops resolving the value any deeper.

It just so happens that this object gives a string as a default value when placed into a string. Some objects give you the type name instead like `System.Collections.Hashtable`. Just something to watch for.

PowerShell allows you to do command execution inside the string with a special syntax. This allows us to get the properties of these objects and run any other command to get a value.

PowerShell

```
$message = "Time: $($directory.CreationTime)"
```

This works great for some situations but it can get just as crazy as concatenation if you have just a few variables.

Command execution

You can run commands inside a string. Even though I have this option, I don't like it. It gets cluttered quickly and hard to debug. I either run the command and save to a variable or use a format string.

PowerShell

```
$message = "Date: $(Get-Date)"
```

Format string

.NET has a way to format strings that I find fairly easy to work with. First let me show you the static method for it before I show you the PowerShell shortcut to do the same thing.

PowerShell

```
# .NET string format string
[string]::Format('Hello, {0} {1}.',$first,$last)

# PowerShell format string
'Hello, {0} {1}.' -f $first, $last
```

What is happening here is that the string is parsed for the tokens `{0}` and `{1}`, then it uses that number to pick from the values provided. If you want to repeat one value some place in the string, you can reuse that values number.

The more complicated the string gets, the more value you get out of this approach.

Format values as arrays

If your format line gets too long, you can place your values into an array first.

PowerShell

```
$values = @(
    "Kevin"
    "Marquette"
)
'Hello, {0} {1}.' -f $values
```

This is not splatting because I'm passing the whole array in, but the idea is similar.

Advanced formatting

I intentionally called these out as coming from .NET because there are lots of formatting options already well documented on it. There are built-in ways to format various data types.

PowerShell

```
"{0:yyyyMMdd}" -f (Get-Date)
"Population {0:N0}" -f  8175133
```

Output

```
20211110
Population 8,175,133
```

I'm not going to go into them but I just wanted to let you know that this is a very powerful formatting engine if you need it.

Joining strings

Sometimes you actually do want to concatenate a list of values together. There's a `-join` operator that can do that for you. It even lets you specify a character to join between the strings.

PowerShell

```
$servers = @(
    'server1'
    'server2'
    'server3'
)

$servers  -join ','
```

If you want to `-join` some strings without a separator, you need to specify an empty string `''`. But if that is all you need, there's a faster option.

PowerShell

```
[string]::Concat('server1','server2','server3')
[string]::Concat($servers)
```

It's also worth pointing out that you can also `-split` strings too.

Join-Path

This is often overlooked but a great cmdlet for building a file path.

PowerShell

```
$folder = 'Temp'
Join-Path -Path 'C:\windows' -ChildPath $folder
```

The great thing about this is it works out the backslashes correctly when it puts the values together. This is especially important if you are taking values from users or config files.

This also goes well with `Split-Path` and `Test-Path`. I also cover these in my post about reading and saving to files.

Strings are arrays

I do need to mention adding strings here before I go on. Remember that a string is just an array of characters. When you add multiple strings together, a new array is created each time.

Look at this example:

PowerShell

```
$message = "Numbers: "
foreach($number in 1..10000)
{
    $message += " $number"
}
```

It looks very basic but what you don't see is that each time a string is added to `$message` that a whole new string is created. Memory gets allocated, data gets copied and the old one is discarded. Not a big deal when it's only done a few times, but a loop like this would really expose the issue.

StringBuilder

StringBuilder is also very popular for building large strings from lots of smaller strings. The reason why is because it just collects all the strings you add to it and only concatenates all of them at the end when you retrieve the value.

PowerShell

```powershell
$stringBuilder = New-Object -TypeName "System.Text.StringBuilder"

[void]$stringBuilder.Append("Numbers: ")
foreach($number in 1..10000)
{
    [void]$stringBuilder.Append(" $number")
}
$message = $stringBuilder.ToString()
```

Again, this is something that I'm reaching out to .NET for. I don't use it often anymore but it's good to know it's there.

Delineation with braces

This is used for suffix concatenation within the string. Sometimes your variable doesn't have a clean word boundary.

PowerShell

```powershell
$test = "Bet"
$tester = "Better"
Write-Host "$test $tester ${test}ter"
```

Thank you Redditor `u/real_parbold` for that one.

Here is an alternate to this approach:

PowerShell

```powershell
Write-Host "$test $tester $($test)ter"
Write-Host "{0} {1} {0}ter" -f $test, $tester
```

I personally use format string for this, but this is good to know in case you see it in the wild.

Find and replace tokens

While most of these features limit your need to roll your own solution, there are times where you may have large template files where you want to replace strings inside.

Let us assume you pulled in a template from a file that has a lot of text.

PowerShell

```
$letter = Get-Content -Path TemplateLetter.txt -RAW
$letter = $letter -replace '#FULL_NAME#', 'Kevin Marquette'
```

You may have lots of tokens to replace. The trick is to use a very distinct token that is easy to find and replace. I tend to use a special character at both ends to help distinguish it.

I recently found a new way to approach this. I decided to leave this section in here because this is a pattern that is commonly used.

Replace multiple tokens

When I have a list of tokens that I need to replace, I take a more generic approach. I would place them in a hashtable and iterate over them to do the replace.

PowerShell

```
$tokenList = @{
    Full_Name = 'Kevin Marquette'
    Location = 'Orange County'
    State = 'CA'
}

$letter = Get-Content -Path TemplateLetter.txt -RAW
foreach( $token in $tokenList.GetEnumerator() )
{
    $pattern = '#{0}#' -f $token.key
    $letter = $letter -replace $pattern, $token.Value
}
```

Those tokens could be loaded from JSON or CSV if needed.

ExecutionContext ExpandString

There's a clever way to define a substitution string with single quotes and expand the variables later. Look at this example:

PowerShell

```
$message = 'Hello, $Name!'
$name = 'Kevin Marquette'
$string = $ExecutionContext.InvokeCommand.ExpandString($message)
```

The call to `InvokeCommand.ExpandString` on the current execution context uses the variables in the current scope for substitution. The key thing here is that the `$message` can be defined very

early before the variables even exist.

If we expand on that just a little bit, we can perform this substitution over and over with different values.

PowerShell

```
$message = 'Hello, $Name!'
$nameList = 'Mark Kraus','Kevin Marquette','Lee Dailey'
foreach($name in $nameList){
    $ExecutionContext.InvokeCommand.ExpandString($message)
}
```

To keep going on this idea; you could be importing a large email template from a text file to do this. I have to thank Mark Kraus for this suggestion.

Whatever works the best for you

I'm a fan of the format string approach. I definitely do this with the more complicated strings or if there are multiple variables. On anything that is very short, I may use any one of these.

Anything else?

I covered a lot of ground on this one. My hope is that you walk away learning something new.

Links

If you'd like to learn more about the methods and features that make string interpolation possible, see the following list for the reference documentation.

- Concatenation uses the addition operator
- Variable and command substitution follow the quoting rules
- Formatting uses the format operator
- Joining strings uses the join operator and references Join-Path, but you could also read about Join-String
- Arrays are documented in About arrays
- StringBuilder is a .NET class, with its own documentation
- Braces in strings is also covered in the quoting rules
- Token replacement uses the replace operator
- The `$ExecutionContext.InvokeCommand.ExpandString()` method has .NET API reference documentation

Last updated on 08/06/2025

Everything you wanted to know about the `if` statement

Like many other languages, PowerShell has statements for conditionally executing code in your scripts. One of those statements is the If statement. Today we will take a deep dive into one of the most fundamental commands in PowerShell.

> **Note**
>
> The original version of this article appeared on the blog written by @KevinMarquette. The PowerShell team thanks Kevin for sharing this content with us. Please check out his blog at PowerShellExplained.com.

Conditional execution

Your scripts often need to make decisions and perform different logic based on those decisions. This is what I mean by conditional execution. You have one statement or value to evaluate, then execute a different section of code based on that evaluation. This is exactly what the `if` statement does.

The `if` statement

Here is a basic example of the `if` statement:

PowerShell

```
$condition = $true
if ( $condition )
{
    Write-Output "The condition was true"
}
```

The first thing the `if` statement does is evaluate the expression in parentheses. If it evaluates to `$true`, then it executes the statements in the braces. If the value was `$false`, then it would skip over that statement block.

In the previous example, the `if` statement was just evaluating the `$condition` variable. It was `$true` and would have executed the `Write-Output` command inside the statement block.

In some languages, you can place a single line of code after the `if` statement and it gets executed. That isn't the case in PowerShell. You must provide a full `statement block` with braces for it to work correctly.

Comparison operators

The most common use of the `if` statement for is comparing two items with each other. PowerShell has special operators for different comparison scenarios. When you use a comparison operator, the value on the left-hand side is compared to the value on the right-hand side.

-eq for equality

The `-eq` does an equality check between two values to make sure they're equal to each other.

PowerShell

```
$value = Get-MysteryValue
if ( 5 -eq $value )
{
    # do something
}
```

In this example, I'm taking a known value of `5` and comparing it to my `$value` to see if they match.

One possible use case is to check the status of a value before you take an action on it. You could get a service and check that the status was running before you called `Restart-Service` on it.

It's common in other languages like C# to use `==` for equality (ex: `5 == $value`) but that doesn't work with PowerShell. Another common mistake that people make is to use the equals sign (ex: `5 = $value`) that is reserved for assigning values to variables. By placing your known value on the left, it makes that mistake more awkward to make.

This operator (and others) has a few variations.

- `-eq` case-insensitive equality
- `-ieq` case-insensitive equality
- `-ceq` case-sensitive equality

-ne not equal

Many operators have a related operator that is checking for the opposite result. `-ne` verifies that the values don't equal each other.

PowerShell

```
if ( 5 -ne $value )
{
    # do something
}
```

Use this to make sure that the action only executes if the value isn't `5`. A good use-cases where would be to check if a service was in the running state before you try to start it.

Variations:

- `-ne` case-insensitive not equal
- `-ine` case-insensitive not equal
- `-cne` case-sensitive not equal

These are inverse variations of `-eq`. I'll group these types together when I list variations for other operators.

-gt -ge -lt -le for greater than or less than

These operators are used when checking to see if a value is larger or smaller than another value. The `-gt -ge -lt -le` stand for GreaterThan, GreaterThanOrEqual, LessThan, and LessThanOrEqual.

PowerShell

```
if ( $value -gt 5 )
{
    # do something
}
```

Variations:

- `-gt` greater than
- `-igt` greater than, case-insensitive
- `-cgt` greater than, case-sensitive
- `-ge` greater than or equal
- `-ige` greater than or equal, case-insensitive
- `-cge` greater than or equal, case-sensitive
- `-lt` less than

- `-ilt` less than, case-insensitive
- `-clt` less than, case-sensitive
- `-le` less than or equal
- `-ile` less than or equal, case-insensitive
- `-cle` less than or equal, case-sensitive

I don't know why you would use case-sensitive and insensitive options for these operators.

-like wildcard matches

PowerShell has its own wildcard-based pattern matching syntax and you can use it with the `-like` operator. These wildcard patterns are fairly basic.

- `?` matches any single character
- `*` matches any number of characters

PowerShell

```
$value = 'S-ATX-SQL01'
if ( $value -like 'S-*-SQL??')
{
    # do something
}
```

It's important to point out that the pattern matches the whole string. If you need to match something in the middle of the string, you need to place the `*` on both ends of the string.

PowerShell

```
$value = 'S-ATX-SQL02'
if ( $value -like '*SQL*')
{
    # do something
}
```

Variations:

- `-like` case-insensitive wildcard
- `-ilike` case-insensitive wildcard
- `-clike` case-sensitive wildcard
- `-notlike` case-insensitive wildcard not matched
- `-inotlike` case-insensitive wildcard not matched
- `-cnotlike` case-sensitive wildcard not matched

-match regular expression

The `-match` operator allows you to check a string for a regular-expression-based match. Use this when the wildcard patterns aren't flexible enough for you.

PowerShell

```
$value = 'S-ATX-SQL01'
if ( $value -match 'S-\w\w\w-SQL\d\d')
{
    # do something
}
```

A regex pattern matches anywhere in the string by default. So you can specify a substring that you want matched like this:

PowerShell

```
$value = 'S-ATX-SQL01'
if ( $value -match 'SQL')
{
    # do something
}
```

Regex is a complex language of its own and worth looking into. I talk more about `-match` and the many ways to use regex in another article.

Variations:

- `-match` case-insensitive regex
- `-imatch` case-insensitive regex
- `-cmatch` case-sensitive regex
- `-notmatch` case-insensitive regex not matched
- `-inotmatch` case-insensitive regex not matched
- `-cnotmatch` case-sensitive regex not matched

-is of type

You can check a value's type with the `-is` operator.

PowerShell

```
if ( $value -is [string] )
{
```

```
    # do something
}
```

You may use this if you're working with classes or accepting various objects over the pipeline. You could have either a service or a service name as your input. Then check to see if you have a service and fetch the service if you only have the name.

PowerShell

```
if ( $Service -isnot [System.ServiceProcess.ServiceController] )
{
    $Service = Get-Service -Name $Service
}
```

Variations:

- `-is` of type
- `-isnot` not of type

Collection operators

When you use the previous operators with a single value, the result is `$true` or `$false`. This is handled slightly differently when working with a collection. Each item in the collection gets evaluated and the operator returns every value that evaluates to `$true`.

PowerShell

```
PS> 1,2,3,4 -eq 3
3
```

This still works correctly in an `if` statement. So a value is returned by your operator, then the whole statement is `$true`.

PowerShell

```
$array = 1..6
if ( $array -gt 3 )
{
    # do something
}
```

There's one small trap hiding in the details here that I need to point out. When using the `-ne` operator this way, it's easy to mistakenly look at the logic backwards. Using `-ne` with a collection returns `$true` if any item in the collection doesn't match your value.

PowerShell

```
PS> 1,2,3 -ne 4
1
2
3
```

This may look like a clever trick, but we have operators `-contains` and `-in` that handle this more efficiently. And `-notcontains` does what you expect.

-contains

The `-contains` operator checks the collection for your value. As soon as it finds a match, it returns `$true`.

PowerShell

```
$array = 1..6
if ( $array -contains 3 )
{
    # do something
}
```

This is the preferred way to see if a collection contains your value. Using `Where-Object` (or `-eq`) walks the entire list every time and is significantly slower.

Variations:

- `-contains` case-insensitive match
- `-icontains` case-insensitive match
- `-ccontains` case-sensitive match
- `-notcontains` case-insensitive not matched
- `-inotcontains` case-insensitive not matched
- `-cnotcontains` case-sensitive not matched

-in

The `-in` operator is just like the `-contains` operator except the collection is on the right-hand side.

PowerShell

```
$array = 1..6
if ( 3 -in $array )
```

```
{
    # do something
}
```

Variations:

- `-in` case-insensitive match
- `-iin` case-insensitive match
- `-cin` case-sensitive match
- `-notin` case-insensitive not matched
- `-inotin` case-insensitive not matched
- `-cnotin` case-sensitive not matched

Logical operators

Logical operators are used to invert or combine other expressions.

-not

The `-not` operator flips an expression from `$false` to `$true` or from `$true` to `$false`. Here is an example where we want to perform an action when `Test-Path` is `$false`.

PowerShell

```
if ( -not ( Test-Path -Path $path ) )
```

Most of the operators we talked about do have a variation where you do not need to use the `-not` operator. But there are still times it is useful.

! operator

You can use `!` as an alias for `-not`.

PowerShell

```
if ( -not $value ){}
if ( !$value ){}
```

You may see `!` used more by people that come from another languages like C#. I prefer to type it out because I find it hard to see when quickly looking at my scripts.

-and

You can combine expressions with the `-and` operator. When you do that, both sides need to be `$true` for the whole expression to be `$true`.

PowerShell

```
if ( ($age -gt 13) -and ($age -lt 55) )
```

In that example, `$age` must be 13 or older for the left side and less than 55 for the right side. I added extra parentheses to make it clearer in that example but they're optional as long as the expression is simple. Here is the same example without them.

PowerShell

```
if ( $age -gt 13 -and $age -lt 55 )
```

Evaluation happens from left to right. If the first item evaluates to `$false`, it exits early and doesn't perform the right comparison. This is handy when you need to make sure a value exists before you use it. For example, `Test-Path` throws an error if you give it a `$null` path.

PowerShell

```
if ( $null -ne $path -and (Test-Path -Path $path) )
```

-or

The `-or` allows for you to specify two expressions and returns `$true` if either one of them is `$true`.

PowerShell

```
if ( $age -le 13 -or $age -ge 55 )
```

Just like with the `-and` operator, the evaluation happens from left to right. Except that if the first part is `$true`, then the whole statement is `$true` and it doesn't process the rest of the expression.

Also make note of how the syntax works for these operators. You need two separate expressions. I have seen users try to do something like this `$value -eq 5 -or 6` without realizing their mistake.

-xor exclusive or

This one is a little unusual. `-xor` allows only one expression to evaluate to `$true`. So if both items are `$false` or both items are `$true`, then the whole expression is `$false`. Another way to look at this is the expression is only `$true` when the results of the expression are different.

It's rare that anyone would ever use this logical operator and I can't think up a good example as to why I would ever use it.

Bitwise operators

Bitwise operators perform calculations on the bits within the values and produce a new value as the result. Teaching bitwise operators is beyond the scope of this article, but here is the list of them.

- `-band` binary AND
- `-bor` binary OR
- `-bxor` binary exclusive OR
- `-bnot` binary NOT
- `-shl` shift left
- `-shr` shift right

PowerShell expressions

We can use normal PowerShell inside the condition statement.

PowerShell

```
if ( Test-Path -Path $Path )
```

`Test-Path` returns `$true` or `$false` when it executes. This also applies to commands that return other values.

PowerShell

```
if ( Get-Process Notepad* )
```

It evaluates to `$true` if there's a returned process and `$false` if there isn't. It's perfectly valid to use pipeline expressions or other PowerShell statements like this:

PowerShell

```
if ( Get-Process | where Name -EQ Notepad )
```

These expressions can be combined with each other with the `-and` and `-or` operators, but you may have to use parenthesis to break them into subexpressions.

PowerShell

```
if ( (Get-Process) -and (Get-Service) )
```

Checking for $null

Having a no result or a `$null` value evaluates to `$false` in the `if` statement. When checking specifically for `$null`, it's a best practice to place the `$null` on the left-hand side.

PowerShell

```
if ( $null -eq $value )
```

There are quite a few nuances when dealing with `$null` values in PowerShell. If you're interested in diving deeper, I have an article about everything you wanted to know about $null.

Variable assignment within the condition

I almost forgot to add this one until Prasoon Karunan V reminded me of it.

PowerShell

```
if ($process=Get-Process notepad -ErrorAction Ignore) {$process} else {$false}
```

Normally when you assign a value to a variable, the value isn't passed onto the pipeline or console. When you do a variable assignment in a sub expression, it does get passed on to the pipeline.

PowerShell

```
PS> $first = 1
PS> ($second = 2)
2
```

See how the `$first` assignment has no output and the `$second` assignment does? When an assignment is done in an `if` statement, it executes just like the `$second` assignment above. Here is a clean example on how you could use it:

PowerShell

```
if ( $process = Get-Process Notepad* )
{
    $process | Stop-Process
}
```

If `$process` gets assigned a value, then the statement is `$true` and `$process` gets stopped.

Make sure you don't confuse this with `-eq` because this isn't an equality check. This is a more obscure feature that most people don't realize works this way.

Variable assignment from the statement block

You can also use the `if` statement statement block to assign a value to a variable.

PowerShell

```
$discount = if ( $age -ge 55 )
{
    Get-SeniorDiscount
}
elseif ( $age -le 13 )
{
    Get-ChildDiscount
}
else
{
    0.00
}
```

Each script block is writing the results of the commands, or the value, as output. We can assign the result of the `if` statement to the `$discount` variable. That example could have just as easily assigned those values to the `$discount` variable directly in each statement block. I can't say that I use this with the `if` statement often, but I do have an example where I used this recently.

Alternate execution path

The `if` statement allows you to specify an action for not only when the statement is `$true`, but also for when it's `$false`. This is where the `else` statement comes into play.

else

The `else` statement is always the last part of the `if` statement when used.

PowerShell

```
if ( Test-Path -Path $Path -PathType Leaf )
{
    Move-Item -Path $Path -Destination $archivePath
}
else
{
    Write-Warning "$path doesn't exist or isn't a file."
}
```

In this example, we check the `$path` to make sure it's a file. If we find the file, we move it. If not, we write a warning. This type of branching logic is very common.

Nested if

The `if` and `else` statements take a script block, so we can place any PowerShell command inside them, including another `if` statement. This allows you to make use of much more complicated logic.

PowerShell

```
if ( Test-Path -Path $Path -PathType Leaf )
{
    Move-Item -Path $Path -Destination $archivePath
}
else
{
    if ( Test-Path -Path $Path )
    {
        Write-Warning "A file was required but a directory was found instead."
    }
    else
    {
        Write-Warning "$path could not be found."
    }
}
```

In this example, we test the happy path first and then take action on it. If that fails, we do another check and to provide more detailed information to the user.

elseif

We aren't limited to just a single conditional check. We can chain `if` and `else` statements together instead of nesting them by using the `elseif` statement.

PowerShell

```
if ( Test-Path -Path $Path -PathType Leaf )
{
    Move-Item -Path $Path -Destination $archivePath
}
elseif ( Test-Path -Path $Path )
{
    Write-Warning "A file was required but a directory was found instead."
}
else
{
    Write-Warning "$path could not be found."
}
```

The execution happens from the top to the bottom. The top `if` statement is evaluated first. If that is `$false`, then it moves down to the next `elseif` or `else` in the list. That last `else` is the default action to take if none of the others return `$true`.

switch

At this point, I need to mention the `switch` statement. It provides an alternate syntax for doing multiple comparisons with a value. With the `switch`, you specify an expression and that result gets compared with several different values. If one of those values match, the matching code block is executed. Take a look at this example:

PowerShell

```
$itemType = 'Role'
switch ( $itemType )
{
    'Component'
    {
        'is a component'
    }
    'Role'
    {
        'is a role'
    }
    'Location'
    {
        'is a location'
    }
}
```

There three possible values that can match the `$itemType`. In this case, it matches with `Role`. I used a simple example just to give you some exposure to the `switch` operator. I talk more about everything you ever wanted to know about the switch statement in another article.

Array inline

I have a function called Invoke-SnowSql that launches an executable with several command-line arguments. Here is a clip from that function where I build the array of arguments.

PowerShell

```
$snowSqlParam = @(
    '--accountname', $Endpoint
    '--username', $Credential.UserName
    '--option', 'exit_on_error=true'
    '--option', 'output_format=csv'
    '--option', 'friendly=false'
    '--option', 'timing=false'
    if ($Debug)
    {
        '--option', 'log_level=DEBUG'
    }
    if ($Path)
    {
        '--filename', $Path
    }
    else
    {
        '--query', $singleLineQuery
    }
)
```

The `$Debug` and `$Path` variables are parameters on the function that are provided by the end user. I evaluate them inline inside the initialization of my array. If `$Debug` is true, then those values fall into the `$snowSqlParam` in the correct place. Same holds true for the `$Path` variable.

Simplify complex operations

It's inevitable that you run into a situation that has way too many comparisons to check and your `if` statement scrolls way off the right side of the screen.

PowerShell

```
$user = Get-ADUser -Identity $UserName
if ( $null -ne $user -and $user.Department -eq 'Finance' -and $user.Title -match
'Senior' -and $user.HomeDrive -notlike '\\server\*' )
{
    # Do Something
}
```

They can be hard to read and that make you more prone to make mistakes. There are a few things we can do about that.

Line continuation

There some operators in PowerShell that let you wrap you command to the next line. The logical operators `-and` and `-or` are good operators to use if you want to break your expression into multiple lines.

PowerShell

```powershell
if ($null -ne $user -and
    $user.Department -eq 'Finance' -and
    $user.Title -match 'Senior' -and
    $user.HomeDrive -notlike '\\server\*'
)
{
    # Do Something
}
```

There's still a lot going on there, but placing each piece on its own line makes a big difference. I generally use this when I get more than two comparisons or if I have to scroll to the right to read any of the logic.

Pre-calculating results

We can take that statement out of the `if` statement and only check the result.

PowerShell

```powershell
$needsSecureHomeDrive = $null -ne $user -and
    $user.Department -eq 'Finance' -and
    $user.Title -match 'Senior' -and
    $user.HomeDrive -notlike '\\server\*'

if ( $needsSecureHomeDrive )
{
    # Do Something
}
```

This just feels much cleaner than the previous example. You also are given an opportunity to use a variable name that explains what it's that you're really checking. This is also and example of self-documenting code that saves unnecessary comments.

Multiple if statements

We can break this up into multiple statements and check them one at a time. In this case, we use a flag or a tracking variable to combine the results.

PowerShell

```
$skipUser = $false

if( $null -eq $user )
{
    $skipUser = $true
}

if( $user.Department -ne 'Finance' )
{
    Write-Verbose "isn't in Finance department"
    $skipUser = $true
}

if( $user.Title -match 'Senior' )
{
    Write-Verbose "Doesn't have Senior title"
    $skipUser = $true
}

if( $user.HomeDrive -like '\\server\*' )
{
    Write-Verbose "Home drive already configured"
    $skipUser = $true
}

if ( -not $skipUser )
{
    # do something
}
```

I did have to invert the logic to make the flag logic work correctly. Each evaluation is an individual `if` statement. The advantage of this is that when you're debugging, you can tell exactly what the logic is doing. I was able to add much better verbosity at the same time.

The obvious downside is that it's so much more code to write. The code is more complex to look at as it takes a single line of logic and explodes it into 25 or more lines.

Using functions

We can also move all that validation logic into a function. Look at how clean this looks at a glance.

PowerShell

```powershell
if ( Test-SecureDriveConfiguration -ADUser $user )
{
    # do something
}
```

You still have to create the function to do the validation, but it makes this code much easier to work with. It makes this code easier to test. In your tests, you can mock the call to `Test-ADDriveConfiguration` and you only need two tests for this function. One where it returns `$true` and one where it returns `$false`. Testing the other function is simpler because it's so small.

The body of that function could still be that one-liner we started with or the exploded logic that we used in the last section. This works well for both scenarios and allows you to easily change that implementation later.

Error handling

One important use of the `if` statement is to check for error conditions before you run into errors. A good example is to check if a folder already exists before you try to create it.

PowerShell

```powershell
if ( -not (Test-Path -Path $folder) )
{
    New-Item -Type Directory -Path $folder
}
```

I like to say that if you expect an exception to happen, then it's not really an exception. So check your values and validate your conditions where you can.

If you want to dive a little more into actual exception handling, I have an article on everything you ever wanted to know about exceptions.

Final words

The `if` statement is such a simple statement but is a fundamental piece of PowerShell. You will find yourself using this multiple times in almost every script you write. I hope you have a better understanding than you had before.

Last updated on 01/18/2026

Everything you ever wanted to know about the switch statement

Like many other languages, PowerShell has commands for controlling the flow of execution within your scripts. One of those statements is the switch statement and in PowerShell, it offers features that aren't found in other languages. Today, we take a deep dive into working with the PowerShell `switch`.

> **Note**
>
> The original version of this article appeared on the blog written by @KevinMarquette. The PowerShell team thanks Kevin for sharing this content with us. Please check out his blog at PowerShellExplained.com.

The `if` statement

One of the first statements that you learn is the `if` statement. It lets you execute a script block if a statement is `$true`.

PowerShell

```
if ( Test-Path $Path )
{
    Remove-Item $Path
}
```

You can have much more complicated logic using `elseif` and `else` statements. Here is an example where I have a numeric value for day of the week and I want to get the name as a string.

PowerShell

```
$day = 3

if ( $day -eq 0 ) { $result = 'Sunday'    }
elseif ( $day -eq 1 ) { $result = 'Monday'    }
elseif ( $day -eq 2 ) { $result = 'Tuesday'   }
elseif ( $day -eq 3 ) { $result = 'Wednesday' }
elseif ( $day -eq 4 ) { $result = 'Thursday'  }
elseif ( $day -eq 5 ) { $result = 'Friday'    }
```

```
elseif ( $day -eq 6 ) { $result = 'Saturday'   }

$result
```

Output

```
Wednesday
```

It turns out that this is a common pattern and there are many ways to deal with this. One of them is with a `switch`.

Switch statement

The `switch` statement allows you to provide a variable and a list of possible values. If the value matches the variable, then its statement block is executed.

PowerShell

```
$day = 3

switch ( $day )
{
    0 { $result = 'Sunday'    }
    1 { $result = 'Monday'    }
    2 { $result = 'Tuesday'   }
    3 { $result = 'Wednesday' }
    4 { $result = 'Thursday'  }
    5 { $result = 'Friday'    }
    6 { $result = 'Saturday'  }
}

$result
```

Output

```
'Wednesday'
```

For this example, the value of `$day` matches one of the numeric values, then the correct name is assigned to `$result`. We're only doing a variable assignment in this example, but any PowerShell can be executed in those script blocks.

Assign to a variable

We can write that last example in another way.

PowerShell

```powershell
$result = switch ( $day )
{
    0 { 'Sunday'    }
    1 { 'Monday'    }
    2 { 'Tuesday'   }
    3 { 'Wednesday' }
    4 { 'Thursday'  }
    5 { 'Friday'    }
    6 { 'Saturday'  }
}
```

We're placing the value on the PowerShell pipeline and assigning it to the `$result`. You can do this same thing with the `if` and `foreach` statements.

Default

We can use the `default` keyword to identify the what should happen if there is no match.

PowerShell

```powershell
$result = switch ( $day )
{
    0 { 'Sunday' }
    # ...
    6 { 'Saturday' }
    default { 'Unknown' }
}
```

Here we return the value `Unknown` in the default case.

Strings

I was matching numbers in those last examples, but you can also match strings.

PowerShell

```powershell
$item = 'Role'

switch ( $item )
{
    Component
    {
        'is a component'
    }
    Role
    {
        'is a role'
```

```
    }
    Location
    {
        'is a location'
    }
}
```

Output

```
is a role
```

I decided not to wrap the `Component`, `Role` and `Location` matches in quotes here to highlight that they're optional. The `switch` treats those as a string in most cases.

Arrays

One of the cool features of the PowerShell `switch` is the way it handles arrays. If you give a `switch` an array, it processes each element in that collection.

PowerShell

```
$roles = @('WEB','Database')

switch ( $roles ) {
    'Database'   { 'Configure SQL' }
    'WEB'        { 'Configure IIS' }
    'FileServer' { 'Configure Share' }
}
```

Output

```
Configure IIS
Configure SQL
```

If you have repeated items in your array, then they're matched multiple times by the appropriate section.

PSItem

You can use the `$PSItem` or `$_` to reference the current item that was processed. When we do a simple match, `$PSItem` is the value that we're matching. I'll be performing some advanced matches in the next section where this variable is used.

Parameters

A unique feature of the PowerShell `switch` is that it has a number of `[switch]` parameters that change how it performs.

-CaseSensitive

The matches aren't case-sensitive by default. If you need to be case-sensitive, you can use `-CaseSensitive`. This can be used in combination with the other `[switch]` parameters.

-Wildcard

We can enable wildcard support with the `-Wildcard` `[switch]` parameter. This uses the same wildcard logic as the `-like` operator to do each match.

PowerShell

```
$Message = 'Warning, out of disk space'

switch -Wildcard ( $message )
{
    'Error*'
    {
        Write-Error -Message $Message
    }
    'Warning*'
    {
        Write-Warning -Message $Message
    }
    default
    {
        Write-Information $message
    }
}
```

Output

```
WARNING: Warning, out of disk space
```

Here we're processing a message and then outputting it on different streams based on the contents.

-Regex

The switch statement supports regex matches just like it does wildcards.

PowerShell

```
switch -Regex ( $message )
{
    '^Error'
    {
        Write-Error -Message $Message
    }
    '^Warning'
    {
        Write-Warning -Message $Message
    }
    default
    {
        Write-Information $message
    }
}
```

I have more examples of using regex in another article I wrote: The many ways to use regex.

-File

A little known feature of the switch statement is that it can process a file with the `-File` parameter. You use `-File` with a path to a file instead of giving it a variable expression.

PowerShell

```
switch -Wildcard -File $path
{
    'Error*'
    {
        Write-Error -Message $PSItem
    }
    'Warning*'
    {
        Write-Warning -Message $PSItem
    }
    default
    {
        Write-Output $PSItem
    }
}
```

It works just like processing an array. In this example, I combine it with wildcard matching and make use of the `$PSItem`. This would process a log file and convert it to warning and error messages depending on the regex matches.

Advanced details

Now that you're aware of all these documented features, we can use them in the context of more advanced processing.

Expressions

The `switch` can be on an expression instead of a variable.

PowerShell

```
switch ( ( Get-Service | where Status -EQ 'running' ).Name ) {...}
```

Whatever the expression evaluates to is the value used for the match.

Multiple matches

You may have already picked up on this, but a `switch` can match to multiple conditions. This is especially true when using `-Wildcard` or `-Regex` matches. You can add the same condition multiple times and all are triggered.

PowerShell

```
switch ( 'Word' )
{
    'word' { 'lower case word match' }
    'Word' { 'mixed case word match' }
    'WORD' { 'upper case word match' }
}
```

Output

```
lower case word match
mixed case word match
upper case word match
```

All three of these statements are fired. This shows that every condition is checked (in order). This holds true for processing arrays where each item checks each condition.

Continue

Normally, this is where I would introduce the `break` statement, but it's better that we learn how to use `continue` first. Just like with a `foreach` loop, `continue` continues onto the next item in the collection or exits the `switch` if there are no more items. We can rewrite that last example with continue statements so that only one statement executes.

PowerShell

```powershell
switch ( 'Word' )
{
    'word'
    {
        'lower case word match'
        continue
    }
    'Word'
    {
        'mixed case word match'
        continue
    }
    'WORD'
    {
        'upper case word match'
        continue
    }
}
```

Output

```
lower case word match
```

Instead of matching all three items, the first one is matched and the switch continues to the next value. Because there are no values left to process, the switch exits. This next example is showing how a wildcard could match multiple items.

PowerShell

```powershell
switch -Wildcard -File $path
{
    '*Error*'
    {
        Write-Error -Message $PSItem
        continue
    }
    '*Warning*'
    {
        Write-Warning -Message $PSItem
        continue
    }
    default
    {
        Write-Output $PSItem
    }
}
```

Because a line in the input file could contain both the word `Error` and `Warning`, we only want the first one to execute and then continue processing the file.

Break

A `break` statement exits the switch. This is the same behavior that `continue` presents for single values. The difference is shown when processing an array. `break` stops all processing in the switch and `continue` moves onto the next item.

PowerShell

```powershell
$Messages = @(
    'Downloading update'
    'Ran into errors downloading file'
    'Error: out of disk space'
    'Sending email'
    '...'
)

switch -Wildcard ($Messages)
{
    'Error*'
    {
        Write-Error -Message $PSItem
        break
    }
    '*Error*'
    {
        Write-Warning -Message $PSItem
        continue
    }
    '*Warning*'
    {
        Write-Warning -Message $PSItem
        continue
    }
    default
    {
        Write-Output $PSItem
    }
}
```

Output

```
Downloading update
WARNING: Ran into errors downloading file
Write-Error -Message $PSItem : Error: out of disk space
+ CategoryInfo          : NotSpecified: (:) [Write-Error], WriteErrorException
+ FullyQualifiedErrorId : Microsoft.PowerShell.Commands.WriteErrorException
```

In this case, if we hit any lines that start with `Error` then we get an error and the switch stops. This is what that `break` statement is doing for us. If we find `Error` inside the string and not just at the beginning, we write it as a warning. We do the same thing for `Warning`. It's possible that a line could have both the word `Error` and `Warning`, but we only need one to process. This is what the `continue` statement is doing for us.

Break labels

The `switch` statement supports `break/continue` labels just like `foreach`.

PowerShell

```
:filelist foreach($path in $logs)
{
    :logFile switch -Wildcard -File $path
    {
        'Error*'
        {
            Write-Error -Message $PSItem
            break filelist
        }
        'Warning*'
        {
            Write-Error -Message $PSItem
            break logFile
        }
        default
        {
            Write-Output $PSItem
        }
    }
}
```

I personally don't like the use of break labels but I wanted to point them out because they're confusing if you've never seen them before. When you have multiple `switch` or `foreach` statements that are nested, you may want to break out of more than the inner most item. You can place a label on a `switch` that can be the target of your `break`.

Enum

PowerShell 5.0 gave us enums and we can use them in a switch.

PowerShell

```
enum Context {
    Component
```

```
    Role
    Location
}

$item = [Context]::Role

switch ( $item )
{
    Component
    {
        'is a component'
    }
    Role
    {
        'is a role'
    }
    Location
    {
        'is a location'
    }
}
```

Output

```
is a role
```

If you want to keep everything as strongly typed enums, then you can place them in parentheses.

PowerShell

```
switch ($item )
{
    ([Context]::Component)
    {
        'is a component'
    }
    ([Context]::Role)
    {
        'is a role'
    }
    ([Context]::Location)
    {
        'is a location'
    }
}
```

The parentheses are needed here so that the switch doesn't treat the value `[Context]::Location` as a literal string.

ScriptBlock

We can use a scriptblock to perform the evaluation for a match if needed.

PowerShell

```powershell
$age = 37

switch ( $age )
{
    {$PSItem -le 18}
    {
        'child'
    }
    {$PSItem -gt 18}
    {
        'adult'
    }
}
```

Output

```
'adult'
```

This adds complexity and can make your `switch` hard to read. In most cases where you would use something like this it would be better to use `if` and `elseif` statements. I would consider using this if I already had a large switch in place and I needed two items to hit the same evaluation block.

One thing that I think helps with legibility is to place the scriptblock in parentheses.

PowerShell

```powershell
switch ( $age )
{
    ({$PSItem -le 18})
    {
        'child'
    }
    ({$PSItem -gt 18})
    {
        'adult'
    }
}
```

It still executes the same way and gives a better visual break when quickly looking at it.

Regex $Matches

We need to revisit regex to touch on something that isn't immediately obvious. The use of regex populates the `$Matches` variable. I do go into the use of `$Matches` more when I talk about The many ways to use regex. Here is a quick sample to show it in action with named matches.

PowerShell

```powershell
$message = 'my ssn is 123-23-3456 and credit card: 1234-5678-1234-5678'

switch -Regex ($message)
{
    '(?<SSN>\d\d\d-\d\d-\d\d\d\d)'
    {
        Write-Warning "message contains a SSN: $($Matches.SSN)"
    }
    '(?<CC>\d\d\d\d-\d\d\d\d-\d\d\d\d-\d\d\d\d)'
    {
        Write-Warning "message contains a credit card number: $($Matches.CC)"
    }
    '(?<Phone>\d\d\d-\d\d\d-\d\d\d\d)'
    {
        Write-Warning "message contains a phone number: $($Matches.Phone)"
    }
}
```

Output

```
WARNING: message may contain a SSN: 123-23-3456
WARNING: message may contain a credit card number: 1234-5678-1234-5678
```

$null

You can match a `$null` value that doesn't have to be the default.

PowerShell

```powershell
$values = '', 5, $null
switch ( $values )
{
    $null           { "Value '$_' is `$null" }
    { '' -eq $_ }   { "Value '$_' is an empty string" }
    default         { "Value [$_] isn't an empty string or `$null" }
}
```

Output

```
Value '' is an empty string
Value [5] isn't an empty string or $null
Value '' is $null
```

When testing for an empty string in a `switch` statement, it's important to use the comparison statement as shown in this example instead of the raw value `''`. In a `switch` statement, the raw value `''` also matches `$null`. For example:

PowerShell

```
$values = '', 5, $null
switch ( $values )
{
    $null          { "Value '$_' is `$null" }
    ''             { "Value '$_' is an empty string" }
    default        { "Value [$_] isn't an empty string or `$null" }
}
```

Output

```
Value '' is an empty string
Value [5] isn't an empty string or $null
Value '' is $null
Value '' is an empty string
```

Also, be careful with empty returns from cmdlets. Cmdlets or pipelines that have no output are treated as an empty array that doesn't match anything, including the `default` case.

PowerShell

```
$file = Get-ChildItem NonExistantFile*
switch ( $file )
{
    $null   { '$file is $null' }
    default { "`$file is type $($file.GetType().Name)" }
}
# No matches
```

Constant expression

Lee Dailey pointed out that we can use a constant `$true` expression to evaluate `[bool]` items. Imagine if we have several boolean checks that need to happen.

PowerShell

```
$isVisible = $false
$isEnabled = $true
$isSecure = $true

switch ( $true )
{
    $isEnabled
    {
        'Do-Action'
    }
    $isVisible
    {
        'Show-Animation'
    }
    $isSecure
    {
        'Enable-AdminMenu'
    }
}
```

Output

```
Do-Action
Enabled-AdminMenu
```

This is a clean way to evaluate and take action on the status of several boolean fields. The cool thing about this is that you can have one match flip the status of a value that hasn't been evaluated yet.

PowerShell

```
$isVisible = $false
$isEnabled = $true
$isAdmin = $false

switch ( $true )
{
    $isEnabled
    {
        'Do-Action'
        $isVisible = $true
    }
    $isVisible
    {
        'Show-Animation'
    }
    $isAdmin
    {
        'Enable-AdminMenu'
```

```
    }
}
```

Output

```
Do-Action
Show-Animation
```

Setting `$isEnabled` to `$true` in this example makes sure that `$isVisible` is also set to `$true`. Then when `$isVisible` gets evaluated, its statement block is invoked. This is a bit counter-intuitive but is a clever use of the mechanics.

$switch automatic variable

When the `switch` is processing its values, it creates an enumerator and calls it `$switch`. This is an automatic variable created by PowerShell and you can manipulate it directly.

PowerShell

```
$a = 1, 2, 3, 4

switch($a) {
    1 { [void]$switch.MoveNext(); $switch.Current }
    3 { [void]$switch.MoveNext(); $switch.Current }
}
```

This gives you the results of:

Output

```
2
4
```

By moving the enumerator forward, the next item doesn't get processed by the `switch` but you can access that value directly. I would call it madness.

Other patterns

Hashtables

One of my most popular posts is the one I did on hashtables. One of the use cases for a `hashtable` is to be a lookup table. That's an alternate approach to a common pattern that a `switch` statement is often addressing.

PowerShell

```
$day = 3

$lookup = @{
    0 = 'Sunday'
    1 = 'Monday'
    2 = 'Tuesday'
    3 = 'Wednesday'
    4 = 'Thursday'
    5 = 'Friday'
    6 = 'Saturday'
}

$lookup[$day]
```

Output

```
Wednesday
```

If I'm only using a `switch` as a lookup, I often use a `hashtable` instead.

Enum

PowerShell 5.0 introduced the `enum` and it's also an option in this case.

PowerShell

```
$day = 3

enum DayOfTheWeek {
    Sunday
    Monday
    Tuesday
    Wednesday
    Thursday
    Friday
    Saturday
}

[DayOfTheWeek]$day
```

Output

```
Wednesday
```

We could go all day looking at different ways to solve this problem. I just wanted to make sure you knew you had options.

Final words

The switch statement is simple on the surface but it offers some advanced features that most people don't realize are available. Stringing those features together makes this a powerful feature. I hope you learned something that you had not realized before.

Last updated on 04/08/2026

Everything you wanted to know about exceptions

Error handling is just part of life when it comes to writing code. We can often check and validate conditions for expected behavior. When the unexpected happens, we turn to exception handling. You can easily handle exceptions generated by other people's code or you can generate your own exceptions for others to handle.

> **Note**
>
> The original version of this article appeared on the blog written by @KevinMarquette. The PowerShell team thanks Kevin for sharing this content with us. Please check out his blog at PowerShellExplained.com.

Basic terminology

We need to cover some basic terms before we jump into this one.

Exception

An Exception is like an event that is created when normal error handling can't deal with the issue. Trying to divide a number by zero or running out of memory are examples of something that creates an exception. Sometimes the author of the code you're using creates exceptions for certain issues when they happen.

Throw and Catch

When an exception happens, we say that an exception is thrown. To handle a thrown exception, you need to catch it. If an exception is thrown and it isn't caught by something, the script stops executing.

The call stack

The call stack is the list of functions that have called each other. When a function is called, it gets added to the stack or the top of the list. When the function exits or returns, it is removed from the stack.

When an exception is thrown, that call stack is checked in order for an exception handler to catch it.

Terminating and non-terminating errors

PowerShell has three categories of errors.

- A *non-terminating error* adds an error to the error stream without stopping execution and doesn't trigger `catch`. By default, `Write-Error` generates non-terminating errors.
- A *statement-terminating error* stops the current statement but allows the script to continue at the next statement. Statement-terminating errors can be generated by engine errors, `$PSCmdlet.ThrowTerminatingError()`, or .NET method exceptions.
- A *script-terminating error* unwinds the entire call stack. Script-terminating errors can be generated by `throw`, parse errors, or `-ErrorAction Stop` escalation.

Both statement-terminating and script-terminating errors can be caught by `try/catch`. For a comprehensive reference, see about_Error_Handling.

Swallowing an exception

This is when you catch an error just to suppress it. Do this with caution because it can make troubleshooting issues very difficult.

Basic command syntax

Here is a quick overview of the basic exception handling syntax used in PowerShell.

Throw

To create our own exception event, we throw an exception with the `throw` keyword.

PowerShell

```powershell
function Start-Something
{
    throw "Bad thing happened"
}
```

This creates a runtime exception that is a script-terminating error. It's handled by a `catch` in a calling function or exits the script with a message like this.

PowerShell

```
PS> Start-Something

Bad thing happened
At line:1 char:1
+ throw "Bad thing happened"
+ ~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : OperationStopped: (Bad thing happened:String) [],
RuntimeException
    + FullyQualifiedErrorId : Bad thing happened
```

Write-Error -ErrorAction Stop

I mentioned that `Write-Error` doesn't throw a terminating error by default. If you specify `-ErrorAction Stop`, `Write-Error` generates a terminating error that can be handled with a `catch`.

PowerShell

```
Write-Error -Message "Houston, we have a problem." -ErrorAction Stop
```

Thank you to Lee Dailey for reminding about using `-ErrorAction Stop` this way.

Cmdlet -ErrorAction Stop

If you specify `-ErrorAction Stop` on any advanced function or cmdlet, it turns all `Write-Error` statements into terminating errors that stop execution or that can be handled by a `catch`.

PowerShell

```
Start-Something -ErrorAction Stop
```

For more information about the **ErrorAction** parameter, see about_CommonParameters. For more information about the `$ErrorActionPreference` variable, see about_Preference_Variables.

Try/Catch

The way exception handling works in PowerShell (and many other languages) is that you first `try` a section of code and if it throws an error, you can `catch` it. Here is a quick sample.

PowerShell

```
try
{
```

```
    Start-Something
}
catch
{
    Write-Output "Something threw an exception"
    Write-Output $_
}

try
{
    Start-Something -ErrorAction Stop
}
catch
{
    Write-Output "Something threw an exception or used Write-Error"
    Write-Output $_
}
```

The `catch` script only runs if there's a terminating error. If the `try` executes correctly, then it skips over the `catch`. You can access the exception information in the `catch` block using the `$_` variable.

Try/Finally

Sometimes you don't need to handle an error but still need some code to execute if an exception happens or not. A `finally` script does exactly that.

Take a look at this example:

PowerShell

```
$command = [System.Data.SqlClient.SqlCommand]::new(queryString, connection)
$command.Connection.Open()
$command.ExecuteNonQuery()
$command.Connection.Close()
```

Anytime you open or connect to a resource, you should close it. If the `ExecuteNonQuery()` throws an exception, the connection isn't closed. Here is the same code inside a `try/finally` block.

PowerShell

```
$command = [System.Data.SqlClient.SqlCommand]::new(queryString, connection)
try
{
    $command.Connection.Open()
    $command.ExecuteNonQuery()
}
```

```
finally
{
    $command.Connection.Close()
}
```

In this example, the connection is closed if there's an error. It also is closed if there's no error. The `finally` script runs every time.

Because you're not catching the exception, it still gets propagated up the call stack.

Try/Catch/Finally

It's perfectly valid to use `catch` and `finally` together. Most of the time you'll use one or the other, but you may find scenarios where you use both.

$PSItem

Now that we got the basics out of the way, we can dig a little deeper.

Inside the `catch` block, there's an automatic variable (`$PSItem` or `$_`) of type `ErrorRecord` that contains the details about the exception. Here is a quick overview of some of the key properties.

For these examples, I used an invalid path in `ReadAllText` to generate this exception.

PowerShell

```
[System.IO.File]::ReadAllText( '\\test\no\filefound.log')
```

PSItem.ToString()

This gives you the cleanest message to use in logging and general output. `ToString()` is automatically called if `$PSItem` is placed inside a string.

PowerShell

```
catch
{
    Write-Output "Ran into an issue: $($PSItem.ToString())"
}

catch
{
```

```
    Write-Output "Ran into an issue: $PSItem"
}
```

$PSItem.InvocationInfo

This property contains additional information collected by PowerShell about the function or script where the exception was thrown. Here is the `InvocationInfo` from the sample exception that I created.

PowerShell

```
PS> $PSItem.InvocationInfo | Format-List *

MyCommand             : Get-Resource
BoundParameters       : {}
UnboundArguments      : {}
ScriptLineNumber      : 5
OffsetInLine          : 5
ScriptName            : C:\blog\throwerror.ps1
Line                  :     Get-Resource
PositionMessage       : At C:\blog\throwerror.ps1:5 char:5
                        +     Get-Resource
                        +     ~~~~~~~~~~~~
PSScriptRoot          : C:\blog
PSCommandPath         : C:\blog\throwerror.ps1
InvocationName        : Get-Resource
```

The important details here show the `ScriptName`, the `Line` of code and the `ScriptLineNumber` where the invocation started.

$PSItem.ScriptStackTrace

This property shows the order of function calls that got you to the code where the exception was generated.

PowerShell

```
PS> $PSItem.ScriptStackTrace
at Get-Resource, C:\blog\throwerror.ps1: line 13
at Start-Something, C:\blog\throwerror.ps1: line 5
at <ScriptBlock>, C:\blog\throwerror.ps1: line 18
```

I'm only making calls to functions in the same script but this would track the calls if multiple scripts were involved.

$PSItem.Exception

This is the actual exception that was thrown.

$PSItem.Exception.Message

This is the general message that describes the exception and is a good starting point when troubleshooting. Most exceptions have a default message but can also be set to something custom when the exception is thrown.

PowerShell

```
PS> $PSItem.Exception.Message

Exception calling "ReadAllText" with "1" argument(s): "The network path was not
found."
```

This is also the message returned when calling `$PSItem.ToString()` if there was not one set on the `ErrorRecord`.

$PSItem.Exception.InnerException

Exceptions can contain inner exceptions. This is often the case when the code you're calling catches an exception and throws a different exception. The original exception is placed inside the new exception.

PowerShell

```
PS> $PSItem.Exception.InnerExceptionMessage
The network path was not found.
```

I will revisit this later when I talk about re-throwing exceptions.

$PSItem.Exception.StackTrace

This is the `StackTrace` for the exception. I showed a `ScriptStackTrace` above, but this one is for the calls to managed code.

Output

```
at System.IO.FileStream.Init(String path, FileMode mode, FileAccess access, Int32
rights, Boolean
 useRights, FileShare share, Int32 bufferSize, FileOptions options,
SECURITY_ATTRIBUTES secAttrs,
```

```
 String msgPath, Boolean bFromProxy, Boolean useLongPath, Boolean checkHost)
at System.IO.FileStream..ctor(String path, FileMode mode, FileAccess access,
FileShare share, Int32
 bufferSize, FileOptions options, String msgPath, Boolean bFromProxy, Boolean
useLongPath, Boolean
 checkHost)
at System.IO.StreamReader..ctor(String path, Encoding encoding, Boolean
detectEncodingFromByteOrderMarks,
 Int32 bufferSize, Boolean checkHost)
at System.IO.File.InternalReadAllText(String path, Encoding encoding, Boolean
checkHost)
at CallSite.Target(Closure , CallSite , Type , String )
```

You only get this stack trace when the event is thrown from managed code. I'm calling a .NET Framework function directly so that is all we can see in this example. Generally when you're looking at a stack trace, you're looking for where your code stops and the system calls begin.

Working with exceptions

There is more to exceptions than the basic syntax and exception properties.

Catching typed exceptions

You can be selective with the exceptions that you catch. Exceptions have a type and you can specify the type of exception you want to catch.

PowerShell

```powershell
try
{
    Start-Something -Path $path
}
catch [System.IO.FileNotFoundException]
{
    Write-Output "Could not find $path"
}
catch [System.IO.IOException]
{
        Write-Output "IO error with the file: $path"
}
```

The exception type is checked for each `catch` block until one is found that matches your exception. It's important to realize that exceptions can inherit from other exceptions. In the example above, `FileNotFoundException` inherits from `IOException`. So if the `IOException` was first, then it would get called instead. Only one catch block is invoked even if there are multiple matches.

If we had a `System.IO.PathTooLongException`, the `IOException` would match but if we had an `InsufficientMemoryException` then nothing would catch it and it would propagate up the stack.

Catch multiple types at once

It's possible to catch multiple exception types with the same `catch` statement.

PowerShell

```
try
{
    Start-Something -Path $path -ErrorAction Stop
}
catch [System.IO.DirectoryNotFoundException],[System.IO.FileNotFoundException]
{
    Write-Output "The path or file was not found: [$path]"
}
catch [System.IO.IOException]
{
    Write-Output "IO error with the file: [$path]"
}
```

Thank you Redditor `u/Sheppard_Ra` for suggesting this addition.

Throwing typed exceptions

You can throw typed exceptions in PowerShell. Instead of calling `throw` with a string:

PowerShell

```
throw "Could not find: $path"
```

Use an exception accelerator like this:

PowerShell

```
throw [System.IO.FileNotFoundException] "Could not find: $path"
```

But you have to specify a message when you do it that way.

You can also create a new instance of an exception to be thrown. The message is optional when you do this because the system has default messages for all built-in exceptions.

PowerShell

```
throw [System.IO.FileNotFoundException]::new()
throw [System.IO.FileNotFoundException]::new("Could not find path: $path")
```

If you're not using PowerShell 5.0 or higher, you must use the older `New-Object` approach.

PowerShell

```
throw (New-Object -TypeName System.IO.FileNotFoundException )
throw (New-Object -TypeName System.IO.FileNotFoundException -ArgumentList "Could
not find path: $path")
```

By using a typed exception, you (or others) can catch the exception by the type as mentioned in the previous section.

Write-Error -Exception

We can add these typed exceptions to `Write-Error` and we can still `catch` the errors by exception type. Use `Write-Error` like in these examples:

PowerShell

```
# with normal message
Write-Error -Message "Could not find path: $path" -Exception
([System.IO.FileNotFoundException]::new()) -ErrorAction Stop

# With message inside new exception
Write-Error -Exception ([System.IO.FileNotFoundException]::new("Could not find
path: $path")) -ErrorAction Stop

# Pre PS 5.0
Write-Error -Exception ([System.IO.FileNotFoundException]"Could not find path:
$path") -ErrorAction Stop

Write-Error -Message "Could not find path: $path" -Exception (New-Object -TypeName
System.IO.FileNotFoundException) -ErrorAction Stop
```

Then we can catch it like this:

PowerShell

```
catch [System.IO.FileNotFoundException]
{
    Write-Log $PSItem.ToString()
}
```

The big list of .NET exceptions

I compiled a master list with the help of the Reddit `r/PowerShell` community that contains hundreds of .NET exceptions to complement this post.

- The big list of .NET exceptions

I start by searching that list for exceptions that feel like they would be a good fit for my situation. You should try to use exceptions in the base `System` namespace.

Exceptions are objects

If you start using a lot of typed exceptions, remember that they are objects. Different exceptions have different constructors and properties. If we look at the FileNotFoundException documentation for `System.IO.FileNotFoundException`, we see that we can pass in a message and a file path.

PowerShell

```
[System.IO.FileNotFoundException]::new("Could not find file", $path)
```

And it has a `FileName` property that exposes that file path.

PowerShell

```
catch [System.IO.FileNotFoundException]
{
    Write-Output $PSItem.Exception.FileName
}
```

You should consult the .NET documentation for other constructors and object properties.

Re-throwing an exception

If all you're going to do in your `catch` block is `throw` the same exception, then don't `catch` it. You should only `catch` an exception that you plan to handle or perform some action when it happens.

There are times where you want to perform an action on an exception but re-throw the exception so something downstream can deal with it. We could write a message or log the problem close to where we discover it but handle the issue further up the stack.

PowerShell

```
catch
{
    Write-Log $PSItem.ToString()
    throw $PSItem
}
```

Interestingly enough, we can call `throw` from within the `catch` and it re-throws the current exception.

PowerShell

```
catch
{
    Write-Log $PSItem.ToString()
    throw
}
```

We want to re-throw the exception to preserve the original execution information like source script and line number. If we throw a new exception at this point, it hides where the exception started.

Re-throwing a new exception

If you catch an exception but you want to throw a different one, then you should nest the original exception inside the new one. This allows someone down the stack to access it as the `$PSItem.Exception.InnerException`.

PowerShell

```
catch
{
    throw [System.MissingFieldException]::new('Could not access
field',$PSItem.Exception)
}
```

$PSCmdlet.ThrowTerminatingError()

The one thing that I don't like about using `throw` for raw exceptions is that the error message points at the `throw` statement and indicates that line is where the problem is.

Output

```
Unable to find the specified file.
At line:31 char:9
+         throw [System.IO.FileNotFoundException]::new()
```

```
+          ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : OperationStopped: (:) [], FileNotFoundException
    + FullyQualifiedErrorId : Unable to find the specified file.
```

Having the error message tell me that my script is broken because I called `throw` on line 31 is a bad message for users of your script to see. It doesn't tell them anything useful.

Dexter Dhami pointed out that I can use `ThrowTerminatingError()` to correct that.

PowerShell

```
$PSCmdlet.ThrowTerminatingError(
    [System.Management.Automation.ErrorRecord]::new(
        ([System.IO.FileNotFoundException]"Could not find $Path"),
        'My.ID',
        [System.Management.Automation.ErrorCategory]::OpenError,
        $MyObject
    )
)
```

If we assume that `ThrowTerminatingError()` was called inside a function called `Get-Resource`, then this is the error that we would see.

Output

```
Get-Resource : Could not find C:\Program Files (x86)\Reference
Assemblies\Microsoft\Framework\.NETPortable\v4.6\System.IO.xml
At line:6 char:5
+     Get-Resource -Path $Path
+     ~~~~~~~~~~~~
    + CategoryInfo          : OpenError: (:) [Get-Resource], FileNotFoundException
    + FullyQualifiedErrorId : My.ID,Get-Resource
```

Do you see how it points to the `Get-Resource` function as the source of the problem? That tells the user something useful.

Because `$PSItem` is an `ErrorRecord`, we can also use `ThrowTerminatingError` this way to re-throw.

PowerShell

```
catch
{
    $PSCmdlet.ThrowTerminatingError($PSItem)
}
```

This changes the source of the error to the Cmdlet and hide the internals of your function from the users of your Cmdlet.

How try/catch changes error propagation

Inside a `try` block, PowerShell sets an internal flag that causes all statement-terminating errors to propagate to the `catch` block. This is by design, not a corner case. The following example demonstrates this behavior.

PowerShell

```
function Start-Something { 1/(1-1) }
```

Outside `try/catch`, a statement-terminating error from a child scope doesn't stop the parent scope. Here is a function that generates a divide by zero runtime exception.

Invoke it like this to see the error reported while the script continues.

PowerShell

```
&{ Start-Something; Write-Output "We did it. Send Email" }
```

But placing that same code inside a `try/catch`, the error propagates to the `catch` block.

PowerShell

```
try {
    &{ Start-Something; Write-Output "We did it. Send Email" }
} catch {
    Write-Output "Notify Admin to fix error and send email"
}
```

The error is caught and the subsequent `Write-Output` inside the script block doesn't run. This is standard `try/catch` behavior — all terminating errors within the `try` block are caught, whether they originate in the current scope or in a child scope.

$PSCmdlet.ThrowTerminatingError() inside try/catch

`$PSCmdlet.ThrowTerminatingError()` creates a statement-terminating error within the cmdlet. After the error leaves the cmdlet, the caller treats it as a non-terminating error by default. The caller can escalate it back to a terminating error by using `-ErrorAction Stop` or calling it from within a `try/catch` block.

Public function templates

One last takeaway I had with my conversation with Kirk Munro was that he places a `try/catch` block inside every `begin`, `process` and `end` block in all his advanced functions. In those generic catch blocks, he has a single line using `$PSCmdlet.ThrowTerminatingError($PSItem)` to deal with all exceptions leaving his functions.

PowerShell

```
function Start-Something
{
    [CmdletBinding()]
    param()

    process
    {
        try {
            ...
        } catch {
            $PSCmdlet.ThrowTerminatingError($PSItem)
        }
    }
}
```

Because everything is in a `try` statement within his functions, everything acts consistently. This also gives clean errors to the end user that hides the internal code from the generated error.

Trap

I focused on the `try/catch` aspect of exceptions. But there's one legacy feature I need to mention before we wrap this up.

A `trap` is placed in a script or function to catch all exceptions that happen in that scope. When an exception happens, the code in the `trap` is executed and then the normal code continues. If multiple exceptions happen, then the trap is called over and over.

PowerShell

```
trap
{
    Write-Log $PSItem.ToString()
}

throw [System.Exception]::new('first')
throw [System.Exception]::new('second')
throw [System.Exception]::new('third')
```

I personally never adopted this approach but I can see the value in admin or controller scripts that log any and all exceptions, then still continue to execute.

Closing remarks

Adding proper exception handling to your scripts not only make them more stable, but also makes it easier for you to troubleshoot those exceptions.

I spent a lot of time talking `throw` because it is a core concept when talking about exception handling. PowerShell also gave us `Write-Error` that handles all the situations where you would use `throw`. So don't think that you need to be using `throw` after reading this.

Now that I have taken the time to write about exception handling in this detail, I'm going to switch over to using `Write-Error -Stop` to generate errors in my code. I'm also going to take Kirk's advice and make `ThrowTerminatingError` my goto exception handler for every function.

Last updated on 04/02/2026

Everything you wanted to know about $null

The PowerShell `$null` often appears to be simple but it has a lot of nuances. Let's take a close look at `$null` so you know what happens when you unexpectedly run into a `$null` value.

> **Note**
>
> The original version of this article appeared on the blog written by @KevinMarquette. The PowerShell team thanks Kevin for sharing this content with us. Please check out his blog at PowerShellExplained.com.

What is NULL?

You can think of NULL as an unknown or empty value. A variable is NULL until you assign a value or an object to it. This can be important because there are some commands that require a value and generate errors if the value is NULL.

PowerShell $null

`$null` is an automatic variable in PowerShell used to represent NULL. You can assign it to variables, use it in comparisons and use it as a place holder for NULL in a collection.

PowerShell treats `$null` as an object with a value of NULL. This is different than what you may expect if you come from another language.

Examples of $null

Anytime you try to use a variable that you have not initialized, the value is `$null`. This is one of the most common ways that `$null` values sneak into your code.

PowerShell

```
PS> $null -eq $undefinedVariable
True
```

If you happen to mistype a variable name then PowerShell sees it as a different variable and the value is `$null`.

The other way you find `$null` values is when they come from other commands that don't give you any results.

PowerShell

```
PS> function Get-Nothing {}
PS> $value = Get-Nothing
PS> $null -eq $value
True
```

Impact of $null

`$null` values impact your code differently depending on where they show up.

In strings

If you use `$null` in a string, then it's a blank value (or empty string).

PowerShell

```
PS> $value = $null
PS> Write-Output "'The value is $value'"
'The value is '
```

This is one of the reasons that I like to place brackets around variables when using them in log messages. It's even more important to identify the edges of your variable values when the value is at the end of the string.

PowerShell

```
PS> $value = $null
PS> Write-Output "The value is [$value]"
The value is []
```

This makes empty strings and `$null` values easy to spot.

In numeric equation

When a `$null` value is used in a numeric equation then your results are invalid if they don't give an error. Sometimes the `$null` evaluates to `0` and other times it makes the whole result

`$null`. Here is an example with multiplication that gives 0 or `$null` depending on the order of the values.

PowerShell

```
PS> $null * 5
PS> $null -eq ( $null * 5 )
True

PS> 5 * $null
0
PS> $null -eq ( 5 * $null )
False
```

In place of a collection

A collection allows you use an index to access values. If you try to index into a collection that is actually `null`, you get this error: `Cannot index into a null array`.

PowerShell

```
PS> $value = $null
PS> $value[10]
Cannot index into a null array.
At line:1 char:1
+ $value[10]
+ ~~~~~~~~~~
    + CategoryInfo          : InvalidOperation: (:) [], RuntimeException
    + FullyQualifiedErrorId : NullArray
```

If you have a collection but try to access an element that is not in the collection, you get a `$null` result.

PowerShell

```
$array = @( 'one','two','three' )
$null -eq $array[100]
True
```

In place of an object

If you try to access a property or sub property of an object that doesn't have the specified property, you get a `$null` value like you would for an undefined variable. It doesn't matter if the variable is `$null` or an actual object in this case.

PowerShell

```
PS> $null -eq $undefined.Some.Fake.Property
True

PS> $date = Get-Date
PS> $null -eq $date.Some.Fake.Property
True
```

Method on a null-valued expression

Calling a method on a `$null` object throws a `RuntimeException`.

PowerShell

```
PS> $value = $null
PS> $value.ToString()
You cannot call a method on a null-valued expression.
At line:1 char:1
+ $value.ToString()
+ ~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation: (:) [], RuntimeException
    + FullyQualifiedErrorId : InvokeMethodOnNull
```

Whenever I see the phrase `You cannot call a method on a null-valued expression` then the first thing I look for are places where I am calling a method on a variable without first checking it for `$null`.

Checking for $null

You may have noticed that I always place the `$null` on the left when checking for `$null` in my examples. This is intentional and accepted as a PowerShell best practice. There are some scenarios where placing it on the right doesn't give you the expected result.

Look at this next example and try to predict the results:

PowerShell

```
if ( $value -eq $null )
{
    'The array is $null'
}
if ( $value -ne $null )
{
    'The array is not $null'
}
```

If I do not define `$value`, the first one evaluates to `$true` and our message is `The array is $null`. The trap here is that it's possible to create a `$value` that allows both of them to be `$false`

PowerShell

```
$value = @( $null )
```

In this case, the `$value` is an array that contains a `$null`. The `-eq` checks every value in the array and returns the `$null` that is matched. This evaluates to `$false`. The `-ne` returns everything that doesn't match `$null` and in this case there are no results (This also evaluates to `$false`). Neither one is `$true` even though it looks like one of them should be.

Not only can we create a value that makes both of them evaluate to `$false`, it's possible to create a value where they both evaluate to `$true`. Mathias Jessen (@IISResetMe) has a good post that dives into that scenario.

PSScriptAnalyzer and VS Code

The PSScriptAnalyzer module has a rule that checks for this issue called `PSPossibleIncorrectComparisonWithNull`.

PowerShell

```
PS> Invoke-ScriptAnalyzer ./myscript.ps1

RuleName                            Message
--------                            -------
PSPossibleIncorrectComparisonWithNull $null should be on the left side of equality
comparisons.
```

Because VS Code uses the PSScriptAnalyser rules too, it also highlights or identifies this as a problem in your script.

Simple if check

A common way that people check for a non-$null value is to use a simple `if()` statement without the comparison.

PowerShell

```
if ( $value )
{
```

```
    Do-Something
}
```

If the value is `$null`, this evaluates to `$false`. This is easy to read, but be careful that it's looking for exactly what you're expecting it to look for. I read that line of code as:

> If `$value` has a value.

But that's not the whole story. That line is actually saying:

> If `$value` is not `$null` or `0` or `$false` or an empty string or an empty array.

Here is a more complete sample of that statement.

PowerShell

```
if ( $null -ne $value -and
        $value -ne 0 -and
        $value -ne '' -and
        ($value -isnot [array] -or $value.Length -ne 0) -and
        $value -ne $false )
{
    Do-Something
}
```

It's perfectly OK to use a basic `if` check as long as you remember those other values count as `$false` and not just that a variable has a value.

I ran into this issue when refactoring some code a few days ago. It had a basic property check like this.

PowerShell

```
if ( $object.Property )
{
    $object.Property = $value
}
```

I wanted to assign a value to the object property only if it existed. In most cases, the original object had a value that would evaluate to `$true` in the `if` statement. But I ran into an issue where the value was occasionally not getting set. I debugged the code and found that the object had the property but it was a blank string value. This prevented it from ever getting updated with the previous logic. So I added a proper `$null` check and everything worked.

PowerShell

```powershell
if ( $null -ne $object.Property )
{
    $object.Property = $value
}
```

It's little bugs like these that are hard to spot and make me aggressively check values for `$null`.

$null.Count

If you try to access a property on a `$null` value, that the property is also `$null`. The `Count` property is the exception to this rule.

PowerShell

```powershell
PS> $value = $null
PS> $value.Count
0
```

When you have a `$null` value, then the `Count` is `0`. This special property is added by PowerShell.

[PSCustomObject] Count

Almost all objects in PowerShell have that `Count` property. One important exception is the `[pscustomobject]` in Windows PowerShell 5.1 (This is fixed in PowerShell 6.0). It doesn't have a `Count` property so you get a `$null` value if you try to use it. I call this out here so that you don't try to use `Count` instead of a `$null` check.

Running this example on Windows PowerShell 5.1 and PowerShell 6.0 gives you different results.

PowerShell

```powershell
$value = [pscustomobject]@{Name='MyObject'}
if ( $value.Count -eq 1 )
{
    "We have a value"
}
```

Enumerable null

There is one special type of `$null` that acts differently than the others. I am going to call it the enumerable null but it's really a System.Management.Automation.Internal.AutomationNull. This enumerable null is the one you get as the result of a function or script block that returns nothing (a void result).

PowerShell

```
PS> function Get-Nothing {}
PS> $nothing = Get-Nothing
PS> $null -eq $nothing
True
```

If you compare it with `$null`, you get a `$null` value. When used in an evaluation where a value is required, the value is always `$null`. But if you place it inside an array, it's treated the same as an empty array.

PowerShell

```
PS> $containEmpty = @( @() )
PS> $containNothing = @($nothing)
PS> $containNull = @($null)

PS> $containEmpty.Count
0
PS> $containNothing.Count
0
PS> $containNull.Count
1
```

You can have an array that contains one `$null` value and its `Count` is `1`. But if you place an empty array inside an array then it's not counted as an item. The count is `0`.

If you treat the enumerable null like a collection, then it's empty.

If you pass in an enumerable null to a function parameter that isn't strongly typed, PowerShell coerces the enumerable null into a `$null` value by default. This means inside the function, the value is treated as `$null` instead of the **System.Management.Automation.Internal.AutomationNull** type.

Pipeline

The primary place you see the difference is when using the pipeline. You can pipe a `$null` value but not an enumerable null value.

PowerShell

```
PS> $null | ForEach-Object{ Write-Output 'NULL Value' }
'NULL Value'
PS> $nothing | ForEach-Object{ Write-Output 'No Value' }
```

Depending on your code, you should account for the `$null` in your logic.

Either check for `$null` first

- Filter out null on the pipeline (`... | where {$null -ne $_} | ...`)
- Handle it in the pipeline function

foreach

One of my favorite features of `foreach` is that it doesn't enumerate over a `$null` collection.

PowerShell

```
foreach ( $node in $null )
{
    #skipped
}
```

This saves me from having to `$null` check the collection before I enumerate it. If you have a collection of `$null` values, the `$node` can still be `$null`.

The `foreach` started working this way with PowerShell 3.0. If you happen to be on an older version, then this is not the case. This is one of the important changes to be aware of when back-porting code for 2.0 compatibility.

Value types

Technically, only reference types can be `$null`. But PowerShell is very generous and allows for variables to be any type. If you decide to strongly type a value type, it cannot be `$null`. PowerShell converts `$null` to a default value for many types.

PowerShell

```
PS> [int]$number = $null
PS> $number
0

PS> [bool]$boolean = $null
PS> $boolean
```

```
False

PS> [string]$string = $null
PS> $string -eq ''
True
```

There are some types that do not have a valid conversion from `$null`. These types generate a `Cannot convert null to type` error.

PowerShell

```
PS> [datetime]$date = $null
Cannot convert null to type "System.DateTime".
At line:1 char:1
+ [datetime]$date = $null
+ ~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : MetadataError: (:) [],
ArgumentTransformationMetadataException
    + FullyQualifiedErrorId : RuntimeException
```

Function parameters

Using a strongly typed values in function parameters is very common. We generally learn to define the types of our parameters even if we tend not to define the types of other variables in our scripts. You may already have some strongly typed variables in your functions and not even realize it.

PowerShell

```
function Do-Something
{
    param(
        [string] $Value
    )
}
```

As soon as you set the type of the parameter as a `string`, the value can never be `$null`. It's common to check if a value is `$null` to see if the user provided a value or not.

PowerShell

```
if ( $null -ne $Value ){...}
```

`$Value` is an empty string `''` when no value is provided. Use the automatic variable `$PSBoundParameters.Value` instead.

PowerShell

```powershell
if ( $null -ne $PSBoundParameters.Value ){...}
```

`$PSBoundParameters` only contains the parameters that were specified when the function was called. You can also use the `ContainsKey` method to check for the property.

PowerShell

```powershell
if ( $PSBoundParameters.ContainsKey('Value') ){...}
```

IsNotNullOrEmpty

If the value is a string, you can use a static string function to check if the value is `$null` or an empty string at the same time.

PowerShell

```powershell
if ( -not [string]::IsNullOrEmpty( $value ) ){...}
```

I find myself using this often when I know the value type should be a string.

When I $null check

I am a defensive scripter. Anytime I call a function and assign it to a variable, I check it for `$null`.

PowerShell

```powershell
$userList = Get-ADUser kevmar
if ($null -ne $userList){...}
```

I much prefer using `if` or `foreach` over using `try/catch`. Don't get me wrong, I still use `try/catch` a lot. But if I can test for an error condition or an empty set of results, I can allow my exception handling be for true exceptions.

I also tend to check for `$null` before I index into a value or call methods on an object. These two actions fail for a `$null` object so I find it important to validate them first. I already covered those scenarios earlier in this post.

No results scenario

It's important to know that different functions and commands handle the no results scenario differently. Many PowerShell commands return the enumerable null and an error in the error stream. But others throw exceptions or give you a status object. It's still up to you to know how the commands you use deal with the no results and error scenarios.

Initializing to $null

One habit that I have picked up is initializing all my variables before I use them. You are required to do this in other languages. At the top of my function or as I enter a `foreach` loop, I define all the values that I'm using.

Here is a scenario that I want you to take a close look at. It's an example of a bug I had to chase down before.

PowerShell

```
function Do-Something
{
    foreach ( $node in 1..6 )
    {
        try
        {
            $result = Get-Something -Id $node
        }
        catch
        {
            Write-Verbose "[$result] not valid"
        }

        if ( $null -ne $result )
        {
            Update-Something $result
        }
    }
}
```

The expectation here is that `Get-Something` returns either a result or an enumerable null. If there's an error, we log it. Then we check to make sure we got a valid result before processing it.

The bug hiding in this code is when `Get-Something` throws an exception and doesn't assign a value to `$result`. It fails before the assignment so we don't even assign `$null` to the `$result` variable. `$result` still contains the previous valid `$result` from other iterations. `Update-Something` to execute multiple times on the same object in this example.

I set `$result` to `$null` right inside the `foreach` loop before I use it to mitigate this issue.

PowerShell

```powershell
foreach ( $node in 1..6 )
{
    $result = $null
    try
    {
        ...
```

Scope issues

This also helps mitigate scoping issues. In that example, we assign values to `$result` over and over in a loop. But because PowerShell allows variable values from outside the function to bleed into the scope of the current function, initializing them inside your function mitigates bugs that can be introduced that way.

An uninitialized variable in your function is not `$null` if it's set to a value in a parent scope. The parent scope could be another function that calls your function and uses the same variable names.

If I take that same `Do-something` example and remove the loop, I would end up with something that looks like this example:

PowerShell

```powershell
function Invoke-Something
{
    $result = 'ParentScope'
    Do-Something
}

function Do-Something
{
    try
    {
        $result = Get-Something -Id $node
    }
    catch
    {
        Write-Verbose "[$result] not valid"
    }

    if ( $null -ne $result )
    {
        Update-Something $result
```

```
    }
}
```

If the call to `Get-Something` throws an exception, then my `$null` check finds the `$result` from `Invoke-Something`. Initializing the value inside your function mitigates this issue.

Naming variables is hard and it's common for an author to use the same variable names in multiple functions. I know I use `$node`, `$result`, `$data` all the time. So it would be very easy for values from different scopes to show up in places where they should not be.

Redirect output to $null

I have been talking about `$null` values for this entire article but the topic is not complete if I didn't mention redirecting output to `$null`. There are times when you have commands that output information or objects that you want to suppress. Redirecting output to `$null` does that.

Out-Null

The Out-Null command is the built-in way to redirect pipeline data to `$null`.

PowerShell

```
New-Item -Type Directory -Path $path | Out-Null
```

Assign to $null

You can assign the results of a command to `$null` for the same effect as using `Out-Null`.

PowerShell

```
$null = New-Item -Type Directory -Path $path
```

Because `$null` is a constant value, you can never overwrite it. I don't like the way it looks in my code but it often performs faster than `Out-Null`.

Redirect to $null

You can also use the redirection operator to send output to `$null`.

PowerShell

```
New-Item -Type Directory -Path $path > $null
```

If you're dealing with command-line executables that output on the different streams. You can redirect all output streams to `$null` like this:

PowerShell

```
git status *> $null
```

Summary

I covered a lot of ground on this one and I know this article is more fragmented than most of my deep dives. That is because `$null` values can pop up in many different places in PowerShell and all the nuances are specific to where you find it. I hope you walk away from this with a better understanding of `$null` and an awareness of the more obscure scenarios you may run into.

Last updated on 04/16/2025

Everything you wanted to know about ShouldProcess

PowerShell functions have several features that greatly improve the way users interact with them. One important feature that is often overlooked is `-WhatIf` and `-Confirm` support and it's easy to add to your functions. In this article, we dive deep into how to implement this feature.

> **Note**
>
> The original version of this article appeared on the blog written by @KevinMarquette. The PowerShell team thanks Kevin for sharing this content with us. Please check out his blog at PowerShellExplained.com.

This is a simple feature you can enable in your functions to provide a safety net for the users that need it. There's nothing scarier than running a command that you know can be dangerous for the first time. The option to run it with `-WhatIf` can make a big difference.

CommonParameters

Before we look at implementing these common parameters, I want to take a quick look at how they're used.

Using -WhatIf

When a command supports the `-WhatIf` parameter, it allows you to see what the command would have done instead of making changes. it's a good way to test out the impact of a command, especially before you do something destructive.

PowerShell

```
PS C:\temp> Get-ChildItem
    Directory: C:\temp
Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         4/19/2021   8:59 AM              0 importantfile.txt
-a----         4/19/2021   8:58 AM              0 myfile1.txt
-a----         4/19/2021   8:59 AM              0 myfile2.txt
```

```
PS C:\temp> Remove-Item -Path .\myfile1.txt -WhatIf
What if: Performing the operation "Remove File" on target "C:\Temp\myfile1.txt".
```

If the command correctly implements `ShouldProcess`, it should show you all the changes that it would have made. Here is an example using a wildcard to delete multiple files.

PowerShell

```
PS C:\temp> Remove-Item -Path * -WhatIf
What if: Performing the operation "Remove File" on target "C:\Temp\myfile1.txt".
What if: Performing the operation "Remove File" on target "C:\Temp\myfile2.txt".
What if: Performing the operation "Remove File" on target
"C:\Temp\importantfile.txt".
```

Using -Confirm

Commands that support `-WhatIf` also support `-Confirm`. This gives you a chance confirm an action before performing it.

PowerShell

```
PS C:\temp> Remove-Item .\myfile1.txt -Confirm

Confirm
Are you sure you want to perform this action?
Performing the operation "Remove File" on target "C:\Temp\myfile1.txt".
[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is
"Y"):
```

In this case, you have multiple options that allow you to continue, skip a change, or stop the script. The help prompt describes each of those options like this.

Output

```
Y - Continue with only the next step of the operation.
A - Continue with all the steps of the operation.
N - Skip this operation and proceed with the next operation.
L - Skip this operation and all subsequent operations.
S - Pause the current pipeline and return to the command prompt. Type "exit" to
resume the pipeline.
[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is
"Y"):
```

Localization

This prompt is localized in PowerShell so the language changes based on the language of your operating system. This is one more thing that PowerShell manages for you.

`[switch]` parameters

Let's take a quick moment to look at ways to pass a value to a `[switch]` parameter. The main reason I call this out is that you often want to pass parameter values to functions you call.

The first approach is a specific parameter syntax that can be used for all parameters but you mostly see it used for `[switch]` parameters. You specify a colon to attach a value to the parameter.

PowerShell

```
Remove-Item -Path:* -WhatIf:$true
```

You can do the same with a variable.

PowerShell

```
$DoWhatIf = $true
Remove-Item -Path * -WhatIf:$DoWhatIf
```

The second approach is to use a hashtable to splat the value.

PowerShell

```
$RemoveSplat = @{
    Path = '*'
    WhatIf = $true
}
Remove-Item @RemoveSplat
```

If you're new to hashtables or splatting, I have another article on that covers everything you wanted to know about hashtables.

SupportsShouldProcess

The first step to enable `-WhatIf` and `-Confirm` support is to specify `SupportsShouldProcess` in the `CmdletBinding` of your function.

PowerShell

```
function Test-ShouldProcess {
    [CmdletBinding(SupportsShouldProcess)]
    param()
    Remove-Item .\myfile1.txt
}
```

By specifying `SupportsShouldProcess` in this way, we can now call our function with `-WhatIf` (or `-Confirm`).

PowerShell

```
PS> Test-ShouldProcess -WhatIf
What if: Performing the operation "Remove File" on target "C:\Temp\myfile1.txt".
```

Notice that I did not create a parameter called `-WhatIf`. Specifying `SupportsShouldProcess` automatically creates it for us. When we specify the `-WhatIf` parameter on `Test-ShouldProcess`, some things we call also perform `-WhatIf` processing.

> ⓘ Note
>
> When you use `SupportsShouldProcess`, PowerShell doesn't add the `$WhatIf` variable to the function. You don't need to check the value of `$WhatIf` because the `ShouldProcess()` method takes care of that for you.

Trust but verify

There's some danger here trusting that everything you call inherits `-WhatIf` values. For the rest of the examples, I'm going to assume that it doesn't work and be very explicit when making calls to other commands. I recommend that you do the same.

PowerShell

```
function Test-ShouldProcess {
    [CmdletBinding(SupportsShouldProcess)]
    param()
    Remove-Item .\myfile1.txt -WhatIf:$WhatIfPreference
}
```

I will revisit the nuances much later once you have a better understanding of all the pieces in play.

$PSCmdlet.ShouldProcess

The method that allows you to implement `SupportsShouldProcess` is `$PSCmdlet.ShouldProcess`. You call `$PSCmdlet.ShouldProcess(...)` to see if you should process some logic and PowerShell takes care of the rest. Let's start with an example:

PowerShell

```
function Test-ShouldProcess {
    [CmdletBinding(SupportsShouldProcess)]
    param()

    $file = Get-ChildItem './myfile1.txt'
    if($PSCmdlet.ShouldProcess($file.Name)){
        $file.Delete()
    }
}
```

The call to `$PSCmdlet.ShouldProcess($file.Name)` checks for the `-WhatIf` (and `-Confirm` parameter) then handles it accordingly. The `-WhatIf` causes `ShouldProcess` to output a description of the change and return `$false`:

PowerShell

```
PS> Test-ShouldProcess -WhatIf
What if: Performing the operation "Test-ShouldProcess" on target "myfile1.txt".
```

A call using `-Confirm` pauses the script and prompts the user with the option to continue. It returns `$true` if the user selected `Y`.

PowerShell

```
PS> Test-ShouldProcess -Confirm
Confirm
Are you sure you want to perform this action?
Performing the operation "Test-ShouldProcess" on target "myfile1.txt".
[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is
"Y"):
```

An awesome feature of `$PSCmdlet.ShouldProcess` is that it doubles as verbose output. I depend on this often when implementing `ShouldProcess`.

PowerShell

```
PS> Test-ShouldProcess -Verbose
VERBOSE: Performing the operation "Test-ShouldProcess" on target "myfile1.txt".
```

Overloads

There are a few different overloads for `$PSCmdlet.ShouldProcess` with different parameters for customizing the messaging. We already saw the first one in the example above. Let's take a closer look at it.

PowerShell

```
function Test-ShouldProcess {
    [CmdletBinding(SupportsShouldProcess)]
    param()

    if($PSCmdlet.ShouldProcess('TARGET')){
        # ...
    }
}
```

This produces output that includes both the function name and the target (value of the parameter).

PowerShell

```
What if: Performing the operation "Test-ShouldProcess" on target "TARGET".
```

Specifying a second parameter as the operation uses the operation value instead of the function name in the message.

PowerShell

```
## $PSCmdlet.ShouldProcess('TARGET','OPERATION')
What if: Performing the operation "OPERATION" on target "TARGET".
```

The next option is to specify three parameters to fully customize the message. When three parameters are used, the first one is the entire message. The second two parameters are still used in the `-Confirm` message output.

PowerShell

```
## $PSCmdlet.ShouldProcess('MESSAGE','TARGET','OPERATION')
What if: MESSAGE
```

Quick parameter reference

Just in case you came here only to figure out what parameters you should use, here is a quick reference showing how the parameters change the message in the different `-WhatIf` scenarios.

PowerShell

```
## $PSCmdlet.ShouldProcess('TARGET')
What if: Performing the operation "FUNCTION_NAME" on target "TARGET".

## $PSCmdlet.ShouldProcess('TARGET','OPERATION')
What if: Performing the operation "OPERATION" on target "TARGET".

## $PSCmdlet.ShouldProcess('MESSAGE','TARGET','OPERATION')
What if: MESSAGE
```

I tend to use the one with two parameters.

ShouldProcessReason

We have a fourth overload that's more advanced than the others. It allows you to get the reason `ShouldProcess` was executed. I'm only adding this here for completeness because we can just check if `$WhatIfPreference` is `$true` instead.

PowerShell

```
$reason = ''
if($PSCmdlet.ShouldProcess('MESSAGE','TARGET','OPERATION',[ref]$reason)){
    Write-Output "Some Action"
}
$reason
```

We have to pass the `$reason` variable into the fourth parameter as a reference variable with `[ref]`. `ShouldProcess` populates `$reason` with the value `None` or `WhatIf`. I didn't say this was useful and I have had no reason to ever use it.

Where to place it

You use `ShouldProcess` to make your scripts safer. So you use it when your scripts are making changes. I like to place the `$PSCmdlet.ShouldProcess` call as close to the change as possible.

PowerShell

```
## general logic and variable work
if ($PSCmdlet.ShouldProcess('TARGET','OPERATION')){
    # Change goes here
}
```

If I'm processing a collection of items, I call it for each item. So the call gets placed inside the `foreach` loop.

PowerShell

```
foreach ($node in $collection){
    # general logic and variable work
    if ($PSCmdlet.ShouldProcess($node,'OPERATION')){
        # Change goes here
    }
}
```

The reason why I place `ShouldProcess` tightly around the change, is that I want as much code to execute as possible when `-WhatIf` is specified. I want the setup and validation to run if possible so the user gets to see those errors.

I also like to use this in my Pester tests that validate my projects. If I have a piece of logic that is hard to mock in pester, I can often wrap it in `ShouldProcess` and call it with `-WhatIf` in my tests. It's better to test some of your code than none of it.

$WhatIfPreference

The first preference variable we have is `$WhatIfPreference`. This is `$false` by default. If you set it to `$true` then your function executes as if you specified `-WhatIf`. If you set this in your session, all commands perform `-WhatIf` execution.

When you call a function with `-WhatIf`, the value of `$WhatIfPreference` gets set to `$true` inside the scope of your function.

ConfirmImpact

Most of my examples are for `-WhatIf` but everything so far also works with `-Confirm` to prompt the user. You can set the `ConfirmImpact` of the function to high and it prompts the user as if it was called with `-Confirm`.

PowerShell

```
function Test-ShouldProcess {
    [CmdletBinding(
        SupportsShouldProcess,
        ConfirmImpact = 'High'
    )]
    param()
```

```
    if ($PSCmdlet.ShouldProcess('TARGET')){
        Write-Output "Some Action"
    }
}
```

This call to `Test-ShouldProcess` is performing the `-Confirm` action because of the `High` impact.

PowerShell

```
PS> Test-ShouldProcess

Confirm
Are you sure you want to perform this action?
Performing the operation "Test-ShouldProcess" on target "TARGET".
[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is
"Y"): y
Some Action
```

The obvious issue is that now it's harder to use in other scripts without prompting the user. In this case, we can pass a `$false` to `-Confirm` to suppress the prompt.

PowerShell

```
PS> Test-ShouldProcess -Confirm:$false
Some Action
```

I'll cover how to add `-Force` support in a later section.

$ConfirmPreference

`$ConfirmPreference` is an automatic variable that controls when `ConfirmImpact` asks you to confirm execution. Here are the possible values for both `$ConfirmPreference` and `ConfirmImpact`.

- `High`
- `Medium`
- `Low`
- `None`

With these values, you can specify different levels of impact for each function. If you have `$ConfirmPreference` set to a value higher than `ConfirmImpact`, then you aren't prompted to confirm execution.

By default, `$ConfirmPreference` is set to `High` and `ConfirmImpact` is `Medium`. If you want your function to automatically prompt the user, set your `ConfirmImpact` to `High`. Otherwise set it to `Medium` if its destructive and use `Low` if the command is always safe run in production. If you set it to `none`, it doesn't prompt even if `-Confirm` was specified (but it still gives you `-WhatIf` support).

When calling a function with `-Confirm`, the value of `$ConfirmPreference` gets set to `Low` inside the scope of your function.

Suppressing nested confirm prompts

The `$ConfirmPreference` can get picked up by functions that you call. This can create scenarios where you add a confirm prompt and the function you call also prompts the user.

What I tend to do is specify `-Confirm:$false` on the commands that I call when I have already handled the prompting.

PowerShell

```powershell
function Test-ShouldProcess {
    [CmdletBinding(SupportsShouldProcess)]
    param()

    $file = Get-ChildItem './myfile1.txt'
    if($PSCmdlet.ShouldProcess($file.Name)){
        Remove-Item -Path $file.FullName -Confirm:$false
    }
}
```

This brings us back to an earlier warning: There are nuances as to when `-WhatIf` is not passed to a function and when `-Confirm` passes to a function. I promise I'll get back to this later.

$PSCmdlet.ShouldContinue

If you need more control than `ShouldProcess` provides, you can trigger the prompt directly with `ShouldContinue`. `ShouldContinue` ignores `$ConfirmPreference`, `ConfirmImpact`, `-Confirm`, `$WhatIfPreference`, and `-WhatIf` because it prompts every time it's executed.

At a quick glance, it's easy to confuse `ShouldProcess` and `ShouldContinue`. I tend to remember to use `ShouldProcess` because the parameter is called `SupportsShouldProcess` in the `CmdletBinding`. You should use `ShouldProcess` in almost every scenario. That is why I covered that method first.

Let's take a look at `ShouldContinue` in action.

PowerShell

```
function Test-ShouldContinue {
    [CmdletBinding()]
    param()

    if($PSCmdlet.ShouldContinue('TARGET','OPERATION')){
        Write-Output "Some Action"
    }
}
```

This provides us a simpler prompt with fewer options.

PowerShell

```
Test-ShouldContinue

Second
TARGET
[Y] Yes  [N] No  [S] Suspend  [?] Help (default is "Y"):
```

The biggest issue with `ShouldContinue` is that it requires the user to run it interactively because it always prompts the user. You should always be building tools that can be used by other scripts. The way you do this is by implementing `-Force`. I'll revisit this idea later.

Yes to all

This is automatically handled with `ShouldProcess` but we have to do a little more work for `ShouldContinue`. There's a second method overload where we have to pass in a few values by reference to control the logic.

PowerShell

```
function Test-ShouldContinue {
    [CmdletBinding()]
    param()

    $collection = 1..5
    $yesToAll = $false
    $noToAll = $false

    foreach($target in $collection) {

        $continue = $PSCmdlet.ShouldContinue(
                "TARGET_$target",
```

```
                'OPERATION',
                [ref]$yesToAll,
                [ref]$noToAll
            )

        if ($continue){
            Write-Output "Some Action [$target]"
        }
    }
}
```

I added a `foreach` loop and a collection to show it in action. I pulled the `ShouldContinue` call out of the `if` statement to make it easier to read. Calling a method with four parameters starts to get a little ugly, but I tried to make it look as clean as I could.

Implementing -Force

`ShouldProcess` and `ShouldContinue` need to implement `-Force` in different ways. The trick to these implementations is that `ShouldProcess` should always get executed, but `ShouldContinue` should not get executed if `-Force` is specified.

ShouldProcess -Force

If you set your `ConfirmImpact` to `high`, the first thing your users are going to try is to suppress it with `-Force`. That's the first thing I do anyway.

PowerShell

```
Test-ShouldProcess -Force
Error: Test-ShouldProcess: A parameter cannot be found that matches parameter name
'force'.
```

If you recall from the `ConfirmImpact` section, they actually need to call it like this:

PowerShell

```
Test-ShouldProcess -Confirm:$false
```

Not everyone realizes they need to do that and `-Force` doesn't suppress `ShouldContinue`. So we should implement `-Force` for the sanity of our users. Take a look at this full example here:

PowerShell

```
function Test-ShouldProcess {
    [CmdletBinding(
        SupportsShouldProcess,
        ConfirmImpact = 'High'
    )]
    param(
        [switch]$Force
    )

    if ($Force -and -not $PSBoundParameters.ContainsKey('Confirm')) {
        $ConfirmPreference = 'None'
    }

    if ($PSCmdlet.ShouldProcess('TARGET')) {
        Write-Output "Some Action"
    }
}
```

We add our own `-Force` switch as a parameter. The `-Confirm` parameter is automatically added when using `SupportsShouldProcess` in the `CmdletBinding`. However, when you use `SupportsShouldProcess`, PowerShell doesn't add the `$Confirm` variable to the function. If you are running in Strict Mode and try to use the `$Confirm` variable before it has been defined, you get an error. To avoid the error you can use `$PSBoundParameters` to test if the parameter was passed by the user.

PowerShell

```
if ($Force -and -not $PSBoundParameters.ContainsKey('Confirm')) {
    $ConfirmPreference = 'None'
}
```

If the user specifies `-Force` we set `$ConfirmPreference` to `None` in the local scope. If the user also specifies `-Confirm` then `ShoudProcess()` honors the values of the `-Confirm` parameter.

PowerShell

```
if ($PSCmdlet.ShouldProcess('TARGET')){
    Write-Output "Some Action"
}
```

If someone specifies both `-Force` and `-WhatIf`, then `-WhatIf` needs to take priority. This approach preserves `-WhatIf` processing because `ShouldProcess` always gets executed.

Don't add a test for the `$Force` value inside the `if` statement with the `ShouldProcess`. That is an anti-pattern for this specific scenario even though that's what I show you in the next section

for `ShouldContinue`.

ShouldContinue -Force

This is the correct way to implement `-Force` with `ShouldContinue`.

PowerShell

```
function Test-ShouldContinue {
    [CmdletBinding()]
    param(
        [switch]$Force
    )

    if($Force -or $PSCmdlet.ShouldContinue('TARGET','OPERATION')){
        Write-Output "Some Action"
    }
}
```

By placing the `$Force` to the left of the `-or` operator, it gets evaluated first. Writing it this way short circuits the execution of the `if` statement. If `$Force` is `$true`, then the `ShouldContinue` is not executed.

PowerShell

```
PS> Test-ShouldContinue -Force
Some Action
```

We don't have to worry about `-Confirm` or `-WhatIf` in this scenario because they're not supported by `ShouldContinue`. This is why it needs to be handled differently than `ShouldProcess`.

Scope issues

Using `-WhatIf` and `-Confirm` are supposed to apply to everything inside your functions and everything they call. They do this by setting `$WhatIfPreference` to `$true` or setting `$ConfirmPreference` to `Low` in the local scope of the function. When you call another function, calls to `ShouldProcess` use those values.

This actually works correctly most of the time. Anytime you call built-in cmdlet or a function in your same scope, it works. It also works when you call a script or a function in a script module from the console.

The one specific place where it doesn't work is when a script or a script module calls a function in another script module. This may not sound like a big problem, but most of the modules you create or pull from the PSGallery are script modules.

The core issue is that script modules do not inherit the values for `$WhatIfPreference` or `$ConfirmPreference` (and several others) when called from functions in other script modules.

The best way to summarize this as a general rule is that this works correctly for binary modules and never trust it to work for script modules. If you aren't sure, either test it or just assume it doesn't work correctly.

I personally feel this is very dangerous because it creates scenarios where you add `-WhatIf` support to multiple modules that work correctly in isolation, but fail to work correctly when they call each other.

We do have a GitHub RFC working to get this issue fixed. See Propagate execution preferences beyond script module scope for more details.

In closing

I have to look up how to use `ShouldProcess` every time I need to use it. It took me a long time to distinguish `ShouldProcess` from `ShouldContinue`. I almost always need to look up what parameters to use. So don't worry if you still get confused from time to time. This article will be here when you need it. I'm sure I will reference it often myself.

If you liked this post, please share your thoughts with me on Twitter using the link below. I always like hearing from people that get value from my content.

Last updated on 04/08/2026

Visualize parameter binding

Article • 05/20/2024

Parameter binding is the process that PowerShell uses to determine which parameter set is being used and to associate (bind) values to the parameters of a command. These values can come from the command line and the pipeline.

The parameter binding process starts by binding named and positional command-line arguments. After binding command-line arguments, PowerShell tries to bind any pipeline input. There are two ways that values are bound from the pipeline. Parameters that accept pipeline input have one or both of the following attributes:

- ValueFromPipeline - The value from the pipeline is bound to the parameter based on its type. The type of the argument must match the type of the parameter.
- ValueFromPipelineByPropertyName - The value from the pipeline is bound to the parameter based on its name. The object in the pipeline must have a property that matches the name of the parameter or one of its aliases. The type of the property must match or be convertible to the type of the parameter.

For more information about parameter binding, see about_Parameter_Binding.

Use `Trace-Command` to visualize parameter binding

Troubleshooting parameter binding issues can be challenging. You can use the Trace-Command cmdlet to visualize the parameter binding process.

Consider the following scenario. You have a directory with two text files, `file1.txt` and `[file2].txt`.

PowerShell

```
PS> Get-ChildItem

    Directory: D:\temp\test\binding

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a---           5/17/2024 12:59 PM              0 [file2].txt
-a---           5/17/2024 12:59 PM              0 file1.txt
```

You want to delete the files by passing the filenames, through the pipeline, to the `Remove-Item` cmdlet.

PowerShell

```
PS> 'file1.txt', '[file2].txt' | Remove-Item
PS> Get-ChildItem

    Directory: D:\temp\test\binding

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a---           5/17/2024 12:59 PM              0 [file2].txt
```

Notice that `Remove-Item` only deleted `file1.txt` and not `[file2].txt`. The filename includes square brackets, which is treated as a wildcard expression. Using `Trace-Command`, you can see that the filename is being bound to the **Path** parameter of `Remove-Item`.

PowerShell

```
Trace-Command -PSHost -Name ParameterBinding -Expression {
    '[file2].txt' | Remove-Item
}
```

The output from `Trace-Command` can be verbose. Each line of output is prefixed with a timestamp and trace provider information. For the output of this example, the prefix information has been removed to make it easier to read.

Output

```
BIND NAMED cmd line args [Remove-Item]
BIND POSITIONAL cmd line args [Remove-Item]
BIND cmd line args to DYNAMIC parameters.
    DYNAMIC parameter object:
[Microsoft.PowerShell.Commands.FileSystemProviderRemoveItemDynamicParameters
]
MANDATORY PARAMETER CHECK on cmdlet [Remove-Item]
CALLING BeginProcessing
BIND PIPELINE object to parameters: [Remove-Item]
    PIPELINE object TYPE = [System.String]
    RESTORING pipeline parameter's original values
    Parameter [Path] PIPELINE INPUT ValueFromPipeline NO COERCION
    BIND arg [[file2].txt] to parameter [Path]
        Binding collection parameter Path: argument type [String], parameter
type [System.String[]],
            collection type Array, element type [System.String], no
coerceElementType
```

```
        Creating array with element type [System.String] and 1 elements
        Argument type String is not IList, treating this as scalar
        Adding scalar element of type String to array position 0
        BIND arg [System.String[]] to param [Path] SUCCESSFUL
    Parameter [Credential] PIPELINE INPUT ValueFromPipelineByPropertyName NO
COERCION
    Parameter [Credential] PIPELINE INPUT ValueFromPipelineByPropertyName NO
COERCION
    Parameter [Credential] PIPELINE INPUT ValueFromPipelineByPropertyName
WITH COERCION
    Parameter [Credential] PIPELINE INPUT ValueFromPipelineByPropertyName
WITH COERCION
MANDATORY PARAMETER CHECK on cmdlet [Remove-Item]
CALLING ProcessRecord
CALLING EndProcessing
```

Using `Get-Help`, you can see that the **Path** parameter of `Remove-Item` accepts string objects from the pipeline `ByValue` or `ByPropertyName`. **LiteralPath** accepts string objects from the pipeline `ByPropertyName`.

PowerShell

```
PS> Get-Help Remove-Item -Parameter Path, LiteralPath

-Path <System.String[]>
    Specifies a path of the items being removed. Wildcard characters are
permitted.

    Required?                    true
    Position?                    0
    Default value                None
    Accept pipeline input?       True (ByPropertyName, ByValue)
    Accept wildcard characters?  true


-LiteralPath <System.String[]>
    Specifies a path to one or more locations. The value of LiteralPath is
used exactly as it's
    typed. No characters are interpreted as wildcards. If the path includes
escape characters,
    enclose it in single quotation marks. Single quotation marks tell
PowerShell not to interpret
    any characters as escape sequences.

    Required?                    true
    Position?                    named
    Default value                None
    Accept pipeline input?       True (ByPropertyName)
    Accept wildcard characters?  false
```

The output of `Trace-Command` shows that parameter binding starts by binding command-line parameters followed by the pipeline input. You can see that `Remove-Item` receives a string object from the pipeline. That string object is bound to the **Path** parameter.

```
BIND PIPELINE object to parameters: [Remove-Item]
    PIPELINE object TYPE = [System.String]
    RESTORING pipeline parameter's original values
    Parameter [Path] PIPELINE INPUT ValueFromPipeline NO COERCION
    BIND arg [[file2].txt] to parameter [Path]
    ...
        BIND arg [System.String[]] to param [Path] SUCCESSFUL
```

Since the **Path** parameter accepts wildcard characters, the square brackets represent a wildcard expression. However, that expression doesn't match any files in the directory. You need to use the **LiteralPath** parameter to specify the exact path to the file.

`Get-Command` shows that the **LiteralPath** parameter accepts input from the pipeline `ByPropertyName` or `ByValue`. And, that it has two aliases, `PSPath` and `LP`.

PowerShell

```
PS> (Get-Command Remove-Item).Parameters.LiteralPath.Attributes |
>> Select-Object ValueFrom*, Alias* | Format-List

ValueFromPipeline               : False
ValueFromPipelineByPropertyName : True
ValueFromRemainingArguments     : False

AliasNames : {PSPath, LP}
```

In this next example, `Get-Item` is used to retrieve a **FileInfo** object. That object has a property named **PSPath**.

PowerShell

```
PS> Get-Item *.txt | Select-Object PSPath

PSPath
------
Microsoft.PowerShell.Core\FileSystem::D:\temp\test\binding\[file2].txt
```

The **FileInfo** object is then passed to `Remove-Item`.

PowerShell

```
Trace-Command -PSHost -Name ParameterBinding -Expression {
    Get-Item *.txt | Remove-Item
}
```

For the output of this example, the prefix information has been removed and separated to show parameter binding for both commands.

In this output, you can see that `Get-Item` binds the positional parameter value `*.txt` to the **Path** parameter.

Output

```
BIND NAMED cmd line args [Get-Item]
BIND POSITIONAL cmd line args [Get-Item]
    BIND arg [*.txt] to parameter [Path]
        Binding collection parameter Path: argument type [String], parameter
type [System.String[]],
            collection type Array, element type [System.String], no
coerceElementType
        Creating array with element type [System.String] and 1 elements
        Argument type String is not IList, treating this as scalar
        Adding scalar element of type String to array position 0
        BIND arg [System.String[]] to param [Path] SUCCESSFUL
BIND cmd line args to DYNAMIC parameters.
    DYNAMIC parameter object:
[Microsoft.PowerShell.Commands.FileSystemProviderGetItemDynamicParameters]
MANDATORY PARAMETER CHECK on cmdlet [Get-Item]
```

In the trace output for parameter binding, you can see that `Remove-Item` receives a **FileInfo** object from the pipeline. Since a **FileInfo** object isn't a **String** object, it can't be bound to the **Path** parameter.

The **PSPath** property of the **FileInfo** object matches an alias for the **LiteralPath** parameter. **PSPath** is also a **String** object, so it can be bound to the **LiteralPath** parameter without type coercion.

Output

```
BIND NAMED cmd line args [Remove-Item]
BIND POSITIONAL cmd line args [Remove-Item]
BIND cmd line args to DYNAMIC parameters.
    DYNAMIC parameter object:
[Microsoft.PowerShell.Commands.FileSystemProviderRemoveItemDynamicParameters
]
MANDATORY PARAMETER CHECK on cmdlet [Remove-Item]
CALLING BeginProcessing
CALLING BeginProcessing
CALLING ProcessRecord
```

```
    BIND PIPELINE object to parameters: [Remove-Item]
        PIPELINE object TYPE = [System.IO.FileInfo]
        RESTORING pipeline parameter's original values
        Parameter [Path] PIPELINE INPUT ValueFromPipeline NO COERCION
        BIND arg [D:\temp\test\binding\[file2].txt] to parameter [Path]
            Binding collection parameter Path: argument type [FileInfo],
parameter type [System.String[]],
                collection type Array, element type [System.String], no
coerceElementType
            Creating array with element type [System.String] and 1 elements
            Argument type FileInfo is not IList, treating this as scalar
            BIND arg [D:\temp\test\binding\[file2].txt] to param [Path]
SKIPPED
        Parameter [Credential] PIPELINE INPUT
ValueFromPipelineByPropertyName NO COERCION
        Parameter [Path] PIPELINE INPUT ValueFromPipelineByPropertyName NO
COERCION
        Parameter [Credential] PIPELINE INPUT
ValueFromPipelineByPropertyName NO COERCION
        Parameter [LiteralPath] PIPELINE INPUT
ValueFromPipelineByPropertyName NO COERCION
        BIND arg
[Microsoft.PowerShell.Core\FileSystem::D:\temp\test\binding\[file2].txt] to
parameter [LiteralPath]
            Binding collection parameter LiteralPath: argument type
[String], parameter type [System.String[]],
                collection type Array, element type [System.String], no
coerceElementType
            Creating array with element type [System.String] and 1 elements
            Argument type String is not IList, treating this as scalar
            Adding scalar element of type String to array position 0
            BIND arg [System.String[]] to param [LiteralPath] SUCCESSFUL
        Parameter [Credential] PIPELINE INPUT
ValueFromPipelineByPropertyName WITH COERCION
    MANDATORY PARAMETER CHECK on cmdlet [Remove-Item]
    CALLING ProcessRecord
CALLING EndProcessing
CALLING EndProcessing
```

Writing Progress across multiple threads with `ForEach-Object -Parallel`

Starting in PowerShell 7.0, the ability to work in multiple threads simultaneously is possible using the **Parallel** parameter in the ForEach-Object cmdlet. Monitoring the progress of these threads can be a challenge though. Normally, you can monitor the progress of a process using Write-Progress. However, since PowerShell uses a separate runspace for each thread when using **Parallel**, reporting the progress back to the host isn't as straight forward as normal use of `Write-Progress`.

Using a synced hashtable to track progress

When writing the progress from multiple threads, tracking becomes difficult because when running parallel processes in PowerShell, each process has it's own runspace. To get around this, you can use a synchronized hashtable. A synced hashtable is a thread safe data structure that can be modified by multiple threads simultaneously without throwing an error.

Set up

One of the downsides to this approach is it takes a, somewhat, complex set up to ensure everything runs without error.

PowerShell

```
$dataset = @(
    @{
        Id   = 1
        Wait = 3..10 | Get-Random | ForEach-Object {$_*100}
    }
    @{
        Id   = 2
        Wait = 3..10 | Get-Random | ForEach-Object {$_*100}
    }
    @{
        Id   = 3
        Wait = 3..10 | Get-Random | ForEach-Object {$_*100}
    }
    @{
        Id   = 4
        Wait = 3..10 | Get-Random | ForEach-Object {$_*100}
    }
    @{
```

```
        Id   = 5
        Wait = 3..10 | Get-Random | ForEach-Object {$_*100}
    }
)

# Create a hashtable for process.
# Keys should be ID's of the processes
$origin = @{}
$dataset | ForEach-Object {$origin.($_.Id) = @{}}

# Create synced hashtable
$sync = [System.Collections.Hashtable]::Synchronized($origin)
```

This section creates three different data structures, for three different purposes.

The `$dataSet` variable stores an array of hashtables that is used to coordinate the next steps without the risk of being modified. If an object collection is modified while iterating through the collection, PowerShell throws an error. You must keep the object collection in the loop separate from the objects being modified. The `Id` key in each hashtable is the identifier for a mock process. The `Wait` key simulates the workload of each mock process being tracked.

The `$origin` variable stores a nested hashtable with each key being one of the mock process id's. Then, it is used to hydrate the synchronized hashtable stored in the `$sync` variable. The `$sync` variable is responsible for reporting the progress back to the parent runspace, which displays the progress.

Running the processes

This section runs the multi-threaded processes and creates some of the output used to display progress.

PowerShell

```
$job = $dataset | ForEach-Object -ThrottleLimit 3 -AsJob -Parallel {
    $syncCopy = $Using:sync
    $process = $syncCopy.$($PSItem.Id)

    $process.Id = $PSItem.Id
    $process.Activity = "Id $($PSItem.Id) starting"
    $process.Status = "Processing"

    # Fake workload start up that takes x amount of time to complete
    Start-Sleep -Milliseconds ($PSItem.Wait*5)

    # Process. update activity
    $process.Activity = "Id $($PSItem.Id) processing"
    foreach ($percent in 1..100)
    {
```

```
        # Update process on status
        $process.Status = "Handling $percent/100"
        $process.PercentComplete = (($percent / 100) * 100)

        # Fake workload that takes x amount of time to complete
        Start-Sleep -Milliseconds $PSItem.Wait
    }

    # Mark process as completed
    $process.Completed = $true
}
```

The mock processes are sent to `ForEach-Object` and started as jobs. The **ThrottleLimit** is set to **3** to highlight running multiple processes in a queue. The jobs are stored in the `$job` variable and allows us to know when all the processes have finished later on.

When using the `Using:` statement to reference a parent scope variable in PowerShell, you can't use expressions to make it dynamic. For example, if you tried to create the `$process` variable like this, `$process = $Using:sync.$($PSItem.Id)`, you would get an error stating you can't use expressions there. So, we create the `$syncCopy` variable to be able to reference and modify the `$sync` variable without the risk of it failing.

Next, we build out a hashtable to represent the progress of the process currently in the loop using the `$process` variable by referencing the synchronized hashtable keys. The **Activity** and the **Status** keys are used as parameter values for `Write-Progress` to display the status of a given mock process in the next section.

The `foreach` loop is just a way to simulate the process working and is randomized based on the `$dataSet` **Wait** attribute to set `Start-Sleep` using milliseconds. How you calculate the progress of your process may vary.

Displaying the progress of multiple processes

Now that the mock processes are running as jobs, we can start to write the processes progress to the PowerShell window.

PowerShell

```
while($job.State -eq 'Running')
{
    $sync.Keys | ForEach-Object {
        # If key is not defined, ignore
        if(![string]::IsNullOrEmpty($sync.$_.Keys))
        {
            # Create parameter hashtable to splat
```

```
            $param = $sync.$_

            # Execute Write-Progress
            Write-Progress @param
        }
    }

    # Wait to refresh to not overload gui
    Start-Sleep -Seconds 0.1
}
```

The `$job` variable contains the parent **job** and has a child **job** for each of the mock processes. While any of the child jobs are still running, the parent job **State** will remain "Running". This allows us to use the `while` loop to continually update the progress of every process until all processes are finished.

Within the while loop, we loop through each of the keys in the `$sync` variable. Since this is a synchronized hashtable, it is constantly updated but can still be accessed without throwing any errors.

There is a check to ensure that the process being reported is actually running using the `IsNullOrEmpty()` method. If the process hasn't been started, the loop won't report on it and move on to the next until it gets to a process that has been started. If the process is started, the hashtable from the current key is used to splat the parameters to `Write-Progress`.

Full example

PowerShell

```
# Example workload
$dataset = @(
    @{
        Id   = 1
        Wait = 3..10 | Get-Random | ForEach-Object {$_*100}
    }
    @{
        Id   = 2
        Wait = 3..10 | Get-Random | ForEach-Object {$_*100}
    }
    @{
        Id   = 3
        Wait = 3..10 | Get-Random | ForEach-Object {$_*100}
    }
    @{
        Id   = 4
        Wait = 3..10 | Get-Random | ForEach-Object {$_*100}
    }
    @{
```

```
        Id   = 5
        Wait = 3..10 | Get-Random | ForEach-Object {$_*100}
    }
)

# Create a hashtable for process.
# Keys should be ID's of the processes
$origin = @{}
$dataset | ForEach-Object {$origin.($_.Id) = @{}}

# Create synced hashtable
$sync = [System.Collections.Hashtable]::Synchronized($origin)

$job = $dataset | ForEach-Object -ThrottleLimit 3 -AsJob -Parallel {
    $syncCopy = $Using:sync
    $process = $syncCopy.$($PSItem.Id)

    $process.Id = $PSItem.Id
    $process.Activity = "Id $($PSItem.Id) starting"
    $process.Status = "Processing"

    # Fake workload start up that takes x amount of time to complete
    Start-Sleep -Milliseconds ($PSItem.Wait*5)

    # Process. update activity
    $process.Activity = "Id $($PSItem.Id) processing"
    foreach ($percent in 1..100)
    {
        # Update process on status
        $process.Status = "Handling $percent/100"
        $process.PercentComplete = (($percent / 100) * 100)

        # Fake workload that takes x amount of time to complete
        Start-Sleep -Milliseconds $PSItem.Wait
    }

    # Mark process as completed
    $process.Completed = $true
}

while($job.State -eq 'Running')
{
    $sync.Keys | ForEach-Object {
        # If key is not defined, ignore
        if(![string]::IsNullOrEmpty($sync.$_.Keys))
        {
            # Create parameter hashtable to splat
            $param = $sync.$_

            # Execute Write-Progress
            Write-Progress @param
        }
    }

    # Wait to refresh to not overload gui
```

```
    Start-Sleep -Seconds 0.1
}
```

Related Links

- about_Jobs
- about_Scopes
- about_Splatting

Last updated on 03/24/2025

Add Credential support to PowerShell functions

> **Note**
>
> The original version of this article appeared on the blog written by @joshduffney. This article has been edited for inclusion on this site. The PowerShell team thanks Josh for sharing this content with us. Please check out his blog at duffney.io.

This article shows you how to add credential parameters to PowerShell functions and why you'd want to. A credential parameter is to allow you to run the function or cmdlet as a different user. The most common use is to run the function or cmdlet as an elevated user account.

For example, the cmdlet `New-ADUser` has a **Credential** parameter, which you could provide domain admin credentials to create an account in a domain. Assuming your normal account running the PowerShell session doesn't have that access already.

Creating credential object

The PSCredential object represents a set of security credentials such as a user name and password. The object can be passed as a parameter to a function that runs as the user account in that credential object. There are a few ways that you can create a credential object. The first way to create a credential object is to use the PowerShell cmdlet `Get-Credential`. When you run without parameters, it prompts you for a username and password. Or you can call the cmdlet with some optional parameters.

To specify the domain name and username ahead of time you can use either the **Credential** or **UserName** parameters. When you use the **UserName** parameter, you're also required to provide a **Message** value. The code below demonstrates using the cmdlet. You can also store the credential object in a variable so that you can use the credential multiple times. In the example below, the credential object is stored in the variable `$Cred`.

PowerShell

```
$Cred = Get-Credential
$Cred = Get-Credential -Credential domain\user
$Cred = Get-Credential -UserName domain\user -Message 'Enter Password'
```

Sometimes, you can't use the interactive method of creating credential objects shown in the previous example. Most automation tools require a non-interactive method. To create a credential without user interaction, create a secure string containing the password. Then pass the secure string and user name to the `System.Management.Automation.PSCredential()` method.

Use the following command to create a secure string containing the password:

PowerShell

```
ConvertTo-SecureString "MyPlainTextPassword" -AsPlainText -Force
```

Both the **AsPlainText** and **Force** parameters are required. Without those parameters, you receive a message warning that you shouldn't pass plain text into a secure string. PowerShell returns this warning because the plain text password gets recorded in various logs. Once you have a secure string created, you need to pass it to the `PSCredential()` method to create the credential object. In the following example, the variable `$password` contains the secure string `$Cred` contains the credential object.

PowerShell

```
$password = ConvertTo-SecureString "MyPlainTextPassword" -AsPlainText -Force
$Cred = New-Object System.Management.Automation.PSCredential ("username", $password)
```

Now that you know how to create credential objects, you can add credential parameters to your PowerShell functions.

Adding a Credential Parameter

Just like any other parameter, you start off by adding it in the `param` block of your function. It's recommended that you name the parameter `$Credential` because that's what existing PowerShell cmdlets use. The type of the parameter should be `[System.Management.Automation.PSCredential]`.

The following example shows the parameter block for a function called `Get-Something`. It has two parameters: `$Name` and `$Credential`.

PowerShell

```
function Get-Something {
    param(
        $Name,
```

```
        [System.Management.Automation.PSCredential]$Credential
    )
```

The code in this example is enough to have a working credential parameter, however there are a few things you can add to make it more robust.

- Add the `[ValidateNotNull()]` validation attribute to check that the value being passed to **Credential**. If the parameter value is null, this attribute prevents the function from executing with invalid credentials.
- Add `[System.Management.Automation.Credential()]`. This allows you to pass in a username as a string and have an interactive prompt for the password.
- Set a default value for the `$Credential` parameter to `[System.Management.Automation.PSCredential]::Empty`. Your function you might be passing this `$Credential` object to existing PowerShell cmdlets. Providing a null value to the cmdlet called inside your function causes an error. Providing an empty credential object avoids this error.

> **Tip**
>
> Some cmdlets that accept a credential parameter do not support `[System.Management.Automation.PSCredential]::Empty` as they should. See the Dealing with Legacy Cmdlets section for a workaround.

Using credential parameters

The following example demonstrates how to use credential parameters. This example shows a function called `Set-RemoteRegistryValue`. This function defines the credential parameter using the techniques describe in the previous section. The function calls `Invoke-Command` using the `$Credential` variable created by the function. This allows you to change the user who's running `Invoke-Command`. Because the default value of `$Credential` is an empty credential, the function can run without providing credentials.

PowerShell

```
function Set-RemoteRegistryValue {
    param(
        $ComputerName,
        $Path,
        $Name,
```

```
        $Value,
        [ValidateNotNull()]
        [System.Management.Automation.PSCredential]
        [System.Management.Automation.Credential()]
        $Credential = [System.Management.Automation.PSCredential]::Empty
    )
        $null = Invoke-Command -ComputerName $ComputerName -ScriptBlock {
            Set-ItemProperty -Path $Using:Path -Name $Using:Name -Value $Using:Value
        } -Credential $Credential
}
```

The following sections show different methods of providing credentials to `Set-RemoteRegistryValue`.

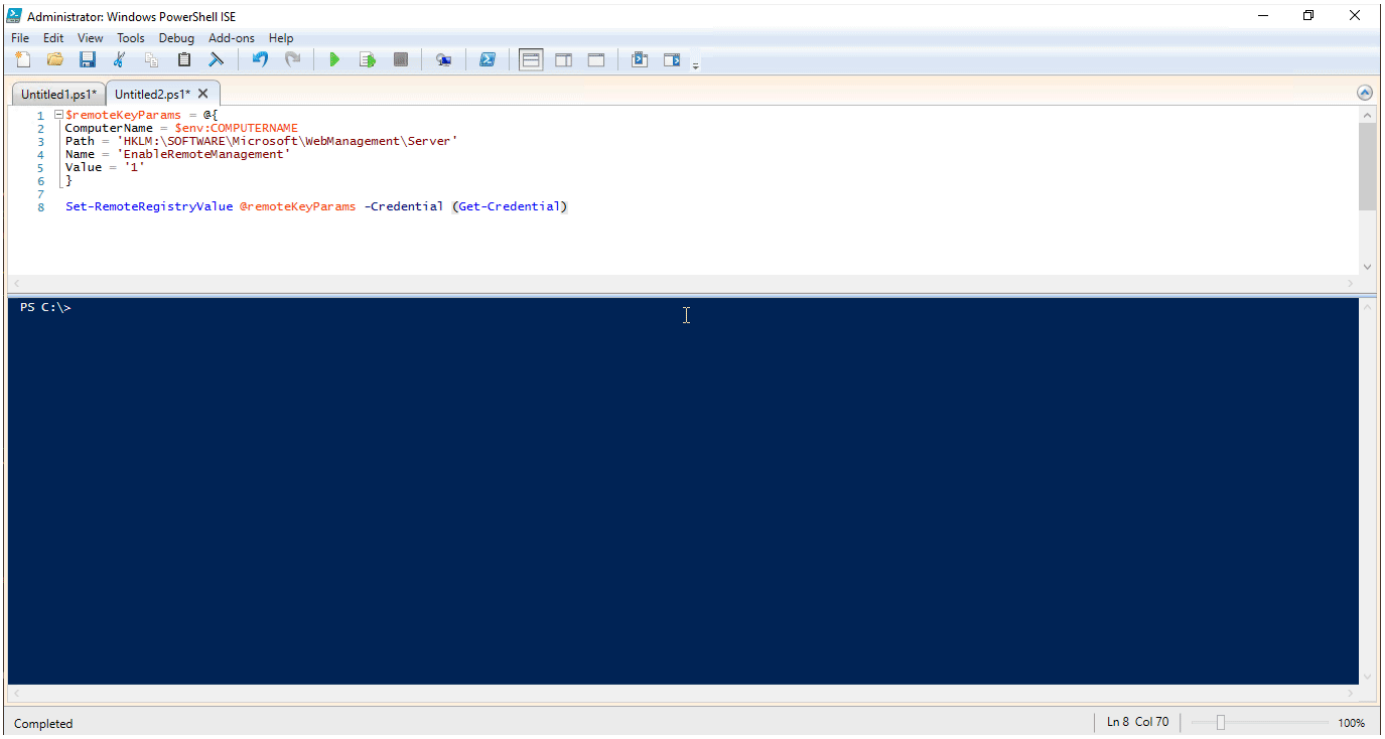
Prompting for credentials

Using `Get-Credential` in parentheses `()` at run time causes the `Get-Credential` to run first. You are prompted for a username and password. You could use the **Credential** or **UserName** parameters of `Get-Credential` to pre-populate the username and domain. The following example uses a technique called splatting to pass parameters to the `Set-RemoteRegistryValue` function. For more information about splatting, check out the about_Splatting article.

PowerShell

```
$remoteKeyParams = @{
    ComputerName = $Env:COMPUTERNAME
    Path = 'HKLM:\SOFTWARE\Microsoft\WebManagement\Server'
    Name = 'EnableRemoteManagement'
    Value = '1'
}

Set-RemoteRegistryValue @remoteKeyParams -Credential (Get-Credential)
```

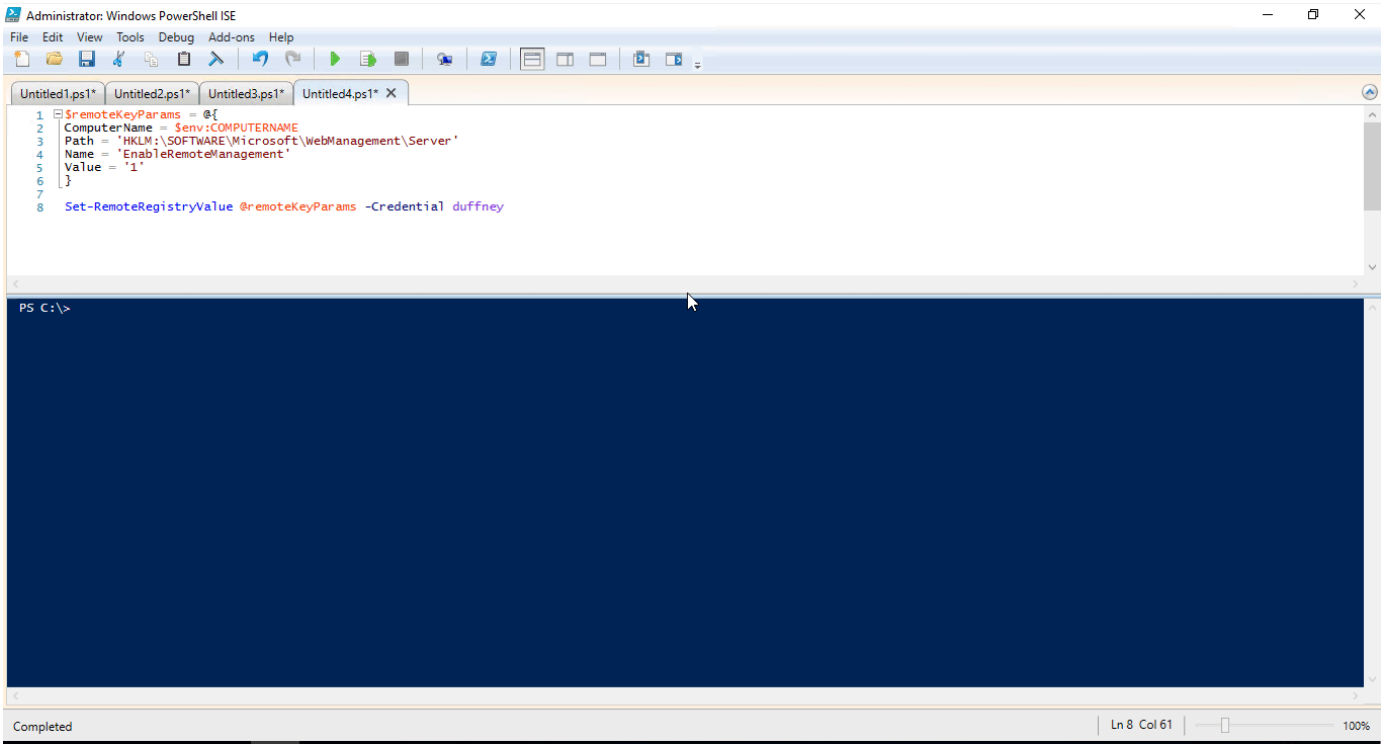


Using `(Get-Credential)` seems cumbersome. Normally, when you use the **Credential** parameter with only a username, the cmdlet automatically prompts for the password. The `[System.Management.Automation.Credential()]` attribute enables this behavior.

PowerShell

```powershell
$remoteKeyParams = @{
    ComputerName = $Env:COMPUTERNAME
    Path = 'HKLM:\SOFTWARE\Microsoft\WebManagement\Server'
    Name = 'EnableRemoteManagement'
    Value = '1'
}

Set-RemoteRegistryValue @remoteKeyParams -Credential duffney
```



> **Note**
>
> To set the registry value shown, these examples assume you have the **Web Server** features of Windows installed. Run `Install-WindowsFeature Web-Server` and `Install-WindowsFeature web-mgmt-tools` if required.

Provide credentials in a variable

You can also populate a credential variable ahead of time and pass it to the **Credential** parameter of `Set-RemoteRegistryValue` function. Use this method with Continuous Integration / Continuous Deployment (CI/CD) tools such as Jenkins, TeamCity, and Octopus Deploy. For an example using Jenkins, check out Hodge's blog post Automating with Jenkins and PowerShell on Windows - Part 2.

This example uses the .NET method to create the credential object and a secure string to pass in the password.

PowerShell

```powershell
$password = ConvertTo-SecureString "P@ssw0rd" -AsPlainText -Force
$Cred = New-Object System.Management.Automation.PSCredential ("duffney", $password)

$remoteKeyParams = @{
    ComputerName = $Env:COMPUTERNAME
    Path = 'HKLM:\SOFTWARE\Microsoft\WebManagement\Server'
    Name = 'EnableRemoteManagement'
```

```
    Value = '1'
}

Set-RemoteRegistryValue @remoteKeyParams -Credential $Cred
```

For this example, the secure string is created using a clear text password. All of the previously mentioned CI/CD have a secure method of providing that password at run time. When using those tools, replace the plain text password with the variable defined within the CI/CD tool you use.

Run without credentials

Since `$Credential` defaults to an empty credential object, you can run the command without credentials, as shown in this example:

PowerShell

```
$remoteKeyParams = @{
    ComputerName = $Env:COMPUTERNAME
    Path = 'HKLM:\SOFTWARE\Microsoft\WebManagement\Server'
    Name = 'EnableRemoteManagement'
    Value = '1'
}

Set-RemoteRegistryValue @remoteKeyParams
```

Dealing with legacy cmdlets

Not all cmdlets support credential objects or allow empty credentials. Instead, the cmdlet wants username and password parameters as strings. There are a few ways to work around this limitation.

Using if-else to handle empty credentials

In this scenario, the cmdlet you want to run doesn't accept an empty credential object. This example adds the **Credential** parameter to `Invoke-Command` only if it's not empty. Otherwise, it runs the `Invoke-Command` without the **Credential** parameter.

PowerShell

```
function Set-RemoteRegistryValue {
    param(
        $ComputerName,
        $Path,
```

```
        $Name,
        $Value,
        [ValidateNotNull()]
        [System.Management.Automation.PSCredential]
        [System.Management.Automation.Credential()]
        $Credential = [System.Management.Automation.PSCredential]::Empty
    )

    if($Credential -ne [System.Management.Automation.PSCredential]::Empty) {
        Invoke-Command -ComputerName:$ComputerName -Credential:$Credential  {
            Set-ItemProperty -Path $Using:Path -Name $Using:Name -Value $Using:Value
        }
    } else {
        Invoke-Command -ComputerName:$ComputerName {
            Set-ItemProperty -Path $Using:Path -Name $Using:Name -Value $Using:Value
        }
    }
}
```

Using splatting to handle empty credentials

This example uses parameter splatting to call the legacy cmdlet. The `$Credential` object is conditionally added to the hash table for splatting and avoids the need to repeat the `Invoke-Command` script block. To learn more about splatting inside functions, see the Splatting Parameters Inside Advanced Functions blog post.

PowerShell

```
function Set-RemoteRegistryValue {
    param(
        $ComputerName,
        $Path,
        $Name,
        $Value,
        [ValidateNotNull()]
        [System.Management.Automation.PSCredential]
        [System.Management.Automation.Credential()]
        $Credential = [System.Management.Automation.PSCredential]::Empty
    )

        $Splat = @{
            ComputerName = $ComputerName
        }

        if ($Credential -ne [System.Management.Automation.PSCredential]::Empty) {
            $Splat['Credential'] = $Credential
        }

        $null = Invoke-Command -ScriptBlock {
            Set-ItemProperty -Path $Using:Path -Name $Using:Name -Value $Using:Value
```

```
        } @splat
}
```

Working with string passwords

The `Invoke-Sqlcmd` cmdlet is an example of a cmdlet that accepts a string as a password. `Invoke-Sqlcmd` allows you to run simple SQL insert, update, and delete statements. `Invoke-Sqlcmd` requires a clear-text username and password rather than a more secure credential object. This example shows how to extract the username and password from a credential object.

The `Get-AllSQLDatabases` function in this example calls the `Invoke-Sqlcmd` cmdlet to query a SQL server for all its databases. The function defines a **Credential** parameter with the same attribute used in the previous examples. Since the username and password exist within the `$Credential` variable, you can extract those values for use with `Invoke-Sqlcmd`.

The user name is available from the **UserName** property of the `$Credential` variable. To obtain the password, you have to use the `GetNetworkCredential()` method of the `$Credential` object. The values are extracted into variables that are added to a hash table used for splatting parameters to `Invoke-Sqlcmd`.

PowerShell

```
function Get-AllSQLDatabases {
    param(
        $SQLServer,
        [ValidateNotNull()]
        [System.Management.Automation.PSCredential]
        [System.Management.Automation.Credential()]
        $Credential = [System.Management.Automation.PSCredential]::Empty
    )

        $UserName = $Credential.UserName
        $Password = $Credential.GetNetworkCredential().Password

        $splat = @{
            UserName = $UserName
            Password = $Password
            ServerInstance = 'SQLServer'
            Query = "Select * from Sys.Databases"
        }

        Invoke-Sqlcmd @splat
}

$credSplat = @{
    TypeName = 'System.Management.Automation.PSCredential'
    ArgumentList = 'duffney',('P@ssw0rd' | ConvertTo-SecureString -AsPlainText -
```

```
Force)
}
$Credential = New-Object @credSplat

Get-AllSQLDatabases -SQLServer SQL01 -Credential $Credential
```

Continued learning credential management

Creating and storing credential objects securely can be difficult. The following resources can help you maintain PowerShell credentials.

- BetterCredentials
- Azure Key Vault
- Vault Project
- SecretManagement module

Last updated on 08/06/2026

Avoid assigning variables in expressions

Article • 11/17/2022

PowerShell allows you to use assignments within expressions by enclosing the assignment in parentheses `()`. PowerShell passes the assigned value through. For example:

PowerShell

```
# In an `if` conditional
if ($foo = Get-Item $PROFILE) { "$foo exists" }

# Property access
($profileFile = Get-Item $PROFILE).LastWriteTime

# You can even *assign* to such expressions.
($profileFile = Get-Item $PROFILE).LastWriteTime = Get-Date
```

> **Note**
>
> While this syntax is allowed, its use is discouraged. There are cases where this does not work and the intent of the code author can be confusing to other code reviewers.

Limitations

The assignment case doesn't always work. When it doesn't work, the assignment is discarded. If you create an instance of a *mutable* value type and attempt to both save the instance in a variable and modify one of its properties in the same expression, the property assignment is discarded.

PowerShell

```
# create mutable value type
PS> Add-Type 'public struct Foo { public int x; }'

# Create an instance, store it in a variable, and try to modify its
property.
# This assignment is effectively IGNORED.
PS> ($var = [Foo]::new()).x = 1
PS> $var.x
0
```

The difference is that you can't return a reference to the value. Essentially, `($var = [Foo]::new())` is equivalent to `$($var = [Foo]::new(); $var)`. You're no longer performing a member access on the variable you're performing a member access on the variable's output, which is a copy.

The workaround is to create the instance and save it in a variable first, and then assign to the property via the variable:

PowerShell

```
# create mutable value type
PS> Add-Type 'public struct Foo { public int x; }'

# Create an instance and store it in a variable first
# and then modify its property via the variable.
PS> $var = [Foo]::new()
PS> $var.x = 1
PS> $var.x
1
```

Avoid using Invoke-Expression

Article • 11/17/2022

The `Invoke-Expression` cmdlet should only be used as a *last resort*. In most scenarios, safer and more robust alternatives are available. Forums like Stack Overflow are filled with examples of `Invoke-Expression` misuse. Also note that **PSScriptAnalyzer** has a rule for this. For more information, see AvoidUsingInvokeExpression.

Carefully consider the security implications. When a string from an untrusted source such as user input is passed directly to `Invoke-Expression`, arbitrary commands can be executed. Always consider a different, more robust and secure solution first.

Common scenarios

Consider the following usage scenarios:

- **It's simpler to redirect PowerShell to execute something naturally.** For example:

  PowerShell

  ```
  Get-Content ./file.ps1 | Invoke-Expression
  ```

  These cases are trivially avoidable. The script or code already exists in file or AST form, so you should write a script with parameters and invoke it directly instead of using `Invoke-Expression` on a string.

- **Running a script from a trusted source.** For example, running the install script from the PowerShell repository:

  PowerShell

  ```
  Invoke-WebRequest https://aka.ms/install-powershell.ps1 | Invoke-
  Expression
  ```

  You should only use this interactively. And, while this does make life simpler, this practice should be discouraged.

- **Testing for parsing errors.** The PowerShell team tests for parse errors in the source code using `Invoke-Expression` because that's the only way to turn a parse-time error into a runtime one.

Conclusion

Most other scripting languages have a way to evaluate a string as code, and as an interpreted language, PowerShell must have a way to dynamically run itself. But there's no good reason to use `Invoke-Expression` in a production environment.

References

- Stack Overflow discussion - In what scenario was Invoke-Expression designed to be used?
- PowerShell Blog post - Invoke-Expression Considered Harmful

Limitations of PowerShell transcripts

Article • 04/30/2025

Mixing `Write-Host` output with the output objects, strings, and PowerShell transcription is complicated. There is a subtle interaction between the script and how transcription works with PowerShell pipelines that can have unexpected results.

When you emit objects from your script the formatting of those objects is handled by `Out-Default`. But the formatting can occur after the script has completed and transcription has stopped. This means that the output doesn't get transcribed. Strings are handled differently. Sometimes string output is passed through formatting, but not always. `Write-Host` makes an immediate write to the host process. `Write-Output` is sent through the formatting system. Combining the output of complex objects with writes to the host makes it difficult to predict what gets logged in the transcript.

Scenario 1 - Output of a structured object after all other operations

Consider the following script and its output:

PowerShell

```
PS> Get-Content scenario1.ps1
Start-Transcript scenario1.log -UseMinimalHeader
Write-Host '1'
Write-Output '2'
Get-Location
Write-Host '4'
Write-Output '5'
Stop-Transcript

PS> ./scenario1.ps1
Transcript started, output file is scenario1.log
1
2

4
Path
----
/Users/user1/src/projects/transcript
5
Transcript stopped, output file is
/Users/user1/src/projects/transcript/scenario1.log
```

The output to the console shows the output you expect, but not in the order you expect it. `Write-Host 4` is visible before `Get-Location` because `Write-Host` is optimized to write directly to the host. There's code in transcription that copies the output to the transcript file and the console. Then we have the regular output of `Get-Location` and `Write-Output 5` sent as output of the script.

PowerShell

```
PS> Get-Content scenario1.log
**********************
PowerShell transcript start
Start time: 20191106114858
**********************
Transcript started, output file is s2
1
2

4
**********************
PowerShell transcript end
End time: 20191106114858
**********************
```

Since transcription is turned off before the script exits, it's not rendered in the transcript. The objects were sent to the next consumer in the pipeline. In this case, it's `Out-Default`, which PowerShell inserted automatically. To complicate things further, the output of strings is also optimized in the formatting system. The first `Write-Output 2` gets emitted and captured by the transcript. But the insertion of the `Get-Location` object causes its output to be pushed into the stack of things that need actual formatting, which sets a bit of state for any remaining objects that also may need formatting. This is why the second `Write-Output 5` doesn't get added to the transcript.

Scenario 2 - Move the object emission to the beginning

Consider the following script and its output:

PowerShell

```
PS> Get-Content scenario2.ps1
Start-Transcript scenario2.log -UseMinimalHeader
Get-Location
Write-Host '1'
Write-Output '2'
Get-Location
```

```
Write-Host '4'
Write-Output '5'
Stop-Transcript

PS> ./scenario2.ps1
Transcript started, output file is scenario2.log

1
4
Path
----
/Users/user1/src/projects/transcript
2
5
Transcript stopped, output file is
/Users/user1/src/projects/transcript/scenario2.log
```

We can see that the `Write-Host` commands happen before anything, and then the objects start to come out. The `Write-Output` of a string forces the object to be rendered to the screen, but notice that the transcript contains only the output of `Write-Host`. That's because those string objects are piped to `Out-Default` for formatting after the script turned off transcription.

PowerShell

```
PS> Get-Content scenario2.log
**********************
PowerShell transcript start
Start time: 20220606094609
**********************
Transcript started, output file is s3

1
4
**********************
PowerShell transcript end
End time: 20220606094609
**********************
```

Scenario 3 - Object emitted at the end of the script

For this scenario, the output of the complex object is at the end of the script.

PowerShell

```
PS> Get-Content scenario3.ps1
Start-Transcript scenario3.log -UseMinimalHeader
Write-Host '1'
Write-Output '2'
Write-Host '4'
```

```
Write-Output '5'
Get-Location
Stop-Transcript

PS> ./scenario3.ps1
Transcript started, output file is scenario3.log
1
2
4
5

Path
----
/Users/user1/src/projects/transcript
Transcript stopped, output file is
/Users/user1/src/projects/transcript/scenario3.log
```

The string output from both `Write-Host` and `Write-Output` makes it into the transcript. However, the output from `Get-Location` occurs after transcription has stopped.

```
**********************
PowerShell transcript start
Start time: 20220606100342
**********************
Transcript started, output file is scenario3.log
1
2
4
5

**********************
PowerShell transcript end
End time: 20220606100342
**********************
```

A way to ensure full transcription

This example is a slight variation on the original scenario, but now everything is logged to the transcript. The original code is wrapped in a script block and the formatter explicitly invoked via `Out-Default`.

PowerShell

```
PS> Get-Content scenario4.ps1
Start-Transcript scenario4.log -UseMinimalHeader
. {
    Write-Host '1'
```

```
    Write-Output '2'
    Get-Location
    Write-Host '4'
    Write-Output '5'
} | Out-Default
Stop-Transcript

PS> ./scenario4.ps1
Transcript started, output file is scenario4.log
1
2

4
Path
----
/Users/user1/src/projects/transcript
5

Transcript stopped, output file is
/Users/user1/src/projects/transcript/scenario4.log
```

Notice that the last `Write-Host` call is still out of order, that's because of the optimization in `Write-Host` that doesn't go into the output stream.

PowerShell

```
PS> Get-Content scenario4.log
**********************
PowerShell transcript start
Start time: 20220606101038
**********************
Transcript started, output file is s5
1
2

4
Path
----
/Users/user1/src/projects/transcript
5

**********************
PowerShell transcript end
End time: 20220606101038
**********************
```

Sample scripts for system administration

A collection of examples walks through scenarios for administering systems with PowerShell.

Working with objects

HOW-TO GUIDE

Viewing object structure

Selecting parts of objects

Removing objects from the pipeline

Sorting objects

Creating .NET and COM objects

Using static classes and methods

Getting WMI objects with Get-CimInstance

Manipulating items directly

Managing computers

HOW-TO GUIDE

Changing computer state

Collecting information about computers

Creating Get-WinEvent queries with FilterHashtable

Managing processes & services

HOW-TO GUIDE

Managing processes with process cmdlets

Managing services

Working with printers

Performing networking tasks

Working with software installations

Decode a PowerShell command from a running process

Working with output

CONCEPT

Redirecting output

Using format commands to change output view

Managing drives & files

HOW-TO GUIDE

Managing current location

Managing PowerShell drives

Working with files and folders

Working with files folders and registry keys

Working with registry entries

Working with registry keys

Creating UI elements

HOW-TO GUIDE

Creating a custom input box

Creating a graphical date picker

Multiple selection list boxes

Selecting items from a list box

Viewing object structure

Article • 12/09/2022

Because objects play such a central role in PowerShell, there are several native commands designed to work with arbitrary object types. The most important one is the `Get-Member` command.

The simplest technique for analyzing the objects that a command returns is to pipe the output of that command to the `Get-Member` cmdlet. The `Get-Member` cmdlet shows you the formal name of the object type and a complete listing of its members. The number of elements that are returned can sometimes be overwhelming. For example, a process object can have over 100 members.

The following command allows you to see all the members of a **Process** object and page through the output.

PowerShell

```powershell
Get-Process | Get-Member | Out-Host -Paging
```

Output

```
TypeName: System.Diagnostics.Process

Name                           MemberType     Definition
----                           ----------     ----------
Handles                        AliasProperty  Handles = Handlecount
Name                           AliasProperty  Name = ProcessName
NPM                            AliasProperty  NPM = NonpagedSystemMemorySize
PM                             AliasProperty  PM = PagedMemorySize
VM                             AliasProperty  VM = VirtualMemorySize
WS                             AliasProperty  WS = WorkingSet
add_Disposed                   Method         System.Void
add_Disposed(Event...
...
```

We can make this long list of information more usable by filtering for elements we want to see. The `Get-Member` command lets you list only members that are properties. There are several forms of properties. The cmdlet displays properties of a type using the **MemberType** parameter with the value `Properties`. The resulting list is still very long, but a more manageable:

PowerShell

```powershell
Get-Process | Get-Member -MemberType Properties
```

Output

```
   TypeName: System.Diagnostics.Process

Name                       MemberType     Definition
----                       ----------     ----------
Handles                    AliasProperty  Handles = Handlecount
Name                       AliasProperty  Name = ProcessName
...
ExitCode                   Property       System.Int32 ExitCode {get;}
...
Handle                     Property       System.IntPtr Handle {get;}
...
CPU                        ScriptProperty System.Object CPU
{get=$this.Total...
...
Path                       ScriptProperty System.Object Path
{get=$this.Main...
...
```

> **Note**
>
> The allowed values of MemberType are AliasProperty, CodeProperty, Property, NoteProperty, ScriptProperty, Properties, PropertySet, Method, CodeMethod, ScriptMethod, Methods, ParameterizedProperty, MemberSet, and All.

There are more than 60 properties for a process. By default, PowerShell determines how to display an object type using information stored in XML files that have names ending in `.format.ps1xml`. The formatting definition for process objects is stored in `DotNetTypes.format.ps1xml`.

If you need to look at properties other than those that PowerShell displays by default, you can format the output using the `Format-*` cmdlets.

Selecting parts of objects

You can use the `Select-Object` cmdlet to create new, custom PowerShell objects that contain properties selected from the objects you use to create them. Type the following command to create a new object that includes only the **Name** and **FreeSpace** properties of the **Win32_LogicalDisk** WMI class:

PowerShell

```powershell
Get-CimInstance -Class Win32_LogicalDisk |
    Select-Object -Property Name, FreeSpace
```

Output

```
Name      FreeSpace
----      ---------
C:      50664845312
```

With `Select-Object` you can create calculated properties to display **FreeSpace** in gigabytes rather than bytes.

PowerShell

```powershell
Get-CimInstance -Class Win32_LogicalDisk |
    Select-Object -Property Name, @{
        Label='FreeSpace'
        Expression={($_.FreeSpace/1GB).ToString('F2')}
    }
```

Output

```
Name    FreeSpace
----    ---------
C:      47.18
```

Last updated on 03/24/2025

Removing objects from the pipeline

Article • 12/09/2022

In PowerShell, you often generate and pass along more objects to a pipeline than you want. You can specify the properties of particular objects to display using the `Format-*` cmdlets, but this doesn't help with the problem of removing entire objects from the display. You may want to filter objects before the end of a pipeline, so you can perform actions on only a subset of the initially generated objects.

PowerShell includes a `Where-Object` cmdlet that allows you to test each object in the pipeline and only pass it along the pipeline if it meets a particular test condition. Objects that don't pass the test are removed from the pipeline. You supply the test condition as the value of the **FilterScript** parameter.

Performing simple tests with Where-Object

The value of **FilterScript** is a *script block* - one or more PowerShell commands surrounded by braces (`{}`) - that evaluates to true or false. These script blocks can be simple, but creating them requires knowing about another PowerShell concept, comparison operators. A comparison operator compares the items that appear on each side of it. Comparison operators begin with a hyphen character (`-`) and are followed by a name. Basic comparison operators work on almost any kind of object. The more advanced comparison operators might only work on text or arrays.

> **Note**
>
> By default, PowerShell comparison operators are case-insensitive.

Due to parsing considerations, symbols such as `<`, `>`, and `=` aren't used as comparison operators. Instead, comparison operators are comprised of letters. The basic comparison operators are listed in the following table.

Expand table

Comparison Operator	Meaning	Example (returns true)
-eq	is equal to	1 -eq 1
-ne	isn't equal to	1 -ne 2
-lt	Is less than	1 -lt 2

Comparison Operator	Meaning	Example (returns true)
-le	Is less than or equal to	1 -le 2
-gt	Is greater than	2 -gt 1
-ge	Is greater than or equal to	2 -ge 1
-like	Is like (wildcard comparison for text)	"file.doc" -like "f*.do?"
-notlike	isn't like (wildcard comparison for text)	"file.doc" -notlike "p*.doc"
-contains	Contains	1,2,3 -contains 1
-notcontains	doesn't contain	1,2,3 -notcontains 4

`Where-Object` script blocks use the special variable `$_` to refer to the current object in the pipeline. Here is an example of how it works. If you have a list of numbers, and only want to return the ones that are less than 3, you can use `Where-Object` to filter the numbers by typing:

```
1,2,3,4 | Where-Object {$_ -lt 3}
1
2
```

Filtering based on object properties

Since `$_` refers to the current pipeline object, we can access its properties for our tests.

As an example, we can look at the **Win32_SystemDriver** class in WMI. There might be hundreds of system drivers on a particular system, but you might only be interested in a particular set of the system drivers, such as those that are running. For the **Win32_SystemDriver** class the relevant property is **State**. You can filter the system drivers, selecting only the running ones by typing:

PowerShell

```
Get-CimInstance -Class Win32_SystemDriver |
    Where-Object {$_.State -eq 'Running'}
```

This still produces a long list. You may want to filter to only select the drivers set to start automatically by testing the **StartMode** value as well:

PowerShell

```
Get-CimInstance -Class Win32_SystemDriver |
    Where-Object {$_.State -eq "Running"} |
    Where-Object {$_.StartMode -eq "Auto"}
```

Output

```
DisplayName : RAS Asynchronous Media Driver
Name        : AsyncMac
State       : Running
Status      : OK
Started     : True

DisplayName : Audio Stub Driver
Name        : audstub
State       : Running
Status      : OK
Started     : True
...
```

This gives us a lot of information we no longer need because we know that the drivers are running. In fact, the only information we probably need at this point are the name and the display name. The following command includes only those two properties, resulting in much simpler output:

PowerShell

```
Get-CimInstance -Class Win32_SystemDriver |
    Where-Object {$_.State -eq "Running"} |
    Where-Object {$_.StartMode -eq "Manual"} |
    Format-Table -Property Name,DisplayName
```

Output

```
Name              DisplayName
----              -----------
AsyncMac               RAS Asynchronous Media Driver
bindflt                Windows Bind Filter Driver
bowser                 Browser
CompositeBus           Composite Bus Enumerator Driver
condrv                 Console Driver
HdAudAddService        Microsoft 1.1 UAA Function Driver for High Definition
Audio Service
HDAudBus               Microsoft UAA Bus Driver for High Definition Audio
HidUsb                 Microsoft HID Class Driver
HTTP                   HTTP Service
igfx                   igfx
IntcDAud               Intel(R) Display Audio
```

```
intelppm               Intel Processor Driver
...
```

There are two `Where-Object` elements in the above command, but they can be expressed in a single `Where-Object` element using the `-and` logical operator, like this:

PowerShell

```powershell
Get-CimInstance -Class Win32_SystemDriver |
    Where-Object {($_.State -eq 'Running') -and ($_.StartMode -eq 'Manual')}
|
    Format-Table -Property Name,DisplayName
```

The standard logical operators are listed in the following table.

Expand table

Logical Operator	Meaning	Example (returns true)
`-and`	Logical and; true if both sides are true	`(1 -eq 1) -and (2 -eq 2)`
`-or`	Logical or; true if either side is true	`(1 -eq 1) -or (1 -eq 2)`
`-not`	Logical not; reverses true and false	`-not (1 -eq 2)`
`!`	Logical not; reverses true and false	`!(1 -eq 2)`

Sorting objects

We can organize displayed data to make it easier to scan using the `Sort-Object` cmdlet. `Sort-Object` takes the name of one or more properties to sort on, and returns data sorted by the values of those properties.

Basic sorting

Consider the problem of listing subdirectories and files in the current directory. If we want to sort by **LastWriteTime** and then by **Name**, we can do it by typing:

PowerShell

```
Get-ChildItem |
    Sort-Object -Property LastWriteTime, Name |
    Format-Table -Property LastWriteTime, Name
```

Output

```
LastWriteTime          Name
-------------          ----
11/6/2017 10:10:11 AM  .localization-config
11/6/2017 10:10:11 AM  .openpublishing.build.ps1
11/6/2017 10:10:11 AM  appveyor.yml
11/6/2017 10:10:11 AM  LICENSE
11/6/2017 10:10:11 AM  LICENSE-CODE
11/6/2017 10:10:11 AM  ThirdPartyNotices
11/6/2017 10:10:15 AM  tests
6/6/2018 7:58:59 PM    CONTRIBUTING.md
6/6/2018 7:58:59 PM    README.md
...
```

You can also sort the objects in reverse order by specifying the **Descending** `[switch]` parameter.

PowerShell

```
Get-ChildItem |
  Sort-Object -Property LastWriteTime, Name -Descending |
  Format-Table -Property LastWriteTime, Name
```

Output

```
LastWriteTime          Name
-------------          ----
12/1/2018 10:13:50 PM  reference
12/1/2018 10:13:50 PM  dsc
...
6/6/2018 7:58:59 PM    README.md
6/6/2018 7:58:59 PM    CONTRIBUTING.md
11/6/2017 10:10:15 AM  tests
11/6/2017 10:10:11 AM  ThirdPartyNotices
11/6/2017 10:10:11 AM  LICENSE-CODE
11/6/2017 10:10:11 AM  LICENSE
11/6/2017 10:10:11 AM  appveyor.yml
11/6/2017 10:10:11 AM  .openpublishing.build.ps1
11/6/2017 10:10:11 AM  .localization-config
```

Using hash tables

You can sort different properties in different orders using hash tables in an array. Each hash table uses an **Expression** key to specify the property name as string and an **Ascending** or **Descending** key to specify the sort order by `$true` or `$false`. The **Expression** key is mandatory. The **Ascending** or **Descending** key is optional.

The following example sorts objects in descending **LastWriteTime** order and ascending **Name** order.

PowerShell

```powershell
Get-ChildItem |
  Sort-Object -Property @{ Expression = 'LastWriteTime'; Descending = $true },
                        @{ Expression = 'Name'; Ascending = $true } |
  Format-Table -Property LastWriteTime, Name
```

Output

```
LastWriteTime          Name
-------------          ----
12/1/2018 10:13:50 PM  dsc
12/1/2018 10:13:50 PM  reference
11/29/2018 6:56:01 PM  .openpublishing.redirection.json
11/29/2018 6:56:01 PM  gallery
11/24/2018 10:33:22 AM developer
11/20/2018 7:22:19 PM  .markdownlint.json
...
```

You can also set a scriptblock to the **Expression** key. When running the `Sort-Object` cmdlet, the scriptblock is executed and the result is used for sorting.

The following example sorts objects in descending order by the time span between **CreationTime** and **LastWriteTime**.

PowerShell

```
Get-ChildItem |
    Sort-Object -Property @{ Exp = { $_.LastWriteTime - $_.CreationTime }; Desc =
$true } |
    Format-Table -Property LastWriteTime, CreationTime
```

Output

```
LastWriteTime          CreationTime
-------------          ------------
12/1/2018 10:13:50 PM  11/6/2017 10:10:11 AM
12/1/2018 10:13:50 PM  11/6/2017 10:10:11 AM
11/7/2018 6:52:24 PM   11/6/2017 10:10:11 AM
11/7/2018 6:52:24 PM   11/6/2017 10:10:15 AM
11/3/2018 9:58:17 AM   11/6/2017 10:10:11 AM
10/26/2018 4:50:21 PM  11/6/2017 10:10:11 AM
11/17/2018 1:10:57 PM  11/29/2017 5:48:30 PM
11/12/2018 6:29:53 PM  12/7/2017 7:57:07 PM
...
```

Tips

You can omit the **Property** parameter name as following:

PowerShell

```
Sort-Object LastWriteTime, Name
```

Besides, you can refer to `Sort-Object` by its built-in alias, `sort`:

PowerShell

```
sort LastWriteTime, Name
```

The keys in the hash tables for sorting can be abbreviated as following:

PowerShell

```
Sort-Object @{ e = 'LastWriteTime'; d = $true }, @{ e = 'Name'; a = $true }
```

In this example, the **e** stands for **Expression**, the **d** stands for **Descending**, and the **a** stands for **Ascending**.

To improve readability, you can place the hash tables into a separate variable:

PowerShell

```
$order = @(
  @{ Expression = 'LastWriteTime'; Descending = $true }
  @{ Expression = 'Name'; Ascending = $true }
)

Get-ChildItem |
    Sort-Object $order |
    Format-Table LastWriteTime, Name
```

Last updated on 04/08/2026

Creating .NET and COM objects

> This sample only runs on Windows platforms.

There are software components with .NET Framework and COM interfaces that enable you to perform many system administration tasks. PowerShell lets you use these components, so you aren't limited to the tasks that can be performed by using cmdlets. Many of the cmdlets in the initial release of PowerShell don't work against remote computers. We will demonstrate how to get around this limitation when managing event logs by using the .NET Framework **System.Diagnostics.EventLog** class directly from PowerShell.

Using New-Object for event log access

The .NET Framework Class Library includes a class named **System.Diagnostics.EventLog** that can be used to manage event logs. You can create a new instance of a .NET Framework class by using the `New-Object` cmdlet with the **TypeName** parameter. For example, the following command creates an event log reference:

PowerShell

```
New-Object -TypeName System.Diagnostics.EventLog
```

Output

```
  Max(K) Retain OverflowAction       Entries Name
  ------ ------ --------------       ------- ----
```

Although the command has created an instance of the **EventLog** class, the instance doesn't include any data. that's because we didn't specify a particular event log. How do you get a real event log?

Using constructors with New-Object

To refer to a specific event log, you need to specify the name of the log. `New-Object` has an **ArgumentList** parameter. The arguments you pass as values to this parameter are used by a special startup method of the object. The method is called a **constructor** because it's used to

construct the object. For example, to get a reference to the Application log, you specify the string 'Application' as an argument:

PowerShell

```
New-Object -TypeName System.Diagnostics.EventLog -ArgumentList Application
```

Output

```
Max(K) Retain OverflowAction       Entries Name
------ ------ --------------       ------- ----
16,384      7 OverwriteOlder         2,160 Application
```

> **Note**
>
> Since most of the .NET classes are contained in the **System** namespace, PowerShell automatically attempts to find classes you specify in the **System** namespace if it can't find a match for the typename you specify. This means that you can specify `Diagnostics.EventLog` instead of `System.Diagnostics.EventLog`.

Storing Objects in Variables

You might want to store a reference to an object, so you can use it in the current shell. Although PowerShell lets you do a lot of work with pipelines, lessening the need for variables, sometimes storing references to objects in variables makes it more convenient to manipulate those objects.

The output from any valid PowerShell command can be stored in a variable. Variable names always begin with `$`. If you want to store the Application log reference in a variable named `$AppLog`, type the name of the variable, followed by an equal sign and then type the command used to create the Application log object:

PowerShell

```
$AppLog = New-Object -TypeName System.Diagnostics.EventLog -ArgumentList
Application
```

If you then type `$AppLog`, you can see that it contains the Application log:

PowerShell

```
$AppLog
```

Output

```
  Max(K) Retain OverflowAction        Entries Name
  ------ ------ --------------        ------- ----
  16,384      7 OverwriteOlder          2,160 Application
```

Accessing a remote event log with New-Object

The commands used in the preceding section target the local computer; the `Get-EventLog` cmdlet can do that. To access the Application log on a remote computer, you must supply both the log name and a computer name (or IP address) as arguments.

PowerShell

```
$RemoteAppLog = New-Object -TypeName System.Diagnostics.EventLog Application,
192.168.1.81
$RemoteAppLog
```

Output

```
  Max(K) Retain OverflowAction        Entries Name
  ------ ------ --------------        ------- ----
     512      7 OverwriteOlder            262 Application
```

Now that we have a reference to an event log stored in the `$RemoteAppLog` variable, what tasks can we perform on it?

Clearing an event log with object methods

Objects often have methods that can be called to perform tasks. You can use `Get-Member` to display the methods associated with an object. The following command and selected output show some the methods of the **EventLog** class:

PowerShell

```
$RemoteAppLog | Get-Member -MemberType Method
```

Output

```
   TypeName: System.Diagnostics.EventLog
```

```
Name                      MemberType Definition
----                      ---------- ----------
...
Clear                     Method     System.Void Clear()
Close                     Method     System.Void Close()
...
GetType                   Method     System.Type GetType()
...
ModifyOverflowPolicy      Method     System.Void ModifyOverflowPolicy(Overfl...
RegisterDisplayName       Method     System.Void RegisterDisplayName(String ...
...
ToString                  Method     System.String ToString()
WriteEntry                Method     System.Void WriteEntry(String message),...
WriteEvent                Method     System.Void WriteEvent(EventInstance in...
```

The `Clear()` method can be used to clear the event log. When calling a method, you must always follow the method name by parentheses, even if the method doesn't require arguments. This lets PowerShell distinguish between the method and a potential property with the same name. Type the following to call the **Clear** method:

PowerShell

```
$RemoteAppLog.Clear()
$RemoteAppLog
```

Output

```
  Max(K) Retain OverflowAction        Entries Name
  ------ ------ --------------        ------- ----
     512      7 OverwriteOlder              0 Application
```

Notice that the event log was cleared and now has 0 entries instead of 262.

Creating COM objects with New-Object

You can use `New-Object` to work with Component Object Model (COM) components. Components range from the various libraries included with Windows Script Host (WSH) to ActiveX applications such as Internet Explorer that are installed on most systems.

`New-Object` uses .NET Framework Runtime-Callable Wrappers to create COM objects, so it has the same limitations that .NET Framework does when calling COM objects. To create a COM object, you need to specify the **ComObject** parameter with the Programmatic Identifier or **ProgId** of the COM class you want to use. A complete discussion of the limitations of COM use and determining what ProgIds are available on a system is beyond the scope of this user's

guide, but most well-known objects from environments such as WSH can be used within PowerShell.

You can create the WSH objects by specifying these progids: **WScript.Shell**, **WScript.Network**, **Scripting.Dictionary**, and **Scripting.FileSystemObject**. The following commands create these objects:

PowerShell

```
New-Object -ComObject WScript.Shell
New-Object -ComObject WScript.Network
New-Object -ComObject Scripting.Dictionary
New-Object -ComObject Scripting.FileSystemObject
```

Although most of the functionality of these classes is made available in other ways in Windows PowerShell, a few tasks such as shortcut creation are still easier to do using the WSH classes.

Creating a desktop shortcut with WScript.Shell

One task that can be performed quickly with a COM object is creating a shortcut. Suppose you want to create a shortcut on your desktop that links to the home folder for PowerShell. You first need to create a reference to **WScript.Shell**, which we will store in a variable named `$WshShell`:

PowerShell

```
$WshShell = New-Object -ComObject WScript.Shell
```

`Get-Member` works with COM objects, so you can explore the members of the object by typing:

PowerShell

```
$WshShell | Get-Member
```

Output

```
   TypeName: System.__ComObject#{41904400-be18-11d3-a28b-00104bd35090}

Name                 MemberType           Definition
----                 ----------           ----------
AppActivate          Method               bool AppActivate (Variant, Va...
CreateShortcut       Method               IDispatch CreateShortcut (str...
...
```

`Get-Member` has an optional **InputObject** parameter you can use instead of piping to provide input to `Get-Member`. You would get the same output as shown above if you instead used the command **Get-Member -InputObject $WshShell**. If you use **InputObject**, it treats its argument as a single item. This means that if you have several objects in a variable, `Get-Member` treats them as an array of objects. For example:

PowerShell

```
$a = 1,2,"three"
Get-Member -InputObject $a
```

Output

```
TypeName: System.Object[]
Name                MemberType    Definition
----                ----------    ----------
Count               AliasProperty Count = Length
...
```

The **WScript.Shell CreateShortcut** method accepts a single argument, the path to the shortcut file to create. We could type in the full path to the desktop, but there is an easier way. The desktop is normally represented by a folder named Desktop inside the home folder of the current user. Windows PowerShell has a variable `$HOME` that contains the path to this folder. We can specify the path to the home folder by using this variable, and then add the name of the Desktop folder and the name for the shortcut to create by typing:

PowerShell

```
$lnk = $WshShell.CreateShortcut("$HOME\Desktop\PSHome.lnk")
```

When you use something that looks like a variable name inside double-quotes, PowerShell tries to substitute a matching value. If you use single-quotes, PowerShell doesn't try to substitute the variable value. For example, try typing the following commands:

PowerShell

```
"$HOME\Desktop\PSHome.lnk"
```

Output

```
C:\Documents and Settings\aka\Desktop\PSHome.lnk
```

PowerShell

```
'$HOME\Desktop\PSHome.lnk'
```

Output

```
$HOME\Desktop\PSHome.lnk
```

We now have a variable named `$lnk` that contains a new shortcut reference. If you want to see its members, you can pipe it to `Get-Member`. The output below shows the members we need to use to finish creating our shortcut:

PowerShell

```
$lnk | Get-Member
```

Output

```
TypeName: System.__ComObject#{f935dc23-1cf0-11d0-adb9-00c04fd58a0b}
Name             MemberType   Definition
----             ----------   ----------
...
Save             Method       void Save ()
...
TargetPath       Property     string TargetPath () {get} {set}
```

We need to specify the **TargetPath**, which is the application folder for PowerShell, and then save the shortcut by calling the `Save` method. The PowerShell application folder path is stored in the variable `$PSHOME`, so we can do this by typing:

PowerShell

```
$lnk.TargetPath = $PSHOME
$lnk.Save()
```

Using Internet Explorer from PowerShell

Many applications, including the Microsoft Office family of applications and Internet Explorer, can be automated by using COM. The following examples illustrate some of the typical techniques and issues involved in working with COM-based applications.

You create an Internet Explorer instance by specifying the Internet Explorer ProgId, **InternetExplorer.Application**:

PowerShell

```
$ie = New-Object -ComObject InternetExplorer.Application
```

This command starts Internet Explorer, but doesn't make it visible. If you type `Get-Process`, you can see that a process named `iexplore` is running. In fact, if you exit PowerShell, the process will continue to run. You must reboot the computer or use a tool like Task Manager to end the `iexplore` process.

> **Note**
>
> COM objects that start as separate processes, commonly called *ActiveX executables*, may or may not display a user interface window when they start up. If they create a window but don't make it visible, like Internet Explorer, the focus usually moves to the Windows desktop. You must make the window visible to interact with it.

By typing `$ie | Get-Member`, you can view properties and methods for Internet Explorer. To see the Internet Explorer window, set the **Visible** property to `$true` by typing:

PowerShell

```
$ie.Visible = $true
```

You can then navigate to a specific Web address using the `Navigate` method:

PowerShell

```
$ie.Navigate("https://devblogs.microsoft.com/scripting/")
```

Using other members of the Internet Explorer object model, it's possible to retrieve text content from the Web page. The following command displays the HTML text in the body of the current Web page:

PowerShell

```
$ie.Document.Body.InnerText
```

To close Internet Explorer from within PowerShell, call its `Quit()` method:

PowerShell

```
$ie.Quit()
```

The `$ie` variable no longer contains a valid reference even though it still appears to be a COM object. If you attempt to use it, PowerShell returns an automation error:

PowerShell

```
$ie | Get-Member
```

Output

```
Get-Member : Exception retrieving the string representation for property "Appli
cation" : "The object invoked has disconnected from its clients. (Exception fro
m HRESULT: 0x80010108 (RPC_E_DISCONNECTED))"
At line:1 char:16
+ $ie | Get-Member <<<<
```

You can either remove the remaining reference with a command like `$ie = $null`, or completely remove the variable by typing:

PowerShell

```
Remove-Variable ie
```

> **Note**
>
> There is no common standard for whether ActiveX executables exit or continue to run when you remove a reference to one. Depending on circumstances, such as whether the application is visible, whether an edited document is running in it, and even whether PowerShell is still running, the application may or may not exit. For this reason, you should test termination behavior for each ActiveX executable you want to use in PowerShell.

Getting warnings about .NET Framework-wrapped COM objects

In some cases, a COM object might have an associated .NET Framework **Runtime-Callable Wrapper** (RCW) that's used by `New-Object`. Since the behavior of the RCW may be different from the behavior of the normal COM object, `New-Object` has a **Strict** parameter to warn you

about RCW access. If you specify the **Strict** parameter and then create a COM object that uses an RCW, you get a warning message:

PowerShell

```powershell
$xl = New-Object -ComObject Excel.Application -Strict
```

Output

```
New-Object : The object written to the pipeline is an instance of the type "Mic
rosoft.Office.Interop.Excel.ApplicationClass" from the component's primary interop
assembly. If
this type exposes different members than the IDispatch members , scripts written to
work with this
object might not work if the primary interop assembly isn't installed. At line:1
char:17 + $xl =
New-Object <<<< -ComObject Excel.Application -Strict
```

Although the object is still created, you are warned that it isn't a standard COM object.

Last updated on 02/24/2025

Using static classes and methods

Not all .NET Framework classes can be created using `New-Object`. For example, if you try to create a **System.Environment** or a **System.Math** object with `New-Object`, you will get the following error messages:

PowerShell

```
New-Object System.Environment
```

Output

```
New-Object : Constructor not found. Cannot find an appropriate constructor for
type System.Environment.
At line:1 char:11
+ New-Object  <<<< System.Environment
```

PowerShell

```
New-Object System.Math
```

Output

```
New-Object : Constructor not found. Cannot find an appropriate constructor for
type System.Math.
At line:1 char:11
+ New-Object  <<<< System.Math
```

These errors occur because there is no way to create a new object from these classes. These classes are reference libraries of methods and properties that don't change state. You don't need to create them, you simply use them. Classes and methods such as these are called *static classes* because they're not created, destroyed, or changed. To make this clear we will provide examples that use static classes.

Getting environment data with System.Environment

Usually, the first step in working with an object in Windows PowerShell is to use Get-Member to find out what members it contains. With static classes, the process is a little different

because the actual class isn't an object.

Referring to the static System.Environment class

You can refer to a static class by surrounding the class name with square brackets. For example, you can refer to **System.Environment** by typing the name within brackets. Doing so displays some generic type information:

PowerShell

```
[System.Environment]
```

Output

```
IsPublic IsSerial Name                                     BaseType
-------- -------- ----                                     --------
True     False    Environment                              System.Object
```

> ⓘ Note
>
> As we mentioned previously, Windows PowerShell automatically prepends '**System.**' to type names when you use `New-Object`. The same thing happens when using a bracketed type name, so you can specify **[System.Environment]** as **[Environment]**.

The **System.Environment** class contains general information about the working environment for the current process, which is `powershell.exe` when working within Windows PowerShell.

If you try to view details of this class by typing **[System.Environment] | Get-Member**, the object type is reported as being **System.RuntimeType** , not **System.Environment**:

PowerShell

```
[System.Environment] | Get-Member
```

Output

```
   TypeName: System.RuntimeType
```

To view static members with Get-Member, specify the **Static** parameter:

PowerShell

```
[System.Environment] | Get-Member -Static
```

Output

```
   TypeName: System.Environment

Name                      MemberType Definition
----                      ---------- ----------
Equals                    Method     static System.Boolean Equals(Object ob...
Exit                      Method     static System.Void Exit(Int32 exitCode)
...
CommandLine               Property   static System.String CommandLine {get;}
CurrentDirectory          Property   static System.String CurrentDirectory ...
ExitCode                  Property   static System.Int32 ExitCode {get;set;}
HasShutdownStarted        Property   static System.Boolean HasShutdownStart...
MachineName               Property   static System.String MachineName {get;}
NewLine                   Property   static System.String NewLine {get;}
OSVersion                 Property   static System.OperatingSystem OSVersio...
ProcessorCount            Property   static System.Int32 ProcessorCount {get;}
StackTrace                Property   static System.String StackTrace {get;}
SystemDirectory           Property   static System.String SystemDirectory {...
TickCount                 Property   static System.Int32 TickCount {get;}
UserDomainName            Property   static System.String UserDomainName {g...
UserInteractive           Property   static System.Boolean UserInteractive ...
UserName                  Property   static System.String UserName {get;}
Version                   Property   static System.Version Version {get;}
WorkingSet                Property   static System.Int64 WorkingSet {get;}
TickCount                              ExitCode
```

We can now select properties to view from System.Environment.

Displaying static properties of System.Environment

The properties of System.Environment are also static, and must be specified in a different way than normal properties. We use `::` to indicate to Windows PowerShell that we want to work with a static method or property. To see the command that was used to launch Windows PowerShell, we check the **CommandLine** property by typing:

PowerShell

```
[System.Environment]::CommandLine
```

Output

```
"C:\Program Files\Windows PowerShell\v1.0\powershell.exe"
```

To check the operating system version, display the **OSVersion** property by typing:

PowerShell

```
[System.Environment]::OSVersion
```

Output

```
          Platform ServicePack         Version             VersionString
          -------- -----------         -------             -------------
           Win32NT Service Pack 2      5.1.2600.131072     Microsoft Windows...
```

We can check whether the computer is in the process of shutting down by displaying the **HasShutdownStarted** property:

PowerShell

```
[System.Environment]::HasShutdownStarted
```

Output

```
False
```

Doing math with System.Math

The **System.Math** static class is useful for performing some mathematical operations. The class includes several useful methods, which we can display using `Get-Member`.

> ⓘ Note
>
> **System.Math** has several methods with the same name, but they're distinguished by the type of their parameters.

Type the following command to list the methods of the **System.Math** class.

PowerShell

```
[System.Math] | Get-Member -Static -MemberType Methods
```

Output

```
   TypeName: System.Math

Name            MemberType Definition
----            ---------- ----------
Abs             Method     static System.Single Abs(Single value), static Sy...
Acos            Method     static System.Double Acos(Double d)
Asin            Method     static System.Double Asin(Double d)
Atan            Method     static System.Double Atan(Double d)
Atan2           Method     static System.Double Atan2(Double y, Double x)
BigMul          Method     static System.Int64 BigMul(Int32 a, Int32 b)
Ceiling         Method     static System.Double Ceiling(Double a), static Sy...
Cos             Method     static System.Double Cos(Double d)
Cosh            Method     static System.Double Cosh(Double value)
DivRem          Method     static System.Int32 DivRem(Int32 a, Int32 b, Int3...
Equals          Method     static System.Boolean Equals(Object objA, Object ...
Exp             Method     static System.Double Exp(Double d)
Floor           Method     static System.Double Floor(Double d), static Syst...
IEEERemainder   Method     static System.Double IEEERemainder(Double x, Doub...
Log             Method     static System.Double Log(Double d), static System...
Log10           Method     static System.Double Log10(Double d)
Max             Method     static System.SByte Max(SByte val1, SByte val2), ...
Min             Method     static System.SByte Min(SByte val1, SByte val2), ...
Pow             Method     static System.Double Pow(Double x, Double y)
ReferenceEquals Method     static System.Boolean ReferenceEquals(Object objA...
Round           Method     static System.Double Round(Double a), static Syst...
Sign            Method     static System.Int32 Sign(SByte value), static Sys...
Sin             Method     static System.Double Sin(Double a)
Sinh            Method     static System.Double Sinh(Double value)
Sqrt            Method     static System.Double Sqrt(Double d)
Tan             Method     static System.Double Tan(Double a)
Tanh            Method     static System.Double Tanh(Double value)
Truncate        Method     static System.Decimal Truncate(Decimal d), static...
```

This displays several mathematical methods. Here is a list of commands that demonstrate how some of the common methods work:

PowerShell

```
[System.Math]::Sqrt(9)
3
[System.Math]::Pow(2,3)
8
[System.Math]::Floor(3.3)
3
[System.Math]::Floor(-3.3)
-4
[System.Math]::Ceiling(3.3)
4
[System.Math]::Ceiling(-3.3)
-3
[System.Math]::Max(2,7)
7
```

```
[System.Math]::Min(2,7)
2
[System.Math]::Truncate(9.3)
9
[System.Math]::Truncate(-9.3)
-9
```

Last updated on 03/24/2025

Getting WMI objects with Get-CimInstance

> This sample only applies to Windows platforms.

Windows Management Instrumentation (WMI) is a core technology for Windows system administration because it exposes a wide range of information in a uniform manner. Because of how much WMI makes possible, the PowerShell cmdlet for accessing WMI objects, `Get-CimInstance`, is one of the most useful for doing real work. We're going to discuss how to use the CIM cmdlets to access WMI objects and then how to use WMI objects to do specific things.

Listing WMI classes

The first problem most WMI users face is trying to find out what can be done with WMI. WMI classes describe the resources that can be managed. There are hundreds of WMI classes, some of which contain dozens of properties.

`Get-CimClass` addresses this problem by making WMI discoverable. You can get a list of the WMI classes available on the local computer by typing:

PowerShell

```powershell
Get-CimClass -Namespace root/CIMV2 |
    Where-Object CimClassName -Like Win32* |
    Select-Object CimClassName
```

Output

```
CimClassName
------------
Win32_DeviceChangeEvent
Win32_SystemConfigurationChangeEvent
Win32_VolumeChangeEvent
Win32_SystemTrace
Win32_ProcessTrace
Win32_ProcessStartTrace
Win32_ProcessStopTrace
Win32_ThreadTrace
Win32_ThreadStartTrace
Win32_ThreadStopTrace
...
```

You can retrieve the same information from a remote computer using the **ComputerName** parameter, specifying a computer name or IP address:

PowerShell

```
Get-CimClass -Namespace root/CIMV2 -ComputerName 192.168.1.29
```

The class listing returned by remote computers may vary due to the specific operating system the computer is running and the particular WMI extensions are added by installed applications.

> ⓘ **Note**
>
> When using CIM cmdlets to connect to a remote computer, the remote computer must be running WMI and the account you are using must be in the local **Administrators** group on the remote computer. The remote system doesn't need to have PowerShell installed. This allows you to administer operating systems that aren't running PowerShell, but do have WMI available.

Displaying WMI class details

If you already know the name of a WMI class, you can use it to get information immediately. For example, one of the WMI classes commonly used for retrieving information about a computer is **Win32_OperatingSystem**.

PowerShell

```
Get-CimInstance -Class Win32_OperatingSystem
```

Output

```
SystemDirectory     Organization BuildNumber RegisteredUser SerialNumber
Version
---------------     ------------ ----------- -------------- ------------
-------
C:\WINDOWS\system32 Microsoft    22621       USER1          00330-80000-00000-AA175
10.0.22621
```

Although we're showing all of the parameters, the command can be expressed in a more succinct way. The **ComputerName** parameter isn't necessary when connecting to the local system. We show it to demonstrate the most general case and remind you about the parameter. The **Namespace** defaults to **root/CIMV2**, and can be omitted as well. Finally, most

cmdlets allow you to omit the name of common parameters. With `Get-CimInstance`, if no name is specified for the first parameter, PowerShell treats it as the **Class** parameter. This means the last command could have been issued by typing:

PowerShell

```
Get-CimInstance Win32_OperatingSystem
```

The **Win32_OperatingSystem** class has many more properties than those displayed here. You can use Get-Member to see all the properties. The properties of a WMI class are automatically available like other object properties:

PowerShell

```
Get-CimInstance -Class Win32_OperatingSystem | Get-Member -MemberType Property
```

Output

```
   TypeName:
Microsoft.Management.Infrastructure.CimInstance#root/cimv2/Win32_OperatingSystem
Name                                       MemberType Definition
----                                       ---------- ----------
BootDevice                                 Property   string BootDevice {get;}
BuildNumber                                Property   string BuildNumber {get;}
BuildType                                  Property   string BuildType {get;}
Caption                                    Property   string Caption {get;}
CodeSet                                    Property   string CodeSet {get;}
CountryCode                                Property   string CountryCode {get;}
CreationClassName                          Property   string CreationClassName
{get;}
CSCreationClassName                        Property   string CSCreationClassName
{get;}
CSDVersion                                 Property   string CSDVersion {get;}
CSName                                     Property   string CSName {get;}
CurrentTimeZone                            Property   int16 CurrentTimeZone {get;}
DataExecutionPrevention_32BitApplications Property   bool
DataExecutionPrevention_32BitApplications {get;}
DataExecutionPrevention_Available          Property   bool
DataExecutionPrevention_Available {get;}
...
```

Displaying non-default properties with Format cmdlets

If you want the information contained in the **Win32_OperatingSystem** class that isn't displayed by default, you can display it by using the **Format** cmdlets. For example, if you want to display available memory data, type:

PowerShell

```
Get-CimInstance -Class Win32_OperatingSystem | Format-Table -Property
TotalVirtualMemorySize, TotalVisibleMemorySize, FreePhysicalMemory,
FreeVirtualMemory, FreeSpaceInPagingFiles
```

Output

```
TotalVirtualMemorySize TotalVisibleMemorySize FreePhysicalMemory FreeVirtualMemory
FreeSpaceInPagingFiles
---------------------- ---------------------- ------------------ -----------------
----------------------
              41787920               16622096            9537952          33071884
25056628
```

> **Note**
>
> Wildcards work with property names in `Format-Table`, so the final pipeline element can be reduced to `Format-Table -Property Total*Memory*, Free*`

The memory data might be more readable if you format it as a list by typing:

PowerShell

```
Get-CimInstance -Class Win32_OperatingSystem | Format-List Total*Memory*, Free*
```

Output

```
TotalVirtualMemorySize : 41787920
TotalVisibleMemorySize : 16622096
FreePhysicalMemory     : 9365296
FreeSpaceInPagingFiles : 25042952
FreeVirtualMemory      : 33013484
Name                   : Microsoft Windows 11
Pro|C:\Windows|\Device\Harddisk0\Partition2
```

Last updated on 03/24/2025

Manipulating items directly

The elements that you see in PowerShell drives, such as the files and folders or registry keys, are called *Items* in PowerShell. The cmdlets for working with them item have the noun **Item** in their names.

The output of the `Get-Command -Noun Item` command shows that there are nine PowerShell item cmdlets.

PowerShell

```powershell
Get-Command -Noun Item
```

Output

```
CommandType     Name                                Definition
-----------     ----                                ----------
Cmdlet          Clear-Item                          Clear-Item [-Path] <String[]...
Cmdlet          Copy-Item                           Copy-Item [-Path] <String[]>...
Cmdlet          Get-Item                            Get-Item [-Path] <String[]> ...
Cmdlet          Invoke-Item                         Invoke-Item [-Path] <String[...
Cmdlet          Move-Item                           Move-Item [-Path] <String[]>...
Cmdlet          New-Item                            New-Item [-Path] <String[]> ...
Cmdlet          Remove-Item                         Remove-Item [-Path] <String[...
Cmdlet          Rename-Item                         Rename-Item [-Path] <String>...
Cmdlet          Set-Item                            Set-Item [-Path] <String[]> ...
```

Creating new items

To create a new item in the filesystem, use the `New-Item` cmdlet. Include the **Path** parameter with path to the item, and the **ItemType** parameter with a value of `File` or `Directory`.

For example, to create a new directory named `New.Directory` in the `C:\Temp` directory, type:

PowerShell

```powershell
New-Item -Path C:\temp\New.Directory -ItemType Directory
```

Output

```
    Directory: Microsoft.PowerShell.Core\FileSystem::C:\temp

Mode                 LastWriteTime     Length Name
```

```
----                 -------------     ------ ----
d----         2006-05-18  11:29 AM            New.Directory
```

To create a file, change the value of the **ItemType** parameter to `File`. For example, to create a file named `file1.txt` in the `New.Directory` directory, type:

PowerShell

```
New-Item -Path C:\temp\New.Directory\file1.txt -ItemType File
```

Output

```
    Directory: Microsoft.PowerShell.Core\FileSystem::C:\temp\New.Directory

Mode                 LastWriteTime     Length Name
----                 -------------     ------ ----
-a---         2006-05-18  11:44 AM          0 file1
```

You can use the same technique to create a new registry key. In fact, a registry key is easier to create because the only item type in the Windows registry is a key. (Registry entries are item *properties*.) For example, to create a key named `_Test` in the `CurrentVersion` subkey, type:

PowerShell

```
New-Item -Path HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\_Test
```

Output

```
   Hive:
Microsoft.PowerShell.Core\Registry::HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\C
urrentVersion

SKC  VC Name                           Property
---  -- ----                           --------
  0   0 _Test                         {}
```

When typing a registry path, be sure to include the colon (`:`) in the PowerShell drive names, `HKLM:` and `HKCU:`. Without the colon, PowerShell doesn't recognize the drive name in the path.

Why registry values aren't items

When you use the `Get-ChildItem` cmdlet to find the items in a registry key, you will never see actual registry entries or their values.

For example, the registry key `HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Run` usually contains several registry entries that represent applications that run when the system starts.

However, when you use `Get-ChildItem` to look for child items in the key, all you will see is the `OptionalComponents` subkey of the key:

PowerShell

```
Get-ChildItem HKLM:\Software\Microsoft\Windows\CurrentVersion\Run
```

Output

```
   Hive:
Microsoft.PowerShell.Core\Registry::HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\C
urrentVersion\Run
SKC  VC Name                           Property
---  -- ----                           --------
  3   0 OptionalComponents             {}
```

Although it would be convenient to treat registry entries as items, you can't specify a path to a registry entry in a way that ensures that it's unique. The path notation doesn't distinguish between the registry subkey named **Run** and the **(Default)** registry entry in the **Run** subkey. Furthermore, because registry entry names can contain the backslash character (`\`), if registry entries were items, then you couldn't use the path notation to distinguish a registry entry named `Windows\CurrentVersion\Run` from the subkey that's located in that path.

Renaming existing items

To change the name of a file or folder, use the `Rename-Item` cmdlet. The following command changes the name of the `file1.txt` file to `fileOne.txt`.

PowerShell

```
Rename-Item -Path C:\temp\New.Directory\file1.txt fileOne.txt
```

The `Rename-Item` cmdlet can change the name of a file or a folder, but it can't move an item. The following command fails because it attempts to move the file from the `New.Directory` directory to the Temp directory.

PowerShell

```
Rename-Item -Path C:\temp\New.Directory\fileOne.txt C:\temp\fileOne.txt
```

Output

```
Rename-Item : can't rename because the target specified isn't a path.
At line:1 char:12
+ Rename-Item  <<<< -Path C:\temp\New.Directory\fileOne C:\temp\fileOne.txt
```

Moving items

To move a file or folder, use the `Move-Item` cmdlet.

For example, the following command moves the New.Directory directory from the `C:\temp` directory to the root of the `C:` drive. To verify that the item was moved, include the **PassThru** parameter of the `Move-Item` cmdlet. Without **PassThru**, the `Move-Item` cmdlet doesn't display any results.

PowerShell

```
Move-Item -Path C:\temp\New.Directory -Destination C:\ -PassThru
```

Output

```
    Directory: Microsoft.PowerShell.Core\FileSystem::C:\

Mode                 LastWriteTime     Length Name
----                 -------------     ------ ----
d----         2006-05-18  12:14 PM            New.Directory
```

Copying items

If you are familiar with the copy operations in other shells, you might find the behavior of the `Copy-Item` cmdlet in PowerShell to be unusual. When you copy an item from one location to another, `Copy-Item` doesn't copy its contents by default.

For example, if you copy the `New.Directory` directory from the C: drive to the `C:\temp` directory, the command succeeds, but the files in the New.Directory directory aren't copied.

PowerShell

```
Copy-Item -Path C:\New.Directory -Destination C:\temp
```

If you display the contents of `C:\temp\New.Directory`, you will find that it contains no files:

```
PS> Get-ChildItem -Path C:\temp\New.Directory
PS>
```

Why doesn't the `Copy-Item` cmdlet copy the contents to the new location?

The `Copy-Item` cmdlet was designed to be generic; it isn't just for copying files and folders. Also, even when copying files and folders, you might want to copy only the container and not the items within it.

To copy all of the contents of a folder, include the **Recurse** parameter of the `Copy-Item` cmdlet in the command. If you have already copied the directory without its contents, add the **Force** parameter, which allows you to overwrite the empty folder.

PowerShell

```
Copy-Item -Path C:\New.Directory -Destination C:\temp -Recurse -Force -PassThru
```

Output

```
    Directory: Microsoft.PowerShell.Core\FileSystem::C:\temp

Mode                 LastWriteTime     Length Name
----                 -------------     ------ ----
d----         2006-05-18   1:53 PM            New.Directory

    Directory: Microsoft.PowerShell.Core\FileSystem::C:\temp\New.Directory

Mode                 LastWriteTime     Length Name
----                 -------------     ------ ----
-a---         2006-05-18  11:44 AM          0 file1
```

Deleting items

To delete files and folders, use the `Remove-Item` cmdlet. PowerShell cmdlets, such as `Remove-Item`, that can make significant, irreversible changes will often prompt for confirmation when you enter its commands. For example, if you try to remove the `New.Directory` folder, you will be prompted to confirm the command, because the folder contains files:

PowerShell

```
Remove-Item C:\temp\New.Directory
```

Output

```
Confirm
The item at C:\temp\New.Directory has children and the -Recurse parameter was not
specified. If you continue, all children will be removed with the item. Are you
 sure you want to continue?
[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help
(default is "Y"):
```

Because `Yes` is the default response, to delete the folder and its files, press the `Enter` key. To remove the folder without confirming, use the **Recurse** parameter.

PowerShell

```
Remove-Item C:\temp\New.Directory -Recurse
```

Executing items

PowerShell uses the `Invoke-Item` cmdlet to perform a default action for a file or folder. This default action is determined by the default application handler in the registry; the effect is the same as if you double-click the item in File Explorer.

For example, suppose you run the following command:

PowerShell

```
Invoke-Item C:\WINDOWS
```

An Explorer window that's located in `C:\Windows` appears, just as if you had double-clicked the `C:\Windows` folder.

If you invoke the `Boot.ini` file on a system prior to Windows Vista:

PowerShell

```
Invoke-Item C:\boot.ini
```

If the `.ini` file type is associated with Notepad, the `boot.ini` file opens in Notepad.

Last updated on 03/24/2025

Changing computer state

Article • 12/18/2023

> This sample only applies to Windows platforms.

To reset a computer in PowerShell, use either a standard command-line tool, WMI, or a CIM class. Although you are using PowerShell only to run the tool, learning how to change a computer's power state in PowerShell illustrates some of the important details about working with external tools in PowerShell.

Locking a computer

The only way to lock a computer directly with the standard available tools is to call the **LockWorkstation()** function in **user32.dll**:

PowerShell

```
rundll32.exe user32.dll,LockWorkStation
```

This command immediately locks the workstation. It uses `rundll32.exe` to call the `LockWorkStation` function in `user32.dll`.

When you lock a workstation while Fast User Switching is enabled, such as on Windows XP, the computer displays the user logon screen rather than starting the current user's screensaver.

To shut down particular sessions on a Terminal Server, use the **tsshutdn.exe** command-line tool.

Logging off the current session

You can use several different techniques to log off of a session on the local system. The simplest way is to use the Remote Desktop/Terminal Services command-line tool, **logoff.exe** (For details, at the PowerShell prompt, type `logoff /?`). To log off the current active session, type `logoff` with no arguments.

You can also use the **shutdown.exe** tool with its logoff option:

PowerShell

```
shutdown.exe -l
```

Another option is to use WMI. The **Win32_OperatingSystem** class has a **Shutdown** method. Invoking the method with the 0 flag initiates logoff:

For more information, see the Shutdown method of the **Win32_OperatingSystem** class.

PowerShell

```
Get-CimInstance -ClassName Win32_OperatingSystem | Invoke-CimMethod -
MethodName Shutdown
```

Shutting down or restarting a computer

Shutting down and restarting computers are similar tasks. Most command-line tools support both actions. Windows includes two command-line tools for restarting a computer. Use either `tsshutdn.exe` or `shutdown.exe` with appropriate arguments. You can get detailed usage information from `tsshutdn.exe /?` or `shutdown.exe /?`.

You can also perform shutdown and restart operations directly from PowerShell.

To shut down the computer, use the `Stop-Computer` command

PowerShell

```
Stop-Computer
```

To restart the operating system, use the Restart-Computer command

PowerShell

```
Restart-Computer
```

To force an immediate restart of the computer, use the -Force parameter.

PowerShell

```
Restart-Computer -Force
```

Collecting information about computers

Article • 12/09/2022

> This sample only applies to Windows platforms.

Cmdlets from **CimCmdlets** module are the most important cmdlets for general system management tasks. All critical subsystem settings are exposed through WMI. Furthermore, WMI treats data as objects that are in collections of one or more items. Because PowerShell also works with objects and has a pipeline that allows you to treat single or multiple objects in the same way, generic WMI access allows you to perform some advanced tasks with very little work.

Listing desktop settings

We'll begin with a command that collects information about the desktops on the local computer.

PowerShell

```
Get-CimInstance -ClassName Win32_Desktop
```

This returns information for all desktops, whether they're in use or not.

> **Note**
>
> Information returned by some WMI classes can be very detailed, and often include metadata about the WMI class.

Because most of these metadata properties have names that begin with **Cim**, you can filter the properties using `Select-Object`. Specify the **-ExcludeProperty** parameter with "Cim*" as the value. For example:

PowerShell

```
Get-CimInstance -ClassName Win32_Desktop | Select-Object -ExcludeProperty
"CIM*"
```

To filter out the metadata, use a pipeline operator (|) to send the results of the `Get-CimInstance` command to `Select-Object -ExcludeProperty "CIM*"`.

Listing BIOS Information

The WMI **Win32_BIOS** class returns fairly compact and complete information about the system BIOS on the local computer:

PowerShell

```
Get-CimInstance -ClassName Win32_BIOS
```

Listing Processor Information

You can retrieve general processor information by using WMI's **Win32_Processor** class, although you will likely want to filter the information:

PowerShell

```
Get-CimInstance -ClassName Win32_Processor | Select-Object -ExcludeProperty
"CIM*"
```

For a generic description string of the processor family, you can just return the **SystemType** property:

PowerShell

```
Get-CimInstance -ClassName Win32_ComputerSystem | Select-Object -Property
SystemType

SystemType
----------
X86-based PC
```

Listing computer manufacturer and model

Computer model information is also available from **Win32_ComputerSystem**. The standard displayed output will not need any filtering to provide OEM data:

PowerShell

```
Get-CimInstance -ClassName Win32_ComputerSystem
```

Output

```
Name PrimaryOwnerName Domain    TotalPhysicalMemory Model
Manufacturer
---- ---------------- ------    ------------------- -----
------------
MyPC Jane Doe         WORKGROUP 804765696           DA243A-ABA 6415cl NA910
Compaq Presario 06
```

Your output from commands such as this, which return information directly from some hardware, is only as good as the data you have. Some information isn't correctly configured by hardware manufacturers and may therefore be unavailable.

Listing installed hotfixes

You can list all installed hotfixes by using **Win32_QuickFixEngineering**:

PowerShell

```
Get-CimInstance -ClassName Win32_QuickFixEngineering
```

This class returns a list of hotfixes that looks like this:

Output

```
Source Description     HotFixID  InstalledBy   InstalledOn PSComputerName
------ -----------     --------  -----------   ----------- --------------
       Security Update KB4048951 Administrator 12/16/2017  .
```

For more succinct output, you may want to exclude some properties. Although you can use the `Get-CimInstance`'s **Property** parameter to choose only the **HotFixID**, doing so will actually return more information, because all the metadata is displayed by default:

PowerShell

```
Get-CimInstance -ClassName Win32_QuickFixEngineering -Property HotFixID
```

Output

```
InstalledOn          :
Caption              :
Description          :
InstallDate          :
Name                 :
Status               :
CSName               :
```

```
FixComments          :
HotFixID             : KB4533002
InstalledBy          :
ServicePackInEffect  :
PSComputerName       :
CimClass             : root/cimv2:Win32_QuickFixEngineering
CimInstanceProperties : {Caption, Description, InstallDate, Name…}
CimSystemProperties  :
Microsoft.Management.Infrastructure.CimSystemProperties
...
```

The additional data is returned, because the **Property** parameter in `Get-CimInstance` restricts the properties returned from WMI class instances, not the object returned to PowerShell. To reduce the output, use `Select-Object`:

PowerShell

```
Get-CimInstance -ClassName Win32_QuickFixEngineering -Property HotFixId |
    Select-Object -Property HotFixId
```

Output

```
HotFixId
--------
KB4048951
```

Listing operating system version information

The **Win32_OperatingSystem** class properties include version and service pack information. You can explicitly select only these properties to get a version information summary from **Win32_OperatingSystem**:

PowerShell

```
Get-CimInstance -ClassName Win32_OperatingSystem |
  Select-Object -Property
BuildNumber,BuildType,OSType,ServicePackMajorVersion,ServicePackMinorVersion
```

You can also use wildcards with the **Property** parameter. Because all the properties beginning with either **Build** or **ServicePack** are important to use here, we can shorten this to the following form:

PowerShell

```
Get-CimInstance -ClassName Win32_OperatingSystem |
    Select-Object -Property Build*,OSType,ServicePack*
```

Output

```
BuildNumber             : 18362
BuildType               : Multiprocessor Free
OSType                  : 18
ServicePackMajorVersion : 0
ServicePackMinorVersion : 0
```

Listing local users and owner

General information about local users can be found with a selection of **Win32_OperatingSystem** class properties. You can explicitly select the properties to display like this:

PowerShell

```
Get-CimInstance -ClassName Win32_OperatingSystem |
    Select-Object -Property NumberOfLicensedUsers, NumberOfUsers,
RegisteredUser
```

A more succinct version using wildcards is:

PowerShell

```
Get-CimInstance -ClassName Win32_OperatingSystem | Select-Object -Property
*user*
```

Getting available disk space

To see the disk space and free space for local drives, you can use the **Win32_LogicalDisk** class. You need to see only instances with a **DriveType** of 3, the value WMI uses for fixed hard disks.

PowerShell

```
Get-CimInstance -ClassName Win32_LogicalDisk -Filter "DriveType=3"
```

Output

```
DeviceID DriveType ProviderName VolumeName Size         FreeSpace
PSComputerName
-------- --------- ------------ ---------- ----         ---------   --------
------
C:       3                     Local Disk 203912880128 65541357568 .
Q:       3                     New Volume 122934034432 44298250240 .
```

PowerShell

```
Get-CimInstance -ClassName Win32_LogicalDisk -Filter "DriveType=3" |
    Measure-Object -Property FreeSpace,Size -Sum |
    Select-Object -Property Property,Sum
```

Output

```
Property           Sum
--------           ---
FreeSpace 109839607808
Size      326846914560
```

Getting logon session information

You can get general information about logon sessions associated with users through the **Win32_LogonSession** WMI class:

PowerShell

```
Get-CimInstance -ClassName Win32_LogonSession
```

Getting the user logged on to a computer

You can display the user logged on to a particular computer system using **Win32_ComputerSystem**. This command returns only the user logged on to the system desktop:

PowerShell

```
Get-CimInstance -ClassName Win32_ComputerSystem -Property UserName
```

Getting local time from a computer